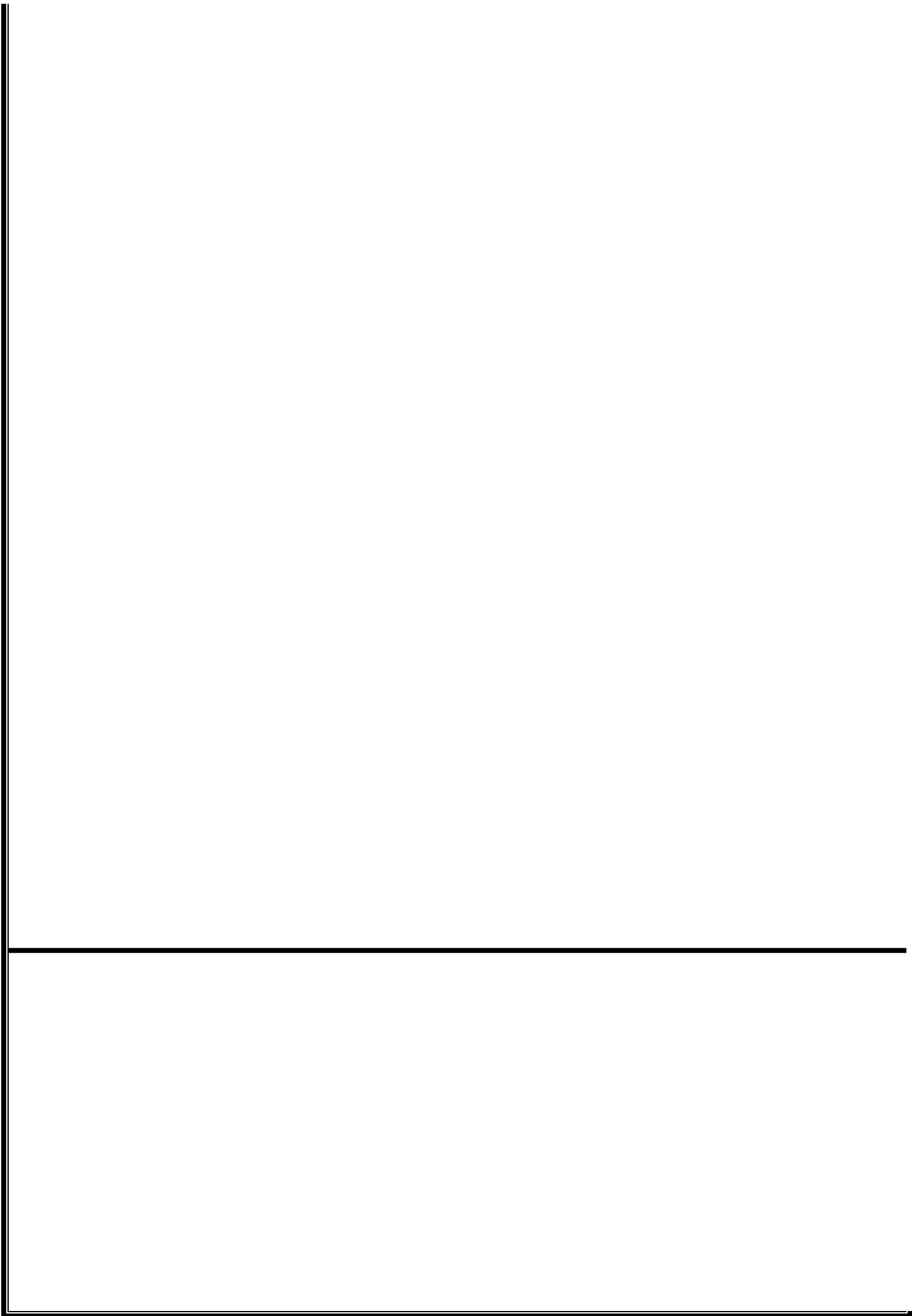




ABHI TECH HOUSE



ORACLE 12C

SQL & PL/SQL



ABHIRAM NAIDU

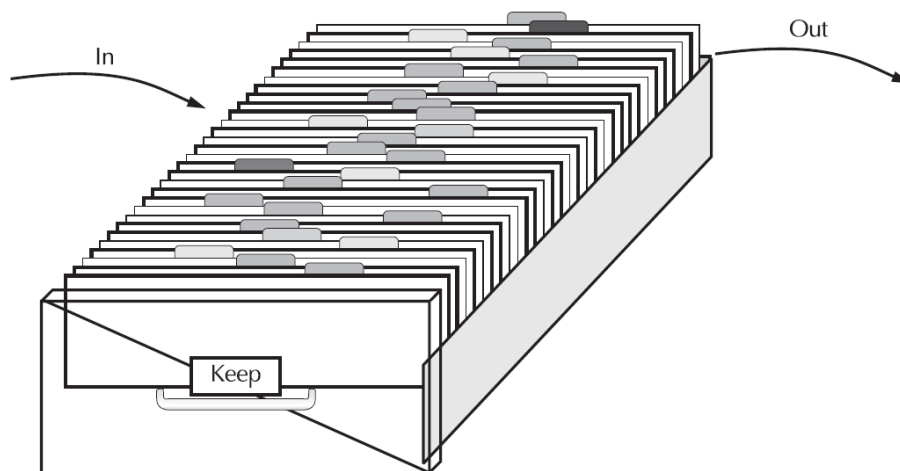
ORACLE

Oracle Database 12c is a significant upgrade from prior releases of Oracle. New features give developers, database administrators, and end users greater control over the storage, processing, and retrieval of their data.

A relational database management system (RDBMS) such as Oracle gives you a way of doing these tasks in an understandable and reasonably uncomplicated way.

- i) Lets you put data into it
- ii) Lets you get the data out
- iii) and work with it

Oracle supports this in/keep/out approach and provides clever tools that allow you considerable sophistication in how the data is captured, edited, modified, and put in; how you keep it securely; and how you get it out to manipulate and report on it.



Oracle is a RELATIONAL DATA BASE MANAGEMENT SYSTEM(RDBMS)
Oracle data base is a software where we can store and process (fetch / insert / change / delete) business data.
Oracle is an RDBMS s/w from oracle corp.

A Timeline of Database History

Ancient Times: Human beings began to store information very long ago. In the ancient times, elaborate database systems were developed by

government offices, libraries, hospitals, and business organizations, and some of the basic principles of these systems are still being used today.

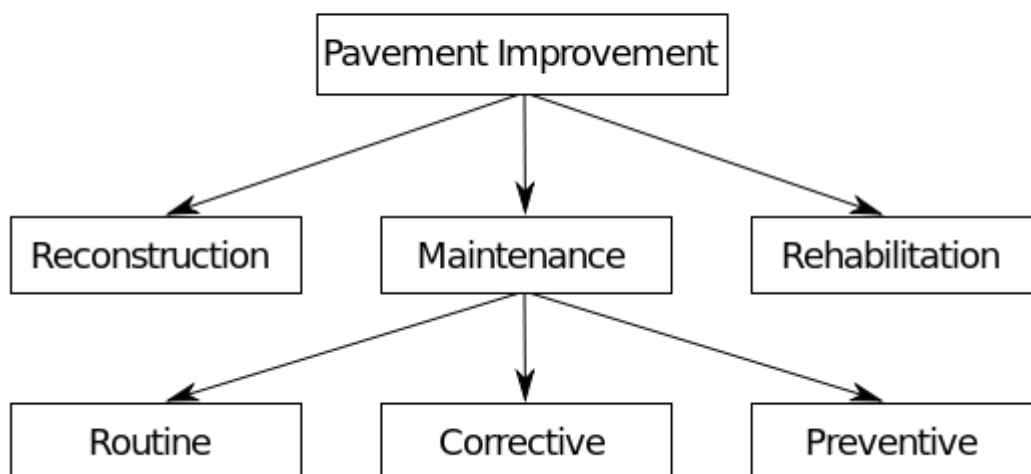
1960s: Computerized database started in the 1960s, when the use of computers became a more cost-effective option for private organizations. There were two popular data models in this decade: **a network model** called **CODASYL** and **a hierarchical model** called **IMS**. One database system that proved to be a commercial success was the SABRE system that was used by IBM to help American Airlines manage its reservations data.

Heirarchical Model

In a [hierarchical model](#), data is organized into a [tree-like structure](#), implying a single parent for each record. A sort field keeps sibling records in a particular order. Hierarchical structures were widely used in the early mainframe database management systems, such as the [Information Management System](#) (IMS) by [IBM](#), and now describe the structure of [XML](#) documents. This structure allows one one-to-many relationship between two types of data. This structure is very efficient to describe many relationships in the real world; recipes, table of contents, ordering of paragraphs/verses, any nested and sorted information.

This hierarchy is used as the physical order of records in storage. Record access is done by navigating downward through the data structure using [pointers](#) combined with sequential accessing. Because of this, the hierarchical structure is inefficient for certain database operations when a full path (as opposed to upward link and sort field) is not also included for each record. Such limitations have been compensated for in later IMS versions by additional logical hierarchies imposed on the base physical hierarchy.

Hierarchical Model



Network Model

The [network model](#) expands upon the hierarchical structure, allowing many-to-many relationships in a tree-like structure that allows multiple parents. It was most popular before being replaced by the relational model, and is defined by the [CODASYL](#) specification.

The network model organizes data using two fundamental concepts, called *records* and *sets*. Records contain fields (which may be organized hierarchically, as in the programming language [COBOL](#)). Sets (not to be confused with mathematical sets) define [one-to-many](#) relationships between records: one owner, many members. A record may be an owner in any number of sets, and a member in any number of sets.

A set consists of circular [linked lists](#) where one record type, the set owner or parent, appears once in each circle, and a second record type, the subordinate or child, may appear multiple times in each circle. In this way a hierarchy may be established between any two record types, e.g., type A is the owner of B. At the same time another set may be defined where B is the owner of A. Thus all the sets comprise a general [directed graph](#) (ownership defines a direction), or *network* construct. **Access to records is either sequential (usually in each record type) or by navigation in the circular linked lists.**

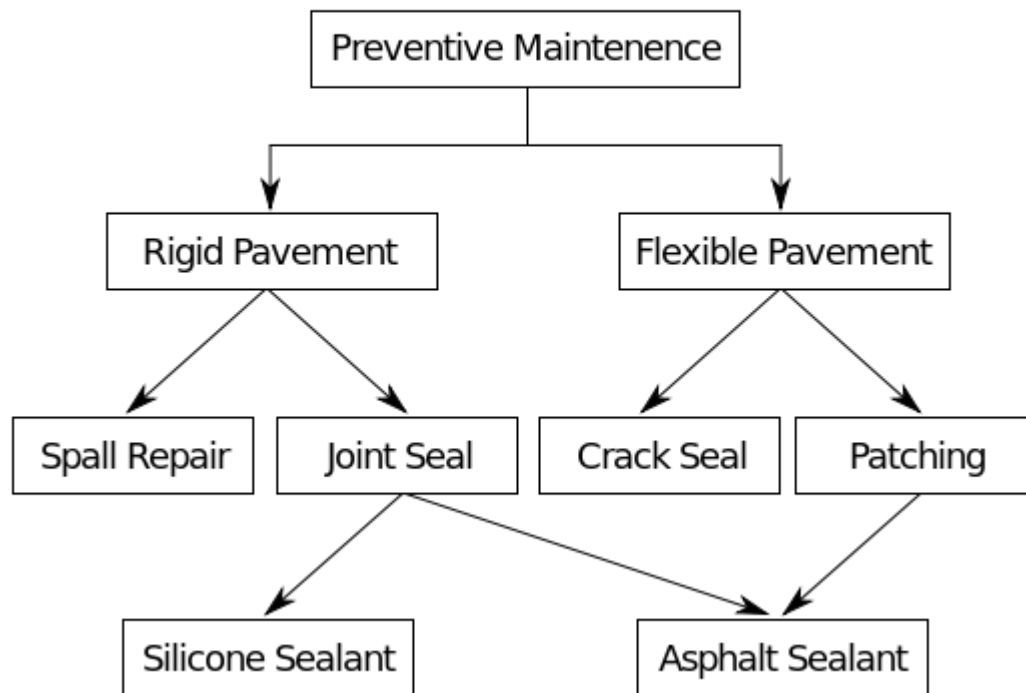
The network model is able to represent redundancy in data more efficiently than in the hierarchical model, and there can be more than one path from an ancestor node to a descendant. The operations of the network model are navigational in style: a program maintains a current position, and navigates from one record to another by following the relationships in which the record participates. Records can also be located by supplying key values.

Although it is not an essential feature of the model, network databases generally implement the set relationships by means of [pointers](#) that directly address the location of a record on disk. This gives excellent retrieval performance, at the expense of operations such as database loading and reorganization.

Popular DBMS products that utilized it were [Cincom Systems'](#) Total and [Cullinet's](#) [IDMS](#). IDMS gained a considerable customer base; in the 1980s, it adopted the relational model and SQL in addition to its original tools and languages.

Most [object databases](#) (invented in the 1990s) use the navigational concept to provide fast navigation across networks of objects, generally using object identifiers as "smart" pointers to related objects. [Objectivity/DB](#), for instance, implements named one-to-one, one-to-many, many-to-one, and many-to-many named relationships that can cross databases. Many object databases also support [SQL](#), combining the strengths of both models.

Network Model



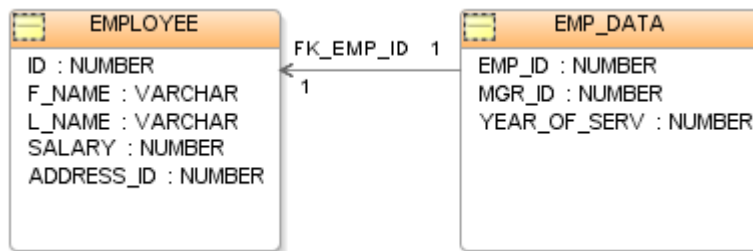
1970 to 1972: E.F. Codd published an important paper to propose the use of a relational database model, and his ideas changed the way people thought about databases. In his model, the database's schema, or logical organization, is disconnected from physical information storage, and this became the standard principle for database systems.

1970s: **Two major relational database system prototypes** were created between the years 1974 and 1977, and they were the **Ingres**, which was developed at UBC, and **System R**, created at IBM San Jose.

Ingres used a query language known as **QUEL**, and it led to the creation of systems such as Ingres Corp., MS SQL Server, Sybase, Wang's PACE, and Britton-Lee. On the other hand, System R used the SEQUEL query language, and it contributed to the development of SQL/DS, DB2, Allbase, Oracle, and Non-Stop SQL. It was also in this decade that Relational Database Management System, or RDBMS, became a recognized term.

1976: A new database model called **Entity-Relationship**, or ER, was proposed by P. Chen this year. This model made it possible for designers to focus on data application, instead of logical table structure.

Relational Model



1980s: Structured Query Language, or SQL, became the standard query language.

Relational database systems became a commercial success as the rapid increase in computer sales boosted the database market, and this caused a major decline in the popularity of network and hierarchical database models. DB2 became the flagship database product for IBM, and the introduction of the IBM PC resulted in the establishments of many new database companies and the development of products such as PARADOX, RBASE 5000, RIM, Dbase III and IV, OS/2 Database Manager, and Watcom SQL.

Early 1990s: After a database industry shakeout, most of the surviving companies sold complex database products at high prices. Around this time, new client tools for application development were released, and these included the Oracle Developer, PowerBuilder, VB, and others. A number of tools for personal productivity, such as ODBC and Excel/Access, were also developed. Prototypes for Object Database Management Systems, or ODBMS, were created in the early 1990s.

Mid 1990s: The advent of the Internet led to exponential growth of the database industry. Average desktop users began to use client-server database systems to access computer systems that contained legacy data.

Late 1990s: Increased investment in online businesses resulted in a rise in demand for Internet database connectors, such as Front Page, Active Server Pages, Java Servlets, Dream Weaver, ColdFusion, Enterprise Java Beans, and Oracle Developer 2000. The use of cgi, gcc, MySQL, Apache, and other systems brought open source solution to the Internet. With the increased use of point-of-sale technology, online transaction processing and online analytic processing began to come of age.

2000s: Although the Internet industry experienced a decline in the early 2000s, database applications continue to grow. New interactive applications were developed for PDAs, point-of-sale transactions, and consolidation of vendors. Presently, the three leading database companies in the western world are Microsoft, IBM, and Oracle.

Then there are many changes to Relational Database like

1) Object Oriented database model

2) Dimensional model

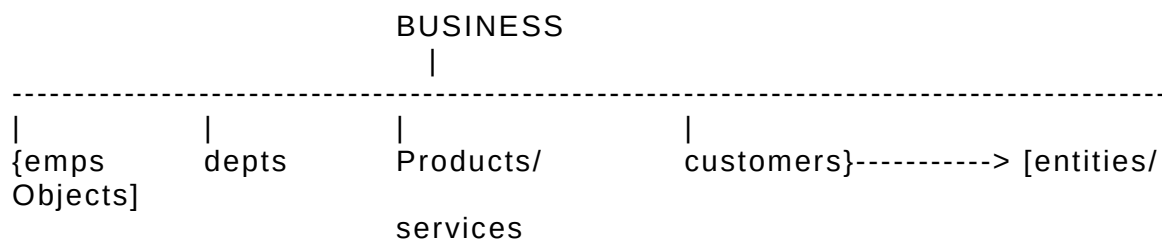
3) Multivalve Model

How the data is generated?

Through business objects and its activities(transactions), data will be generated.

What is a business?

Business is a collection of real world entities and its activities.



In early days of a business, we have very limited data like

- limited number of employees
- limited departments
- limited products
- limited customers (No customers on the very beginning day)

Day-By-Day the business may improve, means need more number of business resources and there is an increase in number of transactions. In this case we need the automated Business system called DBMS, to maintain business data and its activities automatically.

DATA BASE CONCEPTS:

DATA:

Collection of information of any one Business entity is known as data.
[One line of information]

Ex: one employee information
One product information
One sales transaction information

DATA BASE:

It is software which stores and manages the collection of information of all objects in the business.

Technically, it is collection of programs and each program is responsible for performing a specific task.

DATABASE MANAGEMENT SYSTEM:

DB which is comprised with management system services is known as DBMS.

Here the services are

- Entering new data
- Updating old data with new data
- Deleting unwanted data
- Authenticating the users
- Providing security.

RDBMS:

Author of RDBMS IS E.F. CODD and he invented 12 Rules for an RDBMS. Collection of interrelated data of all interrelated objects with in the business is known as RDBMS.

The relation between the tables is implemented by using Referential integrity constraints.

In any **RDBMS**

- Data stored in the form of 2-dimensional tables
- A table is a collection of rows and columns.
- A row is known as record (collection of columns)
- A column is known as a field

Tables of Information

Oracle stores information in tables. Each of these tables has one or more columns. The information is stored row after row. Each unique set of data gets its own row.

Oracle avoids specialized, academic terminology in order to make the product more

approachable. In research papers on relational theory, a **column** may be called an “**attribute**,” a **row** may be called a “**tuple**” and a **table** may be called an “**entity**.”

For an end user, however, these terms are unnecessary.

Ex:WITHOUT RELATION

We are unable to fetch some kind of required data. Consider the below example. By maintaining data in 2 individual tables [no relation between them] we are unable to fetch relevant data from these tables.

Try to find the answers for below questions.

- 1) Number of products from Sony?
- 2) Company details of product id “p001”?
- 3) Total investment made for Asus products?.....etc.

Prod_Dtls

PID	PNAME	COST	MFG	WARRENTY
P001	Mobile	14000	22-oct-14	1 year
P003	Desktop	27000	14-may-15	3 years

P006	Laptop	35990	11-may-12`	2 years
P004	Tablet	12000	21-mar-13	1 year
P005	Smart phone	37000	10-oct-15	1 year

Comp_Dtls

Comp_code	Comp_name	Country
Cmp1	Sony	Japan
Cmp2	Asus	South Korea

Ex:WITH RELATION

By using **Primary key** of one table we need to implement **foreign key** in other table. This is called as implementing Physical relation between the tables.

See Below example:

Comp_Dtls

Comp_code	Comp_name	Country
Cmp1	Sony	Japan
Cmp2	Asus	South Korea

Prod_Dtls

Pid	Pname	Cost	Mfg	Warrenty	Comp_Code
P001	Mobile	14000	22-oct-14	1 year	Cmp1
P003	Desktop	27000	14-may-15	3 years	Cmp2
P006	Laptop	35990	11-may-12`	2 years	Cmp1
P004	Tablet	12000	21-mar-13	1 year	Cmp1
P005	Smart phone	37000	10-oct-15	1 year	Cmp2

Ex: Information for below requirement is easy now.

Find Number of products from Sony?
Find Number of products from Asus?
Average product cost from any company?
Company details of any product?

Advantages:

We can maintain integrity among table data
We can fetch accurate and complete data.

E.F. CODD Rules

Dr Edgar F. Codd, after his extensive research on the Relational Model of database systems, came up with twelve rules of his own, which according to him, a database must obey in order to be regarded as a true relational database.

These rules can be applied on any database system that manages stored data using only its relational capabilities. This is a foundation rule, which acts as a base for all the other rules.

Rule 1: Information Rule

The data stored in a database, may it be user data or metadata, must be a value of some table cell. Everything in a database must be stored in a table format.

Rule 2: Guaranteed Access Rule

Every single data element (value) is guaranteed to be accessible logically with a combination of table-name, primary-key (row value), and attribute-name (column value). No other means, such as pointers, can be used to access data.

Rule 3: Systematic Treatment of NULL Values

The NULL values in a database must be given a systematic and uniform treatment. This is a very important rule because a NULL can be interpreted as one the following – data is missing, data is not known, or data is not applicable.

Rule 4: Active Online Catalog

The structure description of the entire database must be stored in an online catalog, known as **data dictionary**, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself.

Rule 5: Comprehensive Data Sub-Language Rule

A database can only be accessed using a language having linear syntax that supports data definition, data manipulation, and transaction management operations. This language can be used directly or by means of some application. If the database allows access to data without any help of this language, then it is considered as a violation.

Rule 6: View Updating Rule

All the views of a database, which can theoretically be updated, must also be updatable by the system.

Rule 7: High-Level Insert, Update, and Delete Rule

A database must support high-level insertion, updation, and deletion. This must not be limited to a single row, that is, it must also support union, intersection and minus operations to yield sets of data records.

Rule 8: Physical Data Independence

The data stored in a database must be independent of the applications that access the database. Any change in the physical structure of a database must not have any impact on how the data is being accessed by external applications.

Rule 9: Logical Data Independence

The logical data in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application. This is one of the most difficult rule to apply.

Rule 10: Integrity Independence

A database must be independent of the application that uses it. All its integrity constraints can be independently modified without the need of any change in the application. This rule makes a database independent of the front-end application and its interface.

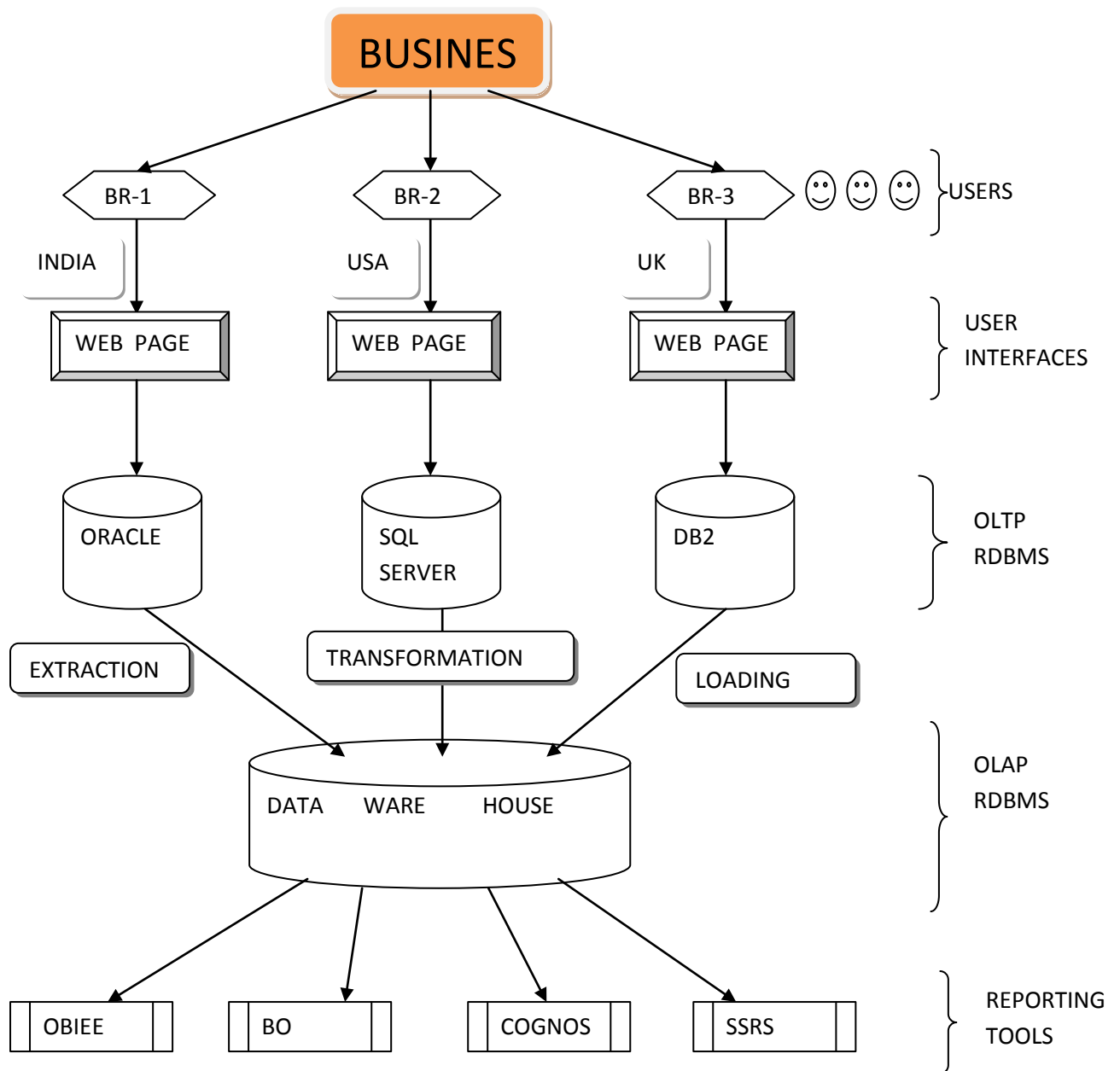
Rule 11: Distribution Independence

The end-user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only. This rule has been regarded as the foundation of distributed database systems.

Rule 12: Non-Subversion Rule

If a system has an interface that provides access to low-level records, then the interface must not be able to subvert the system and bypass security and integrity constraints.

PROJECT DEVELOPMENT PROCESS



OLTP RDBMS: **On Line Transaction Processing**

- It contains transactional data/ day-to-day data/ current data / dynamic data
- It is used to store or process the business data.

OLAP RDBMS: **On Line Analytical Processing**

- It contains historical data / old data.
- It is used to analyze the business.

Brief History of Oracle Database

The current version of Oracle Database is the result of over 35 years of innovative development.

The current version of Oracle Database is the result of over 30 years of innovative development. Highlights in the evolution of Oracle Database include the following:

- Founding of Oracle

In 1977, Larry Ellison, Bob Miner, and Ed Oates started the consultancy Software Development Laboratories, which became Relational Software, Inc. (RSI). In 1983, RSI became Oracle Systems Corporation and then later Oracle Corporation.

- First commercially available RDBMS

In 1979, RSI introduced Oracle V2 (Version 2) as the first commercially available **SQL**-based RDBMS, a landmark event in the history of relational databases.

- Portable version of Oracle Database

Oracle Version 3, released in 1983, was the first relational database to run on mainframes, minicomputers, and PCs. The database was written in C, enabling the database to be ported to multiple platforms.

- Enhancements to concurrency control, data distribution, and scalability

Version 4 introduced multi-version **read consistency**. Version 5, released in 1985, supported client/server computing and **distributed database** systems. Version 6 brought enhancements to disk I/O, row locking, scalability, and backup and recovery. Also, Version 6 introduced the first version of the **PL/SQL** language, a proprietary procedural extension to SQL.

- PL/SQL stored program units

Oracle7, released in 1992, introduced PL/SQL stored procedures and triggers.

- Objects and partitioning

Oracle8 was released in 1997 as the object-relational database, supporting many new data types. Additionally, Oracle8 supported partitioning of large tables.

- Internet computing

Oracle8i Database, released in 1999, provided native support for internet protocols and server-side support for Java. Oracle8i was

designed for internet computing, enabling the database to be deployed in a multitier environment.

- Oracle Real Application Clusters (Oracle RAC)

Oracle9i Database introduced Oracle RAC in 2001, enabling multiple instances to access a single database simultaneously. Additionally, Oracle XML Database (**Oracle XML DB**) introduced the ability to store and query XML.

- Grid computing

Oracle Database 10g introduced **grid computing** in 2003. This release enabled organizations to virtualize computing resources by building a **grid infrastructure** based on low-cost commodity servers. A key goal was to make the database self-managing and self-tuning. **Oracle Automatic Storage Management (Oracle ASM)** helped achieve this goal by vitalizing and simplifying database storage management.

- Manageability, diagnosisability, and availability

Oracle Database 11g, released in 2007, introduced a host of new features that enabled administrators and developers to adapt quickly to changing business requirements. The key to adaptability is simplifying the information infrastructure by consolidating information and using automation wherever possible.

- Plugging In to the Cloud

Oracle Database 12c, released in 2013, was designed for the Cloud, featuring a new Multitenant architecture, In-Memory column store, and support for JSON documents. Oracle Database 12c helps customers make more efficient use of their IT resources, while continuing to reduce costs and improve service levels for users.

Oracle Database Architecture

A **database server** is the key to information management. In general, a **server** reliably manages a large amount of data in a multiuser environment so that users can concurrently access the same data. A database server also prevents unauthorized access and provides efficient solutions for failure recovery.

Database and Instance

An Oracle database server consists of a database and at least one **database instance**, commonly referred to as simply an *instance*. Because an instance and a database are so closely connected, the term **Oracle database** is sometimes used to refer to both instance and database.

- Database

A database is a set of files, located on disk, that store data. These files can exist independently of a database instance.

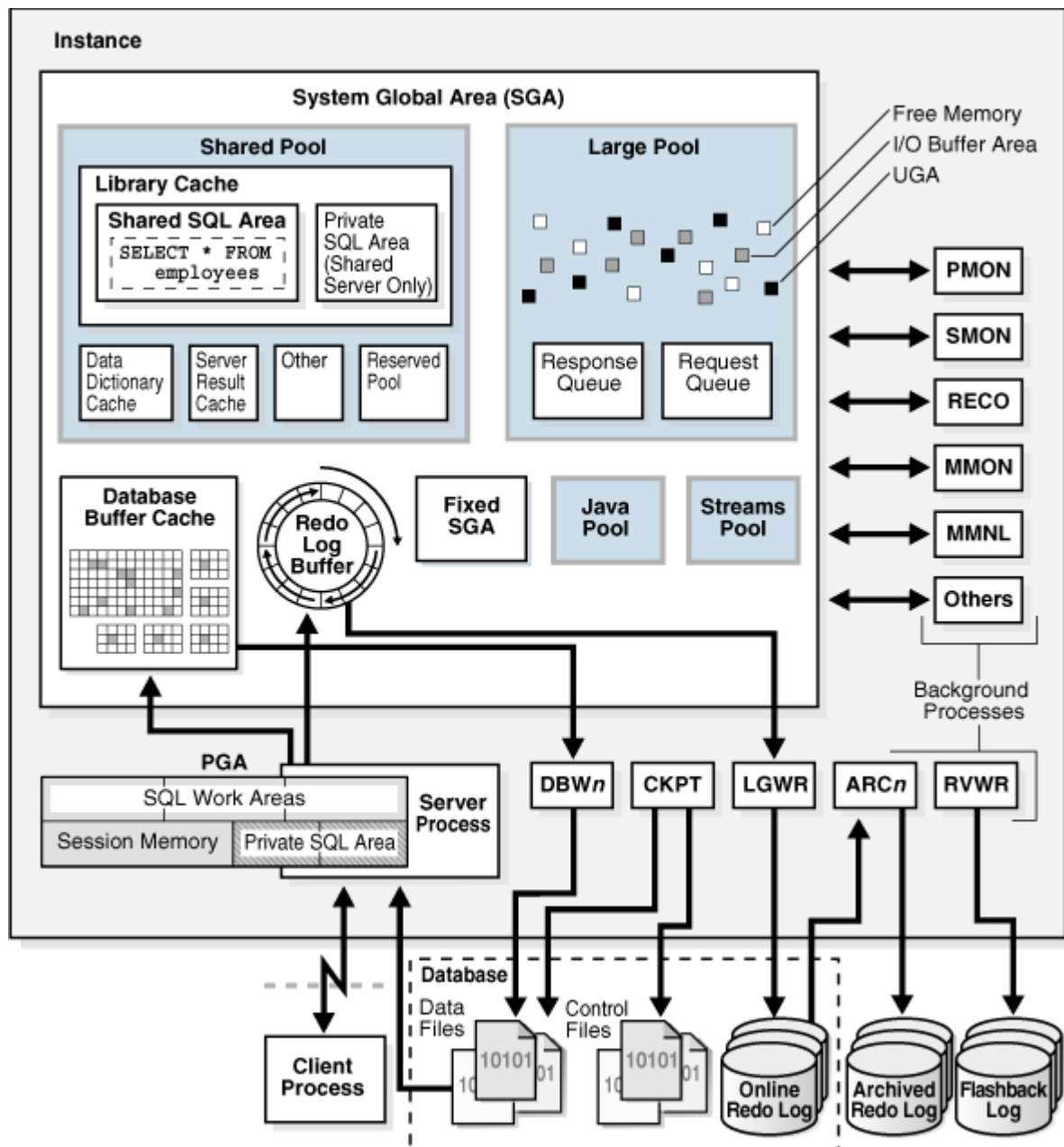
- Database instance

An instance is a set of memory structures that manage database files. The instance consists of a shared memory area, called the **system global area (SGA)**, and a set of background processes. An instance can exist independently of database files.

[Figure 1-1](#) shows a database and its instance.

- For each user connection to the instance, a **client process** runs the application.
- Each client process is associated with its own **server process**.
- The server process has its own private session memory, known as the **program global area (PGA)**.

Oracle Instance and Database



A database can be considered from both a physical and logical perspective. Physical data is data viewable at the operating system level. For example, operating system utilities such as the Linux **ls** and **ps** can list database files and processes. Logical data such as a table is meaningful only for the

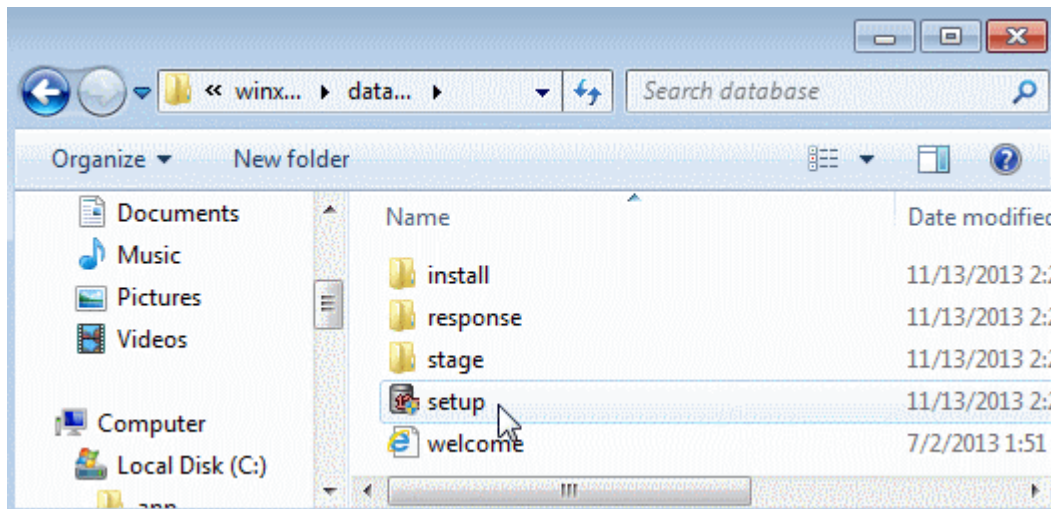
database. A SQL statement can list the tables in an Oracle database, but an operating system utility cannot.

The database has physical structures and logical structures. Because the physical and logical structures are separate, you can manage the physical storage of data without affecting access to logical storage structures. For example, renaming a physical database file does not rename the tables whose data is stored in this file.

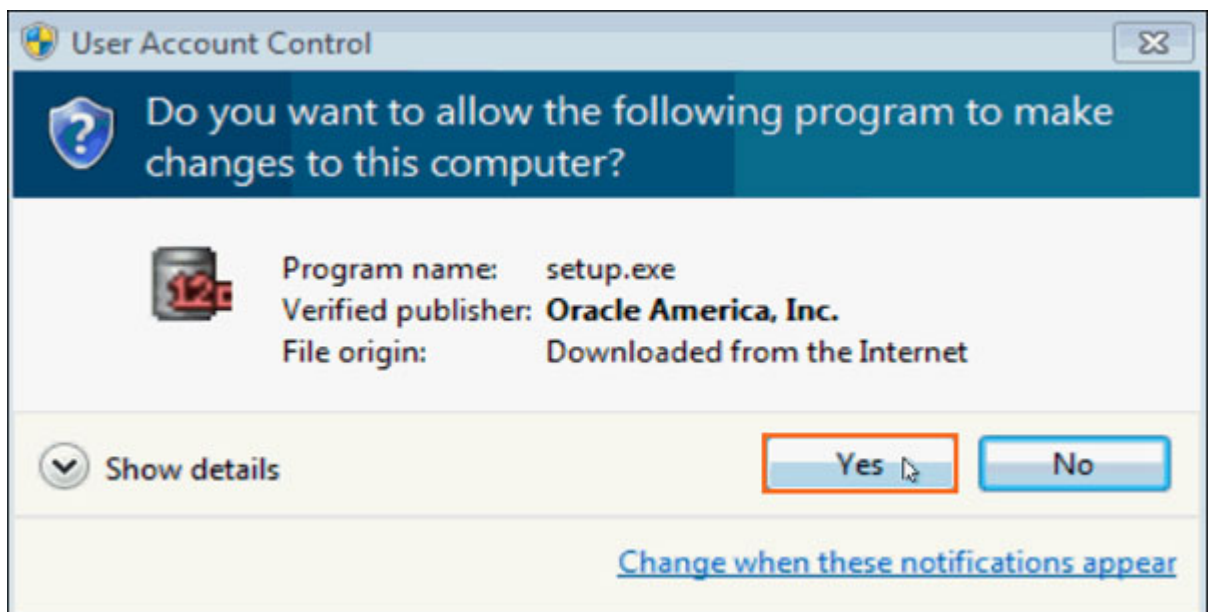
ORACLE-12c Installation

In this section, you will be installing the Oracle Database and creating an Oracle Home User account.

Expand the **database** folder that you extracted in the previous section. Double-click **setup**.



Click **Yes** in the User Account Control window to continue with the installation.



The **Configure Security Updates** window appears. Enter your email address and My Oracle Support password to receive security issue notifications via email. If you do not wish to receive notifications via email, deselect "**I wish to receive security updates via My Oracle**

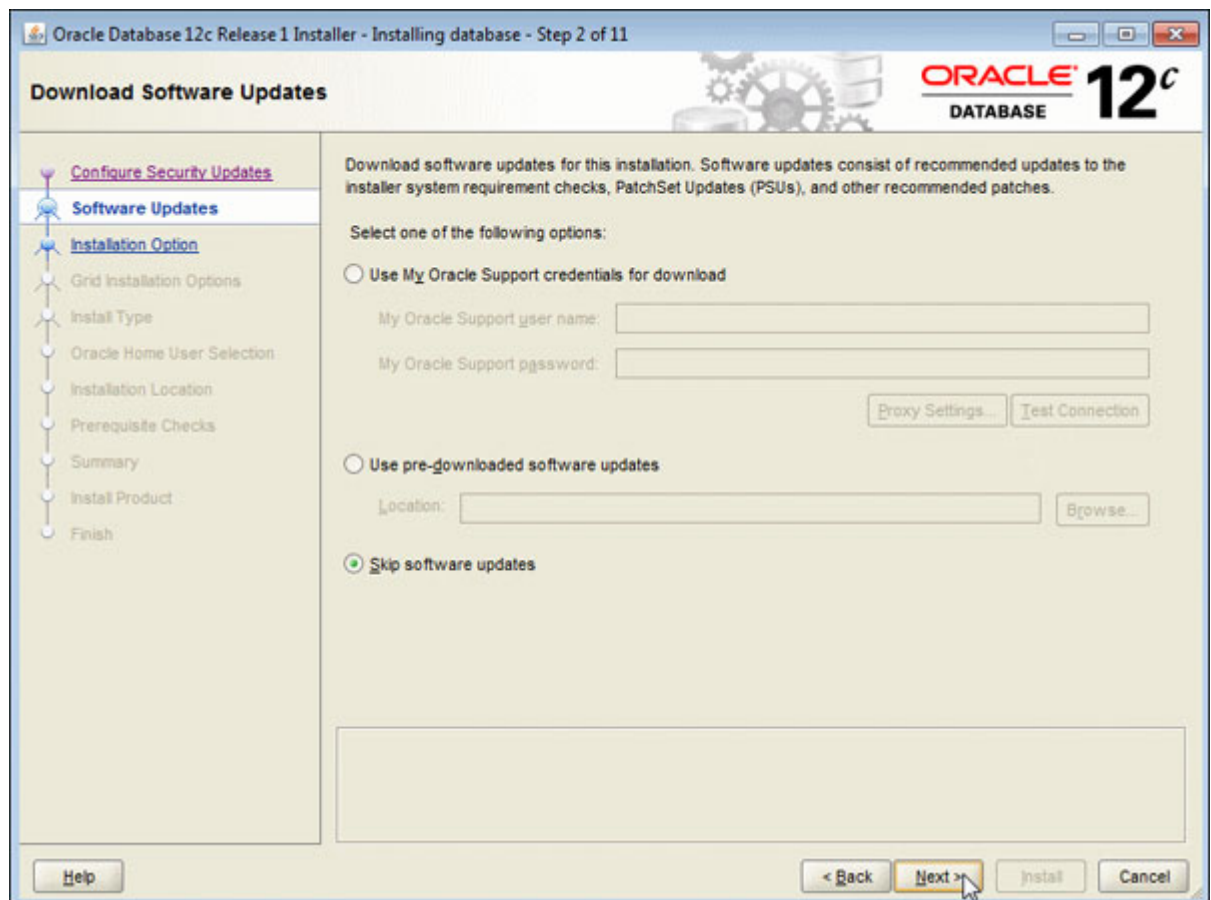
Support". Click **Next** to continue. Click **Yes** in the confirmation window to confirm your preference.



The **Download Software Updates** window appears with the following options:

- o Select "Use My Oracle Support credentials for download" to download and apply the latest software updates.
- o Select "Use pre-downloaded software updates" to apply software updates that you previously downloaded.
- o Select "Skip software updates" if do not want to apply any updates.

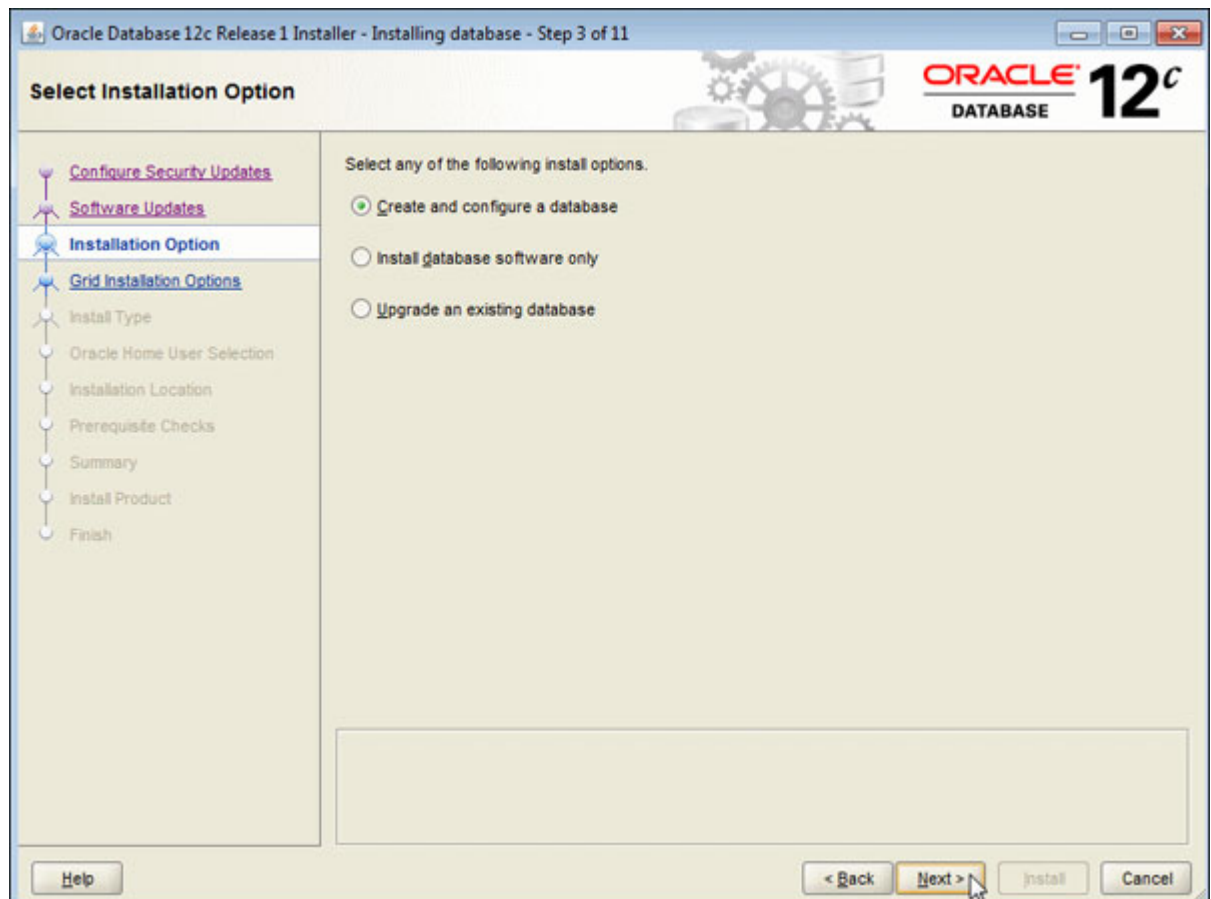
Accept the default and click **Next**.



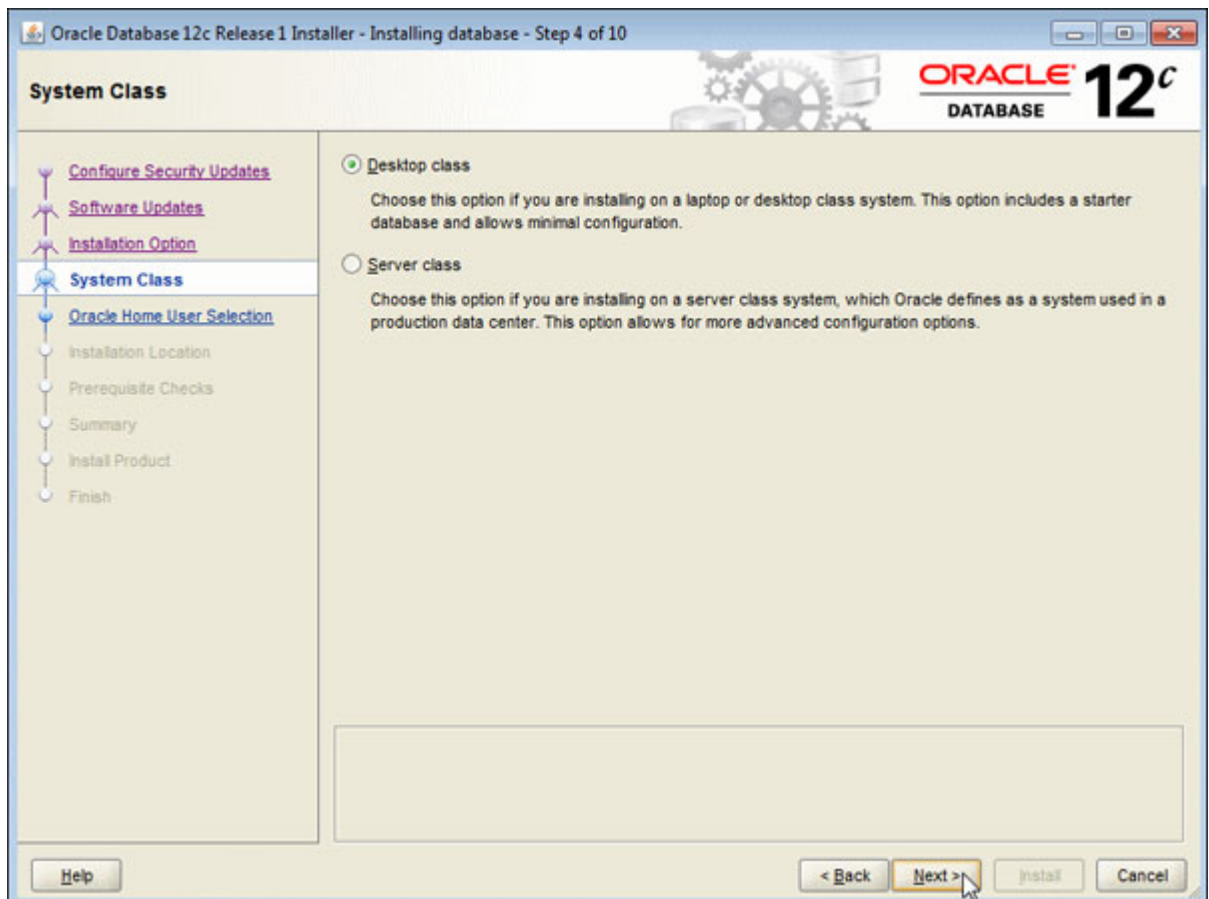
The **Select Installation Option** window appears with the following options:

- o Select "Create and configure a database" to install the database, create database instance and configure the database.
- o Select "Install database software only" to only install the database software.
- o Select "Upgrade an existing database" to upgrade the database that is already installed.

In this OBE, we create and configure the database. Select the **Create and configure a database** option and click **Next**.



The **System Class** window appears. Select Desktop Class or Server Class depending on the type of system you are using. In this OBE, we will perform the installation on a desktop/laptop. Select **Desktop class** and click **Next**.



The **Oracle Home User Selection** window appears. Starting with Oracle Database 12c Release 1 (12.1), Oracle Database on Microsoft Windows supports the use of an Oracle Home User, specified at the time of installation. This Oracle Home User is used to run the Windows services for a Oracle Home, and is similar to the Oracle User on Oracle Database on Linux. This user is associated with an Oracle Home and cannot be changed to a different user post installation.

Note:

- o Different Oracle homes on a system can share the same Oracle Home User or use different Oracle Home Users.
- o The Oracle Home User is different from an Oracle Installation User. The Oracle Installation User is the user who requires administrative privileges to install Oracle products. The Oracle Home User is used to run the Windows services for the Oracle Home.

The window provides the following options:

- o If you select "Use Existing Windows User", the user credentials provided must be a standard Windows user account (not an administrator).

- If this is a single instance database installation, the user can be a local user, a domain user, or a managed services account.
- If this is an Oracle RAC database installation, the existing user must be a Windows domain user. The Oracle installer will display an error if this user has administrator privileges.
- If you select "Create New Windows User", the Oracle installer will create a new standard Windows user account. This user will be assigned as the Oracle Home User. Please note that this user will not have login privileges. This option is not available for an Oracle RAC Database installation.
- If you select "Use Windows Built-in Account", the system uses the Windows Built-in account as the Oracle Home User.

Select the **Create New Windows User** option. Enter the user name as **OracleHomeUser1** and password as **Welcome1**. Click **Next**.

Note: Remember the Windows User password. It will be required later to administer or manage database services.

The **Typical Install Configuration** window appears. Click on a text field and then the balloon icon () to know more about the field.

Note that by default, the installer creates a container database along with a pluggable database called "pdborcl". The pluggable database contains the sample HR schema. Change the Global database name to **orcl**. Enter the "Administrative password" as **Oracle_1**. This password will be used later to log into administrator accounts such as SYS and SYSTEM. Click **Next**.

Oracle Database 12c Release 1 Installer - Installing database - Step 6 of 10

Typical Install Configuration

Perform full database installation with basic configuration.

Oracle base: C:\app\OracleHomeUser1 Browse...

Software location: C:\app\OracleHomeUser1\product12.1.0\dbhome_1 Browse...

Database file location: C:\app\OracleHomeUser1\oradata Browse...

Database edition: Enterprise Edition (6.0GB)

Character set: Default (WE8MSWIN1252)

Global database name: orcl

Administrative password:

Confirm password:

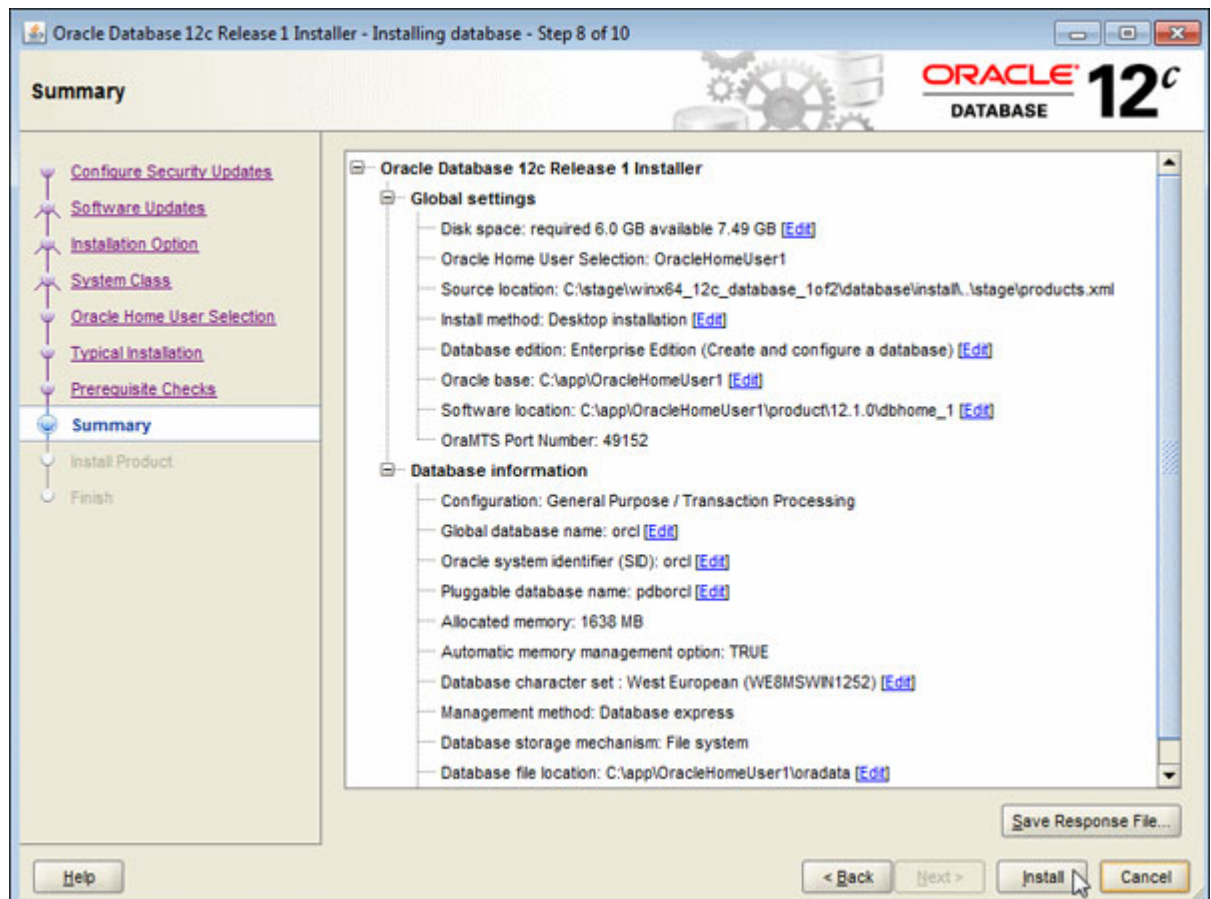
☒ Create as Container database

Pluggable database name: pdborcl

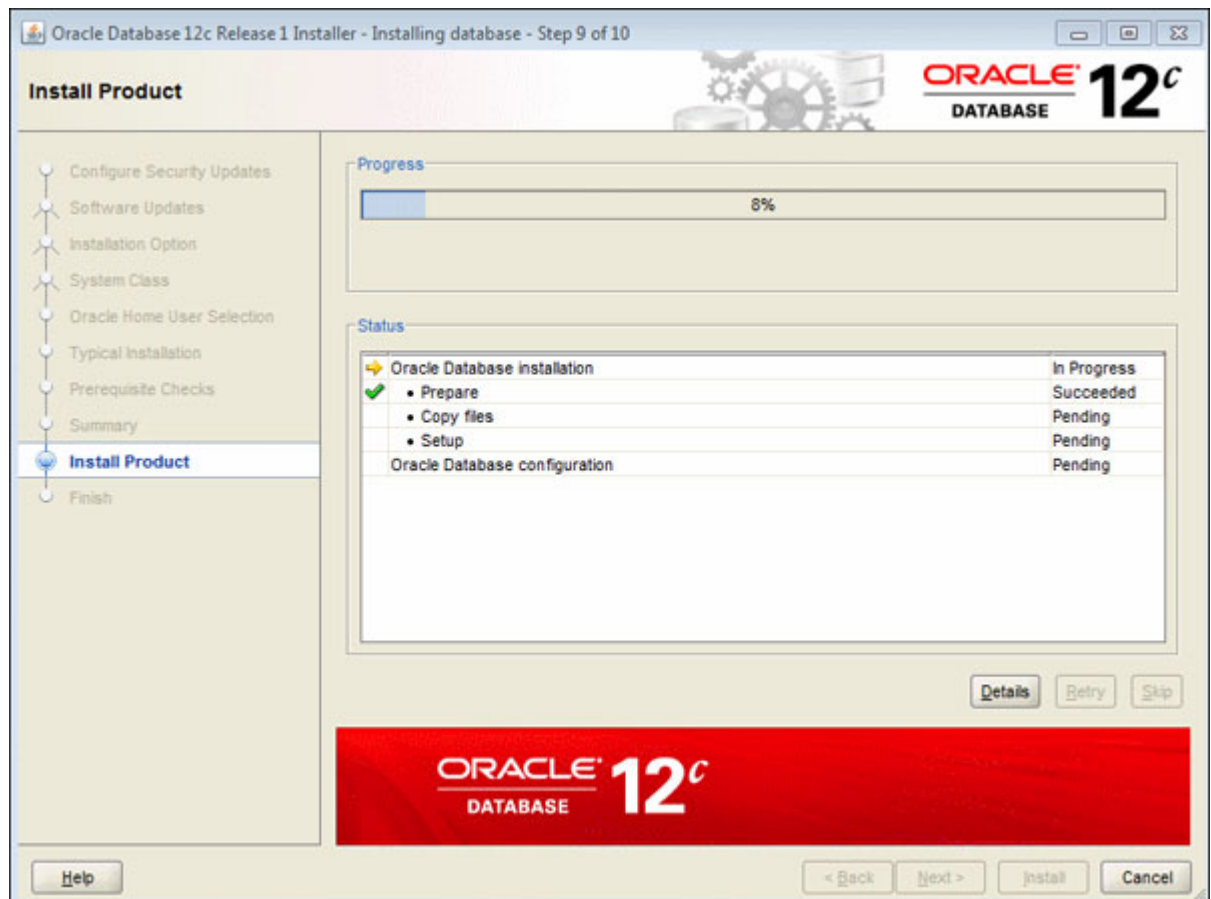
Help < Back Next > Install Cancel

The prerequisite checks are performed and a **Summary** window appears. Review the settings and click **Install**.

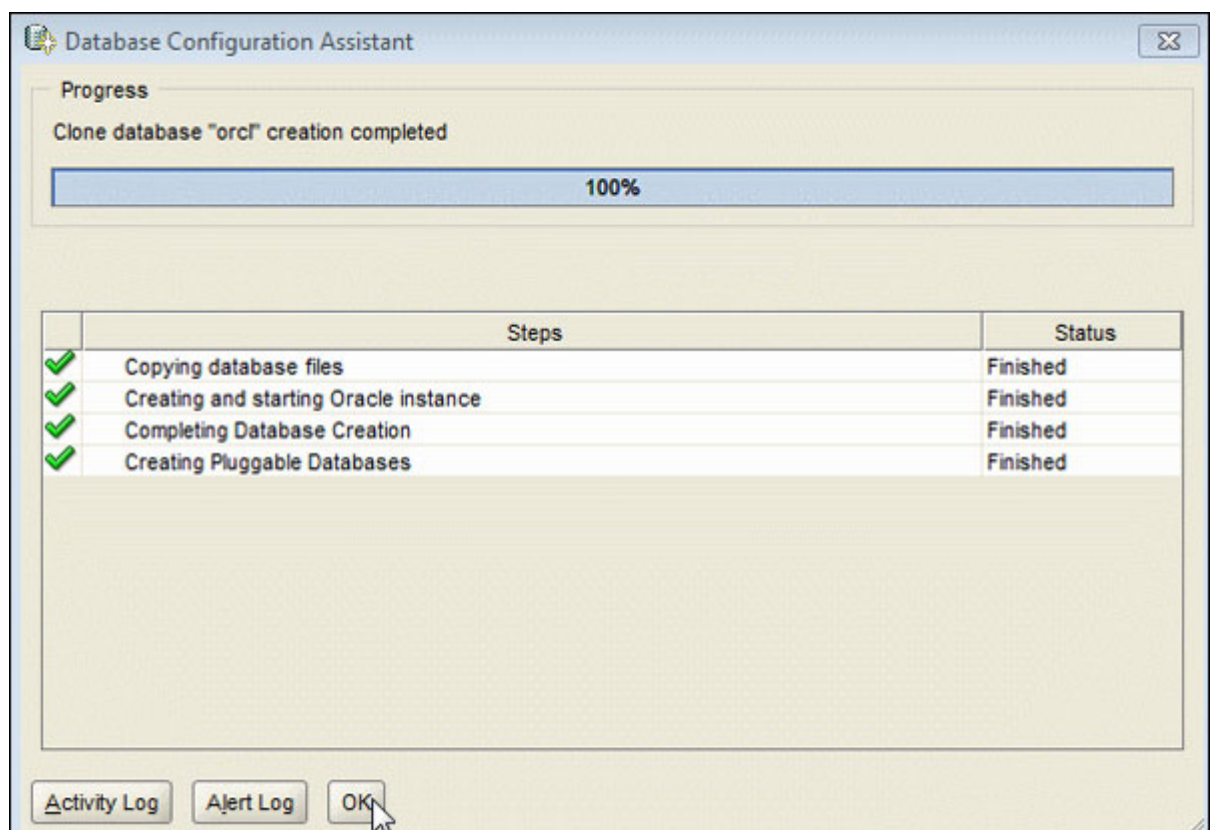
Note: Depending on your firewall settings, you may need to grant permissions to allow java to access the network.



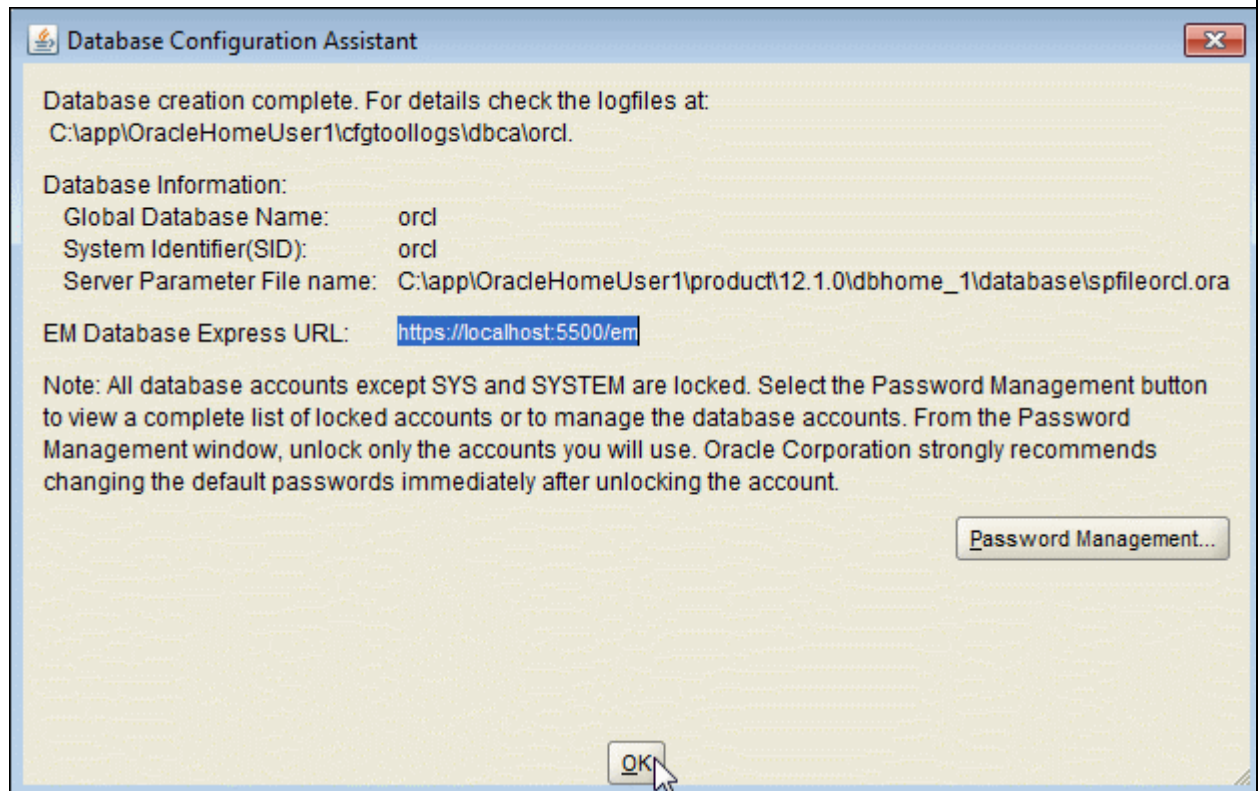
The progress window appears.



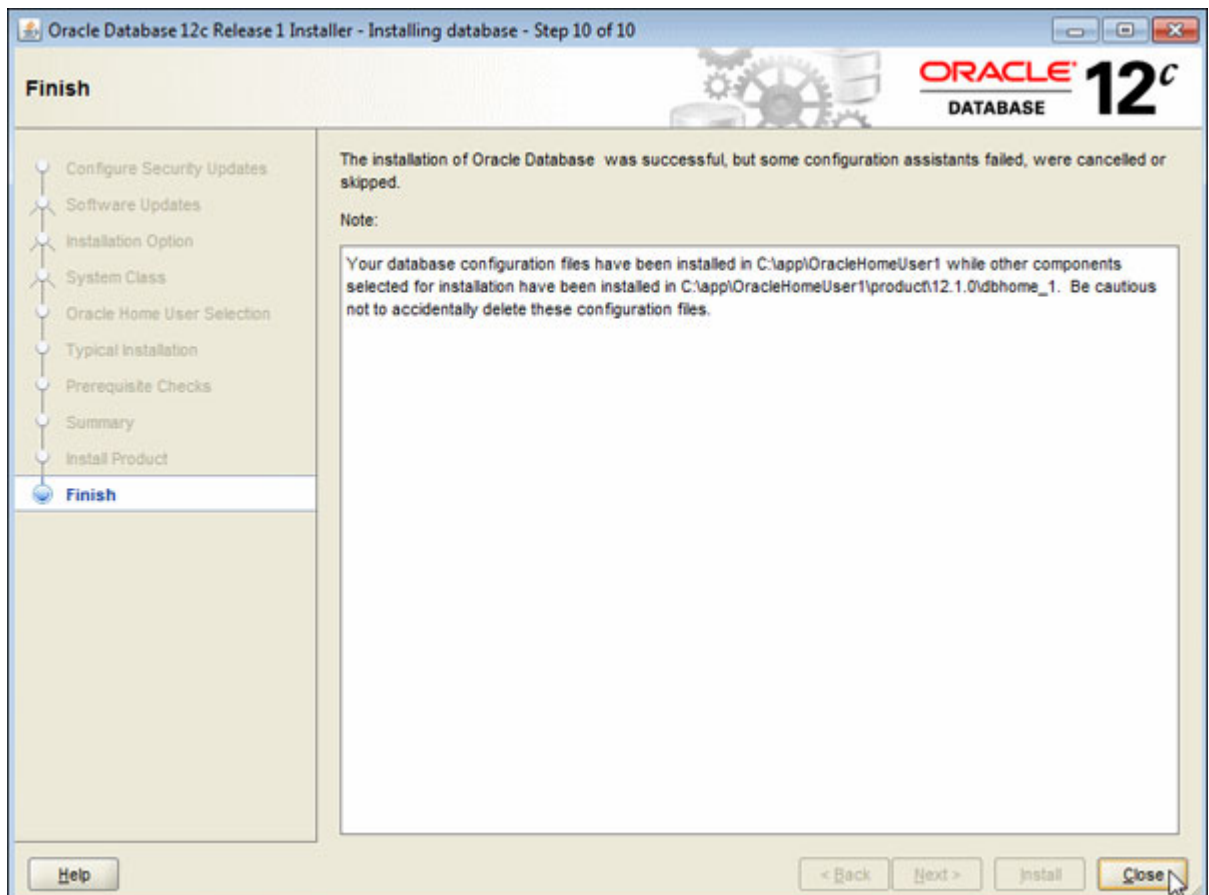
The Database Configuration Assistant creates the database.



After the Database Configuration Assistant creates the database, you can navigate to <https://localhost:5500/em> as a SYS user to manage the database using Enterprise Manager Database Express. You can click "Password Management..." to unlock accounts. Click **OK** to continue.



The **Finish** window appears. Click **Close** to exit the Oracle Universal Installer.



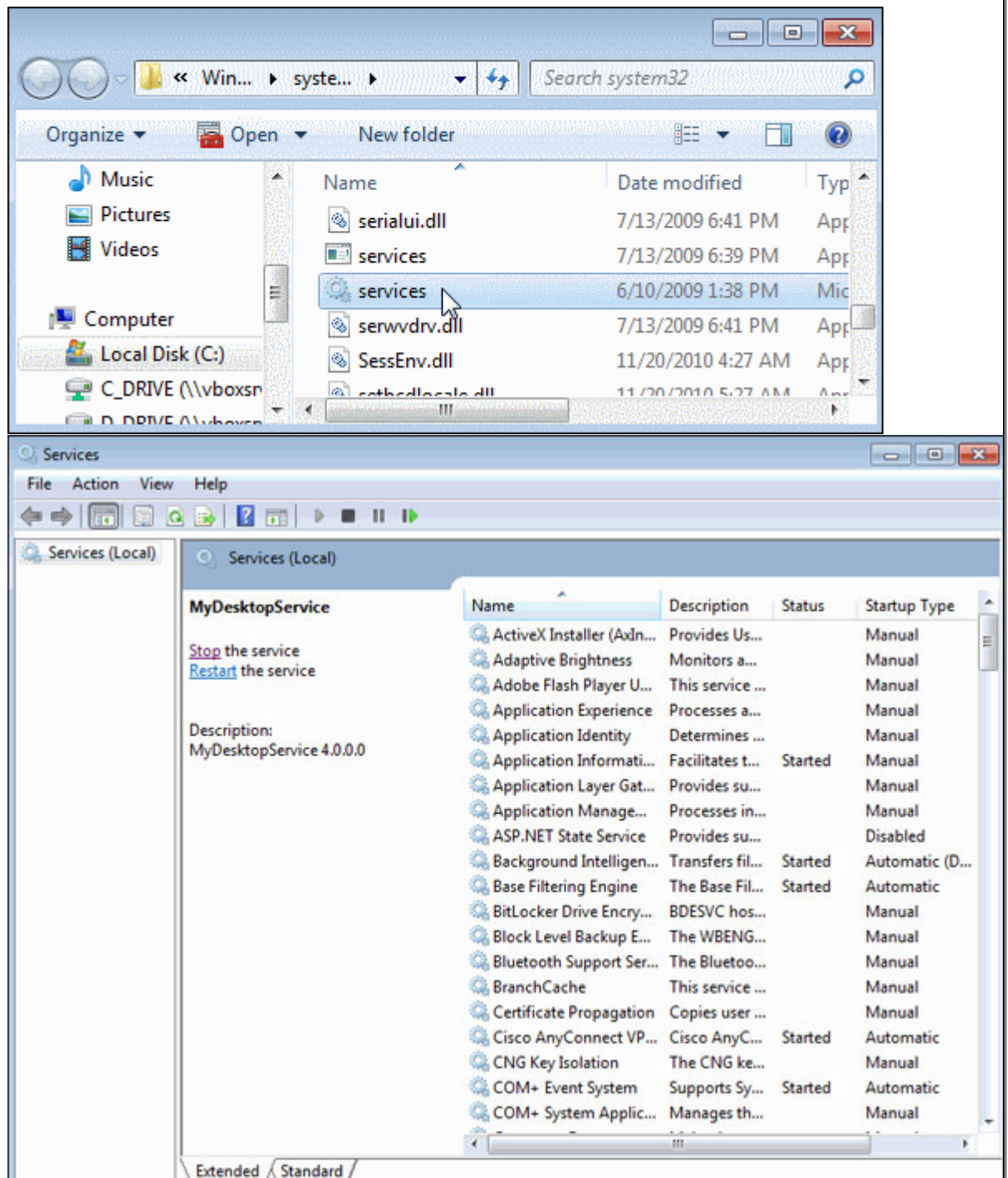
Verifying the Installation

In this section, you will be performing steps to verify the installation of Oracle Database. If you had changed the default location to install the database, make sure to specify the correct location of the files in the following steps. Alternatively, you can also perform a quick search

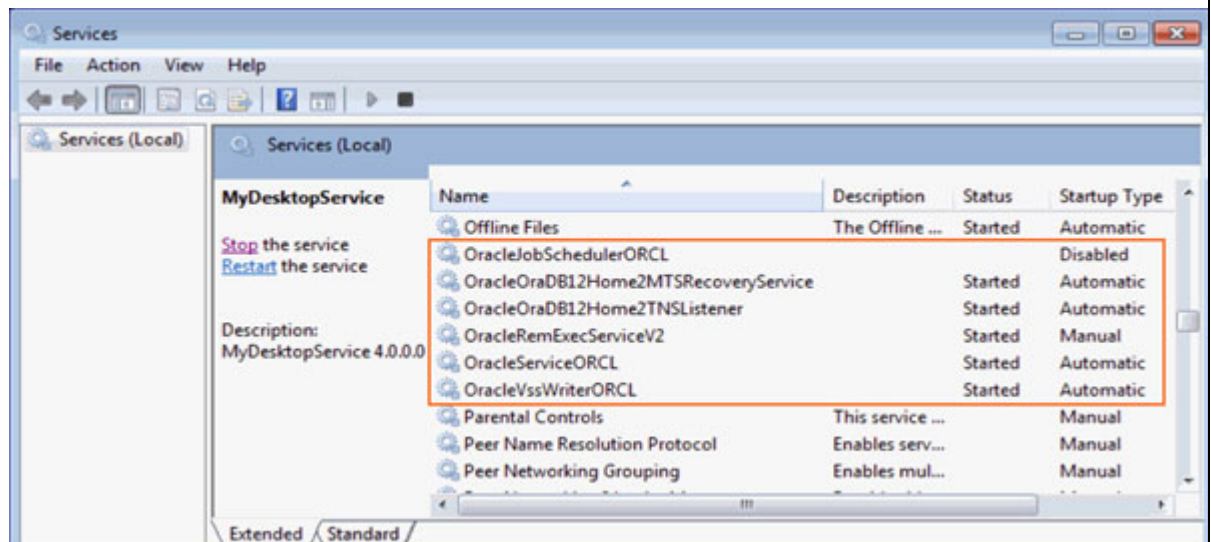
using the Windows Start Menu  to locate files.

View Oracle Services

Navigate to **C:\Windows\system32** using Windows Explorer. Double-click **services**. The **Services** window appears, displaying a list of services.

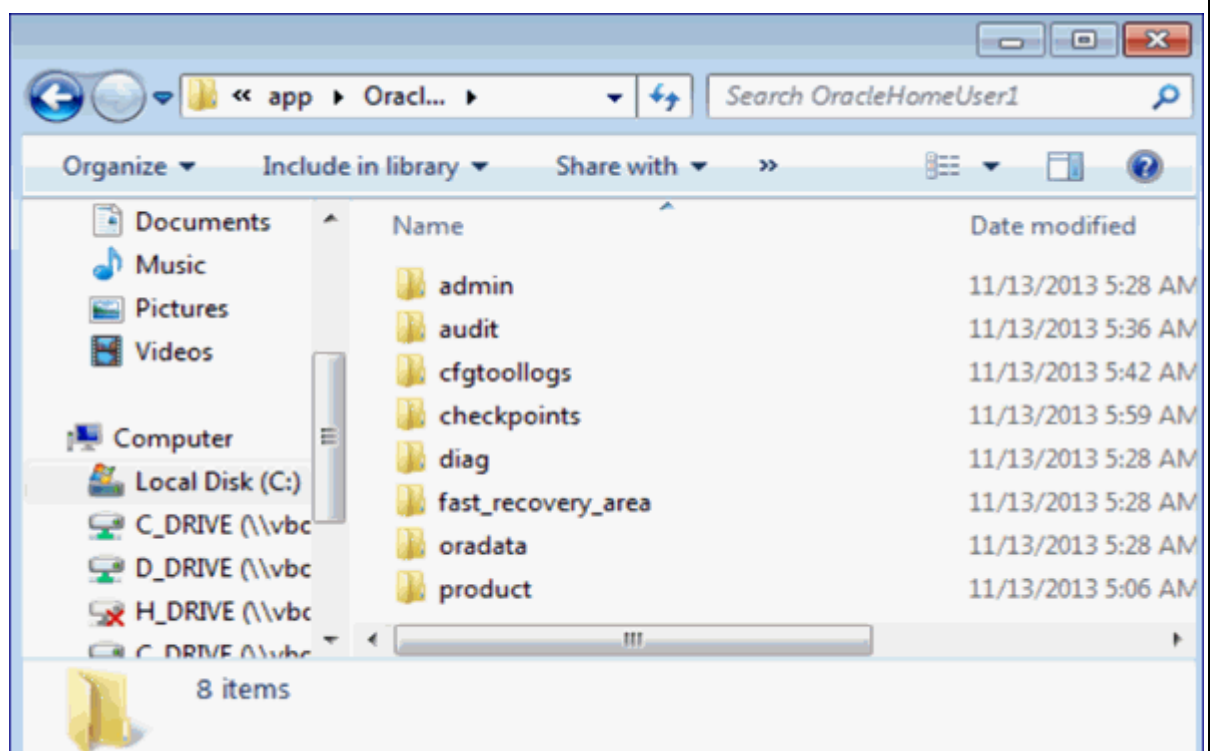


Scroll down to view a list Oracle services. You see that most of the Oracle services are started successfully by the database.

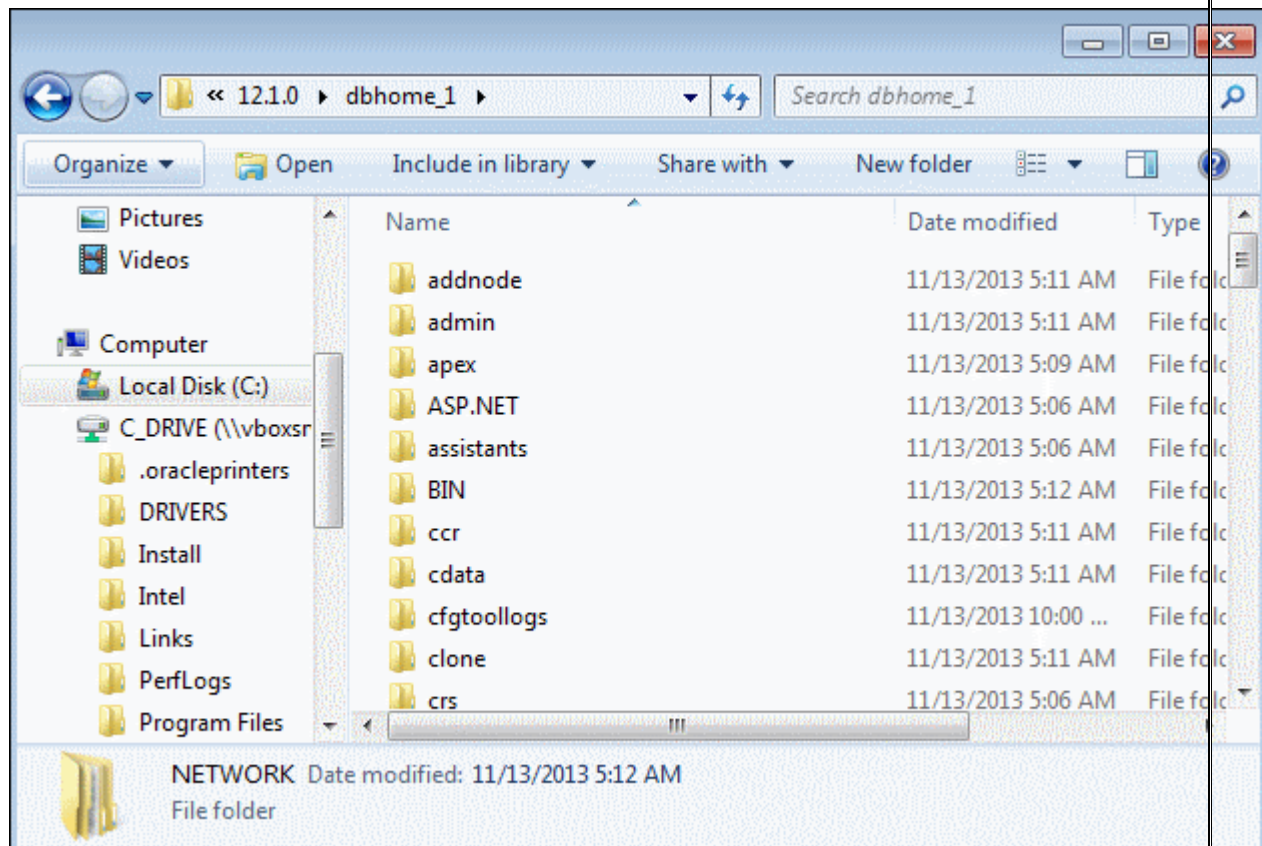


View Oracle Home on the File System

Navigate to the **C:\app\OracleHomeUser1** folder. This folder contains database files (in oradata folder) and the Oracle Database software (in the product folder).



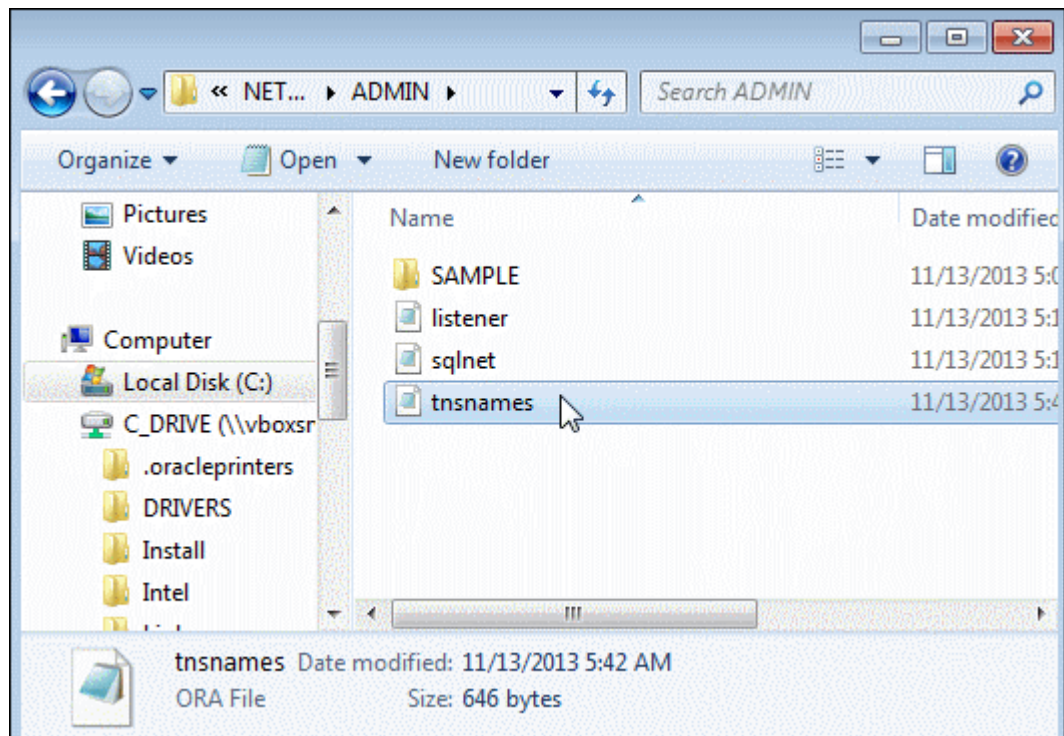
Navigate to **C:\app\OracleHomeUser1\product\12.1.0\dbhome_1** folder. This folder is the new "Oracle Home" created by the installer and contains software files related to the database.



View the tnsnames.ora File

Navigate

to **C:\app\OracleHomeUser1\product\12.1.0\dbhome_1\NETWORK\ADMIN**. Double-click **tnsnames.ora** to view the network configuration settings.



You see that a connect alias called "ORCL" has been created. This "ORCL" alias points to the container database with the service name "ORCL".

A screenshot of a Notepad window titled 'tnsnames - Notepad'. The window displays the contents of the 'tnsnames.ora' file. The text is as follows:

```
# tnsnames.ora Network Configuration File: C:\app
\oracleHomeUser1\product\12.1.0\dbhome_1\network\admin\tnsnames.ora
# Generated by oracle configuration tools.

LISTENER_ORCL =
  (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))

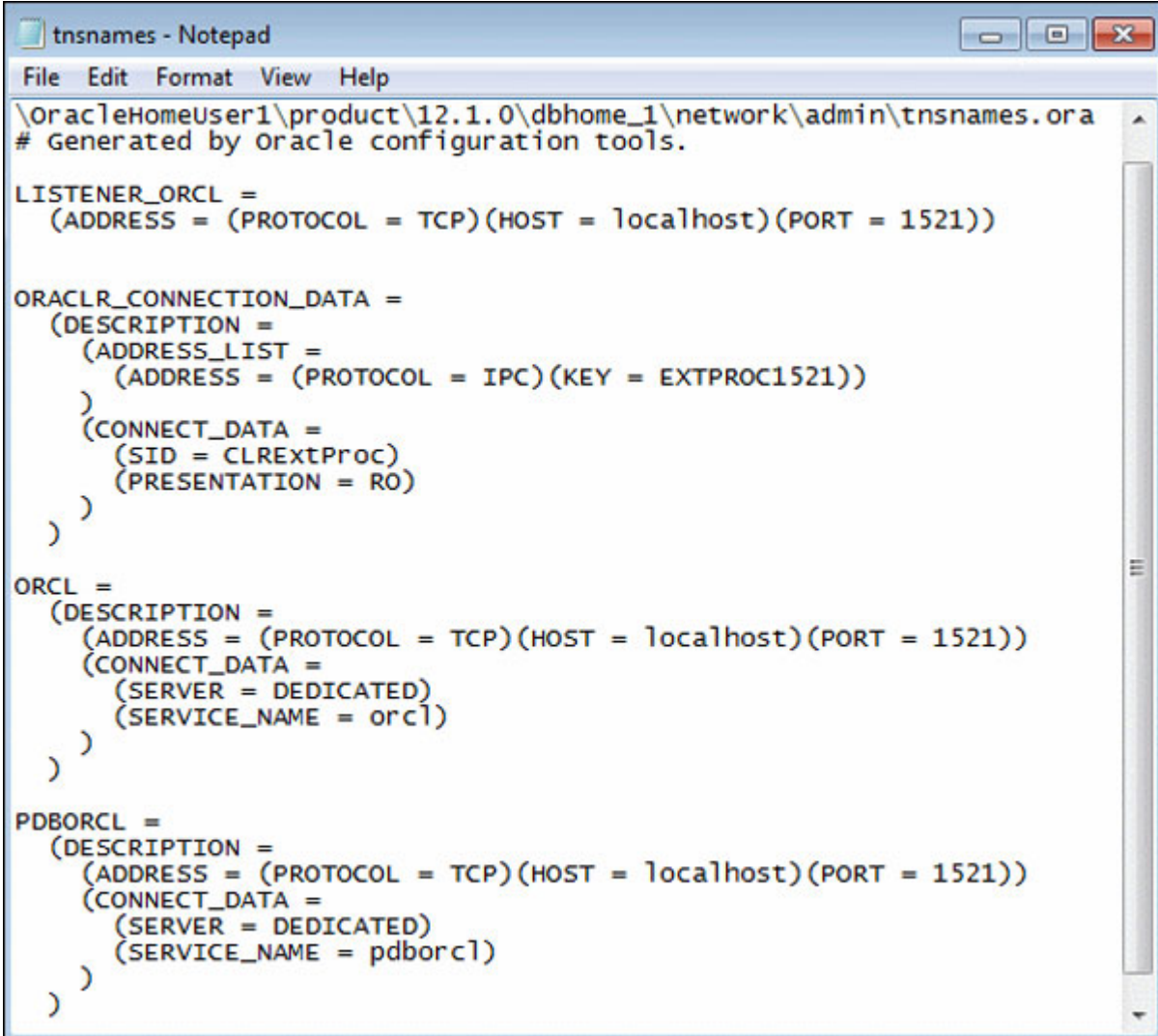
ORACLR_CONNECTION_DATA =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
    )
    (CONNECT_DATA =
      (SID = CLRExtProc)
      (PRESENTATION = RO)
    )
  )

ORCL =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = orcl)
    )
  )
```

The 'ORCL' entry is highlighted with a red rectangular box.

Create a database connect alias called "PDBORCL" and specify the network configuration settings to access the pluggable database "PDBORCL" that we created during installation. Copy the following code and paste it in the **tnsnames.ora** file. If necessary, modify the host and port to match the values in the ORCL alias.

```
PDBORCL =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT =
1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = pdborcl)
    )
  )
```



```
tnsnames - Notepad
File Edit Format View Help
\OracleHomeUser1\product\12.1.0\dbhome_1\network\admin\tnsnames.ora
# Generated by Oracle configuration tools.

LISTENER_ORCL =
  (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))

ORACL_CONNECTION_DATA =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
    )
    (CONNECT_DATA =
      (SID = CLRExtProc)
      (PRESENTATION = RO)
    )
  )

ORCL =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = orcl)
    )
  )

PDBORCL =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = pdborcl)
    )
  )
```

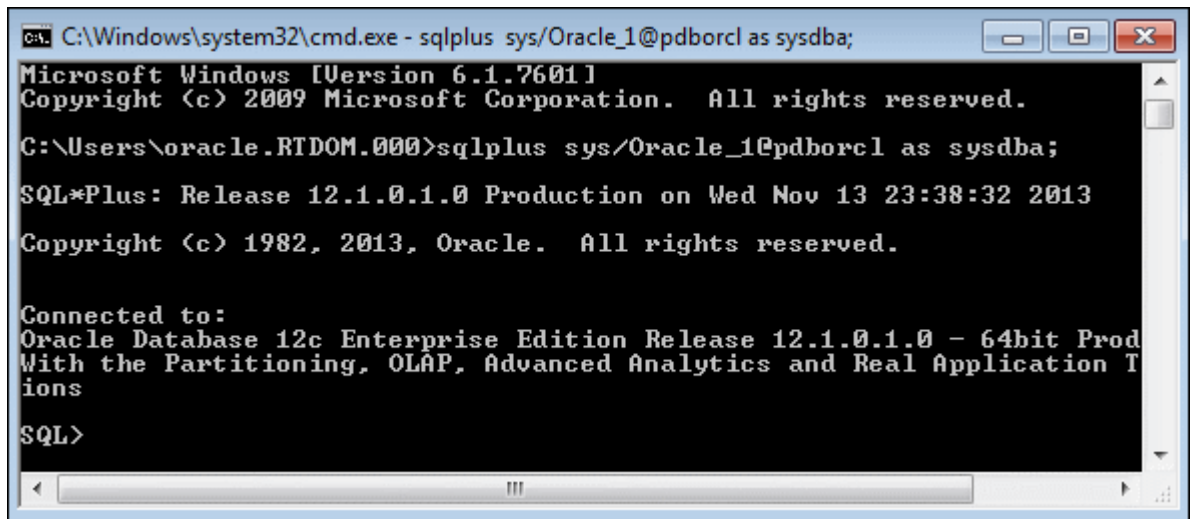
Save and close the file.

In this section, you will be connecting to the pluggable database using the SQL*Plus utility.

Open a command prompt and execute the following command. This command uses SQL*Plus to connect to the pluggable database as a system administrator:

```
sqlplus sys/Oracle_1@pdborcl as sysdba;
```

Note: If you had chosen a different administrative password during installation, replace Oracle_1 with the appropriate password in the command.



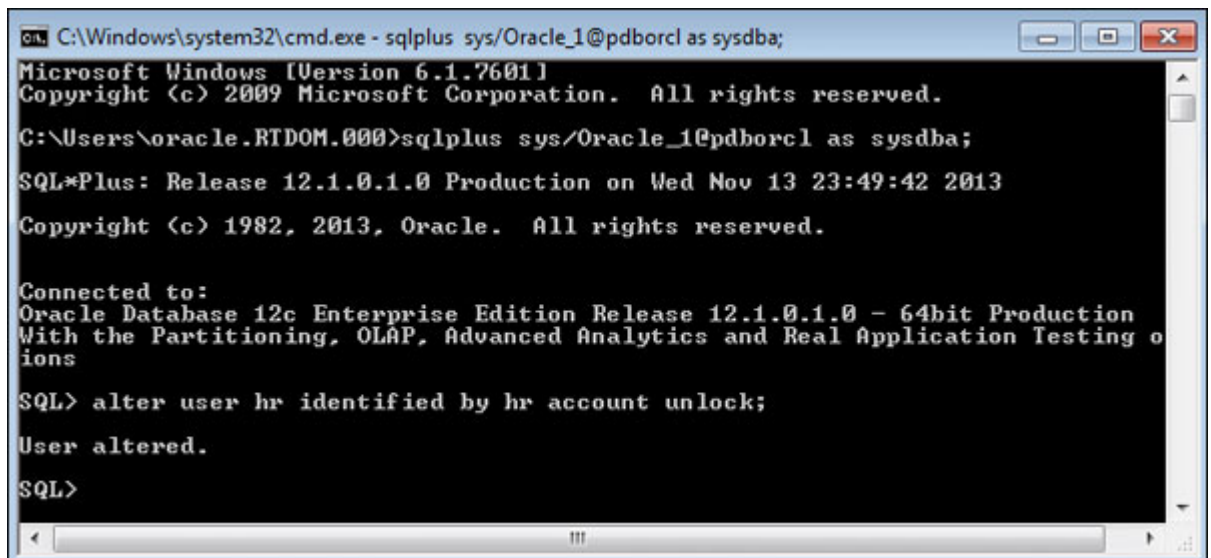
```
Ca. C:\Windows\system32\cmd.exe - sqlplus sys/Oracle_1@pdborcl as sysdba;
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\oracle.RTDOM.000>sqlplus sys/Oracle_1@pdborcl as sysdba;
SQL*Plus: Release 12.1.0.1.0 Production on Wed Nov 13 23:38:32 2013
Copyright (c) 1982, 2013, Oracle. All rights reserved.

Connected to:
Oracle Database 12c Enterprise Edition Release 12.1.0.1.0 - 64bit Prod
With the Partitioning, OLAP, Advanced Analytics and Real Application T
ions
SQL>
```

By default, the HR schema is locked. Execute the following command to unlock the HR schema.

```
alter user hr identified by hr account unlock;
```



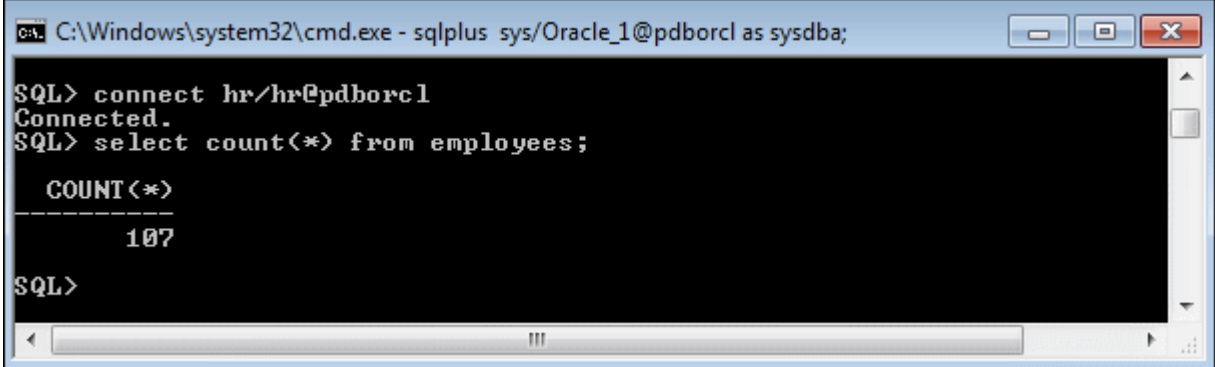
```
Ca. C:\Windows\system32\cmd.exe - sqlplus sys/Oracle_1@pdborcl as sysdba;
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\oracle.RTDOM.000>sqlplus sys/Oracle_1@pdborcl as sysdba;
SQL*Plus: Release 12.1.0.1.0 Production on Wed Nov 13 23:49:42 2013
Copyright (c) 1982, 2013, Oracle. All rights reserved.

Connected to:
Oracle Database 12c Enterprise Edition Release 12.1.0.1.0 - 64bit Production
With the Partitioning, OLAP, Advanced Analytics and Real Application Testing o
ions
SQL> alter user hr identified by hr account unlock;
User altered.
SQL>
```

Execute the following commands to connect to the HR schema in the pluggable database and query the EMPLOYEES table.

```
connect hr/hr@pdborcl
select count(*) from employees;
```



```
C:\Windows\system32\cmd.exe - sqlplus sys/Oracle_1@pdborcl as sysdba;

SQL> connect hr/hr@pdborcl
Connected.
SQL> select count(*) from employees;

  COUNT(*)
-----
        107

SQL>
```

Connecting with DB

We can communicate with database by using a GUI tools or CUI tools. It depends on user type.

Types of users: 2

We can divide the users of data base as follows.

i) **Non-Technical user:**

These users interact with any database through GUI applications or web applications (pages).

ii) **Technical user:**

These users interact with any database through any client tool SQL * PLUS window or any GUI (Graphical User Interface) DataBase tools like **SQL DEVELOPER, PL/SQL DEVELOPER and TOAD.**

Technical users are Developers and DBA.

Parts of Oracle

- 1) SQL
- 2) PL/SQL

SQL (Structured query language)

Oracle was the first company to release a product that used the English-based **Structured Query Language, or SQL**. This language allows end users to extract information themselves, without using a systems group for every little report.

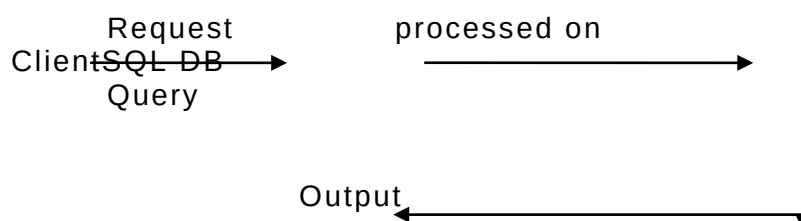
Oracle's query language has structure, just as English or any other language has structure. It has rules of grammar and syntax, but they are basically the normal rules of careful English speech and can be readily understood.

It is known as data base language. It is used to communicate with any database. We can use this language constructs to write SQL QUERIES.

SQL * PLUS is a default client tool and acts as an interface between client and database.

SQL Def:

It is a collection of pre defined commands and constructs with syntactical rules.



1. Sql is a client tool to interact with ORACLE DB /any DB
2. Sql is to be installed in to the system whenever we have installed the db software.
3. Client [Technical] requests should be submitted in the form of "Queries".
4. Queries are executed by SQL STMT EXECUTOR (Oracle Db Engine)
5. Queries are executed against database and output will be displayed on the Sql * plus window.

Features of Sql * Plus:

- At a time only one query is allowed to execute
- Sql queries are not case sensitive
- Each query is terminated with ; (**semi colon**)
- SQL commands are ANSI standard (American National standard institute)

SQL COMMANDS

Types of SQL commands:

1) **DDL**(data definition language) commands:

Used to create or change or delete any data base objects

CREATE
ALTER
DROP
TRUNCATE
RENAME

2) **DML**(data manipulation language) COMMANDS

These commands are used to enter new data/ changing existed data / deleting the data from table.

INSERT
UPDATE
DELETE

3) **DRL**(data retrieval language) Command

SELECT (logicalcommand)

4) **DCL**(DATA CONTROL LANGUAGE) COMMANDS

Used to control the access of data base objects. These commands are used by DBA (databaseadministrator)

GRANT
REVOKE

5) **TCL**(TRANSACTION CONTROL LANGUAGE) COMMANDS

Used to save or cancel the actions/transactions made on table data.

COMMIT ROLLBACK SAVEPOINT

Login into data base

Before communicating with database the user should be authenticated by DBA by creating user account.

HOW TO OPEN SQL * PLUS WINDOW?

NAVIGATION:

This navigation is useful to open sql * plus window

START--> PROGRAMS--> ORACLE ora_Home1_11G--> Application development--> select Sql command

Then it opens Sql command window, then submit user credentials.

HOW TO CREATE A USER ACCOUNT?

DBA (Data Base Administrator) can CREATE and DELETE and manage user accounts

NOTE:

User name is not case sensitive, but password is case sensitive from oracle 11g onwards

General DBA credentials

- 1) In login window, submit the following details

username: system
password: manager
Hoststring: orcl / oracle -----> It is the database service name.

- 2) Now it opens Sql * Plus window

In this window, the DBA has to write queries to create user account.

Syntax: creating a new user account

Create User <user_name> Identified by <password >;

Ex: create user **dinesh** identified by **welcome**;

User created.

3) Giving permissions to use the resources of data base

GRANT resource, connect to <USER_NAME>;

Ex: grant resource,connect to **dinesh**;

Grant succeeded.

Ex: Sql> exit; [Disconnecting from database]

Ex: Connecting to data base as **Dinesh**.

username: dinesh
password: welcome
Hoststring: orcl

LOG ONPROCEDURE:

1) Double click on sqlplus icon on desktop
(In oracle 8i/9i/10g)

Or

Double click SqlPlus Command window

2) It opens a log on window, in this type the following

username: pv
password: welcome
hoststring: oracle / orcl

In Oracle 11g Command window, type as below.

Enter Username: dinesh

Password : welcome

3) It opens Sql * Plus window with prompt

SQL>_

LOG OUT PROCEDURE:

In the sql window, type the command EXIT. It disconnects the current user from the data base and it close the sql window.

Ex: SQL>EXIT;

How to create user Account In Oracle-12c ?

1) Connect as DBA?

Open Command window


```
C:\users\dinesh>sqlplus system/manager [ Hit enter ]
```

Connected.

2) Check your User name

```
SHOW USER;
```

3) Check Container Database

```
SHOW CON_NAME;
```

Note:

If it is CDB\$ROOT, THEN CHANGE IT TO “PDBORCL”[Pluggable DataBase].

4) Changing container to PDBORCL

```
ALTER SESSION SET CONTAINER=pdborcl;
```

5) Now check your container name, usually it is PDBORCL, Create new user account in this container database.

Create User **dinesh** Identified by **welcome** Quota 100M On Users;

Grant connect,resource to dinesh;

6) Now connect as DINESH

```
CONN dinesh/welcome@pdborcl;
```

DATA MODEL

For any OLTP RDBMS the data model is **E-R** (Entity-Relationship) Model. Create an Entity-Relationship (ER) model is to visually represent the structure of a business database, where data equates to entities (or objects) that are linked by defined relationships expressing dependencies and requirements. By nature it is an abstract visualization, the first step in the design process towards creating a logical and functional database.

Entity : Any real time object is known as entity

Ex : emp, dept, orders, transactions, sales, products, service, customers

- For each object in the business we need to create a table.
- Every object is having some Properties.
- For each property we need to maintain a column.

Ex : Consider emp object

Then table name is EMP

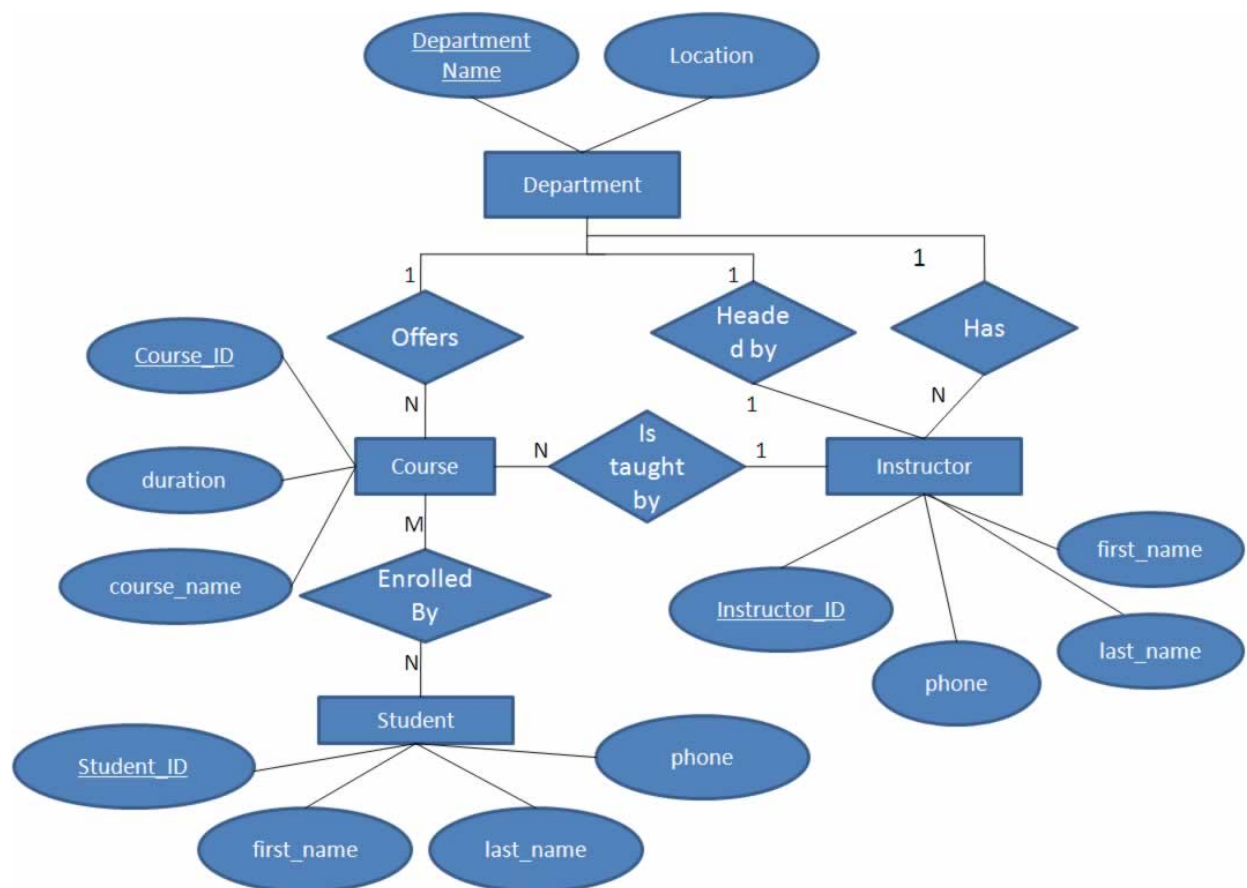
Properties of emp are as follows.

	eid	ename	sal	job	hiredate	gender
mailid	mobile					

Relation: How business activity is working between 2 objects

Ex : emp<-->dept;
company<-->product ;
prod<-->sales ;

Sample E-R Model is as follows



- Entity / Object represented with Rectangle
- Ellipses represents properties of object.
- Rhombus represents relation between objects.

DDL command

1) CREATE

It is used to create any data base object like tables, views, indexes, sequences, synonyms, users, functions, procedures, triggers, packages and so on.

HOW TO CREATE A TABLE?

```
syn: CREATE TABLE <table_name>
      (
        <colname-1> DATATYPE (size),
        <colname-2> DATATYPE (size),
        :      :      :      :,
        :      :      :      :,
      );
```

Naming Rules

Rules to follow before specifying names(Object names, Column Names and Variable Names).

- i) Each name should begins alphabet
- ii) Valid character set is
a-z,A-Z,0-9,@,\$,# and _ (underscore)

Ex: Emp123
Emp_o11

- iii) Names are not case sensitive
- iv) Already existed names are not allowed
- v) Pre defined keywords (ReservedWords) are not allowed as names.
- vi) Blankspace within a name is not allowed.
- vii) Max length of any name is 30 characters.

Ex: valid_names

prod_dtls
emp@sal
emp123

emp_info
emp_table

invalid_names

prod dtls--Since blank space within the name
emp.sal----Since " ." is not valid character
123emp-----Since Name is not beginning with
alphabet
emp-info---Since " -" is not valid char.
table-----Since " table " is a reserved word

DATATYPES:

The data type represents the type of data to be entered into a column and Db engine can assign memory for the value entered into the column.

I) String Data types:

These data types support alphabets, digits and any symbol from keyboard.

1) CHAR(size)

It is used to store fixed length character strings. By default the size is 1 character, and max size is 2000 chars or bytes.

Ex: empid, pnr number, bank account numbers, Policy Numbers and so on

2) VARCHAR2(size)

It is used to store variable length character strings. No default size. we should specify size and max size is 4000 chars /bytes.

Ex: emp names, addresses, descriptions, city names,

3) LONG

It is used to store variable length char data (similar to *varchar2 data type*) but max size is **2 GB**

NOTE: Only one long type column is allowed per a table.

4) NCHAR(size)

It is similar to CHAR(SIZE) data type, but it is used to support any National char set.

5) NVARCHAR2(size)

It is similar to VARCHAR2(SIZE) data type, but it is used to support any National char set.

II) Numeric Data types

The NUMBER datatype stores fixed and floating-point numbers. Numbers of virtually any magnitude can be stored and are guaranteed portable among different systems operating Oracle, up to 38 digits of precision.

The following numbers can be stored in a NUMBER column:

- Positive numbers in the range 1×10^{-130} to $9.99..9 \times 10^{125}$ with up to 38 significant digits

- Negative numbers from -1×10^{-130} to $9.99..99 \times 10^{125}$ with up to 38 significant digits
- Zero

1) **NUMBER(Precision, [Scale])**

It is used to store numbers along with decimal point.

2) **NUMBER(Precision)**

It is used to store numbers without decimal point.

Precision represents total number of digits in the value.

Scale represents the max number of digits after decimal point.

Total number of digits in the value should be less than or equal to Precision value.

Total number of digits in the decimal part should be less than or equal to scale value.

Note: The max value for precision is 38

Ex: prod_price number(7,2)

12.75
123.1
45621.08
99999.99
125

Ex: 123.4567--> Invalid
0.0972----> Invalid

Ex: emp_sal number(6)

12560
100
0
10
999999

III) Date data type

DATE

The DATE datatype stores point-in-time values (dates and times) in a table. The DATE datatype stores the year (including the century), the month, the day, the hours, the minutes, and the seconds (after midnight).

Oracle can store dates in the Julian era, ranging from January 1, 4712 BCE through December 31, 4712 CE (Common Era). Unless BCE ('BC' in the format mask) is specifically used, CE date entries are the default.

Oracle uses its own internal format to store dates. Date data is stored in fixed-length fields of seven bytes each, corresponding to century, year, month, day, hour, minute, and second.

For input and output of dates, the standard Oracle default date format is DD-MON-YY, as below:

'13-NOV-92'

You can change this default date format for an instance with the parameter **NLS_DATE_FORMAT**. You can also change it during a user session with the **ALTER SESSION** statement. To enter dates that are not in standard Oracle date format, use the TO_DATE function with a format mask:

TO_DATE ('November 13, 1992', 'MONTH DD, YYYY')

Note:

If you use the standard date format DD-MON-YY, YY gives the year in the 20th century (for example, 31-DEC-92 is December 31, 1992). If you want to indicate years in any century other than the 20th century, use a different format mask, as shown above.

Oracle stores time in 24-hour format--HH:MI:SS. By default, the time in a date field is 00:00:00 A.M. (midnight) if no time portion is entered. In a time-only entry, the date portion defaults to the first day of the current month. To enter the time portion of a date, use the TO_DATE function with a format mask indicating the time portion, as in

```
INSERT INTO birthdays (bname, bday) VALUES  
('ANDY',TO_DATE('13-AUG-66 12:56 A.M.','DD-MON-YY HH:MI A.M.'));
```

DD-MON-YY

DD Digits of date
MON First 3 chars of month name
YY Last 2 digits of year

Ex: 12-may-13

Ex: 12/may/13 --It is not considered as a date format--

IV) Binary Data types

1) RAW(size)

It is used to store binary data like images, thumb impressions, logos and so on.

Max size is 2000 bytes < 2 KB

2) LONG RAW

It is similar to RAW data type but max size is 2 GB

NOTE: Only one longraw type column is allowed per a table.

V) LOB--Large Objects

The LOB datatypes BLOB, CLOB, NCLOB, and BFILE enable you to store large blocks of unstructured data (such as text, graphic images, video clips, and sound waveforms) up to four gigabytes in size. They provide efficient, random, piece-wise access to the data.

You can perform parallel queries (but not parallel DML or DDL) on LOB columns.

LOB datatypes differ from LONG and LONG RAW datatypes in several ways. For example:

- A table can contain multiple LOB columns but only one LONG column.
- A table containing one or more LOB columns can be partitioned, but a table containing a LONG column cannot be partitioned.
- The maximum size of a LOB is four gigabytes, but the maximum size of a LONG is two gigabytes.
- LOBs support random access to data, but LONGs support only sequential access.
- LOB datatypes (except NCLOB) can be attributes of a user-defined object type but LONG datatypes cannot.
- Temporary LOBs that act like local variables can be used to perform transformations on LOB data. Temporary internal LOBs (BLOBs, CLOBs, and NCLOBs) are created in the user's temporary tablespace and are independent of tables. For LONG datatypes, however, no temporary structures are available.

SQL statements define LOB columns in a table and LOB attributes in a user-defined object type. When defining LOBs in a table, you can explicitly specify the tablespace and storage characteristics for each LOB.

LOB datatypes can be stored inline (within a table), out-of-line (within a tablespace, using a LOB locator), or in an external file (BFILE datatypes).

LOB TYPES -- 3

CLOB--CHAR LOB--used to store character data

BLOB-- BINARY LOB-- Used to store binary data

NCLOB--Fixed length multi char large objects--Used to store both binary and char data.

vi) BFILE Datatype

The BFILE datatype stores unstructured binary data in operating-system files outside the database. A BFILE column or attribute stores a file locator that

points to an external file containing the data. BFILEs can store up to four gigabytes of data.

BFILEs are read-only; you cannot modify them. They support only random (not sequential) reads, and they do not participate in transactions. The underlying operating system must maintain the file integrity and durability for BFILEs. The database administrator must ensure that the file exists and that Oracle processes have operating-system read permissions on the file.

vii) ROWID and UROWID Datatypes

Oracle uses a ROWID datatype to store the address (*rowid*) of every row in the database.

- *Physical rowids* store the addresses of rows in ordinary tables (excluding index-organized tables), clustered tables, table partitions and subpartitions, indexes, and index partitions and subpartitions.
- *Logical rowids* store the addresses of rows in index-organized tables.

A single datatype called the *universal rowid*, or UROWID, supports both logical and physical rowids, as well as rowids of foreign tables such as non-Oracle tables accessed through a gateway.

A column of the UROWID datatype can store all kinds of rowids. The value of the COMPATIBLE initialization parameter must be set to 8.1 or higher to use UROWID columns.

Ex:

Write a query to create emp_info table with columns eid, ename, sal, jdate, desg, and gender with appropriate data types.

```
CREATE TABLE EMP_INFO
(
  EID          NUMBER(4),
  ENAME        VARCHAR2(20),
  SAL          NUMBER(5),
  JDATE        DATE,
  DESG         VARCHAR2(20),
  GENDER       CHAR
);
```

HOW TO INSERT DATA INTO A TABLE?

INSERT:

It is used to insert new records in to the table. Assigns the values in to corresponding columns in the column list. If there is no column list, the first value is inserted into the first column defined by

the CREATE TABLE statement, the second value is inserted into the second column, and so on. There must be one value for each column in the column list. The datatypes of the values being inserted must be compatible with the datatypes of corresponding columns in the column list.

Note:-

Character and date literals in the VALUES list must be enclosed by single quotes ('). Numeric literals are not enclosed by quotes.

Syntax:

```
INSERT INTO <table_name>[(col1, col2, ..., col n)]  
VALUES (val1, val2, ..., val n);
```

Note:

- i) If number of columns in the table and number of values inserting in to the table are equal, then no need to specify column names while inserting records.
- ii) Char, Varchar2 and date type values should be enclosed in Single Quotes.
- iii) If Number of inserting values are less than the number of columns then we must specify column names while inserting records.

Example Insertions:

- a) insert into emp_info(eid,ename,sal,jdate,desg,gender)
values(1111,'Dinesh',75000,'23-may-14','developer','M');
- b) insert into emp_info(eid,ename,sal,jdate,desg,gender)
values(1112,'Madhu',30000,'23-may-14','developer','F');
- c) insert into emp_info(eid,ename,sal,jdate,desg,gender)
values(1191,'Xavier',10000,'02-feb-10','clerk','M');
- d) insert into emp_info
values(1110,'john',11700,'23-may-14','developer','M');
- e) insert into emp_info
Values(1121,'dilroop',61000,'21-oct-13','developer','F');
- f) insert into emp_info
Values(1120,'abhi',10000,'23-may-14','admin','M');
- g) Insert into emp_info
Values('kiran','5000');

Error: Not enough values

- h) Insert into emp_info (ename,sal)
Values('kiran','5000');

The following examples show various forms of INSERT statement:

```
INSERT INTO bonus SELECT ename, job, sal, comm FROM emp
```

```

WHERE comm > sal * 0.25;
...
INSERT INTO emp (empno, ename, job, sal, comm, deptno)
VALUES (4160, 'STURDEVIN', 'SECURITY GUARD', 2045, NULL, 30);
...
INSERT INTO dept
VALUES (my_deptno, UPPER(my_dname), 'CHICAGO');

```

Inserting records using &[Substitution] operator:

“ & “ is known as "substitution operator".

Ex:

```

Insert into emp_info
Values('&eno','&ename','&sal','&jdate', '&desg', '&gender');

```

```

enter value for eno:      232
enter value for ename:    hari
enter value for sal:      23000
enter value for jdate:    22-oct-11
enter value for desg:     admin

```

1 row created.

In sql * plus window

```

sql>/ [ to re-execute the recent Query ]

```

```

enter value for eno:      231
enter value for ename:    samuel
enter value for sal:      12000
enter value for jdate:    10-may-14
enter value for desg:     admin

```

1 row created.

```

sql > /

```

INSERT ALL

The **Oracle INSERT ALL statement** is used to add multiple rows with a single **INSERT statement**. The rows can be **inserted** into one table or multiple tables using only one SQL **command**.

Ex:

Say you have a table COLORS with this structure:

Name	Type
-----	-----
NAME	VARCHAR2(30)
CATEGORY	VARCHAR2(10)

Ex:

```
insert all
into colors(name, category) values('yellow', 1)
into colors(name, category) values('red', 1)
into colors(name, category) values('blue', 1)
into colors(name, category) values('yellow', 2)
into colors(name, category) values('blue', 2)
select * from dual;
```

NULL VALUES:

A missed value in a column is known null value.

Null value is not equal to zero or space or other null values.

NULL values represent missing or unknown data. **NULL values** are used as placeholders or as the default entry in columns to indicate that no actual data is present. The **NULL** is untyped in SQL, meaning that it is not an integer, a character, or any other specific data type.

INSERTING NULL VALUES: 2 Methods

1) Implicit insertion:

If we miss a value in a column then it is dynamically maintains a null value at that place. To insert like this, we need to specify column names while using insert command.

Ex: Insert into emp_info(eno,ename,sal,desg)
Values(555,'john',12000,'salesman');

2) Explicit insertion:

Just specify NULL keyword at the place of missed values.
Here no need to maintain column names while inserting data.

Ex:
Insert into emp_info
Values(888,'martin',12000,'salesman',null,null,NULL,Null);

DESCRIBE / DESC

This command is used to display table structure. A table structure contains column names, data types and sizes.

SYNTAX

DESC <Table_Name>;

EX: **describe** emp_info;

or
desc emp_info;

Ex:

SQL> desc emp;

Name	Null?	Type
-----	-----	-----
EMPNO	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)
MGR		NUMBER(4)
HIREDATE		DATE
SAL		NUMBER(7,2)
COMM		NUMBER(7,2)
DEPTNO		NUMBER(2)

HOW TO DISPLAY LIST OF TABLE NAMES?

EX: SELECT * FROM TAB;

DEFAULT Keyword

We can create a table with DEFAULT Clause. While inserting data in to table , we don't need to enter any value in to default column. The column is populated with the value supplied along with DEFAULT keyword.

Syntax:

```
CREATE TABLE < table_name>
(
Col1          datatype(size) DEFAULT 'Value' ,
"              "              "              ,
"              "              "              ,
);
```

Example:

```
Create Table customer
(
Cno          number(2),
Cname        varchar2(20),
City         varchar2(10) DEFAULT 'Hyderabad'
);
```

Example: Inserting values in to Customer table

```
Insert into customer(cno,cname) values
(1,'kiran');
Insert into customer(cno,cname) values
(2,'Madhu');
Insert into customer values
(3,'dinesh',Null);
Insert into customer values
(4,'john','Texas');
```

Ex: Selecting data from Customer table.

Select * from tab;

1	Kiran	Hyderabad
2	Madhu	Hyderabad
3	Dinesh	
4	john	Texas

Note:-

By Default the column value is "Hyderabad".

If we submit a different value or Null value then the default column is populated with given value.

DATA RETRIEVAL COMMAND (DRL COMMAND)

SELECT

It is used to retrieve a logical copy of required data from a table or columns.

Syntax: SELECT col1, col2,...., coln / *
 FROM table_name;

Retrieving data from single column

Ex: Get employee names?

SQL> select ename from emp;

```
ENAME
-----
SMITH
ALLEN
WARD
JONES
MARTIN
BLAKE
CLARK
SCOTT
```

KING
TURNER
ADAMS

ENAME

JAMES
FORD
MILLER

14 rows selected.

SQL>

Retrieving data from multiple columns

Ex: Get employee id, names and salaries?

SQL> select empno,ename,sal from emp;

EMPNO	ENAME	SAL
-----	-----	-----
7369	SMITH	800
7499	ALLEN	1600
7521	WARD	1250
7566	JONES	2975
7654	MARTIN	1250

Retrieving All Column data

Ex:

SQL> set linesize 300;

SQL> select * from emp;

Note: "*" represents all columns.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	7902	17-DEC-80	800	20	
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000	10	
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

14 rows selected.

SQL>

Retrieving column data in user required order

(Instead of Physical order of table columns)

Ex:

Display employee join_dates, ename, designations and salaries?

SQL> select hiredate, ename, job, sal from emp;

HIREDATE	ENAME	JOB	SAL
17-DEC-80	SMITH	CLERK	800
20-FEB-81	ALLEN	SALESMAN	1600
22-FEB-81	WARD	SALESMAN	1250
02-APR-81	JONES	MANAGER	2975
28-SEP-81	MARTIN	SALESMAN	1250
01-MAY-81	BLAKE	MANAGER	2850
09-JUN-81	CLARK	MANAGER	2450
19-APR-87	SCOTT	ANALYST	3000
17-NOV-81	KING	PRESIDENT	5000
08-SEP-81	TURNER	SALESMAN	1500
23-MAY-87	ADAMS	CLERK	1100

HIREDATE	ENAME	JOB	SAL
03-DEC-81	JAMES	CLERK	950
03-DEC-81	FORD	ANALYST	3000
23-JAN-82	MILLER	CLERK	1300

14 rows selected.

SQL>

DISPLAYING COLUMN DATA WITH USER DEFINED TITLES:

From above examples , By default the column names are output titles in the output.

We can also change these output titles as follows.

Syntax-1 select colname "title", colname "title" ...
 from table;

“In this format we can also maintain spaces in the titles.”

Ex: select ename "Employee Name", desg " Job of Employee"
 from employee_info;

employee name	job of employee
dinesh rao	developer
smitha panday	developer
madhu	admin
madhu	admin
Allen	developer
king	salesman
torjan	clerk
john	salesman
martin	salesman

9 rows selected

Syntax-2

Select col-name *title*, colname *title*,
 From Table_name;

“In this style we cannot maintain spaces in the title”

Ex:

SQL> select ename empname, sal salary from emp;

EMPNAME	SALARY
SMITH	800
ALLEN	1600
WARD	1250
JONES	2975
MARTIN	1250
BLAKE	2850
CLARK	2450
SCOTT	3000
KING	5000
TURNER	1500
ADAMS	1100

14 rows selected.

SQL>

Syntax-3

Select col-name AS title, col-name AS title, . . .
 From table_name;

“In this style we cannot maintain spaces in the title”

Ex:


```
SQL> select empno as eid, ename as empnames from emp;
```

EID EMPNAMES

```
7369 SMITH
7499 ALLEN
7521 WARD
7566 JONES
7654 MARTIN
7698 BLAKE
7782 CLARK
7788 SCOTT
7839 KING
7844 TURNER
7876 ADAMS
```

EID EMPNAMES

```
7900 JAMES
7902 FORD
7934 MILLER
```

14 rows selected.

```
SQL>
```

Assignments:

Create the following tables with your own assumed relevant column names and datatypes.

- i) Create a table Item_dtls (Electronics)
- ii) Create a table Sales_dtls
- iii) create a table manufacturers
- iv) Try to insert at least 10 records in the above tables
- v) Try to insert at least 2 records with null values

Clauses

DISTINCTclause

ORDER BYclause

Ex:Consider the following table and records

```
create table departments
(
did number(2),
```

```
dname varchar2(20),  
city varchar2(20)  
);
```

```
insert into departments  
values(10,'Production','Dallas');  
insert into departments  
values(10,'Production','Dallas');  
insert into departments  
values(10,'Production','Dallas');  
insert into departments  
values(10,'Production','Dallas');  
insert into departments  
values(10,'Production','Dallas');  
insert into departments  
values(10,'Production','Dallas');
```

```
insert into departments  
values(20,'Sales','Texas');  
insert into departments  
values(20,'Sales','Texas');  
insert into departments  
values(20,'Sales','Texas');  
insert into departments  
values(20,'Sales','Texas');
```

```
insert into departments  
values(30,'Finance','Chicago');  
insert into departments  
values(30,'Finance','Chicago');  
insert into departments  
values(30,'Finance','Chicago');  
insert into departments  
values(30,'Finance','Chicago');  
insert into departments  
values(30,'Finance','Chicago');
```

DISTINCTClause

It will display different / unique values from the column and it will also display unique records from the table.

i) Getting Unique values from single column.

Syntax: select **distinct** colname from <table>;

ii) Getting unique combination of values from multiple columns.

Syntax: select **distinct** colname1,colname2,.... from <table>;

iii) Getting Unique records from the table.

Syntax: select **distinct** * from <table>;

Ex: display list of different designations?

SQL> select distinct job from emp;

JOB

CLERK
SALESMAN
PRESIDENT
MANAGER
ANALYST

SQL>

Ex: display distinct department names?

select distinct(dname) from dept;

output:

Production
Sales
Finance

Ex: Get unique records from the above table?

or

Display records from the above table without duplicates?

select distinct did,dname,city from dept;

10	production	chicago
20	Sales	Texas
30	Finance	Dallas

ORDER BY Clause

This clause is used to display the column data or table data in ascending / descending or sorting [a to z] / reverse sorting [z-a] order.

Syntax: select *col1*,..... / *
 from *TableName*
 order by *col1, col2*,..... [asc / desc];

Note:

- i) By default it will display a to z data or ascending order data.
- ii) Incase of char data, if there exists upper and lower case data then order by gives highest priority to the uppercase data and next priority goes to lower case data.

Since upper case data A-Z ascii values are 65-90
lower case data a-z ascii values are 91-122

Ex: Display employee names in alphabetical(sorting) order?

```
select ename from emp ORDER BY ename;
```

sample output:

```
ENAME
-----
ADAMS
ALLEN
BLAKE
CLARK
FORD ...
```

Ex: Display employee names in reverse order?

```
select ename from emp ORDER BY ename DESC;
```

sample output:

```
ENAME
-----
WARD
TURNER
SMITH
SCOTT
MILLER ...
```

Ex: display *ename*, *sal*, *desg* on the order of salary?

```
select ename, sal, job from emp order by sal;
```

sample output:

ENAME	SAL	JOB
SMITH	800	CLERK
JAMES	950	CLERK
ADAMS	1100	CLERK
WARD	1250	SALESMAN
MARTIN	1250	SALESMAN

NOTE:

We can also write the column position number in the ORDER BY clause. The column position number should be from the select column list, not from table column list.

ORDER BY clause on more than one column:

Here the first priority of order by clause given to first column, if first column having duplicates then order by priority goes to second column and so on.

Ex: display sal and ename from emp based on order of salaries?

```
select sal,ename from emp order by sal;
```

sample output:

SAL	ENAME
800	SMITH
950	JAMES
1100	ADAMS
1250	WARD----->
1250	MARTIN ---->
1300	MILLER
1500	TURNER
1600	ALLEN
2450	CLARK
2850	BLAKE
2975	JONES
3000	SCOTT---->
3000	FORD ---->
5000	KING

Ex: display sal and ename from emp based on order of salaries and employee names?

```
select sal,ename from emp order by sal,ename;
```

sample output:

SAL	ENAME
800	SMITH
950	JAMES
1100	ADAMS
1250	MARTIN ----->
1250	WARD ----->
1300	MILLER
1500	TURNER
1600	ALLEN
2450	CLARK
2850	BLAKE
2975	JONES
3000	FORD ----->
3000	SCOTT ----->
5000	KING

Ex:

Get employee data based on deptno order and from each dept least salried emp to highest salaried employee?

```
SQL> select empno,ename,sal, deptno from emp
order by deptno, sal;
```

EMPNO	ENAME	SAL	DEPTNO
7934	MILLER	1300	10
7782	CLARK	2450	10
7839	KING	5000	10
7369	SMITH	800	20
7876	ADAMS	1100	20
7566	JONES	2975	20
7788	SCOTT	3000	20
7902	FORD	3000	20
7900	JAMES	950	30
7654	MARTIN	1250	30
7521	WARD	1250	30

EMPNO	ENAME	SAL	DEPTNO
7844	TURNER	1500	30
7499	ALLEN	1600	30
7698	BLAKE	2850	30

14 rows selected.

SQL>

OPERATORS

Arithmetic Operators:

+ - * /

These operators are used to perform Arithmetic calculations on user's own data and table data.

DUAL table

The **DUAL table** is a special one-row, one-column **table** present by default in **Oracle** and other database installations. In **Oracle**, the **table** has a single VARCHAR2(1) column called DUMMY that has a value of 'X'. It is suitable for use in selecting a pseudo column such as SYSDATE or USER.

It is a system defined table which contains only one column to perform calculations on users own data.

Arithmetic Calculations On Users data:

Ex: select 200+300 from dual;

500

Ex: select (90000*10)/100 "10% of 90000" from dual;

10% of 90000

9000

Ex: select 2000+(0.10*5000)-300 " After calculation" from dual;

2200

Arithmetic Calculations On Table data:

Ex: Display emp salaries and 2% of salary as TA?

select sal " Basic Sal", (0.02*sal) " TA" from emp;

Ex: Display employee salaries, 2% as TA, 5% as DA, 10% HRA, 4% as COMM and final salary?

select Sal " Basic", (0.02*Sal) " TA", (0.05*sal) "DA", (0.10*sal) "HRA",
 (0.04*sal) " Comm",
 (Sal + (0.02*Sal) + (0.05*sal) + (0.10*sal) + (0.04*sal)) " Final

Salary"

from emp;

output

Basic	TA	DA	HRA	Comm	Final Salary
800	16	40	80	32	968
1600	32	80	160	64	1936
1250	25	62.5	125	50	1512.5
2975	59.5	148.75	297.5	119	3599.75
1250	25	62.5	125	50	1512.5
2850	57	142.5	285	114	3448.5
2450	49	122.5	245	98	2964.5
3000	60	150	300	120	3630
5000	100	250	500	200	6050
1500	30	75	150	60	1815
1100	22	55	110	44	1331

Basic	TA	DA	HRA	Comm	Final Salary
950	19	47.5	95	38	1149.5
3000	60	150	300	120	3630
1300	26	65	130	52	1573

14 rows selected.

SQL>

RELATIONAL OPERATORS:-

These are used to compare values by specifying conditions on the columns.

< > = <= >=

WHERE clause:

In **select** query we can write conditions in this clause.

Syntax:

```
select cl1, cl2,.....,cl-n / *  
from table_name  
where <conditions>  
order by cl1, cl2,...,cln [asc/desc];
```

Ex: display salaries below 2000?

```
select sal " emp sal below 2000" from emp_info where sal < 2000;
```


emp sal below 2000

```
-----  
      800  
     1600  
     1250  
     1250  
     1500  
     1100  
      950  
     1300
```

8 rows selected.

Ex: display the details of accounting dept?

```
select * from dept where dname='ACCOUNTING';
```

```
DEPTNO DNAME      LOC  
-----  
     10 ACCOUNTING NEW YORK
```

SQL>

Ex: display the details of managers?

```
select * from emp where job='MANAGER';
```

```
EMPNO ENAME      JOB          MGR HIREDATE      SAL      COMM  
DEPTNO  
-----  
-----  
20    7566 JONES      MANAGER      7839 02-APR-81   2975  
30    7698 BLAKE      MANAGER      7839 01-MAY-81   2850  
10    7782 CLARK      MANAGER      7839 09-JUN-81   2450
```

SQL>

Ex: display employee name and sal of empno 7788?

```
select ename,sal from emp where empno=7788;
```

Ex: Display employee details who joined before 1st jan 1981?

```
select * from emp where hiredate < '01-jan-81';
```

```
SQL> select * from emp where hiredate < '01-jan-81';
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7369	SMITH	CLERK	7902	17-DEC-80	800	

```
SQL>
```

ASSIGNMENTS:

Consider the below tables with estimated columns and then practise below questions.

CUST_DTLS
CUST_Act_DTLS
ACT_TYPES_INFO
PROD_DTLS
EMP
DEPT

- 1) Fetch all clerks information
- 2) Display all departments information located at CHICAGO?
- 3) Display product details manufactured in the current year only?
- 4) Get the details of customers accounts who opened the accounts before this year?
- 5) Get all SALARY account details?
- 6) Display customer names and mobile numbers from the city 'Texas'?

```
select cname,mobile from cust_dtls where city='Texas';
```

- 7) Get the information of Trading account?
 - 8) Display only Expired product details?

```
select * from prod_dtls where exp<sysdate;
```
 - 9) Display different department numbers from emp table?
 - 10) Display customer details from the city HYD?
 - 11) Display customer account details with balance below 5000?
 - 12) Display mobile details?
 - 13) Display products having min warranty 2 years?
 - 14) Display products with below 10000 cost?
-

SPECIAL OPERATORS:

BETWEEN

This operator supports specific range of values from a column. This is applicable on NUMERIC, CHAR and DATE data type columns

Syntax:-1 **BETWEEN**

```
select cl1, cl2,.....,cl-n / *  
from table_name  
where <ColumnName> BETWEEN <start_value> AND <end_value>  
order by cl1, cl2,...,cln [asc/desc];
```

IN

This Operator supports specific list of values from a column.
This is applicable to Numeric, date & character data type columns.

Syntax:

```
select cl1, cl2,.....,cl-n / *  
from table_name  
where <ColumnName> IN(val1, val2,val3,.....)  
order by cl1, cl2,...,cln [asc/desc];
```

IS NULL

This Operator is used to check the column value is null or not, if it is null
display output,
Otherwise it won't display output.

Syntax: **IS NULL**

```
select cl1, cl2,.....,cl-n / *  
from table_name  
where <ColumnName> IS NULL  
order by cl1, cl2,...,cln [asc/desc];
```

Note: It works on only null values and it is independent of data type of column

LIKE

The LIKE conditions specify a test involving pattern matching. Whereas the equality operator (=) exactly matches one character value to another, the LIKE conditions match a portion of one character value to another by searching the first value for the pattern specified by the second. LIKE calculates strings using characters as defined by the input character set.

In the following syntax ESCAPE symbol is any char except _ & % .
A character preceeding escape character remove the meaning of that character.

Syntax: **LIKE**

```
select cl1, cl2,.....,cl-n / *
from table_name
where <ColumnName> LIKE'string' ESCAPE 'literal';
order by cl1, cl2,...,cln [asc/desc];
```

LIKE:

It uses 2 symbols

_ (underscore) represents anyone char

% represents any number of chars

EXAMPLES:

Ex: display salaries between 2000 and 3000 in ascending order?

```
select sal from emp where sal between 2000 and 3000 order by sal;
```

SQL> select sal from emp where sal between 2000 and 3000 order by sal;

```
      SAL
-----
      2450
      2850
      2975
      3000
      3000
```

SQL>

Ex: display employee details who is joined in 1981?

```
select * from emp where hiredate between '01-jan-81' and '31-dec-81';
or
select * from emp where hiredate like'%81';
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
DEPTNO						
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300 30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500 30
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400 30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
7839	KING	PRESIDENT		17-NOV-81	5000	10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0 30
7900	JAMES	CLERK	7698	03-DEC-81	950	30

7902 FORD	ANALYST	7566 03-DEC-81	3000	20
-----------	---------	----------------	------	----

10 rows selected.

SQL>

Ex: display employees working like clerks and managers?

select * from emp where job IN('CLERK','MANAGER');

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7369	SMITH	CLERK	7902	17-DEC-80	800	20
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
7876	ADAMS	CLERK	7788	23-MAY-87	1100	20
7900	JAMES	CLERK	7698	03-DEC-81	950	30
7934	MILLER	CLERK	7782	23-JAN-82	1300	10

7 rows selected.

SQL>

Ex: display employee names and salaries who is getting any one of following salary?

1250,3000,5000

select ename,sal from emp where sal in(1250,3000,5000);

ENAME	SAL
WARD	1250
MARTIN	1250
SCOTT	3000
KING	5000
FORD	3000

SQL>

Ex: display employee id,name,sal,comm who is not getting comission?

select empno,ename,sal,comm from emp
where comm is null;

EMPNO	ENAME	SAL	COMM
7369	SMITH	800	
7566	JONES	2975	

7698 BLAKE	2850
7782 CLARK	2450
7788 SCOTT	3000
7839 KING	5000
7876 ADAMS	1100
7900 JAMES	950
7902 FORD	3000
7934 MILLER	1300

10 rows selected.

SQL>

Ex: display 3 digit salaries?

select sal from emp where sal like '___';

SQL> select sal from emp where sal like '___';

SAL
800
950

SQL>

Ex: display names of emps begins s?

select ename from emp where ename like 'S%';

ENAME
SMITH
SCOTT

SQL>

Ex: display employees joined in 87?

select * from emp where hiredate like '%87';

22-may-87
02-feb-87
11-oct-87

Ex:

Execute the following queries and verify the outputs

select * from emp;

select * from emp where sal between 1000 and 2000;

select * from emp where sal not between 1000 and 2000;

```
select * from emp where hiredate between '01-jan-81' and '31-dec-81' order
by hiredate;
```

```
select * from emp where hiredate not between '01-jan-81' and '31-dec-81'
order by hiredate;
```

```
select * from emp where job in('CLERK','SALESMAN');
```

```
select * from emp where job not in('CLERK','SALESMAN');
```

```
select * from emp where deptno in(10,20);
```

```
select * from emp where hiredate in('19-apr-87','23-jan-82') ;
```

```
create table sample as select * from emp;
select * from sample;
```

```
update sample set deptno=null where empno
in(7499,7566,7698,7788,7876,79007902,7934);
```

```
--Display 3-digit salaries
select sal from emp where sal like'___';
```

```
--Display salaries begining with digit "2"?
select sal from emp where sal like'2%';
```

```
--Display employee names begins with "J" and ends with "S"?
select ename from emp where ename like'J%S';
```

```
--Display 4-char length employee names?
select ename from emp where ename like'____';
```

```
--Display 4-char length employee names ends with "D"?
select ename from emp where ename like'___D';
```

```
--Display employee names,salaries, hiredates joined inn the year " 81"?
select ename,sal,hiredate from emp where hiredate like'%81';
```

RELATION NEGATION OPERATORS:

The following operators are the negation operators for the above special operators.

!= (or) <> (or) ^= (NOT EQUAL TO)

NOT BETWEEN

NOT LIKE

NOT IN

IS NOT NULL

Ex: Display all emps details except SALESMAN?

```
select * from emp where job<>'SALESMAN';
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7369	SMITH	CLERK	7902	17-DEC-80	800	20
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
7788	SCOTT	ANALYST	7566	19-APR-87	3000	20
7839	KING	PRESIDENT		17-NOV-81	5000	10
7876	ADAMS	CLERK	7788	23-MAY-87	1100	20
7900	JAMES	CLERK	7698	03-DEC-81	950	30
7902	FORD	ANALYST	7566	03-DEC-81	3000	20
7934	MILLER	CLERK	7782	23-JAN-82	1300	10

10 rows selected.

SQL>

Ex: Display employee details not joined in the last year?

```
select * from emp where hiredate NOT BETWEEN '01-jan-14' and '31-dec-14';
```

ASSIGNMENTS:

- 1) Display customer account details whose balance is at least 10000 and at most 100000?
- 2) Display unknown account details?
- 3) Display customer details whose gender is unknown?
- 4) Display customers from the citites 'TEXAS ' and 'CHICAGO'?
- 5) Display Product details manufactured in january of this year?
- 6) Display product details whose warrenty is finished in the last year?
- 7) Display customer names having a char 'K'?
- 8) Display customer details who is living in 6 char length cities?

LOGICAL OPERATORS:

These Operators are used to specify Multiple conditions in the where clause.

AND Display output if all conditions are true.
If any one condition was failed then it will not display output.

OR Display output if anyone condition is true.
If all conditions are false then it will not display output.

Syntax:

```
SELECT cl1,cl2,....., / *  
FROM <table_name>  
WHERE <cond-1> [ AND / OR ] <cond-2> [ AND / OR ] <cond-3> [  
AND / OR ].....  
ORDER BY cl1, cl2,..... [ ASC /DESC];
```

Ex: Display manager details getting above 2500 sal?

```
select * from emp where job='MANAGER' and sal>2500;
```

Ex: Display clerks and salesman details if their salary at least 1000 and atmost 1500?

```
select ename,sal,job from emp  
where job in('CLERK','SALESMAN') AND sal between 1000 and 1500;
```

Ex: Display salary account details having below 100000 balance?

```
select * from cust_act_dtls where act_type='sal' and bal <100000;
```

Assignments:

Ex: Display tablet or mobile information if their cost min 10000 and max 15000?

Ex: Display product details if they were manufactured in current year and min cost 2000 and max cost 10000?

CUST_ACT_DTLS				

ACTNO	ACT_TYPE	ACT_OPEN_DT	ACT_BAL	CUST_ID

Ex: Display "male" customers from "texas" and "female" customers from "chicago"?

CUST_DTLS			

CUST_ID	CUST_NAME	CUST_CITY	CUST_GENDER
CUST_MOBILE			

```
select * from cust_dtls
where (gender='male' and city='texas')
      or
      ( gender='female' and city='chicago');
```

EX:

Display employee details joined in 87 year or working under deptno 10?

```
select * from emp where hiredate like '%87' or deptno=10;
```

Ex:

Display trading account details having min balance 10000 and savings account details having min balance 100000?

DML COMMANDS

UPDATE

It is used to update old values with new values within the table. By default it updates all values in a column. We can also update column values by verifying conditions.

Updating single column value:

Syntax: **update**<table_name>
 set colname= value / expression / select. . .Query
 where <condition>;

Note: Without condition update command change all values in the column.

Updating multiple column values:

Syntax:
 update <table_name>
 set colname1= value / expression,
 colname2= value / expression,
 colname3= value / expression
 :

where <condition>;

Ex: update the commission of 7369 as 500?

```
update emp set comm=500 where empno=7369;
```

Ex: update all emps commissions as 1000?

```
update emp set comm=1000;
```

Ex: update the salesman salary with 20% increment , change their designation as Sr.SALES who joined before 2005?

```
update emp
set sal=sal+(0.20*sal),
  job='Sr.SALES'
where job='SALESMAN' AND hiredate < '01-jan'05';
```

Ex: Update the customer account details with 25% additional interest?

```
Update cust_act_dtls
Set act_bal=act_bal + (act_bal*0.25);
```

DELETE:

This command is used to delete the records from the table. By default it deletes all the records. We can also delete the records conditionally.

Syntax:

```
delete from <table_name> where <cond>;
```

Ex: delete all customer details?

```
delete from cust_dtls;
```

Ex: delete employees information who is not getting any commission?

```
delete from emp where comm is null;
```

Note:

We can get back the deleted records / data within the current session by using *ROLLBACK*.

DDL COMMANDS:-

CREATE
ALTER
DROP
TRUNCATE
RENAME

ALTER:

This command is used to change the structure of the table by

i) adding new columns

syn:

```
alter table <table_name> ADD
(
    <col_name> datatype(size),
    <col_name> datatype(size),
    :
    :
);
```

ii) deleting existed columns

syn: alter table <table_name> DROP COLUMN <col_name>;

iii) changing the datatype and size (increasing / decreasing) of column

syn: alter table <table_name> MODIFY <colname>
new_datatype(new_size);

Note:

- A) If the column is empty then we can do the following
 - We can change from any data type to any data type.
 - We can change any size to any size .

- B) If column is not empty then,
 - Number type and CHAR type column size can be decreased but can be increased.
 - Data types can be changed from CHAR TO VARCHAR2 AND vice versa,
 - NUMBER(p) to NUMBER(p,s) BUT NOT VICE VERSA.

iv) Rename a column with New Name.

syn: alter table <table_name>
RENAME COLUMN<oldcolname>**to**<newcolname>;

Ex: alter table emp rename column ename to empnames;

3) DROP

This command is used to delete any data base object.

How to delete the table?

Ex: drop table <table_name>;

4) TRUNCATE:

It deletes all the data from the table, it cannot deletes the partial data from a table.

We cannot get back or restore deleted data.

Syntax: truncate table <table_name>;

Ex: truncate table customers;

Ex: let us consider a table stud_dtls with columns rno ,sname, fee

stud_dtls		

rno	sname	fee
number(2)	varchar2(5)	number(5)

Ex: change the above table to maintain course name for each student?

alter table stud_dtls add course_name varchar2(10);

Ex: change the above table to maintain gender and mobile number for each student?

```
alter table stud_dtls
add
(
  gender char,
  mobile number(10)
);
```

Ex: from the above table delete student mobile number column?

```
alter table stud_dtls
drop column mobile;
```

Ex: increase the student name column size to 20?

```
alter table stud_dtls modify sname varchar2(20);
```

Ex: change the name of column sname to stud_name?

```
alter table stud_dtls RENAME sname to STUD_NAME;
```

NOTE:

We can also change the table name by using ALTER command as follows.

```
ALTER TABLE emp  
RENAME emp empinfo;
```

TCL (TRANSACTION CONTROL LANGUAGE) COMMANDS:

Generally, DML operations on table data are considered as transactions. Transaction Control Language(TCL) commands are used to manage transactions in database. These are used to manage the changes made by DML statements. It also allows statements to be grouped together into logical transactions.

1) COMMIT

It is used to make permanent the user transactions(DML operations) on the table.

Note: Once a transaction made permanent then we cannot cancel it

2) ROLLBACK:

It is used to cancel the user transaction.

3)SAVEPOINT

This command is used to save a set of transactions under a name temporarily. So that if we want to rollback transactions to a save point we can rollback up to a save point.

```
SAVEPOINT <name>;
```

Examples:

```
create table cust  
(  
cid char(3),  
cname varchar2(20)  
);
```

```
insert into cust values('c00','Sanju');
insert into cust values('c01','Manoj');
```

```
select * from cust;
```

```
rollback;
```

```
select * from cust;
```

```
insert into cust values('c00','Sanju');
insert into cust values('c01','Manoj');
```

```
commit;
select * from cust;
```

```
rollback;
select * from cust;
```

Ex-2:

```
Insert into cust values('c02','hellen','Female','xx','hyd');
```

```
delete from cust where cname='Sanju';
```

```
savepoint s1;
```

```
update cust set phone='aa' where cname='hellen';
```

```
savepoint s2;
```

```
delete from cust;
```

```
select * from cust;
```

```
rollback to s2;
```

```
rollback;
```

DCL commands: [Data Control Language commands]

GRANTING AND REVOKING PERMISSIONS:

Oracle provides extensive security features in order to safeguard information stored in its tables from unauthorized viewing and damage. Depending on a user's status and responsibility, appropriate rights on Oracle's resources can be assigned to the user by the DBA. The rights that allow the use of some or all of oracle's resources on the Server are called **PRIVILEGES**.

Objects that are created by a user are owned and controlled by that user. If a user wishes to access any of the objects belonging to another user, the owner of the object will have to give permissions for such access. This is called **GRANTING** of **PRIVILEGES**.

Privileges once given can be taken back by the owner of the object. This is called **REVOKING** of **PRIVILEGES**.

GRANT Statement:

Syntax:

```
GRANT <object privileges>  
ON <object_name>  
TO <User_Name>  
[WITH GRANT OPTION]
```

eg:

```
GRANT ALL  
ON Student TO Mohd Imran  
WITH GRANT OPTION
```

The **WITH GRANT OPTION** allows the grantee to in turn grant object privileges to other users.

eg:

```
GRANT SELECT, UPDATE  
ON Student TO Fareen  
WITH GRANT OPTION
```

The user **Fareen** has been given permission to view and modify records in the table **Student**.

To view the contents of the student table that belongs to Mohd Imran.

eg:

```
SELECT * FROM Mohd Imran.Student;
```

Mohd Imran is the owner of the table, and he has given privileges to other user for his table.

Objects Privileges that can be given to users.

Each object privileges that is granted authorized the grantee to perform some operation on the object. A user can grant all the privileges or grant only specific object privileges.

The list of object privileges is as follows:

Alter: Allows the grantee to change the table definition with the ALTER TABLE command.

DELETE: Allows the grantee to remove the records from the table with the DELETE command.

INDEX: Allows the grantee to create an index on the table with the CREATE INDEX command.

INSERT: Allows the grantee to add records to the table with the INSERT command.

SELECT: Allows the grantee to query the table with the SELECT command.

UPDATE: Allows the grantee to modify the records in the tables with the UPDATE command.

REVOKING PRIVILEGES GIVEN:

Privileges once given can be denied to a user using the REVOKE command. The object owner can revoke privileges granted to another user. A user of an object who is not the owner, but has been granted the GRANT privileges, has the power to REVOKE the privileges from grantee.

Revoking the Permission using the REVOKE command.

Syntax:

```
REVOKE <Object_Privileges>  
ON <Object_Name>  
FROM <User_Name>
```

Eg:

```
REVOKE UPDATE  
ON Student  
FROM Fareen;
```

Ex: Assign SELECT previllage to the user-2 on the table PRODUCTS created by USER-1

```
GRANT SELECT ON ORCL.USER_1.PRODUCTS TO USER_2;
```

Ex: How do i cancel the above permission?

```
REVOKE SELECT ON ORCL.USER_1.PRODUCTS FROM USER-2;
```

Ex: How to assign CREATE VIEW permission to user-2?

```
GRANT CREATE VIEW To user_2;
```

Ex: How do i cancel all previllages on all objects to the user-1?

```
REVOKE ALL from USER_1;
```

Data Integrity Constraints

CONSTRAINTS:

Constraints are set of rules / business rules which will be defined at DDL level.

Constraints enforce the data base to allow only valid values in to the tables.

Constraints ensure the user to fetch only valid / complete and accurate data from the database.

Categories of Constraints: 3

- 1) **Key Constraints**
- 2) **Domain Constraints**
- 3) **Referential Integrity constraints**

1) KEY CONSTRAINTS:

These constraints check the individual values in to a column according to Business.

These are divided into 3 types

a) **UNIQUE**

It doesn't allow duplicates but allows null values.

Ex: This constraint is suitable for maintaining phone numbers, mailid, etc...

b) **NOT NULL**

It doesn't allow null values but allows duplicates.

Ex: EmpNames, CustNames,

c) **PRIMARY KEY**

It doesn't allow duplicates and null values. Generally a Primary key is used to identify any record in a table **uniquely**. *"Only one primary key is allowed per a table"*.

Primary key is of 2 types.

Simple primary key

If a Primary key Defined on a single column then it is known as Simple Primary key.

Composite primary key

If a Primary key constraint defined on more than one column then it is known as Composite Primary Key.

(Max numbers of columns in to a composite Primary key are 32 columns)

Composite Primary Key: primary key (custid,prodid,timeid)

SALES_DTLS				
=====				
custid	prodid	timeid	qty	sales_amount
-----	-----	-----	----	-----
c1	p3	jan5-15	100	100000
c1	p2	jan5-15	100	200000
c2	p3	jan5-15	100	100000
c1	p3	jan7-15	100	150000
c5	p1	jan5-15	200	300000

Syntax: Creating a table with **key** constraints:

```
Create table <table name>
(
col1 data type(size) <constraint_name>,
col2 data type(size) <constraint_name>,
:      :      :      :
);
```

Ex: create a table student with columns rno,sname,course,fee and mobile along with constraints pk,nn,nn,nn and unique respectively?

```
Create table student
(
rno    number(2) primary key,
sname        varchar2(10) not null,
course        varchar2(15) not null,
fee    number(5) not null,
mobile        char(10) unique
);
```

```
insert into student values(1,'a','oracle',9000,'8989898989');
insert into student values(0,'b','java',2000,'8787878787');
insert into student values(2,'x','oracle',9000,null);
insert into student values(11,'s','abc',100,null);
```

data:-

RNO	SNAME	COURSE	FEE	MOBILE
-----	-----	-----	-----	-----
1	a	oracle	9000	8989898989

0		b	java	2000	8787878787
2	x		oracle	9000	
11	s		abc	100	

ERROR GENERATING RECORDS:

```

insert into student values(1,'kiran','java',2300,null);
insert into student values(null,'kiran','java',2300,null);
insert into student values(12,null,'java',2300,null);
insert into student values(1,'kiran',null,2300,null);
insert into student values(1,'kiran','java',null,null);
insert into student values(1,'kiran','java',2300,8989898989);

```

Note: Even after the key constraints on the table, still we have invalid values.

We can eliminate them by using **DOMAIN** constraints.

Displaying constraints information on a table:-

In oracle database, all constraints stored in a system defined table called **USER_CONSTRAINTS**.

Each constraint named and numbered uniquely like, **SYS_Cn**(System defined Constraint).

To Fetch data from this table use the below example.

Ex:

```

select constraint_name,constraint_type from USER_CONSTRAINTS where
table_name='STUDENT';

```

CONSTRAINT_NAME	CONSTRAINT_TYPE
SYS_C007050	C --either Not null or Check
SYS_C007051	C
SYS_C007052	C
SYS_C007053	P --Primary key
SYS_C007054	U --Unique key
SYS_C007055	R --Foreign key

CONSTRAINTS with user-defined names:

```

SYN: <col> datatype(size) Constraint <User defined name><actual
constraint name>,
      <col> datatype(size) Constraint <User defined name><actual
constraint name>,

```

Ex: <col> datatype(size) Constraint pk_rno_student Primary key,

```
create table stud_dtls
```

```
(
  rno    number(2) constraint pk_rno_stud primary key,
  sname   varchar2(20) constraint nn_sname_stud not null,
  course  varchar2(7) constraint nn_course_stud not null,
  fee     number(5) constraint nn_fee_stud not null,
  mobile  number(10) constraint uk_mobile_stud unique
);
```

```
create table student_dtls
```

```
(
  rno    number(2) constraint pk_rno_student primary key,
  constraint ck_rno_student check (rno between 1 and 60),
  sname   varchar2(20) constraint nn_sname_student not null,
  course  varchar2(7) constraint nn_course_student not null,
  constraint ck_course_student check (course in('cse','ece','eee','it')),
  fee     number(5) constraint nn_fee_student not null,
  constraint ck_fee_student check (fee between 30000 and 40000),
  mobile  number(10) constraint uk_mobile_student unique
);
```

```
select constraint_name,constraint_type from user_constraints where
table_name='STUDENT_DTLS';
```

CONSTRAINT_NAME	CONSTRAINT_TYPE
NN_SNAME_STUDENT	C
NN_COURSE_STUDENT	C
NN_FEE_STUDENT	C
CK_RNO_STUDENT	C
CK_COURSE_STUDENT	C
CK_FEE_STUDENT	C
PK_RNO_STUDENT	P
UK_MOBILE_STUDENT	U

8 rows selected

DOMAIN constraints:

It is used to define a valid range / valid list of values on a column by using the keyword *CHECK*.

CHECK uses 2 operators.

BETWEEN to define range

IN to define list of values.

Syntax:

```
create table <table_name>
```

```
(
col1 datatype(size) <constraint_name>,
col2 datatype(size) <constraint_name>,
:      :      :      :,

CHECK (col1 BETWEEN begin_value AND end_value),
CHECK (col2 IN (val1, val2,.....)),
....
....

);
```

Ex: create the above table along with below domain constraints:

```
--rno should be between 1 and 60
--course names are oracle, sql server and unix
--Min fee is 5000 and max fee 10000
```

```
create table stud_dtls
(
rno number(2) constraint pk_rno_stud primary key,
sname varchar2(10) constraint nn_name_stud not null,
course varchar2(15) constraint nn_course_stud not null,
fee number(5) constraint nn_fee_stud not null,
mobile char(10) constraint uk_mobile_stud unique,
constraint ck_rno_stud check (rno between 1 and 60),
constraint ck_course_stud check (course in('oracle','sql server','unix')),
constraint ck_fee_stud check (fee between 5000 and 10000)
);
```

Records:

```
insert into stud_dtls values(1,'a','oracle',7000,1212);
insert into stud_dtls values(0,'b','sql server',7000,null);
insert into stud_dtls values(61,'a','oracle',7000,null);
insert into stud_dtls values(12,'b','sql server',7000,null);
insert into stud_dtls values(11,'ajay','unics',10000,2212);
insert into stud_dtls values(21,'hari','unix',17000,1211);
insert into stud_dtls values(21,'hari','unix',11000,1211);
```

RNO	SNAME	COURSE	FEE	MOBILE
-----	-----	-----	-----	-----
1	a	oracle	7000	1212
12	b	sql server	7000	
21	hari	unix	11000	1211

CREATING A TABLE WITH USER-FRIENDLY NAMES TO THE CONSTRAINTS:-

```
create table s_dtls
```

```
(
rno number(2)          CONSTRAINT PK_RNO_S_DTLS      primary key,
sname varchar2(10)     CONSTRAINT NN_SNAME_S_DTLS   not null,
course varchar2(15)    CONSTRAINT NN_COURSE_S_DTLS   not null,
fee number(5)          CONSTRAINT NN_FEE_S_DTLS      not null,
mobile char(10)        CONSTRAINT UK_MOB_S_DTLS      unique,
CONSTRAINT CK_RNO_S_DTLS check (rno between 1 and 60),
CONSTRAINT CK_COURSE_S_DTLS check (course in('oracle','sql
server','unix')),
CONSTRAINT CK_FEE_S_DTLS check (fee between 10000 and 20000)
);
```

How do i display constraints information of a table?

```
SELECT CONSTRAINT_NAME,CONSTRAINT_TYPE FROM
USER_CONSTRAINTS WHERE TABLE_NAME='s_dtls'; ----> this won't work
```

```
SELECT CONSTRAINT_NAME,CONSTRAINT_TYPE FROM
USER_CONSTRAINTS WHERE TABLE_NAME='S_DTLS';
```

CONSTRAINT_NAME	CONSTRAINT_TYPE
NN_SNAME_S_DTLS	C
NN_COURSE_S_DTLS	C
NN_FEE_S_DTLS	C
CK_RNO_S_DTLS	C
CK_COURSE_S_DTLS	C
CK_FEE_S_DTLS	C
PK_RNO_S_DTLS	P
UK_MOB_S_DTLS	U

constraint_type	Meaning
C	check or not null
P	Primary Key
U	Unique key
R	Foreign key

Assignment:

i) create customers table with columns
custid,custname,city,gender,mailid,phone, Address with the constraints
Pk,NN,NN,NN, and Unique and Unique Respectively.

NORMALIZATION AND DENORMALIZATION CONCEPTS:

Consider the below tables and data.

DEPT

DNO	DNAME	LOC
10	Production	Hyderabad
20	Sales	Hyderabad
30	Finance	Chennai

EMP

Eid	Ename	Salary
1	A	2000
2	X	1200
3	A	3400
4	Z	5000
5	C	1000
6	S	1300
7	D	2300
8	X	1200
9	B	2200

Note:

By using above tables we are unable to fetch the complete data of an object, like, department name of any employee, number of employees in dept and etc.

The solution for such kind of requirements we have 2 methods.

- 1) Maintaining all the information in one big table [*DENORMALIZED DATA*]
- 2) Maintaining data in different tables and implementing Physical relationships between the tables [*NORMALIZED DATA*]

1) DENORMALIZATION

Maintaining all information in one big table is known as De-normalized method.

emp_dept_details

Eid	Loc	Ename	Salary	Dno	Dname
1		A	2000	10	Production
2		X	1200	10	Production
3		A	3400	10	Production
4		Z	5000	10	Production
5		C	1000	20	Sales
6		S	1300	20	Sales
7		D	2300	20	Sales
8		X	1200	30	Finance
9		B	2200	30	Finance

Note:

From the above table we will get the required information, but it has

- data duplicacy
- it occupies more disk space
- data retrieval time is very high.

Disk space: 6X9=54 kb

2) NORMALIZATION:

The Process of dividing the above big table in to sub tables until the data duplicacy is maximum reduced is called normalization process.

i) 1st NF(normal form):

Dividing the table into sub tables based on repeated groups of data.

emp			dept		
eid	ename	sal	dno	dname	loc
1	a	2000	10	production	hyderabad
2	x	1200	10	production	hyderabad
3	a	3400	10	production	hyderabad
4	z	5000	10	production	hyderabad
5	c	1000	20	sales	hyderabad
6	s	1300	20	sales	hyderabad
7	d	2300	20	sales	hyderabad
8	x	1200	30	fin	chennai
9	b	2200	30	fin	chennai

Ex:

Emp Table:

```
create table emp
as
select eid,ename,sal from emp_dept_details;
```

DEPT table:

```
create table dept
as
select dno,dname,loc from emp_dept_details;
```

ii) IInd NF:

Eliminating duplicates and defining primary keys

emp			dept		
eid	ename	sal	dno	dname	loc
1	a	2000	10	production	hyderabad
2	x	1200	20	sales	hyderabad
3	a	3400	30	fin	chennai
4	z	5000	PK		
5	c	1000			
6	s	1300			
7	d	2300			
8	x	1200			
9	b	2200			
PK					

Ex: Eliminating Duplicate records

```
create table dept1
as
select distinct dno,dname,loc from dept;
```

iii) IIIrd NF:

Implementing relationships between the tables by using Primary key of dept table , define the foreign key column under emp table.

emp				dept		
eid	ename	sal	dno	dno	dname	loc
1	a	2000	10	10	production	hyderabad
2	x	1200	10	20	sales	hyderabad
3	a	3400	10	30	fin	chennai
4	z	5000	10	PK		

5	c	1000	20
6	s	1300	20
7	d	2300	20
8	x	1200	30
9	b	2200	30
PK			FK

It occupies less disk space and max .data duplicacy is reduced.

disk space: $4 \times 9 = 36$
 $3 \times 3 = 9$

45KB

Advantages:

- Searching for required data is as much as fast
- And data retrieval is fast
- Max data duplicacy is eleminated.
- Occupy less Disk space.

3) REFERENTIAL INTEGRITY CONSTRAINTS:

This constraint is used to implement PHYSICAL relationship between the tables by using primary key of one table and we can define foriegn key in other table.

Foriegn key column contains only values from primary key. Foriegn key contains duplicates and null values also.

--A table which contains primary key is considered as parent /Master/Base table.

--A table which contains foriegn key is known as child table /Detailed table/ Derived table.

REFERENCES

we can use this keyword in the "creation of child table and to define foriegn key column".

Ex: create comp_dtls as parent table

Ex: create prod_dtls as child table

```
create table comp_dtls
(
  cmpId char(5) constraint pk_cmpid_comp_dtls primary key,
  cmpName varchar2(20) not null,
  cmpCountry varchar2(20) not null,
  constraint ck_country_cmp
  check (cmpcountry IN('india','usa','japan','uk'))
);
```

```

insert into comp_dtls values('cmp01','sony','japan');
insert into comp_dtls values('cmp02','wipro','india') ;
insert into comp_dtls values('cmp03','Philips','india');
insert into comp_dtls values('cmp04','semantic','usa');

```

```

create table prod_DTLS
(
  pid      char(4) primary key,
  pname    varchar2(20) not null,
  cost     number(7,2),
  mfg      date,
  warrenty varchar2(10),
  cmpId    char(5),
  constraint fk_prod_cmpId FOREIGN KEY(cmpId) REFERENCES
comp_dtls(cmpId) /* foreign key column */
);

```

```

insert into prod_dtls values
('p001','smart phone',34000,'12-may-14','1 year','cmp01');
insert into prod_dtls values
('p002','laptop',54000,'03-feb-14','3 years','cmp01');
insert into prod_dtls values
('p003','Television',24000,'08-aug-14','5 years','cmp03');
insert into prod_dtls values
('p004','Home Theatre',55000,'11-aug-13','2 years','cmp03');
insert into prod_dtls values
('p005','Mobile',24000,'08-aug-14','1 year',null);
insert into prod_dtls values
('p006','vmware',64000,'22-oct-10','1 year','cmp04');

```

Ex: create the following tables and Implement relationships accordingly?

1) Cust_dtls (parent table)

cno	cname	city	gender	mobile

Primary key

```

CREATE TABLE cust_dtls
(
  Cno char(5) constraint pk_cno PRIMARY KEY,
  Cname varchar2(20) constraint nn_cname NOT NULL,
  City varchar2(20) ,
  MOBILE number(10)
);

```

2) Act_types (parent table)

<u>Act type</u>	<u>act name</u>	<u>desc</u>
-----------------	-----------------	-------------

SB	Savings Bank
DEMAT	Trading account
CA	Current account

```
CREATE TABLE act_types
(
Act_type char(5) constraint pk_act_type PRIMARY KEY,
Act_name varchar2(20)
);
```

3) cust_act_dtls (Child table)

Actno	Act_type	Act_open_date	Act_bal	cno
PK	Foreign key			Foreign key

```
CREATE TABLE cust_act_dtls
(
Actno number(10) constraint pk_actno primary key,
Act_type char(5),
Constraint fk_act_type FOREIGN KEY(act_type)
REFERENCES act_types(act_type),
Act_open_date date,
Act_bal number(8,2),
Cno char(5),
Constraint fk_cno FOREIGN KEY(cno) REFERENCES cust_dtls(cno)
);
```

ON DELETE CASCADE

It allows deletion of parent table records even the parent is having childs. As an effect the dependant child records are automatically deleted. This known as "*Maintaining Data Integrity among the tables.*"

ALTERING CONSTRAINTS

Altering constraints means,
i) Adding constraints to the existed columns

Keyword: ADD CONSTRAINT

ii) Deleting constraints from the columns

Keyword: DROP CONSTRAINT

iii) Modify constraints / changing existing constraints with new constraints

To do this ,First drop old constraint and then add new constraints.

ENABLING / DISABLING CONSTRAINTS

We can also temporarily enable or disable constraints on the columns based on business requirement.

Syn:- Enable constraint

Alter table <table_name>

ENABLE CONSTRAINT<constraint_name>;

Syn:- Disable constraint

Alter table <table_name>

DISABLE CONSTRAINT<constraint_name>;

NOTE:

Q: Can we enable any constraint if a column having invalid data according to constraint?

Ans: YES ,we can enable , but the column should be defined with DIFFERABLE keyword.

MERGE statement

Oracle's MERGE statement is used for situations when you want to do an "upsert" i.e. update existing rows in a table or insert new rows depending on a match condition. This is typically the case when you have to synchronize a table periodically with data from another source (table/view/query).

Syntax:

```
MERGE into <target table>
USING
<source table/view/result of subquery>
ON
<match condition>
WHEN MATCHED THEN
<update clause>
<delete clause>
WHEN NOT MATCHED THEN
<insert clause>
```

Example:

SQL> select * from student;

ID	NAME	SCORE
1	Jack	540
2	Rose	
3	William	650
4	Caledon	620
5	Fabrizio	600
6	Thomas	

7 Ruth	680
8 Spacer	555

8 rows selected.

SQL> select * from student_n;

ID NAME	SCORE

7 Ruth	690
8 Spicer	620
9 Wallace	600
10 Lizzy	
11 Brock	705

As you can see, the following actions are required on table STUDENT:

- 1 row for id#7 to be corrected for score: Ruth had scored 690, not 680.
- 1 row for id#8 to be corrected for name: the student is called Spicer, not Spacer.
- 3 new rows (ids#9,10,11) to be inserted into STUDENT table.

Note: 5 rows should get processed in all.

```
merge into student a
using
  (select id, name, score
   from student_n) b
on (a.id = b.id)
when matched then
  update set a.name = b.name
    , a.score = b.score
when not matched then
  insert (a.id, a.name, a.score)
  values (b.id, b.name, b.score);
```

5 rows merged.

SQL> select * from student;

ID NAME	SCORE

1 Jack	540
2 Rose	
3 William	650
4 Caledon	620
5 Fabrizio	600
6 Thomas	
7 Ruth	690
11 Brock	705
10 Lizzy	
9 Wallace	600
8 Spicer	620

11 rows selected.

Sure enough, 5 rows have got merged as expected – 2 updates + 3 inserts.

SET OPERATORS

You can combine multiple queries using the set operators **UNION**, **UNION ALL**, **INTERSECT**, and **MINUS**. All set operators have equal precedence. If a SQL statement contains multiple set operators, then Oracle Database evaluates them from the left to right unless parentheses explicitly specify another order.

The corresponding expressions in the select lists of the component queries of a compound query must match in number and must be in the same datatype group (such as numeric or character).

Or

These operators will display combined data from multiple tables.

1) UNION

It will display combined data from multiple tables without duplicates

2) UNION ALL

It will display combined data from multiple tables with duplicates

3) INTERSECT

It will display common data from multiple tables (From multiple Select stmts)

4) MINUS

It will display values from first selection by eliminating values which are repeating in second selection

For example:

s1={a,b,c,d} s2={x,y,z,c,b}

1) select * from s1
union
select * from s2;

output: {a,b,c,d,x,y,z}

2) select * from s1
union all
select * from s2;

output: {a,b,c,d,x,y,z,c,b}

3)select * from s1
intersect

```
select * from s2;
```

output: {c,b}

```
4)select * from s1
    minus
    select * from s2;
```

output: {a,d}

Sample Tables:

```
CREATE TABLE CUST_BR1
(
CID CHAR(3),
CNAME VARCHAR2(20),
MOBILE NUMBER(10),
CITY VARCHAR2(20),
GENDER VARCHAR2(10)
);
```

```
INSERT INTO CUST_BR1 VALUES('C1','VIJAY',1212121212,'HYD','MALE');
INSERT INTO CUST_BR1 VALUES('C2','JOHN',1313131313,'DELHI','MALE');
INSERT INTO CUST_BR1
VALUES('C3','SWATHI',1414141414,'HYD','FEMALE');
```

```
CREATE TABLE CUST_BR2
(
CID CHAR(3),
CNAME VARCHAR2(20),
MOBILE NUMBER(10),
CITY VARCHAR2(20),
GENDER VARCHAR2(10)
);
```

```
INSERT INTO CUST_BR2 VALUES('C1','KIRAN',9898989898,'HYD','MALE');
INSERT INTO CUST_BR2 VALUES('C2','JOHN',1313131313,'DELHI','MALE');
INSERT INTO CUST_BR2
VALUES('C3','LAKSHMI',8989898989,'DELHI','FEMALE');
```

```
CREATE TABLE CUST_BR3
(
CID CHAR(3),
CNAME VARCHAR2(20),
MOBILE NUMBER(10),
CITY VARCHAR2(20),
GENDER VARCHAR2(10)
);
```

```
INSERT INTO CUST_BR3 VALUES('C1','KIRAN',9898989898,'HYD','MALE');
INSERT INTO CUST_BR3 VALUES('C2','JOHN',1313131313,'DELHI','MALE');
INSERT INTO CUST_BR3
VALUES('C5','VINAY',7878787878,'DELHI','MALE');
```

Examples:

→ Display all customers info from all branches

```
select * from cust_br1
union all
select * from cust_br2
union all
select * from cust_br3;
```

→ Get the customer details without duplicates

```
select * from cust_br1
union
select * from cust_br2
union
select * from cust_br3;
```

→ DISPLAY COMMON CUSTOMER NAMES AND MOBILE NUMBERS FROM ALL BRANCHES

```
SELECT CNAME,MOBILE FROM CUST_BR1
INTERSECT
SELECT CNAME,MOBILE FROM CUST_BR2
INTERSECT
SELECT CNAME,MOBILE FROM CUST_BR3;
```

→ DISPLAY CUSTOMERS DETAILS WHO IS THE ONLY CUSTOMER FOR BRANCH 2

```
SELECT * FROM CUST_BR2
MINUS
(
SELECT * FROM CUST_BR1
UNION ALL
SELECT * FROM CUST_BR3
);
```

Note:

What are the limitations of set operators?

***We need to select equal number of columns from each table
We need to select same data type of data in the same sequence under each select query.***

Ex:

```
select cname,mobile from cust_br1
union all
select cid from cust_br2;
```

```
select cname,mobile from cust_br1
```

```
*
```

ERROR at line 1:

ORA-01789: query block has incorrect number of result columns

```
SQL> select cname,mobile from cust_br1
```

```
2 union all
```

```
3 select mobile,cname from cust_br2;
```

```
select cname,mobile from cust_br1
```

```
*
```

ERROR at line 1:

ORA-01790: expression must have same datatype as corresponding expression

Restrictions on the Set Operators The set operators are subject to the following restrictions:

- The set operators are not valid on columns of type BLOB, CLOB, BFILE, VARRAY, or nested table.
- The UNION, INTERSECT, and MINUS operators are not valid on LONG columns.
- If the select list preceding the set operator contains an expression, then you must provide a column alias for the expression in order to refer to it in the *order_by_clause*.
- You cannot also specify the *for_update_clause* with the set operators.
- You cannot specify the *order_by_clause* in the *subquery* of these operators.
- You cannot use these operators in SELECT statements containing TABLE collection expressions.

JOINS

Joins are used to display multiple data types of data from multiple tables. A **join** is a query that combines rows from two or more tables, views, or materialized views. Oracle Database performs a join whenever multiple tables appear in the **FROM** clause of the query. The select list of the query can select any columns from any of these tables. If any two of these tables have a column name in common, then you must qualify all references to these columns throughout the query with table names to avoid ambiguity.

Types of joins: 4

- i) **CROSS JOIN**
- ii) **EQUI JOIN / INNER JOIN**
- iii) **SELF JOIN**
- iv) **OUTER JOINS**

CROSS JOIN / CARTESIAN PRODUCT

It will display combination of data from multiple tables. In this join, each value in the first table is mapped with all values in the second table. It will display all possible combinations of data from multiple tables.

If two tables in a join query have no join condition, then Oracle Database returns their **Cartesian product**. Oracle combines each row of one table with each row of the other. A Cartesian product always generates many rows and is rarely useful.

Example:

consider
s1={a,b,c,d} s2={d1,d2}

s1Xs2={(a,d1),(a,d2),(b,d1),(b,d2),.....}

syn:

```
select col1, col2,...,coln  
from table1, table2,...  
where <cond>  
order by col1, col2,...[ desc ];
```

Ex: Display employee names , salaries and their department names?

```
select ename,sal,dname from emp,dept;
```

ENAME	SAL	DNAME
SMITH	800	ACCOUNTING
ALLEN	1600	ACCOUNTING
WARD	1250	ACCOUNTING
JONES	2975	ACCOUNTING
.....		
.....		
.....		
SMITH	800	RESEARCH
ALLEN	1600	RESEARCH
WARD	1250	RESEARCH
JONES	2975	RESEARCH
MARTIN	1250	RESEARCH
.....		
.....		
.....		
SMITH	800	SALES
ALLEN	1600	SALES
WARD	1250	SALES
JONES	2975	SALES
MARTIN	1250	SALES
.....		
.....		
.....		
SMITH	800	OPERATIONS
ALLEN	1600	OPERATIONS
.....		
ENAME	SAL	DNAME
WARD	1250	OPERATIONS
JONES	2975	OPERATIONS
MARTIN	1250	OPERATIONS
BLAKE	2850	OPERATIONS
.....		
.....		

56 rows selected.

SQL>

Note:

In the above output some combinations are valid and remaining are invalid according to physical table data.

Ex:

Get manager details and their dept details?

```
SQL> select * from
2 emp,dept
3 where job='MANAGER';
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	DEPTNO
DNAME	LOC							
-								
7566	JONES	MANAGER		7839 02-APR-81	2975		20	10
ACCOUNTING	NEW YORK							
7566	JONES	MANAGER	7839	02-APR-81	2975		20	20
RESEARCH	DALLAS							
7566	JONES	MANAGER		7839 02-APR-81	2975		20	30 SALES
CHICAGO								
7566	JONES	MANAGER		7839 02-APR-81	2975		20	40
OPERATIONS	BOSTON							
7698	BLAKE	MANAGER		7839 01-MAY-81	2850		30	10
ACCOUNTING	NEW YORK							
7698	BLAKE	MANAGER		7839 01-MAY-81	2850		30	20
RESEARCH	DALLAS							
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30	30 SALES
CHICAGO								
7698	BLAKE	MANAGER		7839 01-MAY-81	2850		30	40
OPERATIONS	BOSTON							
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10	10
ACCOUNTING	NEW YORK							
7782	CLARK	MANAGER		7839 09-JUN-81	2450		10	20
RESEARCH	DALLAS							
7782	CLARK	MANAGER		7839 09-JUN-81	2450		10	30 SALES
CHICAGO								
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	
DEPTNO	DNAME	LOC						
-								
7782	CLARK	MANAGER		7839 09-JUN-81	2450		10	40
OPERATIONS	BOSTON							

12 rows selected.

SQL>

"In the above output, bold faced lines are valid , and remaining are invalid."

So, Then in which case cross join will display only valid combination?

Ex: for all managers display 13% increment details?

```
select ename,sal,job,per_id,per_incr,desc
from emp,Percent_incr_dtls
where job='manager' and per_incr='13%';
```

Cross join examples:

```
CREATE TABLE "PER_INCR"
(
  "INCRID" NUMBER,
  "INCRVAL" VARCHAR2(20 BYTE),
  "DESCRIPTION" VARCHAR2(40 BYTE) DEFAULT null
);
```

```
insert into per_incr values
(101,'5%','Min INcrement');
```

```

insert into per_incr values
(102,'10%','Second level INcrement');
insert into per_incr values
(103,'15%','3rd level INcrement');
insert into per_incr values
(104,'25%','Max level INcrement');

```

```

select * from emp;
select * from per_incr;

```

INCRID	INCRVAL	DESCRIPTION
101	5%	Min INcrement
102	10%	Second level INcrement
103	15%	3rd level INcrement
104	25%	Max level INcrement

Ex: display managers details with 15% increment?

```

select e.ename,e.sal,e.job,i.incrval,i.description
from emp e,per_incr i
where e.job='MANAGER' AND i.incrval='15%';

```

Ex: Apply 5% for all clerks

```

select e.*, i.* from emp e, per_incr i
where e.job='CLERK' and i.incrval='5%';

```

EQUI JOIN / INNER JOIN

A cross join is known as equi join if we specify join condition using '=' operator.

It will display only matched data from all tables.

A condition is known as join condition if it is specified between primary key of one table and foreign key of other table.

```

syn: select col1, col2,...,coln / *
      from table1, table2,...
      where table1.pk=table2.fk and table2.pk=table3.fk.....
      order by col1,col2,...[ asc/ desc];

```

Ex: display employee names , salaries and corresponding department details?


```
select ename, sal,dept.deptno,dname,loc
from emp,dept
where emp.deptno=dept.deptno;
```

ENAME	SAL	DEPTNO	DNAME	LOC
CLARK	2450	10	ACCOUNTING	NEW YORK
KING	5000	10	ACCOUNTING	NEW YORK
MILLER	1300	10	ACCOUNTING	NEW YORK
JONES	2975	20	RESEARCH	DALLAS
FORD	3000	20	RESEARCH	DALLAS
ADAMS	1100	20	RESEARCH	DALLAS
SMITH	800	20	RESEARCH	DALLAS
SCOTT	3000	20	RESEARCH	DALLAS
WARD	1250	30	SALES	CHICAGO
TURNER	1500	30	SALES	CHICAGO
ALLEN	1600	30	SALES	CHICAGO

ENAME	SAL	DEPTNO	DNAME	LOC
JAMES	950	30	SALES	CHICAGO
BLAKE	2850	30	SALES	CHICAGO
MARTIN	1250	30	SALES	CHICAGO

14 rows selected.

SQL>

WRITING JOIN QUERIES WITH ALIAS NAMES

Alias name is a temporary name for the table and it is valid with in the query.

Ex: Display employee details and dept details ?

```
select e.ename,e.sal,e.job,e.deptno,d.deptno,d.dname,d.loc from emp e,
dept d
where e.deptno=d.deptno;
```

Ex:

Display employee details and dept details for all managers?

```
select e.*,d.* from emp e, dept d
where e.job='MANAGER' and e.DEPTNO = d.DEPTNO;
```

OUTPUT:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	DEPTNO
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10	10
	ACCOUNTING	NEW YORK						
7566	JONES	MANAGER	7839	02-APR-81	2975		20	20
	RESEARCH	DALLAS						
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30	30 SALES
	CHICAGO							

SQL>

Ex:

Display all clerks details and their department names like highest salaried clerk to least salaried clerk?

```
SELECT E.*,D.DEPTNO,D.DNAME FROM EMP E, DEPT D
WHERE E.JOB='CLERK' AND E.DEPTNO=D.DEPTNO
ORDER BY SAL DESC;
```

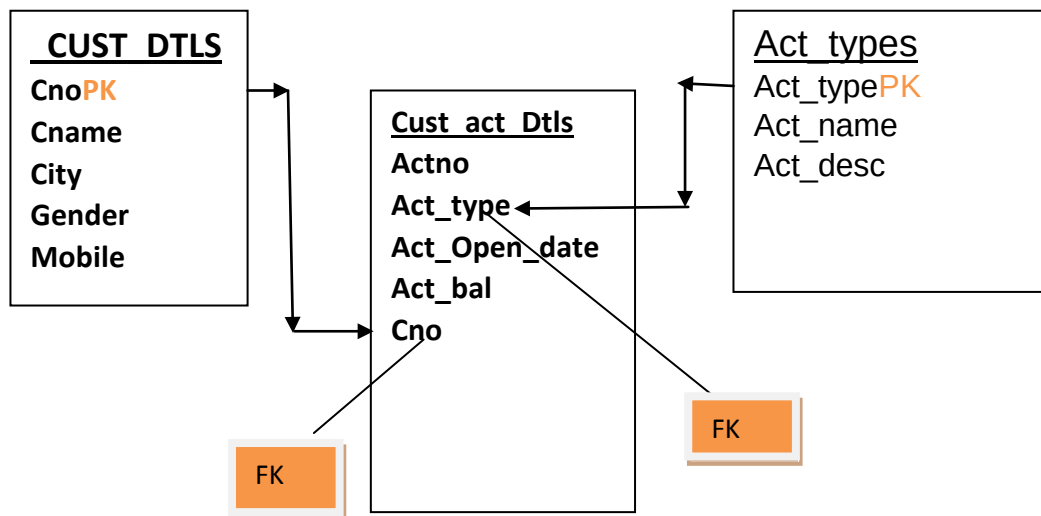
Output:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	DEPTNO
7934	MILLER	CLERK	7782	23-JAN-82	1300		10	10
ACCOUNTING								
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20	20
RESEARCH								
7900	JAMES	CLERK	7698	03-DEC-81	950		30	30 SALES
7369	SMITH	CLERK	7902	17-DEC-80	800		20	20 RESEARCH

SQL>

Examples :

Consider the following tables.



EQUI JOIN EXAMPLES WITH MORETHAN 2 TABLES

Ex:

Get customer details with account balance and account name?

```
Select cd.*,cad.act_bal,at.act_name
From cust_dtls cd, cust_act_dtls cad, act_types at
Where cd.cno=cad.cno
```

And
Cad.act_type=at.act_type;

OUTPUT:

CNO	CNAME	CITY	G	ACT_BAL	ACT_NAME
cust-1	Ajay	Texas	M	32000	Salary A/c.
cust-2	Kiran	Chicago	M	23000	Savings Bank A/c.
cust-3	vinod	Delhi	M	49000	Savings Bank A/c.
cust-3	vinod	Delhi	M	123000	Trading A/c.
cust-4	Madhu	Delhi	M	11000	Salary A/c.
cust-5	Rocky	Texas	M	13000	Savings Bank A/c.
cust-6	Ching Fu	Chicago	F	23000	Salary A/c.

7 rows selected.

SQL>

Ex:

Get product name, cost, warranty, company name and sale date?

```
Select p.pname,p.cost,p.warr,c.cname,s.sale_dt
From prod_dtls p, comp_dtls c, sales s
Where c.cmpid=p.cmpid
And p.pid=s.pid;
```

INNER JOIN

Ex: display customer name and city, customer actno, acttype and bal, act_name for all customers?

```
select
cd.cname,cd.city,cad.actno,cad.act_type,cad.act_bal,at.act_name
from cust_dtls cd INNER JOIN cust_act_dtls cad
ON cd.cno=cad.cno
INNER JOIN act_types at
ON cad.act_type=at.act_type;
```

OUTPUT:

CNAME	CITY	ACTNO	ACT_T	ACT_BAL	ACT_NAME
Ajay	Texas	20035201471	SAL	32000	Salary A/c.
Kiran	Chicago	20035201473	SB	23000	Savings Bank A/c.
vinod	Delhi	20035201470	SB	49000	Savings Bank A/c.
vinod	Delhi	20035201472	DEMAT	123000	Trading A/c.
Madhu	Delhi	20035201474	SAL	11000	Salary A/c.

Rocky	Texas	20035201475	SB	13000	Savings Bank A/c.
Ching Fu	Chicago	20035201476	SAL	23000	Salary A/c.

7 rows selected.

SQL>

SQL>

Ex: Applying equi join on 5 tables?

```
select t1.*,t2.*...
from t1,t2,t3,t4,t5
where
t1.col1=t2.col4
and
t2.col3=t3.col1
and
t3.col2=t4.col31
and
t4.col31=t5.col30;
```

Ex: Applying Inner join on 5 tables?

```
select t1.*,t2.*...
from t1 Inner Join t2
ON
t1.col1=t2.col4      Inner Join t3

ON
t2.col3=t3.col1      Inner Join t4

ON
t3.col2=t4.col31     Inner Join t5

ON
t4.col31=t5.col30;
```

Ex: Display manager details and corresponding department details?

```
select e.*,d.*
from emp e, dept d
where job='MANAGER'
      AND
      e.deptno=d.deptno;

or
```

```
Ex:  display product and corresponding company details?
Ex:  display customer details and corresponding account details?
Ex:  display company name and its products details if the products are from
the company "sony"?
Ex:  display expired product details and their corresponding company
details?
```

Ex: Display customer names, mobile, product names, cost, warrenty and company names?

```
select c.cname, c.mobile, p.pname,p.cost , p.warrenty,
cmp.comp_name
from cust_dtls c, prod_dtls p, comp_dtls cmp, sales_dtls s
where
c.cid=s.cid
and
s.pid=p.pid
and
p.comp_id=cmp.comp_id;
```

Ex: Display product details along with its sales informations from the year 2014?

```
select p.*,s.* from prod_dtls p, sales_dtls s
where s.tid IN('Q1-2014','Q2-2014','Q3-2014','Q4-2014')
and
s.pid=p.pid;
```

Ex: Display customer details and their account details who is from the city 'CHICAGO'?

Ex: Display above information if the customer held DEMAT account and with min balance 100000?

```
select c.*,a.* from cust_dtls c, cust_act_dtls a
where (c.city='CHICAGO'
and
(a.act_type='DEMAT' and a.act_bal>=100000))
and
c.cid=a.cid;
```

Practical Examples:

Ex: Write a query to display customer names, account names and respective Act balances?

By Using Equi Join:

```
select cd.cname,at.act_name,cad.act_bal
from CUST_DTLS cd,ACT_TYPES at,CUST_ACT_DTLS cad
where cd.cno=cad.cust_code AND cad.act_type=at.act_type;
```

By using Inner Join:

```
select cd.cname,at.act_name,cad.act_bal
from CUST_DTLS cd Inner Join CUST_ACT_DTLS cad
```

```
ON cd.cno=cad.cust_code Inner Join act_types at
ON cad.act_type=at.act_type;
```

```
SQL> select cd.cname,cad.actno
       from cust_dtls cd inner join cust_act_dtls cad
       on cd.cno=cad.cust_code;
```

CNAME	ACTNO
Anil	20035201471
Kiran	20035201473
vinod	20035201472
vinod	20035201470
Madhu	20035201474
Rocky	20035201475
Ching Fu	20035201476

7 rows selected.

```
SQL> select cd.cname,cd.city,cad.actno,cad.act_type,cad.act_bal
       2 from cust_dtls cd inner join cust_act_dtls cad
       3 on cd.cno=cad.cust_code;
```

CNAME	CITY	ACTNO	ACT_T	ACT_BAL
Anil	Texas	20035201471	SAL	32000
Kiran	Chicago	20035201473	SB	23000
vinod	Delhi	20035201472	DEMAT	123000
vinod	Delhi	20035201470	SB	49000
Madhu	Delhi	20035201474	SAL	11000
Rocky	Texas	20035201475	SB	13000
Ching Fu	Chicago	20035201476	SAL	23000

7 rows selected.

```
SQL> select cd.cname,cd.city,cad.actno,cad.act_type,cad.act_bal
       2 from cust_dtls cd inner join cust_act_dtls cad
       3 on cd.cno=cad.cust_code
       4 order by cad.act_bal;
```

CNAME	CITY	ACTNO	ACT_T	ACT_BAL
Madhu	Delhi	20035201474	SAL	11000
Rocky	Texas	20035201475	SB	13000
Kiran	Chicago	20035201473	SB	23000
Ching Fu	Chicago	20035201476	SAL	23000
Anil	Texas	20035201471	SAL	32000
vinod	Delhi	20035201470	SB	49000
vinod	Delhi	20035201472	DEMAT	123000

7 rows selected.

```
SQL> select cd.cname,cd.city,cad.actno,cad.act_type,cad.act_bal
  2 from cust_dtls cd inner join cust_act_dtls cad
  3 on cd.cno=cad.cust_code
  4 order by cad.act_bal desc;
```

CNAME	CITY	ACTNO	ACT_T	ACT_BAL
vinod	Delhi	20035201472	DEMAT	123000
vinod	Delhi	20035201470	SB	49000
Anil	Texas	20035201471	SAL	32000
Ching Fu	Chicago	20035201476	SAL	23000
Kiran	Chicago	20035201473	SB	23000
Rocky	Texas	20035201475	SB	13000
Madhu	Delhi	20035201474	SAL	11000

7 rows selected.

```
SQL> select cd.cname,at.act_name,cad.actno,cad.act_bal
  2 from cust_dtls cd,act_types_info at,cust_act_dtls cad
  3 where cd.cno=cad.cust_code
  4 and
  5 at.act_type=cad.act_type;
```

CNAME	ACT_NAME	ACTNO	ACT_BAL
Anil	Salary A/c.	20035201471	32000
Kiran	Savings Bank A/c.	20035201473	23000
vinod	Savings Bank A/c.	20035201470	49000
vinod	Trading A/c.	20035201472	123000
Madhu	Salary A/c.	20035201474	11000
Rocky	Savings Bank A/c.	20035201475	13000
Ching Fu	Salary A/c.	20035201476	23000

7 rows selected.

```
SQL> select cd.cname,at.act_name,cad.actno,cad.act_bal
  2 from cust_dtls cd Inner Join cust_act_dtls cad
  3 on cd.cno=cad.cust_code
  4 Inner join act_types_info at
  5 on cad.act_type=at.act_type;
```

CNAME	ACT_NAME	ACTNO	ACT_BAL
Anil	Salary A/c.	20035201471	32000
Kiran	Savings Bank A/c.	20035201473	23000
vinod	Savings Bank A/c.	20035201470	49000
vinod	Trading A/c.	20035201472	123000
Madhu	Salary A/c.	20035201474	11000
Rocky	Savings Bank A/c.	20035201475	13000
Ching Fu	Salary A/c.	20035201476	23000

7 rows selected.

Assignment:

- 1) Write a query to display employee names, salaries, job titles, hiredate and respective dept names based on salary order?
 - 2) Write a query to display all "salesman and clerk" names, salaries, job titles, hiredate and respective dept names based on salary and job title order?
 - 3) Write a query to display product details and company details of each product with min cost 5000 and max cost 40000 also manufactured in current year with warranty?
 - 4) Write a query to display the customer names, product names, cost, quantity and sales amount generated by the customers from the city "Delhi" and with min sales amount 10000?
 - 5) Write a query to display customer code, customer name, phone_number and actno, act_bal who is maintaining below 5000 act_balance?
 - 6) Write a query to display customer name, Phone number, actno, act_bal and act_name who has opted for loan account?
-

SELF JOIN

A **self join** is a join of a table to itself. This table appears twice in the **FROM** clause and is followed by table aliases that qualify column names in the join condition. To perform a self join, Oracle Database combines and returns rows of the table that satisfy the join condition.

employee	

ename	city

kiran	mumbai
hari	hyd
madhu	hyd
smith	delhi
scott	mumbai
allen	hyd
soumya	chennai
john	delhi

Ex: select * from employee where ename='john'; [NOT CORRECT]

Ex: display employee details who is living in a city where "john" is living?

e1		e2	
---		---	
city	ename	city	ename
-----		-----	

	kiran	mumbai		kiran	mumbai
	hari	hyd		hari	hyd
	madhu	hyd		madhu	
hyd					
	smith	delhi		smith	delhi(matched)
	scott	mumbai		scott	mumbai
	allen	hyd		allen	hyd
	soumya	chennai		soumya	chennai
	john	delhi----- (matched)		john	
delhi(matched)					

→ goto first alias table and check the employee name "john"
→ If it is there then get his "city"
→ then the city from first alias table is compared with all city names in 2nd alias table
→ If the city values are equal then get the records from second alias table.

```
select e2.ename,e2.city
from employee e1, employee e2
where e1.ename='john'
and
e1.city=e2.city;
```

(or)

```
select e2.*
from employee e1, employee e2
where e1.ename='john'
and
e1.city=e2.city;
```

output:

```
smith delhi
john delhi
```

Ex:

```
select e1.*
from employee e1, employee e2
where e1.ename='john'
and
e1.city=e2.city;
```

output: If you display output from First alias table then you will get Duplicate data

```
john delhi
john delhi
```

Ex: display the employee details who is working like 'smith'?

```
select e2.* from emp e1, emp e2
where e1.ename='SMITH' AND e1.job=e2.job;
```

Ex: display customer details who is living in a city where c5 is living?

```
select c2.* from customers c1, customers c2
where c1.cid='c5' and c1.city=c2.city;
```

Ex:--DISPLAY EMPLOYEE DETAILS WHO IS WORKING LIKE "CLARK"

```
SELECT e2.* from emp e1, emp e2
where e1.ename='CLARK'
AND
E1.JOB=E2.JOB;
```

output:

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
20	7566	JONES	MANAGER	7839	02-APR-81	2975	
30	7698	BLAKE	MANAGER	7839	01-MAY-81	2850	
10	7782	CLARK	MANAGER	7839	09-JUN-81	2450	

OUTER JOINS

These are used to display **all data** from one table and **only matched data** from other table.

An **outer join** extends the result of a simple join. An outer join returns all rows that satisfy the join condition and also returns some or all of those rows from one table for which no rows from the other satisfy the join condition.

Types of outer joins: 3

1) Left outer join / left join

Display all the data from left table and only matched data from right table.

2) Right outer join / Right join

Display complete data from right table and only matched data from left table.

3) Full outer join / Full join:

Display
--matched data from both the tables
--unmatched data from left table
--unmatched data from right table

syn-1:

```
select col1, col2, col..... / *  
from table_1 [left join / right join / full join] table_2  
ON table1.pk=table2.fk;
```

Create a new table **EMPCP** from the table **emp** and delete deptno of emps working under deptno 30?

Create table empcp

As

Select * from emp;

Update empcp set deptno=NULL where deptno=30;

Ex:

Display all employee details and if he is working under a dept then display his department details also?

```
select e.*,d.*  
from emp e left outer join dept d on e.deptno=d.deptno;  
output:
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
DEPTNO	DNAME	LOC					
-							
7934	MILLER	CLERK	7782	23-JAN-82	1300	10	10
ACCOUNTING	NEW YORK						
7839	KING	PRESIDENT		17-NOV-81	5000	10	10
ACCOUNTING	NEW YORK						
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10	10
ACCOUNTING	NEW YORK						
7902	FORD	ANALYST	7566	03-DEC-81	3000	20	20
RESEARCH	DALLAS						
7876	ADAMS	CLERK	7788	23-MAY-87	1100	20	20
RESEARCH	DALLAS						
7788	SCOTT	ANALYST	7566	19-APR-87	3000	20	20
RESEARCH	DALLAS						
7566	JONES	MANAGER	7839	02-APR-81	2975	20	20
RESEARCH	DALLAS						
7369	SMITH	CLERK	7902	17-DEC-80	800	20	20 RESEARCH
DALLAS							
7900	JAMES	CLERK	7698	03-DEC-81	950	30	30 SALES
CHICAGO							
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30 30 SALES
CHICAGO							
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30	30 SALES
CHICAGO							
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
DEPTNO	DNAME	LOC					

```

-----
-
  7654 MARTIN    SALESMAN    7698 28-SEP-81    1250    1400    30    30 SALES
CHICAGO
  7521 WARD      SALESMAN    7698 22-FEB-81    1250    500    30    30 SALES
CHICAGO
  7499 ALLEN     SALESMAN    7698 20-FEB-81    1600    300    30    30 SALES
CHICAGO

```

14 rows selected.

SQL>

Ex:

Display all employee details and if he is working under a dept then display his department details also?

```

SQL> select e.*,d.*
2  from empcp e left join dept d
3  on e.deptno=d.deptno;

```

```

      EMPNO ENAME      JOB              MGR HIREDATE          SAL      COMM      DEPTNO
DEPTNO DNAME          LOC
-----
-
  7934 MILLER    CLERK              7782 23-JAN-82      1300              10        10
ACCOUNTING      NEW YORK
  7839 KING      PRESIDENT          17-NOV-81          5000              10        10
ACCOUNTING      NEW YORK
  7782 CLARK      MANAGER            7839 09-JUN-81      2450              10        10
ACCOUNTING      NEW YORK
  7902 FORD      ANALYST            7566 03-DEC-81      3000              20        20
RESEARCH        DALLAS
  7876 ADAMS      CLERK              7788 23-MAY-87      1100              20        20
RESEARCH        DALLAS
  7788 SCOTT      ANALYST            7566 19-APR-87      3000              20        20
RESEARCH        DALLAS
  7566 JONES      MANAGER            7839 02-APR-81      2975              20        20
RESEARCH        DALLAS
  7369 SMITH      CLERK              7902 17-DEC-80       800              20        20 RESEARCH
DALLAS
  7900 JAMES      CLERK              7698 03-DEC-81       950
  7844 TURNER     SALESMAN            7698 08-SEP-81      1500              0
  7698 BLAKE      MANAGER            7839 01-MAY-81      2850

```

```

      EMPNO ENAME      JOB              MGR HIREDATE          SAL      COMM      DEPTNO
DEPTNO DNAME          LOC
-----
-
  7654 MARTIN    SALESMAN    7698 28-SEP-81    1250    1400
  7521 WARD      SALESMAN    7698 22-FEB-81    1250    500
  7499 ALLEN     SALESMAN    7698 20-FEB-81    1600    300

```

14 rows selected.

SQL>

NOTE

Check the above output , we are getting employees even they are not working under any department, it is not possible using EQUI or Inner joins.

Ex:

Get all department details and also if a dept having emps then display its emps details also?

ex:

10 rows selected.

Ex:

```
SQL> select * from empcp e full outer join dept d
2 on e.deptno=d.deptno;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	DEPTNO
DNAME	LOC							

7369	SMITH	CLERK	7902	17-DEC-80	800		20	20 RESEARCH
DALLAS								
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300		
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500		
7566	JONES	MANAGER	7839	02-APR-81	2975		20	20
RESEARCH	DALLAS							
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400		
7698	BLAKE	MANAGER	7839	01-MAY-81	2850			
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10	10
ACCOUNTING	NEW YORK							
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20	20
RESEARCH	DALLAS							
7839	KING	PRESIDENT		17-NOV-81	5000		10	10
ACCOUNTING	NEW YORK							
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0		
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20	20
RESEARCH	DALLAS							
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	
DEPTNO	DNAME	LOC						

7900 JAMES	CLERK	7698 03-DEC-81	950		
7902 FORD	ANALYST	7566 03-DEC-81	3000	20	20
RESEARCH	DALLAS				
7934 MILLER	CLERK	7782 23-JAN-82	1300	10	10
ACCOUNTING	NEW YORK				
				30 SALES	CHICAGO
				40 OPERATIONS	BOSTON

16 rows selected.

SQL>

Ex: Display customer name, actno, act type and act name?

Ex: Display Female customers from CHICAGO, with their accounts information who have DEMAT account?

Ex: Display all customer details and if a customer having SALARY account then display account details also?

Ex: Display all customers personnel details and all accounts list of information?

If a customer have an account then display his details along with his account details?

NON-EQUI JOIN

An **nonequi (or theta) join** is an [inner join](#) statement that uses an unequal operation (i.e: <>, >, <, !=, BETWEEN, etc.) to match [rows](#) from different [tables](#). The converse of an nonequi join is a [equi join](#) operation.

Ex:

```
SELECT e.ename, e.sal, s.grade
2    FROM emp e, salgrade s
3    WHERE e.sal BETWEEN s.losal AND s.hisal;
```

NATURAL JOIN

The join is based on all the columns in the two tables that have the same name and data types. The join creates, by using the NATURAL JOIN keywords. It selects rows from the two tables that have equal values in all matched columns. When specifying columns that are involved in the natural join, do not qualify the column name with a table name or table alias.

SQL: Syntax

```
SELECT table1.column, table2.column
```

```
FROM table1
```

```
NATURAL JOIN table2
```

A NATURAL JOIN is a [JOIN operation](#) that creates an implicit join clause for you based on the common columns in the two tables being joined. Common columns are columns that have the same name in both tables.

A NATURAL JOIN can be an INNER join, a LEFT OUTER join, or a RIGHT OUTER join. The default is INNER join.

Assignments

- a)Get product details and respective sales details?
- b)Get customer details and respective account details?
- c)Get Bookdetails along with author details?
- d)Get account types along with account details?

SUB QUERIES

A query with in other query is known as sub query.

Sub queries are preferable to display output from one table and having an input value from other table.

Syntax:

```
select ..... from table...where [=/IN/exists/not exists]( select ..... from
table.....where ....[=/IN/exits/not exists]( select.....).....);
```

Execution process:

Here the execution process is always from innermost query to the outermost.

Outer query<-----o/p<-----Inner query<-----o/p<-----Inner query

TYPES OF SUBQUERIES: 2

1) Single row subquery:

A sub query which returns single output value.

In this case in between the outer and inner queries we can use = operator.

2) Multi row sub query:

A sub query which returns multiple output values.

In this case in between the outer and inner queries we can use IN operator.

***CORRELATED SUBQUERY:

A sub query which depends on a value generated by outer query. Here the execution process is as follows.

```

outer query-----o/p-----> Inner query----->o/p----->
      ^                               |
      |                               v
      <-----

```

- First Outer query has to be executed and generates some output values

- Second based on these output values, sub query has to be executed
- From subquery we have some output and it is passed to outer query again
- Then outer query has to be executed again.
- Correlated subqueries reduce performance.
- Correlated subqueries are useful in reverse business processes.

EXAMPLES

Ex: display department details of employee smith?

```
select * from dept
where deptno IN(select deptno from emp where ename='SMITH');
```

DEPTNO	DNAME	LOC
20	RESEARCH	DALLAS

By using join Query:

```
select d.* from emp e, dept d
where e.ename='SMITH' and e.deptno=d.deptno;
```

DEPTNO	DNAME	LOC
20	RESEARCH	DALLAS

Ex:

Display employee details who is working under ACCOUNTING?

```
select * from emp
where deptno=(select deptno from dept where
dname='ACCOUNTING');
```

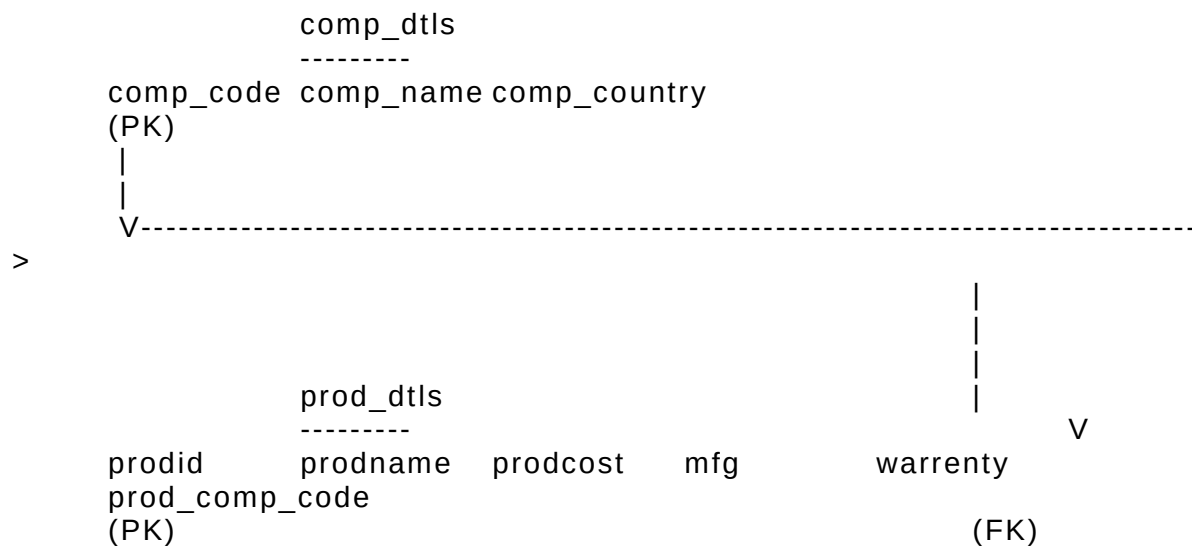
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7839	KING	PRESIDENT		17-NOV-81	5000	10	
7934	MILLER	CLERK	7782	23-JAN-82	1300	10	

Ex: display the department details of all managers?

```
select * from dept
where deptno IN(select deptno from emp where job='MANAGER');
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO

sample tables:



SQL> select * from comp_dtls;

COMP_CODE	COMP_NAME	CITY
I101	IBM	NEW YORK
W202	WIPRO	CHICAGO
D303	DELL	TEXAS
S404	SAMSUNG	KOREA

SQL>

SQL> select * from prod_dtls;

PROD_CODE	PROD_NAME	COST	MFG	WARRENTY	COMP_CODE
ILTPQS	LAPTOP	45000	11-FEB-12	4 YEARS	I101
ILTPXS	LAPTOP	34000	21-MAR-12	4 YEARS	I101
ILTPZS	LAPTOP	40000	12-FEB-13	7 YEARS	I101
WDTPG	DESKTOP	23000	30-OCT-12	1 YEAR	W202
WCFL	LIGHTS	3400	22-JAN-13	2 YEARS	W202
DDTPW	DESKTOP	27000	13-NOV-12	5 YEARS	D303
DLTPG	LAPTOP	46000	08-JAN-13	2 YEARS	D303
DMBLY	MOBILE	32000	10-JAN-13		D303
SLTPC	LAPTOP	34000	04-DEC-12	6 MONTHS	S404
SLTPD	LAPTOP	45000	09-DEC-12	2 YEARS	S404
SMBLGLXY	MOBILE	24000	10-JAN-12	1 YEAR	S404

PROD_CODE	PROD_NAME	COST	MFG	WARRENTY	COMP_CODE
SMBLGLXG	MOBILE	21500	02-FEB-13	1 YEAR	S404

12 rows selected.

Ex: display product details from the company IBM?

```

select * from prod_dtls
where comp_code=
(select comp_code from comp_dtls where comp_name='IBM');
  
```

PROD_CODE	PROD_NAME	COST	MFG	WARRENTY	COMP_CODE
ILTPQS	LAPTOP	45000	11-FEB-12	4 YEARS	I101
ILTPXS	LAPTOP	34000	21-MAR-12	4 YEARS	I101
ILTPZS	LAPTOP	40000	12-FEB-13	7 YEARS	I101

```
SQL> SELECT * FROM PROD_DTLS
2 WHERE COMP_CODE IN
3 (
4 SELECT COMP_CODE FROM COMP_DTLS
5 WHERE COMP_NAME IN('IBM','SAMSUNG')
6 );
```

PROD_CODE	PROD_NAME	COST	MFG	WARRENTY	COMP_CODE
ILTPQS	LAPTOP	45000	11-FEB-12	4 YEARS	I101
ILTPXS	LAPTOP	34000	21-MAR-12	4 YEARS	I101
ILTPZS	LAPTOP	40000	12-FEB-13	7 YEARS	I101
SLTPC	LAPTOP	34000	04-DEC-12	6 MONTHS	S404
SLTPD	LAPTOP	45000	09-DEC-12	2 YEARS	S404
SMBLGLXY	MOBILE	24000	10-JAN-12	1 YEAR	S404
SMBLGLXG	MOBILE	21500	02-FEB-13	1 YEAR	S404

7 rows selected.

Ex: display product details from the companies other than IBM, samsung?

```
SELECT * FROM PROD_DTLS
WHERE COMP_CODE NOT IN
(
SELECT COMP_CODE FROM COMP_DTLS
WHERE COMP_NAME IN('IBM','SAMSUNG')
);
```

PROD_CODE	PROD_NAME	COST	MFG	WARRENTY	COMP_CODE
WCFL	LIGHTS	3400	22-JAN-13	2 YEARS	W202
WDTPG	DESKTOP	23000	30-OCT-12	1 YEAR	W202
DMBLY	MOBILE	32000	10-JAN-13		D303
DLTPG	LAPTOP	46000	08-JAN-13	2 YEARS	D303
DDTPW	DESKTOP	27000	13-NOV-12	5 YEARS	D303

Ex: display company names of the prod ids 'WDTPG','DMBLY'?

```
SQL> select comp_name from comp_dtls
2 where comp_code in
3 (
4 select comp_code from prod_dtls
5 where prod_code in('WDTPG','DMBLY')
6 );
```

COMP_NAME
DELL
WIPRO

CORRELATED SUBQUERIES

These sub queries use 2 operators either EXISTS OR NOT EXISTS

EXISTS

It returns true if a sub query fetches at least one value. If it returns TRUE then outer query will display the result.

NOT EXISTS

It returns true if a sub query fetches no values at all. If it returns TRUE then outer query will display the result.

Ex: display department details which is having at least one employee with in it?

```
select d.* from dept d
where exists( select 10 from empcp e where e.deptno=d.deptno);
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS

Ex: display department details which not having at least one employee with in it?

```
select d.* from dept d
where not exists( select 10 from empcp e where e.deptno=d.deptno);
```

DEPTNO	DNAME	LOC
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

ex: Display compnies information from which we are maintaining at least one product

Ex: Display company details from which we have no products at all?

```
SQL> select c.* from comp_dtls c
2 where not exists
3 (select 1 from prod_dtls p where p.comp_code=c.comp_code);
```

no rows selected

Assignments:

Get customer details have no accounts at all?

Get customer details who has at least one account?

Get account types information having at least one customer for that?

Get account details opted by the customers from the city "HYD"?
Get account details of "vinod"?
Get dept details of empnos 7654, 7902?

ALL

The ALL comparison condition is used to compare a value to a list or subquery. It must be preceded by =, !=, >, <, <=, >= and followed by a list or subquery.

When the ALL condition is followed by a list, the optimizer expands the initial condition to all elements of the list and strings them together with AND operators, as shown below.

```
SELECT empno, sal
FROM emp
WHERE sal > ALL (2000, 3000, 4000);
```

EMPNO	SAL
7839	5000

SQL>

-- Transformed to equivalent statement without ALL.

```
SELECT empno, sal
FROM emp
WHERE sal > 2000 AND sal > 3000 AND sal > 4000;
```

EMPNO	SAL
7839	5000

SQL>

When the ALL condition is followed by a subquery, the optimizer performs a two-step transformation as shown below.

```

SELECT e1.empno, e1.sal
FROM emp e1
WHERE e1.sal > ALL (SELECT e2.sal
                    FROM emp e2
                    WHERE e2.deptno = 20);

```

EMPNO	SAL
7839	5000

SQL>

-- Transformed to equivalent statement using ANY.

```

SELECT e1.empno, e1.sal
FROM emp e1
WHERE NOT (e1.sal <= ANY (SELECT e2.sal
                          FROM emp e2
                          WHERE e2.deptno = 20));

```

EMPNO	SAL
7839	5000

SQL>

-- Transformed to equivalent statement without ANY.

```

SELECT e1.empno, e1.sal
FROM emp e1
WHERE NOT EXISTS (SELECT e2.sal
                  FROM emp e2

```

```
WHERE e2.deptno = 20
AND e1.sal <= e2.sal);
```

EMPNO	SAL
7839	5000

```
SQL>
```

Assuming subqueries don't return zero rows, the following statements can be made for both list and subquery versions:/p>

- "x = ALL (...)": The value must match all the values in the list to evaluate to TRUE.
- "x != ALL (...)": The value must not match any values in the list to evaluate to TRUE.
- "x > ALL (...)": The value must be greater than the biggest value in the list to evaluate to TRUE.
- "x < ALL (...)": The value must be smaller than the smallest value in the list to evaluate to TRUE.
- "x >= ALL (...)": The value must be greater than or equal to the biggest value in the list to evaluate to TRUE.
- "x <= ALL (...)": The value must be smaller than or equal to the smallest value in the list to evaluate to TRUE.

If a subquery returns zero rows, the condition evaluates to TRUE. In the following example, the subquery returns zero rows, which means the whole expression "sal > ALL (zero rows)" evaluates to TRUE, so all rows are displayed.

```
-- The following query returns zero rows.
```

```
SELECT e2.sal FROM emp e2 WHERE e2.deptno = 100;
```

```
no rows selected
```

```
SQL>
```

```
-- Place it in the subquery of an ALL and the
```

```
-- condition "sal > ALL (zero rows)" evaluates to TRUE
```

```
-- so all rows are displayed.
```

```
SELECT e1.empno, e1.sal
```

```
FROM emp e1
```



```
WHERE e1.sal > ALL (SELECT e2.sal FROM emp e2 WHERE e2.deptno = 100);
```

EMPNO	SAL
7369	800
7900	950
7876	1100
7521	1250
7654	1250
7934	1300
7844	1500
7499	1600
7782	2450
7698	2850
7566	2975
7788	3000
7902	3000
7839	5000

```
SQL>
```

ANY

The ANY comparison condition is used to compare a value to a list or subquery. It must be preceded by =, !=, >, <, <=, >= and followed by a list or subquery.

When the ANY condition is followed by a list, the optimizer expands the initial condition to all elements of the list and strings them together with OR operators, as shown below.

```
SELECT empno, sal
FROM emp
WHERE sal > ANY (2000, 3000, 4000);
```

EMPNO	SAL
-------	-----

7566	2975
7698	2850
7782	2450
7788	3000
7839	5000
7902	3000

SQL>

-- Transformed to equivalent statement without ANY.

```
SELECT empno, sal
FROM emp
WHERE sal > 2000 OR sal > 3000 OR sal > 4000;
```

EMPNO	SAL
-------	-----

7566	2975
7698	2850
7782	2450
7788	3000
7839	5000
7902	3000

SQL>

When the ANY condition is followed by a subquery, the optimizer performs a single transformation as shown below.

```
SELECT e1.empno, e1.sal
FROM emp e1
WHERE e1.sal > ANY (SELECT e2.sal
```

```
FROM emp e2
WHERE e2.deptno = 10);
```

EMPNO	SAL
7839	5000
7902	3000
7788	3000
7566	2975
7698	2850
7782	2450
7499	1600
7844	1500

SQL>

-- Transformed to equivalent statement without ANY.

```
SELECT e1.empno, e1.sal
FROM emp e1
WHERE EXISTS (SELECT e2.sal
              FROM emp e2
              WHERE e2.deptno = 10
              AND e1.sal > e2.sal);
```

EMPNO	SAL
7839	5000
7902	3000
7788	3000
7566	2975
7698	2850

7782	2450
7499	1600
7844	1500

SQL>

Assuming subqueries don't return zero rows, the following statements can be made for both list and subquery versions:

- "x = ANY (...)": The value must match one or more values in the list to evaluate to TRUE.
- "x != ANY (...)": The value must not match one or more values in the list to evaluate to TRUE.
- "x > ANY (...)": The value must be greater than the smallest value in the list to evaluate to TRUE.
- "x < ANY (...)": The value must be smaller than the biggest value in the list to evaluate to TRUE.
- "x >= ANY (...)": The value must be greater than or equal to the smallest value in the list to evaluate to TRUE.
- "x <= ANY (...)": The value must be smaller than or equal to the biggest value in the list to evaluate to TRUE.

If a subquery returns zero rows, the condition evaluates to FALSE. In the following example, the subquery returns zero rows, which means the whole expression "sal > ANY (zero rows)" evaluates to FALSE, so no rows are displayed.

-- The following query returns zero rows.

```
SELECT e2.sal FROM emp e2 WHERE e2.deptno = 100;
```

no rows selected

SQL>

-- Place it in the subquery of an ANY and the

-- condition "sal > ANY (zero rows)" evaluates to FALSE

-- so no rows are displayed.

```
SELECT e1.empno, e1.sal
```

```
FROM emp e1
```

```
WHERE e1.sal > ANY (SELECT e2.sal FROM emp e2 WHERE e2.deptno = 100);
```

no rows selected

SQL>

SOME

The SOME and ANY comparison conditions do exactly the same thing and are completely interchangeable.

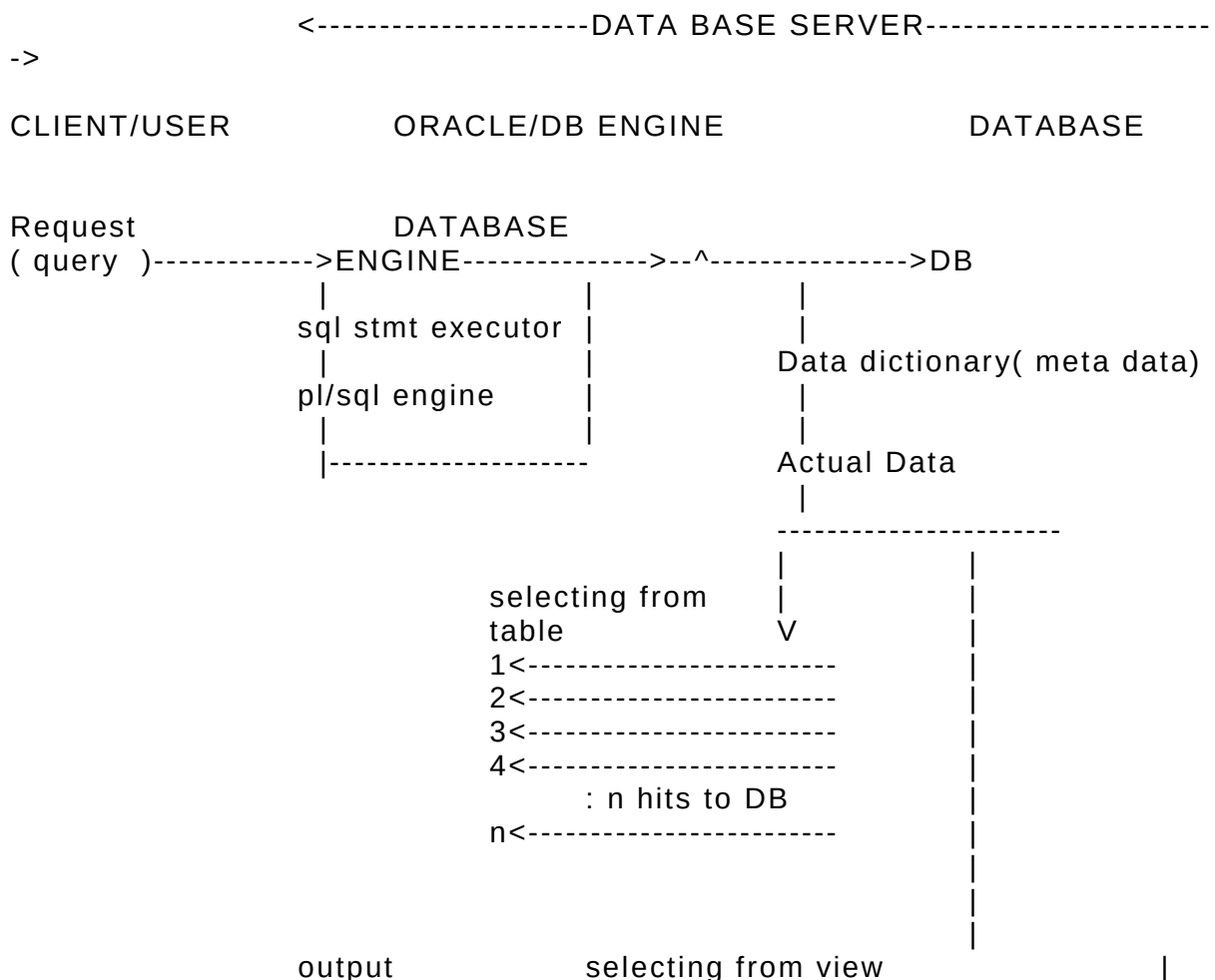
VIEWS

Views are known as logical tables. They represent the data of one or more tables. A view derives its data from the tables on which it is based. These tables are called base tables. Views can be based on actual tables or another view also.

Whatever DML operations you performed on a view they actually affect the base table of the view. You can treat views same as any other table. You can Query, Insert, Update and delete from views, just as any other table.

Views are very powerful and handy since they can be treated just like any other table but do not occupy the space of a table.

QUERY EXECUTION PROCESS



```

<-----CACHE <-----
<-----MEMORY Only one HIT to the DB
<-----
<-----
:
<-----

```

View:

It is a database object contains logical copy of selected table data.

It can be created based on frequently using data.

It reduces number of hits to the database.

It improves the performance of queries and database.

Types of Views: 5

1) Simple view / updateable view:

It is created based on single table data. It allows dml operations.

syn: create view <view_name>
AS select from table_name where <cond>;

2) Composite view or read only view:

It is created based on morethan one table data. It doesnt allow dml operations.

Advantage: It will reduce writing number of join queries again and again.

Syntax: create view <vw_name>
AS select from table1, table2,..... where <join-cond>;

NOTE:

By default the **Client user** not having permission to create views. That privillage is assigned by DBA.

Ex: write a query to create a view which contains managers information from emp table?

```

create view vw_mgr_info
as select * from emp where job='MANAGER';

```

Error: Insufficient privillages

ex: connect to dba

```

system
manager

```

orcl

Ex: grant create view to manju;

grant succeeded

Ex: conn manju /welcome@orcl
connected.

Ex: create view vw_mgr_info
as select * from emp where job='MANAGER';

view created.

HOW TO SELECT THE DATA FROM VIEW?

SYN: SELECT * from <vw_name>;

Ex: select * from vw_mgr_info;

sample output:

EMPNO DEPTNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10

Ex: display manager names and salaries?

select ename,sal from vw_mgr_info;

ENAME	SAL
JONES	2975
BLAKE	2850
CLARK	2450

NOTE

→SUB TABLES MAINTAINS DATA STATICALLY
→VIEWS MAINTAINS DATA DYNAMICALLY.

sample hands-on:

sub table:

create table emp_mgr as select * from emp where job='MANAGER';

Ex:

select * from emp_mgr;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
1122	dinesh	MANAGER		12-MAY-13	4500	700 40

Ex: creating a view containing managers info?

create view vw_mgr_info as select * from emp where job='MANAGER';

Ex:

select * from vw_mgr_info;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
1122	dinesh	MANAGER		12-MAY-13	4500	700 40

Ex:

insert into emp values(2233,'naresh','MANAGER',NULL,'21-MAY-14',3400,200,10);

1 rows inserted.

Ex:

In the below sub table data we cannot see any new manager information.

select * from emp_mgr;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
1122	dinesh	MANAGER		12-MAY-13	4500	700 40

In the below data we are seeing manager information since we are taking data from the view.

select * from vw_mgr_info;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
-------	-------	-----	-----	----------	-----	------


```

-----
7566 JONES      MANAGER      7839 02-APR-81      2975              20
7698 BLAKE      MANAGER      7839 01-MAY-81      2850              30
7782 CLARK      MANAGER      7839 09-JUN-81      2450              10
1122 dinesh     MANAGER      12-MAY-13          4500            700            40
2233 naresh     MANAGER      21-MAY-14          3400            200            10

```

Ex: delete from vw_mgr_info where empno in(1122,2233);

```

select * from emp_mgr;
select * from vw_mgr_info;
select * from emp where job='MANAGER';

```

```

EMPNO ENAME      JOB      MGR HIREDATE      SAL      COMM
DEPTNO
-----
7566 JONES      MANAGER      7839 02-APR-81      2975              20
7698 BLAKE      MANAGER      7839 01-MAY-81      2850              30
7782 CLARK      MANAGER      7839 09-JUN-81      2450              10
1122 dinesh     MANAGER      12-MAY-13          4500            700            40

```

```

EMPNO ENAME      JOB      MGR HIREDATE      SAL      COMM
DEPTNO
-----
7566 JONES      MANAGER      7839 02-APR-81      2975              20
7698 BLAKE      MANAGER      7839 01-MAY-81      2850              30
7782 CLARK      MANAGER      7839 09-JUN-81      2450              10

```

```

EMPNO ENAME      JOB      MGR HIREDATE      SAL      COMM
DEPTNO
-----
7566 JONES      MANAGER      7839 02-APR-81      2975              20
7698 BLAKE      MANAGER      7839 01-MAY-81      2850              30
7782 CLARK      MANAGER      7839 09-JUN-81      2450              10

```

2) READ ONLY VIEWS / COMPOSITE VIEWS

EXAMPLES

Ex:

write a query to maintain a view containing managers ename, sal, job, dept name and dept loc?

```

create view vw_emp_dept_info
as select ename,sal,job,dname,loc from emp,dept
      where job='MANAGER' and
emp.deptno=dept.deptno;

```

Ex: Display manager details and their dept details?

```
select * from VW_EMP_DEPT_INFo;
```

ENAME	SAL	JOB	DNAME	LOC
JONES	2975	MANAGER	RESEARCH	DALLAS
BLAKE	2850	MANAGER	SALES	CHICAGO
CLARK	2450	MANAGER	ACCOUNTING	NEW YORK

Ex: display manager names and their department names?

```
select ename,dname from vw_emp_dept_info;
```

ENAME	DNAME
JONES	RESEARCH
BLAKE	SALES
CLARK	ACCOUNTING

3) Inline view

An inline view is a SELECT statement in the FROM-clause of another SELECT statement. In-line views are commonly used to simplify complex queries by removing join operations and condensing several separate queries into a single query.

Ex: sub query

Generally sub query is to be written inside the WHERE clause.

Display employee details working under accounting department?

```
select * from emp where deptno=( select deptno from dept where  
dname='ACCOUNTING');
```

Ex: Inline view

```
SELECT *  
FROM ( SELECT deptno, count(*) emp_count  
      FROM emp  
      GROUP BY deptno ) emp,  
dept  
WHERE dept.deptno = emp.deptno;
```

Explanation:

deptno	emp_count
10	4

20	6
30	7

Ex:

deptno	dname	loc	emp_count
10	Accounting	chicago	4
20	REsearch	Texas	6
30	Sales	Dallas	7

Ex: display the employees who earn the highest salary in each department?

```
SELECT *
FROM
  ( SELECT deptno, max(sal) maxsal
    FROM emp
    GROUP BY deptno ) b, emp a
WHERE
a.sal= b.maxsal
and
a.deptno = b.deptno;
```

output:

DEPTNO	MAXSAL	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM
30	2850	7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
20	3000	7788	SCOTT	ANALYST	7566	19-APR-87	3000	20
10	5000	7839	KING	PRESIDENT		17-NOV-81	5000	10
20	3000	7902	FORD	ANALYST	7566	03-DEC-81	3000	20

Ex:

```
SELECT d.dept_id, d.name, emp_cnt.tot
FROM department d INNER JOIN
  (SELECT dept_id, COUNT(*) tot
   FROM employee
   GROUP BY dept_id) emp_cnt
ON d.dept_id = emp_cnt.dept_id;
```

DEPT_ID	NAME	TOT
-----	-----	-----

10 ACCOUNTING	3
20 RESEARCH	5
30 SALES	6

Ex:

Display comp_codes, number of products from each company code and also display company details?

```
select * from (select comp_code,count(*) from prod_dtls group by
comp_code) p,comp_dtls C
where p.comp_code=c.comp_code;
```

ex:

```
select d.deptno,d.dname,d.loc,e.deptno,count(e.empno) from emp e ,dept d
where e.deptno=d.deptno
group by d.deptno,d.dname,d.loc,e.deptno;
```

iv) Creating FORCE VIEWS:

A view can be created even if the defining query of the view cannot be executed, as long as the CREATE VIEW command has no syntax errors. We call such a view a view with errors.

For example,

if a view refers

- to a non-existent table or
- an invalid column of an existing table, or
- if the owner of the view does not have the required privileges,

then the view can still be created and entered into the data dictionary.

You can only create a view with errors by using the FORCE option of the CREATE VIEW command:

```
CREATE FORCE VIEW AS ...;
```

Ex:

```
create FORCE view vw_act_dtls_Sb
as
select * from cust_act_dtls where act_type='SB' order by act_bal;
```

Materialized Views in Oracle

A materialized view in Oracle is a database object that contains the results of a query. They are local copies of data located remotely, or are used to create summary tables based on aggregations of a table's data. Materialized views, which store data based on remote tables are also, known as snapshots.

A materialized view can query tables, views, and other materialized views. Collectively these are called master tables (a replication term) or detail tables (a data warehouse term).

For replication purposes, materialized views allow you to maintain copies of remote data on your local node. These copies are read-only. If you want to update the local copies, you have to use the Advanced Replication feature. You can select data from a materialized view as you would from a table or view.

For data warehousing purposes, the materialized views commonly created are aggregate views, single-table aggregate views, and join views.

In this article, we shall see how to create a Materialized View in Oracle and discuss Refresh Option of the view.

In replication environments, the materialized views commonly created are primary key, rowid, and subquery materialized views.

Primary Key Materialized Views

The following statement creates the primary-key materialized view on the table emp located on a remote database.

```
SQL> CREATE MATERIALIZED VIEW mv_emp_pk  
      REFRESH FAST START WITH SYSDATE  
      NEXT SYSDATE + 1/48  
      WITH PRIMARY KEY  
      AS SELECT * FROM emp@remote_db;
```

Materialized view created.

Note: When you create a materialized view using the FAST option you will need to create a view log on the master table(s) as shown below:

```
SQL> CREATE MATERIALIZED VIEW LOG ON emp;  
Materialized view log created.
```

Rowid Materialized Views

The following statement creates the rowid materialized view on table emp located on a remote database:

```
SQL> CREATE MATERIALIZED VIEW mv_emp_rowid  
      REFRESH WITH ROWID
```

```
AS SELECT * FROM emp@remote_db;
```

Materialized view log created.

Subquery Materialized Views

The following statement creates a subquery materialized view based on the emp and dept tables located on the remote database:

```
SQL> CREATE MATERIALIZED VIEW mv_empdept
AS SELECT * FROM emp@remote_db e
WHERE EXISTS
  (SELECT * FROM dept@remote_db d
   WHERE e.dept_no = d.dept_no)
```

REFRESH CLAUSE

```
[refresh [fast|complete|force]
        [on demand | commit]
        [start with date] [next date]
        [with {primary key|rowid}]]
```

The refresh option specifies:

- a. The refresh method used by Oracle to refresh data in materialized view
- b. Whether the view is primary key based or row-id based
- c. The time and interval at which the view is to be refreshed

Refresh Method - FAST Clause

The FAST refreshes use the materialized view logs (as seen above) to send the rows that have changed from master tables to the materialized view.

You should create a materialized view log for the master tables if you specify the REFRESH FAST clause.

```
SQL> CREATE MATERIALIZED VIEW LOG ON emp;
```

Materialized view log created.

Materialized views are not eligible for fast refresh if the defined subquery contains an analytic function.

Refresh Method - COMPLETE Clause

The complete refresh re-creates the entire materialized view. If you request a complete refresh, Oracle performs a complete refresh even if a fast refresh is possible.

Refresh Method - FORCE Clause

When you specify a FORCE clause, Oracle will perform a fast refresh if one is possible or a complete refresh otherwise. If you do not specify a refresh method (FAST, COMPLETE, or FORCE), FORCE is the default.

PRIMARY KEY and ROWID Clause

WITH PRIMARY KEY is used to create a primary key materialized view i.e. the materialized view is based on the primary key of the master table instead of ROWID (for ROWID clause). PRIMARY KEY is the default option. To use the PRIMARY KEY clause you should have defined PRIMARY KEY on the master table or else you should use ROWID based materialized views.

Primary key materialized views allow materialized view master tables to be reorganized without affecting the eligibility of the materialized view for fast refresh.

Rowid materialized views should have a single master table and cannot contain any of the following:

- Distinct or aggregate functions
- GROUP BY Subqueries , Joins & Set operations

Timing the refresh

The START WITH clause tells the database when to perform the first replication from the master table to the local base table. It should evaluate to a future point in time. The NEXT clause specifies the interval between refreshes

```
SQL> CREATE MATERIALIZED VIEW mv_emp_pk
      REFRESH FAST
      START WITH SYSDATE
      NEXT SYSDATE + 2
      WITH PRIMARY KEY
      AS SELECT * FROM emp@remote_db;
```

Materialized view created.

In the above example, the first copy of the materialized view is made at SYSDATE and the interval at which the refresh has to be performed is every two days.

Summary

Materialized Views thus offer us flexibility of basing a view on Primary key or ROWID, specifying refresh methods and specifying time of automatic refreshes.

Replacing/Altering Views

To alter the definition of a view, you must replace the view using one of the following methods:

- A view can be dropped and then re-created. When a view is dropped, all grants of corresponding view privileges are revoked from roles and users. After the view is re-created, necessary privileges must be regranted.
- A view can be replaced by redefining it with a CREATE VIEW statement that contains the OR REPLACE option. This option replaces the current definition of a view, but preserves the present security authorizations.

For example, assume that you create the ACCOUNTS_STAFF view, as given in a previous example. You also grant several object privileges to roles and other users. However, now you realize that you must redefine the ACCOUNTS_STAFF view to correct the department number specified in the WHERE clause of the defining query, because it should have been 30. To preserve the grants of object privileges that you have made, you can replace the current version of the ACCOUNTS_STAFF view with the following statement:

```
CREATE OR REPLACE VIEW Accounts_staff AS
  SELECT Empno, Ename, Deptno
  FROM Emp
  WHERE Deptno = 30
  WITH CHECK OPTION CONSTRAINT ica_Accounts_cnst;
```

Replacing a view has the following effects:

- Replacing a view replaces the view's definition in the data dictionary. All underlying objects referenced by the view are not affected.
- If previously defined but not included in the new view definition, then the constraint associated with the WITH CHECK OPTION for a view's definition is dropped.
- All views and PL/SQL program units dependent on a replaced view become invalid.

Referencing Invalid Views

When a user attempts to reference an invalid view, Oracle returns an error message to the user:

```
ORA-04063: view 'view_name' has errors
```

This error message is returned when a view exists but is unusable due to errors in its query (whether it had errors when originally created or it was

created successfully but became unusable later because underlying objects were altered or dropped).

Dropping Views

Use the SQL command DROP VIEW to drop a view. For example:

```
DROP VIEW Accounts_staff;
```

Listing Information about VIEWS.

To see how many views are there in your schema. Give the following query.

```
select * from user_views;
```

To see which columns are updatable in join views.

Data Dictionaries which shows which columns are updatable.

View Name	Description
USER_UPDATABLE_COLUMNS	Shows all columns in all tables and views in the user's schema that are modifiable
DBA_UPDATABLE_COLUMNS	Shows all columns in all tables and views in the DBA schema that are modifiable
ALL_UPDATABLE_VIEWS	Shows all columns in all tables and views that are modifiable

INDEXES

Indexes are optional structures associated with tables and clusters that allow SQL queries to execute more quickly against a table.

Just as the index in this manual helps you locate information faster than if there were no index, an Oracle Database index provides a faster access path to table data. You can use indexes without rewriting any queries. Your results are the same, but you see them more quickly.

Oracle Database provides several indexing schemes that provide complementary performance functionality. These are:

- B-tree indexes: the default and the most common
- B-tree cluster indexes: defined specifically for cluster
- Hash cluster indexes: defined specifically for a hash cluster
- Global and local indexes: relate to partitioned tables and indexes
- Reverse key indexes: most useful for Oracle Real Application Clusters applications
- Bitmap indexes: compact; work best for columns with a small set of values
- Function-based indexes: contain the precomputed value of a function/expression
- Domain indexes: specific to an application or cartridge.

Indexes are logically and physically independent of the data in the associated table. Being independent structures, they require storage space. You can create or drop an index without affecting the base tables, database applications, or other indexes. The database automatically maintains indexes when you insert, update, and delete rows of the associated table. If you drop an index, all applications continue to work. However, access to previously indexed data might be slower.

Index is a table like object which maintains ordered data of the column physically.

It reduces number of comparisons to make, to fetch the required data. It fastens the search as much as fast. It occupies physical disk space.

Index contains 2 parts

- i) Data part ii) Address part (**ROWID**)

ROWID is a pseudo column / virtual column. It contains Physical address of each record.

We can access the values of rowid , but we cannot modify them.

This (**ROWID**) address is a combination of the following.

{fileno datablock_no record_no}.

This first data block of a table is indicated with 0.

And the first record in each data block is indicated with 0.

Ex: let us consider the file [table] number is 555.

select * from prod_dtls;

555.000.000

555.000.001

555.000.002

:

555.000.099-----> this is the last record in first data block of table no 555. (100 records)

555.001.000

555.001.001

555.001.002

:

555.001.999-----> this is the last record in second data block of table no 555 (1000 records)

555.002.000

555.002.001

:

555.002.009-----> this is the last record in the third data block of table no 555

ex: salaries(table)

Idx_sal(indexed table)

1--9000

2--1000

3--2000

4--9000

5--1000

6--2500

7--2000

8--1000

9--2000

10--9000

2--1000

5--1000

8--1000

3--2000

7--2000

9--2000

6--2500

1--9000

4--9000

10-9000

Before INDEX creation:

select * from emp
where sal <=2000;

10 comparisions

6 salaries

After INDEX creation:

select * from emp where sal <=2000;

7 comparisions

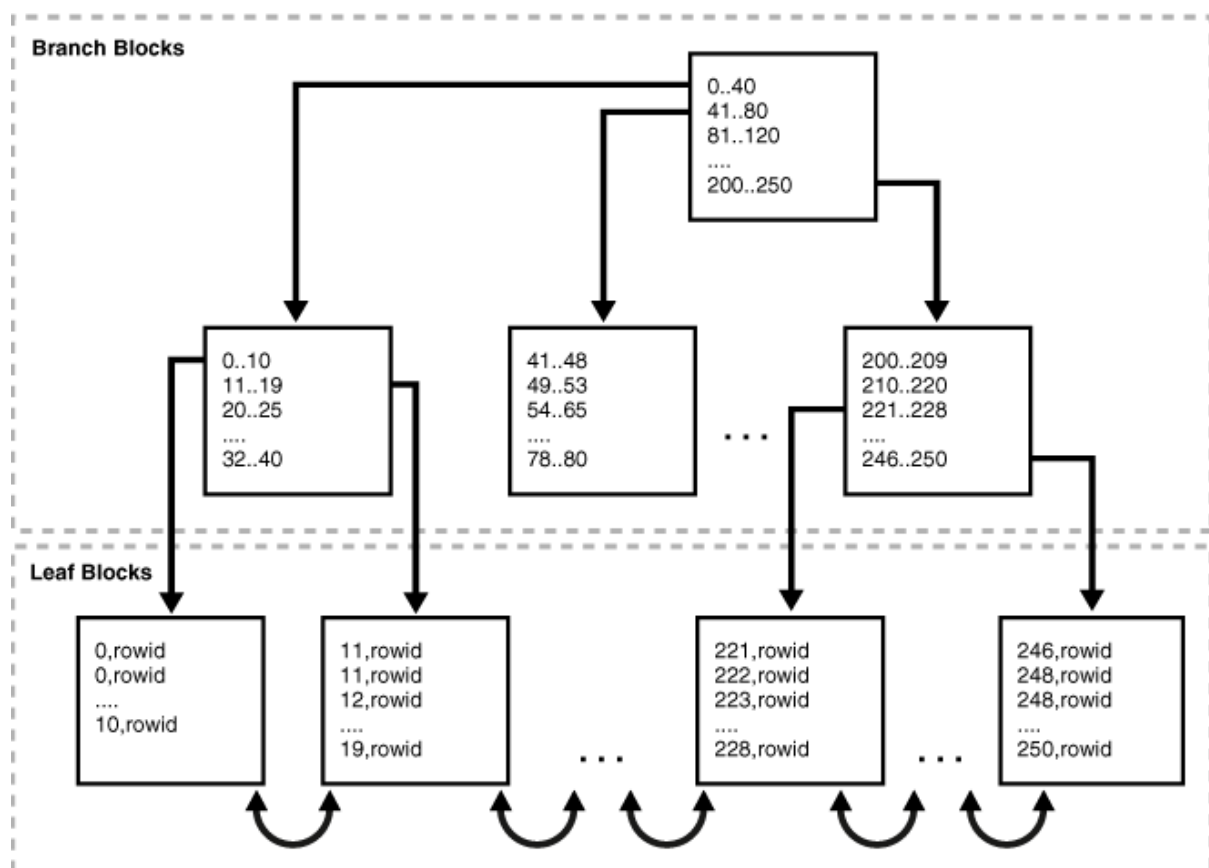
6 records

B-Tree Indexes

B-trees, short for **balanced trees**, are the most common type of database index. A B-tree index is an ordered list of values divided into ranges. By associating a key with a row or range of rows, B-trees provide excellent retrieval performance for a wide range of queries, including exact match and range searches.

[Figure 3-1](#) illustrates the structure of a B-tree index. The example shows an index on the department_id column, which is a foreign key column in the employees table.

Figure 3-1 Internal Structure of a B-tree Index



TYPES OF INDEXES: 2

1) Simple index:

It is created on a table on single column.

Syntax: create index <idx_name>
 on table_name(colname);

2) Composite index

It is created on multiple columns of a table.

Syntax: create index <idx_name>
 on table_name(col1 ,col2,...);

Ex: create an index on table emp on the column sal?

 create index idx_sal on emp(sal);

Ex: create an index on prod_dtls on the columns cost,comp_code?

 create index idx_prod_cost_comp_code on prod_dtls(cost,comp_code);

3) Function Based Index

In the index definition, If we specify any **calculations or functions on table columns** then it is known as Function Based Index.

syn: create index <idx_name>
 on table_name(col+100,col2*0.10,Func(col3));

Ex: create index idx_sal_ename
 on emp((0.20*sal),Initcap(ename));

Ex: select sal,(0.20*sal),initcap(ename) from emp;

4) REVERSE KEY INDEX

In this index , the search criteria is from right most leaf to left.
This index is preferable to search for highest values frequently.

Syntax: create index <idx_name> on table_name(col) **REVERSE**;

Ex: create a reverse index on sales transaction table on sales amount column?

 Create index idx_higher_sales on sales(sales_amt) REVERSE;

5) UNIQUE INDEX

If an index is creating on Unique column then it is known as unique index.

Ex: create unique index idx_comm
 on emp(comm);

6) Bitmap Index

Use this index if you have very less number of different values in a column like gender, emp_job_status

Ex: create bitmap index ix_gender
 on cust_dtls(gender);

Note

By default the oracle engine create and maintains a default index on each primary key column of a table.

Note

In Oracle indexes information is maintained under a system defined table called **USER_INDEXES**

HOW TO DELETE AN INDEX?

drop index idx_name;

Multiple Indexes on same set of columns:

You can create multiple indexes on the same set of columns when the indexes are different in some way. For example, you can create a B-tree index and a bitmap index on the same set of columns.

When you have multiple indexes on the same set of columns, only one of these indexes can be visible at a time, and any other indexes must be invisible.

You might create different indexes on the same set of columns because they provide the flexibility to meet your requirements. You can also create multiple indexes on the same set of columns to perform application migrations without dropping an existing index and recreating it with different attributes.

Creating an Invisible Index

An invisible index is an index that is ignored by the optimizer unless you explicitly set the **OPTIMIZER_USE_INVISIBLE_INDEXES** initialization parameter to **TRUE** at the session or system level.

To create an invisible index:

- Use the **CREATE INDEX** statement with the **INVISIBLE** keyword.

The following statement creates an invisible index named `emp_ename` for the `ename` column of the `emp` table:

```
CREATE INDEX emp_ename ON emp(ename)
    TABLESPACE users
    STORAGE (INITIAL 20K
    NEXT 20k)
INVISIBLE;
```

USER_INDEXES

`USER_INDEXES` describes the indexes owned by the current user. This view does not display the `OWNER` column.

CLUSTERS

Creating Clusters

To create a cluster in your schema, you must have the **CREATE CLUSTER** system privilege and a quota for the tablespace intended to contain the cluster or the **UNLIMITED TABLESPACE** system privilege.

To create a cluster in another user's schema you must have the **CREATE ANY CLUSTER** system privilege, and the owner must have a quota for the tablespace intended to contain the cluster or the **UNLIMITED TABLESPACE** system privilege.

You create a cluster using the **CREATE CLUSTER** statement. The following statement creates a cluster named **emp_dept**, which stores the **emp** and **dept** tables, clustered by the **deptno** column:

```
CREATE CLUSTER emp_dept (deptno NUMBER(3))
```

If no **INDEX** keyword is specified, as is true in this example, an index cluster is created by default. You can also create a **HASH** cluster, when hash parameters (**HASHKEYS**, **HASH IS**, or **SINGLE TABLE HASHKEYS**) are specified

Creating Clustered Tables

To create a table in a cluster, you must have either the **CREATE TABLE** or **CREATE ANY TABLE** system privilege. You do not need a tablespace quota or the **UNLIMITED TABLESPACE** system privilege to create a table in a cluster.

You create a table in a cluster using the **CREATE TABLE** statement with the **CLUSTER** clause. The **emp** and **dept** tables can be created in the **emp_dept** cluster using the following statements:

```
CREATE TABLE emp (  
    empno NUMBER(5) PRIMARY KEY,
```

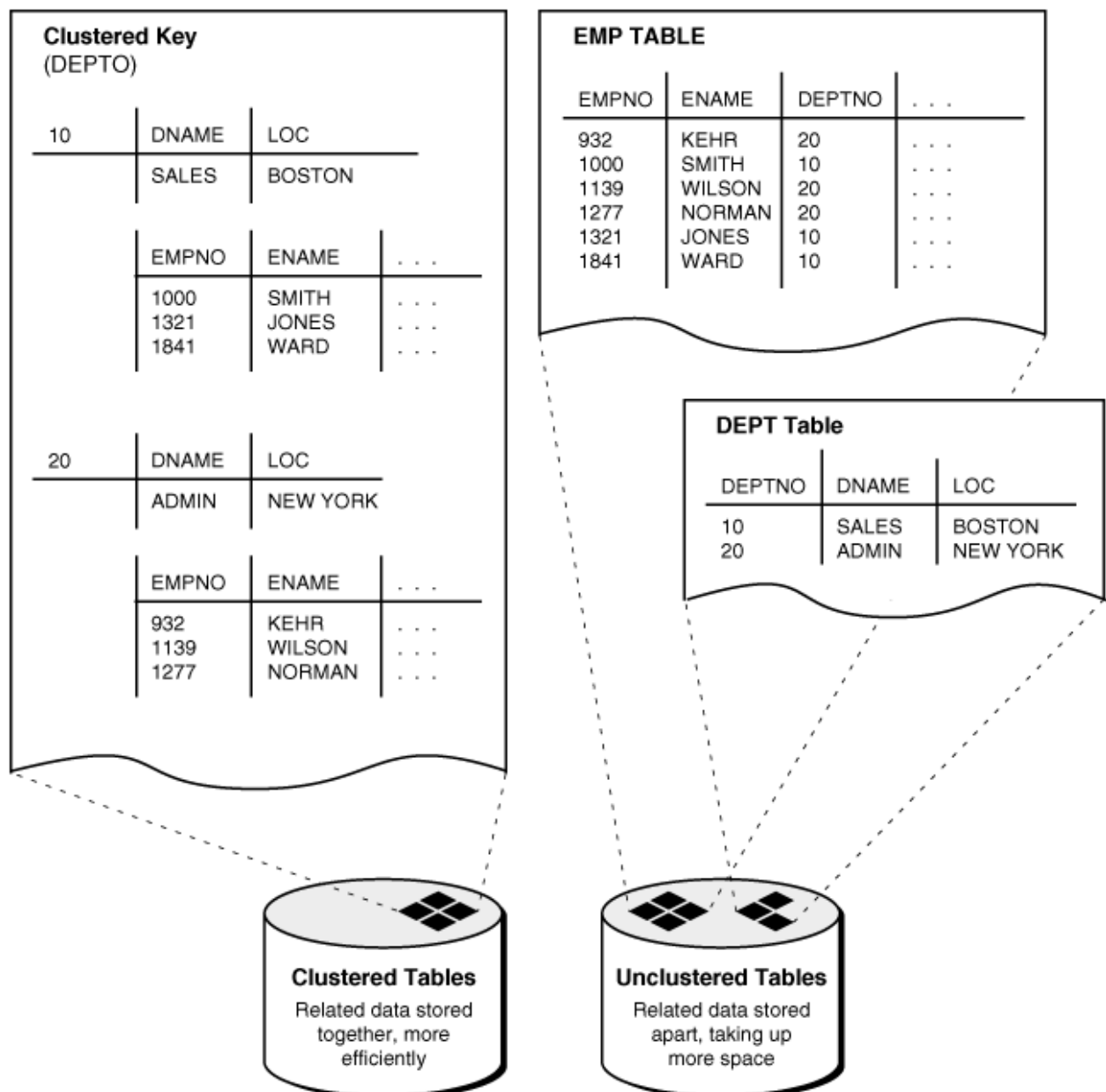


```

ename VARCHAR2(15) NOT NULL,
...
deptno NUMBER(3) REFERENCES dept)
CLUSTER emp_dept (deptno);

CREATE TABLE dept (
  deptno NUMBER(3) PRIMARY KEY, ... )
CLUSTER emp_dept (deptno);

```



Cust_dtls (custid,cname,city)

Act_types (act_type,act_name,desc)

Cust_act_dtls(actno,act_type,act_open_dt,act_bal,custid)

Example:

```
create cluster emp_dept (deptno number(2));
```

```
create table deptcp
```

```
(
```

```
deptno number(2) primary key,
```

```
dname varchar2(20),
```

```
loc varchar2(20)
```

```
)
```

```
cluster emp_dept(deptno);
```

```
create table empcp
```

```
(
```

```
eid number(4) primary key,
```

```
ename varchar2(20),
```

```
sal number(5),
```

```
deptno number(2) references deptcp(deptno)
```

```
)
```

```
cluster emp_dept(deptno);
```

ALL_CLUSTERS

`ALL_CLUSTERS` describes all clusters accessible to the current user.

Related Views

- `DBA_CLUSTERS` describes all clusters in the database.
- `USER_CLUSTERS` describes all clusters owned by the current user. This view does not display the `OWNER` column.

Column	Datatype	NULL
<code>OWNER</code>	<code>VARCHAR2(30)</code>	<code>NOT NULL</code>
<code>CLUSTER_NAME</code>	<code>VARCHAR2(30)</code>	<code>NOT NULL</code>
<code>TABLESPACE_NAME</code>	<code>VARCHAR2(30)</code>	<code>NOT NULL</code>

SEQUENCES

Use the `CREATE SEQUENCE` statement to create a **sequence**, which is a database object from which multiple users may generate unique integers. You can use sequences to automatically generate primary key values.

When a sequence number is generated, the sequence is incremented, independent of the transaction committing or rolling back. If two users concurrently increment the same sequence, then the sequence numbers each user acquires may have gaps, because sequence numbers are being generated by the other user. One user can never acquire the sequence number generated by another user. After a sequence value is generated by one user, that user can continue to access that value regardless of whether the sequence is incremented by another user.

Sequence numbers are generated independently of tables, so the same sequence can be used for one or for multiple tables. It is possible that individual sequence numbers will appear to be skipped, because they were generated and used in a transaction that ultimately rolled back. Additionally, a single user may not realize that other users are drawing from the same sequence.

After a sequence is created, you can access its values in SQL statements with the `CURRVAL` pseudocolumn, which returns the current value of the sequence, or the `NEXTVAL` pseudocolumn, which increments the sequence and returns the new value.

It is a data base object which is used to generate sequential integers with the specified interval value.
Generally the sequences are used to generate primary key values.

The syntax to create a sequence in Oracle is:

```
CREATE SEQUENCE sequence_name
  MINVALUE value
  MAXVALUE value
  START WITH value
```

```
INCREMENT BY value
CACHE value;
```

Note: by default the sequence starts with 1 and increment value is also 1.

**** Sequence is an independent object.(It is not depending on any table)**

Ex: create sequence srno1;

Pseudo columns:

<seq_name>.CURRVAL Display current value of the sequence

<seq_name>.NEXTVAL Display nextvalue of sequence

Ex: select srno1.currval from dual;

```
CURRVAL
-----
      1
```

Ex: select srno1.nextval from dual;

```
NEXTVAL
-----
```

2

Ex: create a sequence starts with 1001?

```
create sequence custno start with 1001;
```

Ex: update the cust_dtls table under that update the cust_srno column with sequence values?

```
update cust_dtls set cust_srno=srno1.nextval;
```

Ex: adding a primary key constraint on the column cust_srno?

```
alter table cust_dtls add constraint pk_custsrno primary
key(cust_srno);
```

Ex: Insert new record in the table cust_dtls along with sequence values?

```
insert into cust_dtls
values(4,'abc','hyd',6767676765,'F',null,srno1.nextval);
*****
```

Hands-on:

```
CREATE TABLE CUSTOMERS
(
ID INT,
NAME VARCHAR2(20),
SRNO NUMBER)
```

```
CREATE SEQUENCE ID START WITH 6600 INCREMENT BY 5
CREATE SEQUENCE SRNO
```

```
INSERT INTO CUSTOMERS VALUES(ID.NEXTVAL,'AJAY',NULL)
INSERT INTO CUSTOMERS VALUES(ID.NEXTVAL,'KIRAN',NULL)
INSERT INTO CUSTOMERS VALUES(ID.NEXTVAL,'MADHU',NULL)
```

```
SELECT * FROM CUSTOMERS
```

```
UPDATE CUSTOMERS SET SRNO=SRNO.NEXTVAL
```

results:

```
CREATE TABLE succeeded.
CREATE SEQUENCE succeeded.
CREATE SEQUENCE succeeded.
1 rows inserted
1 rows inserted
1 rows inserted
```

ID	NAME	SRNO
6605	AJAY	
6610	KIRAN	
6615	MADHU	

3 rows selected

ID	NAME	SRNO
6605	AJAY	1
6610	KIRAN	2
6615	MADHU	3

3 rows selected

Ex:

```
CREATE SEQUENCE t1_seq;
```

```

CREATE TABLE t1 (
  id          NUMBER DEFAULT t1_seq.NEXTVAL,
  description VARCHAR2(30)
);

INSERT INTO t1 (description) VALUES ('DESCRIPTION only');
INSERT INTO t1 (id, description) VALUES (999, 'ID=999 and
DESCRIPTION');
INSERT INTO t1 (id, description) VALUES (NULL, 'ID=NULL and
DESCRIPTION');

SELECT * FROM t1;

```

```

      ID DESCRIPTION
-----
      1 DESCRIPTION only
      999 ID=999 and DESCRIPTION
           ID=NULL and DESCRIPTION

```

3 rows selected.

DEFAULT Values Using Sequences

In Oracle 12c, it is now possible to specify the CURRVAL and NEXTVAL sequence pseudocolumns as the default values for a column. You should also consider using [Identity columns](#) for this purpose.

In the following example you can see the effect of specifying a sequence as the default value for a column. The default value is only used when the column is not referenced by the insert. This behaviour can be modified using the ON NULL clause described in the next section.

```

CREATE SEQUENCE t1_seq;

CREATE TABLE t1 (
  id          NUMBER DEFAULT t1_seq.NEXTVAL,
  description VARCHAR2(30)
);

INSERT INTO t1 (description) VALUES ('DESCRIPTION only');
INSERT INTO t1 (id, description) VALUES (999, 'ID=999 and
DESCRIPTION');
INSERT INTO t1 (id, description) VALUES (NULL, 'ID=NULL and
DESCRIPTION');

SELECT * FROM t1;

```

```

      ID DESCRIPTION
-----
      1 DESCRIPTION only
    999 ID=999 and DESCRIPTION
      ID=NULL and DESCRIPTION

3 rows selected.

SQL>

```

IDENTITY COLUMNS IN ORACLE 12c

```

CREATE TABLE identity_test_tab (
  id          NUMBER GENERATED ALWAYS AS IDENTITY,
  description VARCHAR2(30)
);

SQL> INSERT INTO identity_test_tab (description) VALUES
('Just DESCRIPTION');

1 row created.

SQL> INSERT INTO identity_test_tab (id, description) VALUES
(NULL, 'ID=NULL and DESCRIPTION');
INSERT INTO identity_test_tab (id, description) VALUES (NULL,
'ID=NULL and DESCRIPTION')

*

ERROR at line 1:
ORA-32795: cannot insert into a generated always identity
column

SQL> INSERT INTO identity_test_tab (id, description) VALUES
(999, 'ID=999 and DESCRIPTION');
INSERT INTO identity_test_tab (id, description) VALUES (999,
'ID=999 and DESCRIPTION')

*

```


ERROR at line 1:

ORA-32795: cannot insert into a generated always identity column

SQL>

Using BY DEFAULT allows you to use the identity if the column isn't referenced in the insert statement, but if the column is referenced, the specified value will be used in place of the identity. Attempting to specify the value NULL in this case results in an error, since identity columns are always NOT NULL.

```
DROP TABLE identity_test_tab PURGE;
```

```
CREATE TABLE identity_test_tab (  
    id          NUMBER GENERATED BY DEFAULT AS IDENTITY,  
    description VARCHAR2(30)  
);
```

```
SQL> INSERT INTO identity_test_tab (description) VALUES  
( 'Just DESCRIPTION');
```

1 row created.

```
SQL> INSERT INTO identity_test_tab (id, description) VALUES  
(999, 'ID=999 and DESCRIPTION');
```

1 row created.

```
SQL> INSERT INTO identity_test_tab (id, description) VALUES  
(NULL, 'ID=NULL and DESCRIPTION');  
  
INSERT INTO identity_test_tab (id, description) VALUES (NULL,  
'ID=NULL and DESCRIPTION')
```

*

ERROR at line 1:

```
ORA-01400: cannot insert NULL into
("TEST"."IDENTITY_TEST_TAB"."ID")
```

```
SQL>
```

Using BY DEFAULT ON NULL allows the identity to be used if the identity column is referenced, but a value of NULL is specified.

```
DROP TABLE identity_test_tab PURGE;
```

```
CREATE TABLE identity_test_tab (
  id          NUMBER GENERATED BY DEFAULT ON NULL AS
IDENTITY,
  description VARCHAR2(30)
);
```

```
SQL> INSERT INTO identity_test_tab (description) VALUES
('Just DESCRIPTION');
```

```
1 row created.
```

```
SQL> INSERT INTO identity_test_tab (id, description) VALUES
(999, 'ID=999 and DESCRIPTION');
```

```
1 row created.
```

```
SQL> INSERT INTO identity_test_tab (id, description) VALUES
(NULL, 'ID=NULL and DESCRIPTION');
```

```
1 row created.
```

```
SQL> SELECT * FROM identity_test_tab;
```

```
  ID DESCRIPTION
```

1 Just DESCRIPTION

999 ID=999 and DESCRIPTION

2 ID=NULL and DESCRIPTION

SQL>

HOW TO DELETE THE SEQUENCE?

Ex: drop sequence srno;

Note:

System defined view to get sequences information is "user_sequences".

SYNONYMS

Use the `CREATE SYNONYM` statement to create a **synonym**, which is an alternative name for a table, view, sequence, operator, procedure, stored function, package, materialized view, Java class schema object, user-defined object type, or another synonym. A synonym places a dependency on its target object and becomes invalid if the target object is changed or dropped.

Synonyms provide both data independence and location transparency. Synonyms permit applications to function without modification regardless of which user owns the table or view and regardless of which database holds the table or view. However, synonyms are not a substitute for privileges on database objects. Appropriate privileges must be granted to a user before the user can use the synonym.

You can refer to synonyms in the following DML statements:

`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `FLASHBACK TABLE`, `EXPLAIN PLAN`, and `LOCK TABLE`.

You can refer to synonyms in the following DDL statements:

`AUDIT`, `NOAUDIT`, `GRANT`, `REVOKE`, and `COMMENT`.

Synonyms are used to create permanent alias names for the tables.

Types of Synonyms: 2

i) Private synonym

It is a default synonym. and it is used by the owner only.

ii) Public synonym

It is created by DBA. And it can be accessed by permitted authenticated user.

syn: `create [ public ] synonym <syn_name> FOR <table_name>;`

Ex: `create synonym cd for cust_dtls;`

Note:

Once a synonym is created then we can use synonym name instead of table name for any operations on the table.

Ex: display cust details data?

```
select * from cust_dtls;
```

or

```
select * from cd;
```

Ex: insert into cd values((10,'c','mumbai',null,'M',null,custno.nextval);

Ex: deleting the data from table using alias name?

```
delete from cd where mobile is null;
```

Ex: how to delete a synonym?

```
drop synonym cd;
```

USER_SYNONYMS

It is a system defined view, from where we can retrieve the synonyms information

PARTITIONS

Partitioning enables you to decompose very large tables and indexes into smaller and more manageable pieces called **partitions**. Each partition is an independent object with its own name and optionally its own storage characteristics.

For an analogy that illustrates partitioning, suppose an HR manager has one big box that contains employee folders. Each folder lists the employee hire date. Queries are often made for employees hired in a particular month. One approach to satisfying such requests is to create an index on employee hire date that specifies the locations of the folders scattered throughout the box. In contrast, a partitioning strategy uses many smaller boxes, with each box containing folders for employees hired in a given month.

Using smaller boxes has several advantages. When asked to retrieve the folders for employees hired in June, the HR manager can retrieve the June box. Furthermore, if any small box is temporarily damaged, the other small boxes remain available. Moving offices also becomes easier because instead of moving a single heavy box, the manager can move several small boxes.

From the perspective of an application, only one schema object exists. **DML** statements require no modification to access partitioned tables. Partitioning is useful for many different types of database applications, particularly those that manage large volumes of data. Benefits include:

- Increased availability

The unavailability of a partition does not entail the unavailability of the object. The query **optimizer** automatically removes unreferenced partitions from the **query plan** so queries are not affected when the partitions are unavailable.

- Easier administration of schema objects

A partitioned object has pieces that can be managed either collectively or individually. **DDL** statements can manipulate partitions rather than entire tables or indexes. Thus, you can break up resource-intensive

tasks such as rebuilding an index or table. For example, you can move one table partition at a time. If a problem occurs, then only the partition move must be redone, not the table move. Also, dropping a partition avoids executing numerous DELETE statements.

- Reduced contention for shared resources in **OLTP** systems

In some OLTP systems, partitions can decrease contention for a shared resource. For example, DML is distributed over many segments rather than one segment.

- Enhanced query performance in data warehouses

In a **data warehouse**, partitioning can speed processing of ad hoc queries. For example, a sales table containing a million rows can be partitioned by quarter.

Partition Characteristics

Each partition of a table or index must have the same logical attributes, such as **column** names, **data types**, and constraints. For example, all partitions in a table share the same column and constraint definitions, and all partitions in an index share the same indexed columns. However, each partition can have separate physical attributes, such as the tablespace to which it belongs.

Partition Key

The **partition key** is a set of one or more columns that determines the partition in which each **row** in a partitioned table should go. Each row is unambiguously assigned to a single partition.

In the sales table, you could specify the time_id column as the key of a range partition. The database assigns rows to partitions based on whether the date in this column falls in a specified range. Oracle Database automatically directs insert, update, and delete operations to the appropriate partition by using the partition key.

Partitioning Strategies

Oracle Partitioning offers several partitioning strategies that control how the database places data into partitions. The basic strategies are range, list, and hash partitioning.

A **single-level** partitioning strategy uses only one method of data distribution, for example, only list partitioning or only range partitioning. In **composite partitioning**, a table is partitioned by one data distribution method and then each partition is further divided into

subpartitions using a second data distribution method. For example, you could use a list partition for `channel_id` and a range subpartition for `time_id`.

Range Partitioning

In **range partitioning**, the database maps rows to partitions based on ranges of values of the partitioning key. Range partitioning is the most common type of partitioning and is often used with dates.

Suppose that you want to populate a partitioned table with the sales rows shown in [Example 4-1](#).

Example 4-1 Sample Row Set for Partitioned Table

PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
116	11393	05-JUN-99	2	999	1	12.18
40	100530	30-NOV-98	9	33	1	44.99
118	133	06-JUN-01	2	999	1	17.12
133	9450	01-DEC-00	2	999	1	31.28
36	4523	27-JAN-99	3	999	1	53.89
125	9417	04-FEB-98	3	999	1	16.86
30	170	23-FEB-01	2	999	1	8.8
24	11899	26-JUN-99	4	999	1	43.04
35	2606	17-FEB-00	3	999	1	54.94
45	9491	28-AUG-98	4	350	1	47.45

You create `time_range_sales` as a partitioned table using the statement in [Example 4-2](#). The `time_id` column is the partition key.

Example 4-2 Range-Partitioned Table

```
CREATE TABLE time_range_sales
( prod_id      NUMBER(6)
, cust_id      NUMBER
, time_id      DATE
, channel_id   CHAR(1)
, promo_id     NUMBER(6)
, quantity_sold NUMBER(3)
, amount_sold  NUMBER(10,2)
)
```



```
PARTITION BY RANGE (time_id)
(PARTITION SALES_1998 VALUES LESS THAN (TO_DATE('01-JAN-
1999','DD-MON-YYYY')),
PARTITION SALES_1999 VALUES LESS THAN (TO_DATE('01-JAN-
2000','DD-MON-YYYY')),
PARTITION SALES_2000 VALUES LESS THAN (TO_DATE('01-JAN-
2001','DD-MON-YYYY')),
PARTITION SALES_2001 VALUES LESS THAN (MAXVALUE)
);
```

Afterward, you load time_range_sales with the rows from [Example 4-1](#). [Figure 4-1](#) shows the row distributions in the four partitions. The database chooses the partition for each row based on the time_id value according to the rules specified in the PARTITION BY RANGE clause.

Figure 4-1 Range Partitions

Table Partition SALES_1998

PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
40	100530	30-NOV-98	9	33	1	44.99
125	9417	04-FEB-98	3	999	1	16.86
45	9491	28-AUG-98	4	350	1	47.45

Table Partition SALES_1999

PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
116	11393	05-JUN-99	2	999	1	12.18
36	4523	27-JAN-99	3	999	1	53.89
24	11899	26-JUN-99	4	999	1	43.04

Table Partition SALES_2000

PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
133	9450	01-DEC-00	2	999	1	31.28
35	2606	17-FEB-00	3	999	1	54.94

Table Partition SALES_2001

PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
118	133	06-JUN-01	2	999	1	17.12
30	170	23-FEB-01	2	999	1	8.8

Description of "Figure 4-1 Range Partitions"

The range partition key value determines the high value of the range partitions, which is called the **transition point**. In [Figure 4-1](#), the SALES_1998 partition contains rows with partitioning key time_id values less than the transition point 01-JAN-1999.

The database creates **interval partitions** for data beyond that transition point. Interval partitions extend range partitioning by instructing the database to create partitions of the specified range or interval automatically when data inserted into the table exceeds all of the range partitions.

In [Figure 4-1](#), the SALES_2001 partition contains rows with partitioning key time_id values greater than or equal to 01-JAN-2001.

List Partitioning

In **list partitioning**, the database uses a list of discrete values as the partition key for each partition. You can use list partitioning to control how individual rows map to specific partitions. By using lists, you can group and organize related sets of data when the key used to identify them is not conveniently ordered.

Assume that you create list_sales as a list-partitioned table using the statement in [Example 4-3](#). The channel_id column is the partition key.

Example 4-3 List-Partitioned Table

```
CREATE TABLE list_sales
( prod_id      NUMBER(6)
, cust_id      NUMBER
, time_id      DATE
, channel_id    CHAR(1)
, promo_id     NUMBER(6)
, quantity_sold NUMBER(3)
, amount_sold  NUMBER(10,2)
)
PARTITION BY LIST (channel_id)
(PARTITION even_channels VALUES (2,4),
 PARTITION odd_channels VALUES (3,9)
);
```

Afterward, you load the table with the rows from [Example 4-1](#). [Figure 4-2](#) shows the row distribution in the two partitions. The database chooses the partition for each row based on the channel_id value according to the rules specified in the PARTITION BY LIST clause. Rows with a channel_id value of 2 or 4 are stored in the EVEN_CHANNELS partitions, while rows with a channel_id value of 3 or 9 are stored in the ODD_CHANNELS partition.

Figure 4-2 List Partitions

Table Partition EVEN_CHANNELS						
PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
116	11393	05-JUN-99	2	999	1	12.18
118	133	06-JUN-01	2	999	1	17.12
133	9450	01-DEC-00	2	999	1	31.28
30	170	23-FEB-01	2	999	1	8.8
24	11899	26-JUN-99	4	999	1	43.04
45	9491	28-AUG-98	4	350	1	47.45

Table Partition ODD_CHANNELS						
PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
40	100530	30-NOV-98	9	33	1	44.99
36	4523	27-JAN-99	3	999	1	53.89
125	9417	04-FEB-98	3	999	1	16.86
35	2606	17-FEB-00	3	999	1	54.94

Description of "Figure 4-2 List Partitions"

Hash Partitioning

In **hash partitioning**, the database maps rows to partitions based on a **hashing** algorithm that the database applies to the user-specified partitioning key. The destination of a row is determined by the internal **hash function** applied to the row by the database. The hashing algorithm is designed to evenly distributes rows across devices so that each partition contains about the same number of rows.

Hash partitioning is useful for dividing large tables to increase manageability. Instead of one large table to manage, you have several smaller pieces. The loss of a single hash partition does not affect the remaining partitions and can be recovered independently. Hash partitioning is also useful in **OLTP** systems with high update contention. For example, a segment is divided into several pieces, each of which is updated, instead of a single segment that experiences contention.

Assume that you create the partitioned hash_sales table using the statement in [Example 4-4](#). The prod_id column is the partition key.

Example 4-4 Hash-Partitioned Table

```
CREATE TABLE hash_sales
( prod_id      NUMBER(6)
, cust_id      NUMBER
, time_id      DATE
, channel_id   CHAR(1)
```

```

, promo_id    NUMBER(6)
, quantity_sold NUMBER(3)
, amount_sold  NUMBER(10,2)
)
PARTITION BY HASH (prod_id)
PARTITIONS 2;

```

Afterward, you load the table with the rows from [Example 4-1](#). [Figure 4-3](#) shows a possible row distribution in the two partitions. Note that the names of these partitions are system-generated.

As you insert rows, the database attempts to randomly and evenly distribute them across partitions. You cannot specify the partition into which a row is placed. The database applies the hash function, whose outcome determines which partition contains the row. If you change the number of partitions, then the database redistributes the data over all of the partitions.

Figure 4-3 Hash Partitions

Table Partition SYS_P33						
PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
40	100530	30-NOV-98	9	33	1	44.99
118	133	06-JUN-01	2	999	1	17.12
36	4523	27-JAN-99	3	999	1	53.89
30	170	23-FEB-01	2	999	1	8.8
35	2606	17-FEB-00	3	999	1	54.94

Table Partition SYS_P34						
PROD_ID	CUST_ID	TIME_ID	CHANNEL_ID	PROMO_ID	QUANTITY_SOLD	AMOUNT_SOLD
116	11393	05-JUN-99	2	999	1	12.18
133	9450	01-DEC-00	2	999	1	31.28
125	9417	04-FEB-98	3	999	1	16.86
24	11899	26-JUN-99	4	999	1	43.04
45	9491	28-AUG-98	4	350	1	47.45

SQL FUNCTIONS

Oracle provides set of ***built-in*** functions to perform user required operations. These functions are divided into 2 categories.

CATEGORIES: 2

1) GROUP OR AGGREGATE FUNCTIONS

These functions can acts on group of values display single output value. These functions act on column level / field level.

Ex: sum(), avg()...

2) SCALAR /SINGLE ROW FUNCTIONS

These functions acts on group of values and display a set of output values. These functions act on Record level / row level.

Ex: lower(), length(), trim()

The above 2 categories of functions are further divided into following 4 types.

TYPES OF FUNCTIONS: 4

i) Numeric functions

Acts on numeric data

ii) String functions

Acts on char data

iii) Date functions

Acts on date data

iv) Conversion functions

It acts on one data type and display result in other data type.

NUMERIC FUNCTIONS [GROUP FUNCTIONS]

These functions act on column data only.

i) SUM(colname)

It will display addition of values from the column

Ex: display addition of all salaries?
select sum(sal) from emp;

output:
SUM(SAL)

29025

Ex: Find the sum of salaries for managers?

Ex: Find the sum of salaries of emps working under RESEARCH department?

Ex: Find the total investment amount for the products from sony?

Ex: Find the total balance from the account types SB and SAL?

Output:
SQL> select sum(act_bal)
2 from cust_act_dtls
3 where act_type in('SB','SAL');

SUM(ACT_BAL)

151000

ii) AVG(colname)

It will display average value from the column

Ex: display average product cost?

select avg(cost) AvgCostOfProduct
from prod_dtls;

output:
SQL> select avg(cost) AvgCostOfProduct
2 from prod_dtls;

AVGCOSTOFPRODUCT

31241.6667

Ex: Find the average sales amount in the last year?

```
select avg(sales_amount) from sales
where tid like'%14';
```

iii) MAX(colname)

Display higher value from the column.

Ex: Display highest salary among all salesman?

```
SQL> select max(sal ) from emp where job='SALESMAN';
```

output:

```
MAX(SAL)
-----
1600
```

Ex: display employee details who is getting higher salary?

```
select * from emp where sal=max(sal);-----WRONG
```

Or

```
select * from emp where sal IN(select max(sal) from emp);
```

iv) MIN(colname)

Display lower value from the column

Ex: display least cost product details among all mobiles?

```
SQL> select * from prod_dtls where cost IN(select min(cost) from prod_dtls
where prod_name='MOBILE');
```

PROD_CODE	PROD_NAME	COST	MFG	WARRENTY
COMP_CODE				
SMBLGLXG	MOBILE	21500	02-FEB-13	1 YEAR S404

v) COUNT(colname)

Display number of values from the column except null values

Ex: find how many emps getting commission?

```
select count(empno) from emp where comm is not null;
```

OR

```
select count(comm) from emp;
```

```
COUNT(COMM)
-----
4
```

vi) COUNT(*)

Display number of records from a table.

Ex: display number of transactions on current day?


```
select count(*) from trans_dtls where trans_date=sysdate;
```

Note:

" Aggregate functions executes on field /column level data "

Ex: Display number of emps under deptno 10?

```
select count(*) from emp where deptno=10;
```

```
10    5
```

Ex: Display number of emps under deptno 20?

```
select count(*) from emp where deptno=20;
```

```
20    7
```

Expected output:

```
10    5
20    7
30    5
40    2
50    1
```

GROUP BY Clause:

It is used to group related data by considering distinct values from the column. On each group the oracle engine executes the aggregate function and display result individually.

Syntax: select colname, colname,..., aggfunction1, aggregate2,.....
 from table
 where <cond>
 GROUP BY <col1>,<col2>,...
 HAVING aggfunc1,...
 ORDER BY cl1,col2,.....;

Ex: find out number of emps working under each dept ?

```
select deptno, count(*) " No. of emps"
from emp
GROUP BY deptno;
```

output:

deptno	No. of emps
-----	-----
30	6
20	7

10 5

Ex: find out number of emps working under each dept on order of deptno?

```
select deptno, count(*) " No. of emps"
from emp
GROUP BY deptno
order by deptno;
```

output: 10 5
 20 7
 30 6

Ex:
Find out number of customers for each account type?

```
SQL> select act_type, count(cno) " No of customers"
2 from cust_act_dtls
3 group by act_type;
```

ACT_T No of customers

```
-----
SB                      3
SAL                     3
DEMAT                   1
```

Ex:
Find number of emps under each job category for each dept number?

```
SQL> select deptno, job, count(empno) "No of emps"
2 from emp
3 group by deptno, job;
```

```
DEPTNO JOB        No of emps
-----
20 CLERK           2
30 SALESMAN        4
20 MANAGER        1
30 CLERK           1
10 PRESIDENT       1
30 MANAGER        1
10 CLERK           1
10 MANAGER        1
20 ANALYST         2
```

9 rows selected.

SQL>

Ex: findout max sal under each deptno?

Ex: findout min sal for each job category based on the order of job?

Ex: findout number of emps under each dept getting morethan 5000 salary based on the order of deptno?

```
select deptno, count(*)
from emp
where sal> 5000
group by deptno
order by deptno;
```

Ex: findout number of customers from each city based on order of city?

Ex: findout number of customers for each account type?

Ex findout number of products from each product category?

HAVING clause:

It is used to specify conditions on group by output.

Ex: find out number of emps working under each dept on order of deptno if a dept contains at least 10 emps?

```
select deptno, count(*) " No. of emps"
from emp
GROUP BY deptno
HAVING count(*)>=10
order by deptno;
```

output: 20 70
 30 60

Examples:

```
select deptno,count(empno) EmpCount from emp group by deptno;
```

```
select deptno,count(empno) EmpCount
from emp
group by deptno
Having count(empno)>=5 ;
```

```
select job,count(empno) EmpCount
from emp
group by job;
```

--Having count(emEx: Display number of emps under deptno 10?

```
select count(*) from emp where deptno=10;pno)>=5 ;
```

```
select job,count(empno) EmpCount
from emp
```

group by job
Having count(empno)<=3 ;

select d.dname,avg(e.sal) "Average Salary" from dept d, emp e
where d.deptno=e.deptno
group by d.dname;

OUTPUT:

DEPTNO	EMPCOUNT
30	6
20	5
10	2

DEPTNO	EMPCOUNT
30	6
20	5
10	2

DEPTNO	EMPCOUNT
30	6
20	5

JOB	EMPCOUNT
CLERK	4
SALESMAN	4
PRESIDENT	1
MANAGER	2
ANALYST	2

JOB	EMPCOUNT
PRESIDENT	1
MANAGER	2
ANALYST	2

DNAME	Average Salary
ACCOUNTING	3150
RESEARCH	2175
SALES	1925

NUMERIC FUNCTIONS (SCALAR FUNCTIONS)

These functions are acting on record level.

1) **ABS(n) [ABSOLUTE]**

Display absolute value of n.

Ex: select abs(-9) from dual;
 9

Ex: select abs(13.23) from dual;
 13.23

Ex: select abs(round((months_between(hiredate,sysdate)/12))) from emp;

2) **mod(m,n)**

Display remainder value after m divides n.

Ex: select mod(25,5) from dual;
 0

Ex: select mod(17,3) from dual;
 2

3) **power(m,n)**

Display m power nth value

Ex: select power(5,3) from dual;
 125

4) **SQRT(n) [square root]**

Display square root value of n

Ex: select sqrt(64) from dual;
 8

5) **ROUND(m,n)**

Display value "m" which is rounded to the "n" number of decimal places.

Before displaying "nth" Decimal digit it will check " n+1 th" decimal digit, if it is > or = 5 then "nth" digit incremented by 1.

Ex: select round(63.354,2) from dual;
 63.35

Ex: select round(63.354,1) from dual;
 63.4

Ex: select round(63.354) from dual;
 63

Ex: select round(69.554) from dual;
 70

6) TRUNC(m,n)

Display value m which is truncated to the n number of decimal places.

Ex: select trunc(63.354,1) from dual;
63.3

Ex: select trunc(69.554) from dual;
69

7) FLOOR(n)

Display highest integer value which is less than or equal to given value.

Ex: select floor(64.2) from dual;
{0,1,2,.....,61,62,63,64}= 64

8) CEIL(n)

Display lowest integer value which is greater than or equal to given value.

Ex: select ceil(64.2) from dual;
{65,66,67,.....}=65

9) LEAST(val/expr,val/expr,....)

Display minimum value from the given values or expression results.

Ex: select least(32,(6*5), (20-10), (36/2)) from dual;
10

10) GREATEST (val/expr, val/expr,.....)

Display maximum value from the given values or expressions

Ex: select greatest (32,(6*5), (20-10), (36/2)) from dual;
32

STRING FUNCTIONS (scalar functions)

1) ASCII('ch')

display ascii value of the character

Ex: select ascii('a') from dual;
 97

Ex: select ascii('A') from dual;
 65

Ex: select ascii('@') from dual;
 64

2) LENGTH('str'/col)

Display number of chars from the given string or column values

Ex:

Get all employee names and also each name length?

select ename, length(ename) " length of name" from emp;

ENAME	length of name
-------	----------------

SMITH	5
ALLEN	5
WARD	4
JONES	5
MARTIN	6
BLAKE	5
CLARK	5
SCOTT	5
KING	4
TURNER	6
ADAMS	5

ENAME	length of name
-------	----------------

JAMES	5
FORD	4
MILLER	6

Ex: select length('oracle') from dual;
 6

3) LOWER('str'/col)

Display the given string chars or column values in lower case.

Ex:

Get all employee names in lower case?

select lower(ename) from emp;

ENAME	LOWER(ENAM
SMITH	smith
ALLEN	allen
WARD	ward
JONES	jones
MARTIN	martin
BLAKE	blake
CLARK	clark
SCOTT	scott
KING	king
TURNER	turner
ADAMS	adams

ENAME	LOWER(ENAM
JAMES	james
FORD	ford
MILLER	miller

14 rows selected.

SQL>

Ex: select lower('HAI') from dual;

hai

4) UPPER('str'/col)

Display given string chars or column values in upper case

Ex: select upper (pname) from products;

5) INITCAP('str'/col) [initial capital]

Display the given string or column values with begining char as capital.

Ex: select initcap('welcome to oracle') from dual;

Welcome To Oracle

Ex:

Get employee names with beginning character as capital?

SQL> select initcap(ename) from emp;

INITCAP(EN
Smith
Allen
Ward
Jones
Martin
Blake

Clark
Scott
King
Turner
Adams

INITCAP(EN

James
Ford
Miller

14 rows selected.

6) SUBSTR('str'/col,m,n) (substring)

Display a substring from the given string. Here the substring started with "m" th char and through "n" number of chars.

Ex: select substr('secure',3,4) from dual;
cure

Ex: select substr('welcome to oracle functions',12,6) from dual;
oracle

Ex: select substr('welcome to oracle functions',12) from dual;
oracle functions

7) INSTR('str'/col,'ch',m,n) [instring]

Display the position of char in the given string or col.

Here "m" value is either +1(default), or -1
+1 Means search the character position from the begining of string.
-1 Means search the character position from the end of string.

Here "n" is nth occurance of give character.

ex: select instr('welcome','e') from dual;
2
or

ex: select instr('welcome','e',1,1) from dual;
2

Ex: display second occurance of 'e' from the begining of string?

select instr('welcome','e',1,2) from dual;

7

Ex: Find the length of username in a mail id?

select email, instr(email,'@') - 1 " Length of Mail" from emp;

Ex: select instr('dineshp.ora@gmail.com','@')-1 from dual;

output:- 11

select instr('welcome','e') from dual;

O/P: 2

select instr('welcome','e',+1) from dual;

O/P: 2

select instr('welcome','e',-1,1) from dual;

O/P: 7

select instr('welcome','e',-1,2) from dual;

O/P: 2

8) TRANSLATE('str'/col, 'sourcechars','targetchars')

It will display given string chars by translating source chars with corresponding target chars.

Ex: select translate('welcome','em','xy') from dual;
wxlcoyx

Ex: select translate('welcome','em','x') from dual;
wxlcox

9) REPLACE('str'/col, 'source string','target string')

Display given string by replacing source string with target string.

Ex: select replace('welcome','come','sys') from dual;
welsys

Ex: select replace('welcome','come','X') from dual;
welX

10) TRIM('str'/col)

Display given string by eliminating blank spaces before and after the string.

Ex:
select trim(' welcome to ') " trim" || initcap('oracle')
from dual;

:welcome to Oracle

11) LTRIM('str'/col) [left trim]

Display given string by removing blank spaces from the left of string only.

```
:welcome to      Oracle
```

Display given string by removing blank spaces from right of string only.

```
: welcome to Oracle
```

Display given string by removing similar occurrnaces of specific char from left of string

Display given string by removing similar occurrences of specific char from right of string

Display given string by eliminating similar occurrences of specific char from both sides of string.

Display given string along with the specific char in the left of the string.

Display given string along with specific char in the right of string.

Ex: `select rpad('page 1',12,'*') from dual;`

18) CONCAT(str1,str2)

CONCAT returns *char1* concatenated with *char2*. Both *char1* and *char2* can be any of the datatypes CHAR, VARCHAR2, NCHAR, NVARCHAR2, CLOB, or NCLOB. The string returned is in the same character set as *char1*. Its datatype depends on the datatypes of the arguments.

In concatenations of two different datatypes, Oracle Database returns the datatype that results in a lossless conversion. Therefore, if one of the arguments is a LOB, then the returned value is a LOB. If one of the arguments is a national datatype, then the returned value is a national datatype. For example:

- CONCAT(CLOB, NCLOB) returns NCLOB
- CONCAT(NCLOB, NCHAR) returns NCLOB
- CONCAT(NCLOB, CHAR) returns NCLOB
- CONCAT(NCHAR, CLOB) returns NCLOB

This function is equivalent to the concatenation operator (||).

Ex:

```
SELECT CONCAT(CONCAT(last_name, ''s job category is '),
              job_id) "Job"
FROM employees
WHERE employee_id = 152;
```

Job

Hall's job category is SA_REP

Ex:

Get the below output?

SMITH working as CLERK with salary 1200
CLARK working as MANAGER with salary 3400

```
SQL> select ename||' working as '||job||' with salary '||sal
2 from emp;
```

ENAME||'WORKINGAS'||JOB||'WITHSALARY'||SAL

SMITH working as CLERK with salary 800

ALLEN working as SALESMAN with salary 1600
WARD working as SALESMAN with salary 1250
JONES working as MANAGER with salary 2975
MARTIN working as SALESMAN with salary 1250
BLAKE working as MANAGER with salary 2850
CLARK working as MANAGER with salary 2450
SCOTT working as ANALYST with salary 3000
KING working as PRESIDENT with salary 5000
TURNER working as SALESMAN with salary 1500
ADAMS working as CLERK with salary 1100

ENAME||'WORKINGAS'||JOB||'WITHSALARY'||SAL

JAMES working as CLERK with salary 950
FORD working as ANALYST with salary 3000
MILLER working as CLERK with salary 1300

14 rows selected.

SQL>

DATE FUNCTIONS

1) TO_DATE('char fmt of date',date)

It will display any non-Oracle date format value in oracle's date format.

It accepts any char format of date(dd/mm/yy or dd-mm-yyyy or dd:mon:yyyy or yyyy-mm-dd) and converts it into oracle's default date format.

Ex: select to_date('22/03/2015','dd/mm/yyyy') from dual;
 22-mar-15

Ex: select to_date('2015:03:12','yyyy:mm:dd') from dual;
 12-mar-15

2) ADD_MONTHS(d,n)

Display a date value after adding " n " number of months to the specified date

Ex: select add_months(sysdate,6) from dual;

Ex: select add_months(mfg,24) " exp " from products;

pid	pname	cost	mfg	exp
--	---	---	--	
--	---	---	--	
--	--	---	--	

update the expdt for all products like 24 months from the date of mfg?

update prod_dtls set exp=add_months(mfg,24);

3) MONTHS_BETWEEN(d1,d2)

it shows number of months between dates.

Ex: select months_between(sysdate,'21-may-13') from dual;

Ex: select months_between('01-jan-13','01-jan-14') from dual;
 -12

4) LAST_DAY(d)

Display the date value of last day in the month.

Ex: select last_day('06-jul-10') from dual;
 31-jul-10

5) NEXT_DAY(d,'weekdayname')

it will display the date value of given weekdayname after the specified date.

Ex: select next_day(sysdate,'saturday') from dual;

Ex: select next_day(sysdate,'monday') from dual;

6) EXTRACT(DATE PART, DATE_VALUE)

The Oracle/PLSQL EXTRACT function extracts a value from a date or interval value.

Syntax

The syntax for the EXTRACT function in Oracle/PLSQL is:

```
EXTRACT (
{ YEAR | MONTH | DAY | HOUR | MINUTE | SECOND }
| { TIMEZONE_HOUR | TIMEZONE_MINUTE }
| { TIMEZONE_REGION | TIMEZONE_ABBR }
FROM { date_value | interval_value } )
```

Note

- You can only extract YEAR, MONTH, and DAY from a DATE.
- You can only extract TIMEZONE_HOUR and TIMEZONE_MINUTE from a timestamp with a time zone datatype.

Applies To

The EXTRACT function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i

```
SQL> select extract( year from sysdate) from dual;
```

```
EXTRACT(YEARFROMSYSDATE)
```

```
-----
                2017
```

7) TRUNC Function (with dates)

Description

The Oracle/PLSQL TRUNC function returns a date truncated to a specific unit of measure.

Syntax (with dates)

The syntax for the TRUNC function in Oracle/PLSQL is:

```
TRUNC ( date [, format ] )
```

Parameters or Arguments

date

The date to truncate.

format

Optional. The unit of measure to apply for truncating. If the *format* parameter is omitted, the TRUNC function will truncate the date to the day value, so that any hours, minutes, or seconds will be truncated off. It can be one of the following values:

Unit	Valid format parameters
Year	SYYYYY, YYYYY, YEAR, SYEAR, YYY, YY, Y
ISO Year	IYYYY, IY, I
Quarter	Q
Month	MONTH, MON, MM, RM
Week	WW
IW	IW
W	W
Day	DDD, DD, J
Start day of the week	DAY, DY, D
Hour	HH, HH12, HH24
Minute	MI

Applies To

The TRUNC function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i, Oracle 8i

Example - with dates

Let's look at some Oracle TRUNC function examples and explore how to use the TRUNC function in Oracle/PLSQL.

For example:

```
TRUNC(TO_DATE('22-AUG-03'), 'YEAR')
```

Result: '01-JAN-03'

```
TRUNC(TO_DATE('22-AUG-03'), 'Q')
```

Result: '01-JUL-03'

```
TRUNC(TO_DATE('22-AUG-03'), 'MONTH')
```

Result: '01-AUG-03'

```
TRUNC(TO_DATE('22-AUG-03'), 'DDD')
```

Result: '22-AUG-03'

```
TRUNC(TO_DATE('22-AUG-03'), 'DAY')
```

Result: '17-AUG-03'

8) CURRENT_DATE function

Description

The Oracle/PLSQL CURRENT_DATE function returns the current date in the time zone of the current SQL session as set by the ALTER SESSION command.

Syntax

The syntax for the CURRENT_DATE function in Oracle/PLSQL is:

```
CURRENT_DATE
```

Parameters or Arguments

There are no parameters or arguments for the CURRENT_DATE function.

Applies To

The CURRENT_DATE function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i

Example

The CURRENT_DATE function can be used in Oracle/PLSQL.

If the following ALTER SESSION command was issued:

```
ALTER SESSION SET TIME_ZONE = '-7:0';
```

And then the following SQL statement was executed:

```
select CURRENT_DATE  
from dual;
```

You might get the following result:

```
9/10/2005 10:58:24 PM
```

You then modified the session time zone with the following ALTER SESSION command:

```
ALTER SESSION SET TIME_ZONE = '-2:0';
```

And then the following SQL statement was executed:

```
select CURRENT_DATE  
from dual;
```

You would now get the following result:

```
9/11/2005 3:58:24 AM
```

The session time zone value has changed from -7:0 to -2:0, causing the CURRENT_DATE function to return the current date as a value 5 hours ahead.

9) CURRENT_TIMESTAMP

The Oracle/PLSQL CURRENT_TIMESTAMP function returns the current date and time in the time zone of the current SQL session as set by the ALTER SESSION command. It returns a TIMESTAMP WITH TIME ZONE value.

Syntax

The syntax for the CURRENT_TIMESTAMP function in Oracle/PLSQL is:

```
CURRENT_TIMESTAMP
```

Parameters or Arguments

There are no parameters or arguments for the CURRENT_TIMESTAMP function.

Note

- A similar function to the CURRENT_TIMESTAMP function is the [LOCALTIMESTAMP function](#).

- The difference between these two functions is that the `CURRENT_TIMESTAMP` function returns a `TIMESTAMP WITH TIME ZONE` value while the [LOCALTIMESTAMP function](#) returns a `TIMESTAMP` value.

Applies To

The `CURRENT_TIMESTAMP` function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i

Example

The `CURRENT_TIMESTAMP` function can be used in Oracle/PLSQL. If the following `ALTER SESSION` command was issued:

```
ALTER SESSION SET TIME_ZONE = '-7:0';
```

And then the following SQL statement was executed:

```
select CURRENT_TIMESTAMP  
from dual;
```

You might get the following result:

```
10-Sep-05 10.58.24.853421 PM -07:00
```

You then modified the session time zone with the following `ALTER SESSION` command:

```
ALTER SESSION SET TIME_ZONE = '-2:0';
```

And then the following SQL statement was executed:

```
select CURRENT_TIMESTAMP  
from dual;
```

You would now get the following result:

```
10-Sep-05 03.58.24.853421 AM -02:00
```

The session time zone value has changed from `-7:0` to `-2:0`, causing the `CURRENT_TIMESTAMP` function to return the current date and time as a value 5 hours ahead.

10) LOCALTIMESTAMP

The Oracle/PLSQL `LOCALTIMESTAMP` function returns the current date and time in the time zone of the current SQL session as set by the `ALTER SESSION` command. It returns a `TIMESTAMP` value.

Syntax

The syntax for the `LOCALTIMESTAMP` function in Oracle/PLSQL is:

```
LOCALTIMESTAMP
```

Parameters or Arguments

There are no parameters or arguments for the `LOCALTIMESTAMP` function.

Note

- A similar function to the LOCALTIMESTAMP function is the [CURRENT_TIMESTAMP function](#).
- The difference between these two functions is that the LOCALTIMESTAMP function returns a TIMESTAMP value while the [CURRENT_TIMESTAMP function](#) returns a TIMESTAMP WITH TIME ZONE value.

Applies To

The LOCALTIMESTAMP function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i, Oracle 8i

Example

Let's look at some Oracle LOCALTIMESTAMP function examples and explore how to use the LOCALTIMESTAMP function in Oracle/PLSQL.

For example:

If the following ALTER SESSION command was issued:

```
ALTER SESSION SET TIME_ZONE = '-7:0';
```

And then the following SQL statement was executed:

```
select LOCALTIMESTAMP  
from dual;
```

You might get the following result:

```
10-Sep-05 10.58.24 PM
```

You then modified the session time zone with the following ALTER SESSION command:

```
ALTER SESSION SET TIME_ZONE = '-2:0';
```

And then the following SQL statement was executed:

```
select LOCALTIMESTAMP  
from dual;
```

You would now get the following result:

```
10-Sep-05 03.58.24 AM
```

The session time zone value has changed from -7:0 to -2:0, causing the LOCALTIMESTAMP function to return the current date and time as a value 5 hours ahead.

11) SYSTIMESTAMP

The Oracle/PLSQL SYSTIMESTAMP function returns the current system date and time (including fractional seconds and time zone) on your local database.

Syntax

The syntax for the SYSTIMESTAMP function in Oracle/PLSQL is:

SYSTIMESTAMP

Parameters or Arguments

There are no parameters or arguments for the SYSTIMESTAMP function.

Applies To

The SYSTIMESTAMP function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i

Example

Let's look at some Oracle SYSTIMESTAMP function examples and explore how to use the SYSTIMESTAMP function in Oracle/PLSQL.

For example:

```
SELECT SYSTIMESTAMP  
INTO v_time  
FROM dual;
```

The variable called *v_time* will now contain the date and time (including fractional seconds and time zone) at the moment the command is executed. The SYSTIMESTAMP function might return a value like this:

```
2015-07-22 10:35:07.417849 -06:00
```

You can also choose to use the [TO_CHAR function](#) with the SYSTIMESTAMP function.

For example:

```
SELECT TO_CHAR(SYSTIMESTAMP, 'SSSS.FF')  
FROM dual;
```

The function above may return a value such as:

```
4141.550774
```

You could also use the SYSTIMESTAMP function in any SQL statement. For example:

```
SELECT supplier_id, SYSTIMESTAMP  
FROM suppliers  
WHERE supplier_id > 5000;
```

CONVERSION FUNCTIONS

1) TO_NUMBER(chardata,[numberdata])

It accepts the chardata which contains a sequence of digits and convert it into number type data.

Ex: salaries

 \$1200
 \$11005 (char type data)
 \$107069.12

target: find the sum of salaries

1) substr(sal,2)

 1200
 11005(chardata)
 107069.12

2)to_number(substr(sal,2))

converted into number type data

3) sum(to_number(substr(sal,2)))

Ex: select sum(to_number(substr(sal,2))) from emp;

2) TO_CHAR(num, [char])

it accepts number type data and convert it into character type data.

Ex:

price_list		target_format
-----		-----
100.78		00,00,100.78
1200.12		00,01,200.12
5463.00	(number type data)	00,05,463.00
(char format)		
100700.00		01,00,700.00
1223501.01		12,23,501.01

Ex: select to_char(price_list,'00,00,000.00') from products;

3) TO_CHAR(date,[char])

It accepts Oracle's date type data and convert it into required char format.

Date Formats:

DD digits of month
Day Dayname
day dayname
DAY DAYNAME
Mon 3-chars of month with begining char capital
MON "
mon "
monthfull month name(in lower case)
MONTH full month name(caps)
Monthfull month name with begining char capital
yy last 2 digits of year
YY "
yyyy complete year number

Ex:

input	output
sysdate	27 September 2014

Ex: select to_char(sysdate,'dd month yyyy') from dual;

SAMPLE EXECUTIONS:

SQL> select to_char(sysdate,'dd month yyyy') from dual;

TO_CHAR(SYSDATE,'DDMO

28 february 2015

SQL> select to_char(sysdate,'dd mon yyyy') from dual;

TO_CHAR(SYSDATE

28 feb 2015

SQL> select to_char(sysdate,'dd MON yyyy') from dual;

TO_CHAR(SYSDATE

28 FEB 2015

SQL> select to_char(sysdate,'dd Mon yyyy') from dual;

TO_CHAR(SYSDATE

28 Feb 2015

SQL> select to_char(sysdate,'dd day Mon yyyy') from dual;

TO_CHAR(SYSDATE,'DDDAYMO

28 saturday Feb 2015

```
SQL> select to_char(sysdate,'dd Day Mon yyyy') from dual;
```

```
TO_CHAR(SYSDATE,'DDDAYMO
```

```
-----  
28 Saturday Feb 2015
```

```
SQL> select to_char(sysdate,'dd DAY Mon yyyy') from dual;
```

```
TO_CHAR(SYSDATE,'DDDAYMO
```

```
-----  
28 SATURDAY Feb 2015
```

4) CAST

The Oracle/PLSQL CAST function converts one datatype to another.

Syntax

The syntax for the CAST function in Oracle/PLSQL is:

```
CAST ( { expr | ( subquery ) | MULTISET ( subquery ) } AS type_name )
```

Applies To

The CAST function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i

Example

Let's look at some Oracle CAST function examples and explore how to use the CAST function in Oracle/PLSQL.

For example:

```
select CAST( '22-Aug-2003' AS varchar2(30) )  
from dual;
```

This would convert the date (ie: 22-Aug-2003) into a varchar2(30) value.

5) TO_TIMESTAMP

The Oracle/PLSQL TO_TIMESTAMP function converts a string to a timestamp.

Syntax

The syntax for the TO_TIMESTAMP function in Oracle/PLSQL is:

```
TO_TIMESTAMP( string1 [, format_mask] ['nlsparam'] )
```

Parameters or Arguments

string1

The string that will be converted to a timestamp.

format_mask

Optional. This is the format that will be used to convert *string1* to a timestamp. It can be one or a combination of the following values

Parameter	Explanation
YYYY	4-digit year
MM	Month (01-12; JAN = 01).
MON	Abbreviated name of month.
MONTH	Name of month, padded with blanks to length of 9 characters.
DD	Day of month (1-31).
HH	Hour of day (1-12).
HH12	Hour of day (1-12).
HH24	Hour of day (0-23).
MI	Minute (0-59).
SS	Second (0-59).

Applies To

The TO_TIMESTAMP function can be used in the following versions of Oracle/PLSQL:

- Oracle 12c, Oracle 11g, Oracle 10g, Oracle 9i

Example

Let's look at some Oracle TO_TIMESTAMP function examples and explore how to use the TO_TIMESTAMP function in Oracle/PLSQL.

For example:

```
TO_TIMESTAMP('2003/12/13 10:13:18', 'YYYY/MM/DD HH:MI:SS')
```

would return '13-DEC-03 10.13.18.000000000 AM' as a timestamp value.

```
TO_TIMESTAMP('2003/DEC/13 10:13:18', 'YYYY/MON/DD HH:MI:SS')
```

would also return '13-DEC-03 10.13.18.000000000 AM' as a timestamp value.

ORACLE MISLLANEOUS CONCEPTS

SQL CASE statement

The CASE statement is like a series of IF statements, only using the key word WHEN. A CASE statement is evaluated from top to bottom. If a condition is true, then corresponding THEN clause is executed and execution jumps to the END CASE (short circuit evaluation) clause.

Oracle SQL allows you to add "Boolean logic" and branching using the decode and CASE clauses. The case statement is a more flexible extension of the Decode statement. In its simplest form the Oracle CASE function is used to return a value when a match is found:

Ex:

```
SELECT last_name, commission_pct,  
(CASE commission_pct  
WHEN 0.1 THEN 'Low'  
WHEN 0.15 THEN 'Average'  
WHEN 0.2 THEN 'High'  
ELSE 'N/A' END ) Commission  
FROM  
    employees  
ORDER BY  
    last_name;
```

Ex:

```
select case when sal<1000 then 'low'  
            when sal >=1000 and sal <2000 then 'Medium'  
            else 'High'  
        end  
from emp;
```

COALESCE function

The COALESCE function takes two or more compatible arguments and returns the first argument that is not null.

The result is null only if all the arguments are null.

If all the parameters of the function call are dynamic, an error occurs.

Syntax

COALESCE (*expression*, *expression* [, *expression*]*)

The function must have at least two arguments.

Example

create table with three different integer types

create table temp(smallintcol smallint, bigintcol bigint, intcol integer);

0 rows inserted/updated/deleted

ij> insert into temp values (1, null, null);

1 row inserted/updated/deleted

ij> insert into temp values (null, 2, null);

1 row inserted/updated/deleted

```
ij> insert into temp values (null, null, 3);
```

1 row inserted/updated/deleted

```
ij> select * from temp;
```

SMALL& BIGINTCOL	INTCOL
------------------	--------

1	NULL	NULL
NULL	2	NULL
NULL	NULL	3

3 rows selected

```
ij> -- the return data type of coalesce is bigint
```

```
ij> select coalesce (smallintcol, bigintcol) from temp;
```

1

1

2

NULL

3 rows selected

```
ij> -- the return data type of coalesce is bigint
```

```
ij> select coalesce (smallintcol, bigintcol, intcol) from temp;
```

1

1

2

3

3 rows selected

```
ij> -- the return data type of coalesce is integer
```

```
ij> select coalesce (smallintcol, intcol) from temp;
```

1

1

NULL

3

3 rows selected

DEFAULT Values Using Sequences

In Oracle 12c, it is now possible to specify the CURRVAL and NEXTVAL sequence pseudo columns as the default values for a column.

```
CREATE SEQUENCE t1_seq;

CREATE TABLE t1 (
  id      NUMBER DEFAULT t1_seq.NEXTVAL,
  description VARCHAR2(30)
);

INSERT INTO t1 (description) VALUES ('DESCRIPTION only');
INSERT INTO t1 (id, description) VALUES (999, 'ID=999 and
DESCRIPTION');
INSERT INTO t1 (id, description) VALUES (NULL, 'ID=NULL and
DESCRIPTION');

SELECT * FROM t1;
```

ID	DESCRIPTION
1	DESCRIPTION only
999	ID=999 and DESCRIPTION
	ID=NULL and DESCRIPTION

3 rows selected.

LAG

The LAG function is used to access data from a previous row. The following query returns the salary from the previous row to calculate the difference between the salary of the current row and that of the previous row. Notice that the ORDER BY of the LAG function is used to order the data by salary.

```
SELECT empno,
       ename,
       job,
       sal,
       LAG(sal, 1, 0) OVER (ORDER BY sal) AS sal_prev,
       sal - LAG(sal, 1, 0) OVER (ORDER BY sal) AS sal_diff
FROM   emp;
```

EMPNO	ENAME	JOB	SAL	SAL_PREV	SAL_DIFF
7369	SMITH	CLERK	800	0	800
7900	JAMES	CLERK	950	800	150

7876	ADAMS	CLERK	1100	950	150
7521	WARD	SALESMAN	1250	1100	150
7654	MARTIN	SALESMAN	1250	1250	0
7934	MILLER	CLERK	1300	1250	50
7844	TURNER	SALESMAN	1500	1300	200
7499	ALLEN	SALESMAN	1600	1500	100
7782	CLARK	MANAGER	2450	1600	850
7698	BLAKE	MANAGER	2850	2450	400
7566	JONES	MANAGER	2975	2850	125
7788	SCOTT	ANALYST	3000	2975	25
7902	FORD	ANALYST	3000	3000	0
7839	KING	PRESIDENT	5000	3000	2000

SQL>

LEAD

The LEAD function is used to return data from the next row. The following query returns the salary from the next row to calculate the difference between the salary of the current row and the following row.

```
SELECT empno,
       ename,
       job,
       sal,
       LEAD(sal, 1, 0) OVER (ORDER BY sal) AS sal_next,
       LEAD(sal, 1, 0) OVER (ORDER BY sal) - sal AS sal_diff
FROM emp;
```

EMPNO	ENAME	JOB	SAL	SAL_NEXT	SAL_DIFF
7369	SMITH	CLERK	800	950	150
7900	JAMES	CLERK	950	1100	150
7876	ADAMS	CLERK	1100	1250	150
7521	WARD	SALESMAN	1250	1250	0
7654	MARTIN	SALESMAN	1250	1300	50
7934	MILLER	CLERK	1300	1500	200
7844	TURNER	SALESMAN	1500	1600	100
7499	ALLEN	SALESMAN	1600	2450	850
7782	CLARK	MANAGER	2450	2850	400
7698	BLAKE	MANAGER	2850	2975	125
7566	JONES	MANAGER	2975	3000	25
7788	SCOTT	ANALYST	3000	3000	0
7902	FORD	ANALYST	3000	5000	2000
7839	KING	PRESIDENT	5000	0	-5000

RANK

Let's assume we want to assign a sequential order, or rank, to people within a department based on salary, we might use the RANK function like.

```
SELECT empno,  
       deptno,  
       sal,  
       RANK() OVER (PARTITION BY deptno ORDER BY sal) "rank"  
FROM   emp;
```

EMPNO	DEPTNO	SAL	rank

7934	10	1300	1
7782	10	2450	2
7839	10	5000	3
7369	20	800	1
7876	20	1100	2
7566	20	2975	3
7788	20	3000	4
7902	20	3000	4
7900	30	950	1
7654	30	1250	2
7521	30	1250	2
7844	30	1500	4
7499	30	1600	5
7698	30	2850	6

SQL>

What we see here is where two people have the same salary they are assigned the same rank. When multiple rows share the same rank the next rank in the sequence is not consecutive.

DENSE_RANK

The DENSE_RANK function acts like the RANK function except that it assigns consecutive ranks.

```
SELECT empno,  
       deptno,  
       sal,  
       DENSE_RANK() OVER (PARTITION BY deptno ORDER BY sal) "rank"  
FROM   emp;
```

EMPNO	DEPTNO	SAL	rank

7934	10	1300	1
7782	10	2450	2
7839	10	5000	3
7369	20	800	1

7876	20	1100	2
7566	20	2975	3
7788	20	3000	4
7902	20	3000	4
7900	30	950	1
7654	30	1250	2
7521	30	1250	2
7844	30	1500	3
7499	30	1600	4
7698	30	2850	5

SQL>

FIRST and LAST

The FIRST and LAST functions can be used to return the first or last value from an ordered sequence. Say we want to display the salary of each employee, along with the lowest and highest within their department we may use something like.

```
SELECT empno,
       deptno,
       sal,
       MIN(sal) KEEP (DENSE_RANK FIRST ORDER BY sal) OVER
(PARTITION BY deptno) "Lowest",
       MAX(sal) KEEP (DENSE_RANK LAST ORDER BY sal) OVER
(PARTITION BY deptno) "Highest"
FROM   emp
ORDER BY deptno, sal;
```

EMPNO	DEPTNO	SAL	Lowest	Highest
7934	10	1300	1300	5000
7782	10	2450	1300	5000
7839	10	5000	1300	5000
7369	20	800	800	3000
7876	20	1100	800	3000
7566	20	2975	800	3000
7788	20	3000	800	3000
7902	20	3000	800	3000
7900	30	950	950	2850
7654	30	1250	950	2850
7521	30	1250	950	2850
7844	30	1500	950	2850
7499	30	1600	950	2850
7698	30	2850	950	2850

SQL>

NVL

The NVL function allows you to replace null values with a default value. If the value in the first parameter is null, the function returns the value in the second parameter. If the first parameter is any value other than null, it is returned unchanged.

We know that COL1 in the test table contains null in all rows except the first. Using the NVL function we replace the null values with 'ZERO'.

```
SQL> SELECT id, NVL(col1, 'ZERO') AS output FROM null_test_tab ORDER BY id;
```

ID	OUTPUT
1	ONE
2	ZERO
3	ZERO
4	ZERO

4 rows selected.

SQL>

DECODE

The DECODE function is not specifically for handling null values, but it can be used in a similar way to the NVL function, as shown by the following example.

```
SQL> SELECT id, DECODE(col1, NULL, 'ZERO', col1) AS output FROM null_test_tab ORDER BY id;
```

ID	OUTPUT
1	ONE
2	ZERO
3	ZERO
4	ZERO

4 rows selected.

SQL>

NVL2

The NVL2 function accepts three parameters. If the first parameter value is not null it returns the value in the second parameter. If the first parameter value is null, it returns the third parameter.

The following query shows NVL2 in action.

```
SQL> SELECT id, NVL2(col1, col2, col3) AS output FROM null_test_tab ORDER BY id;
```


ID OUTPUT

```
-----  
1 TWO  
2 THREE  
3 THREE  
4 THREE
```

4 rows selected.

SQL>

The first row in the test table has a not null value in COL1, so the value of COL2 is returned. All other rows contains null in COL1, so the value of COL3 is returned.

COALESCE

The COALESCE function was introduced in Oracle 9i. It accepts two or more parameters and returns the first non-null value in a list. If all parameters contain null values, it returns null.

```
SQL> SELECT id, COALESCE(col1, col2, col3) AS output FROM  
null_test_tab ORDER BY id;
```

ID OUTPUT

```
-----  
1 ONE  
2 TWO  
3 THREE  
4 THREE
```

4 rows selected.

SQL>

NULLIF

The NULLIF function was introduced in Oracle 9i. It accepts two parameters and returns null if both parameters are equal. If they are not equal, the first parameter value is returned.

In our test table the values of COL3 and COL4 are equal in row 4, so we would only expect null returned for that row using the following query.

```
SQL> SELECT id, NULLIF(col3, col4) AS output FROM null_test_tab ORDER  
BY id;
```

ID OUTPUT

```
-----  
1 THREE  
2 THREE
```

3 THREE

4

4 rows selected.

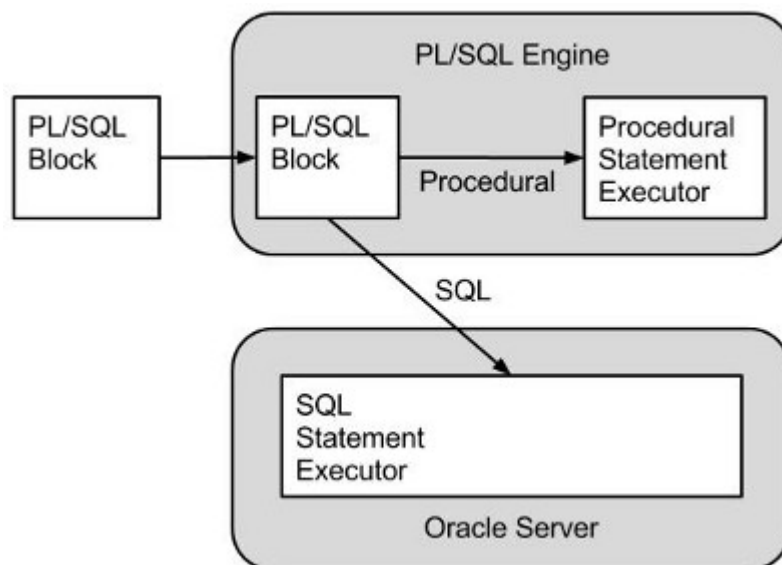
SQL>

PL/SQL

PL/SQL is a procedural extension to Oracle SQL. PL/SQL is integrated with Oracle Database, enabling you to use all of the Oracle Database SQL statements, functions, and data types. You can use PL/SQL to control the flow of a SQL program, use variables, and write error-handling procedures.

A primary benefit of PL/SQL is the ability to store application logic in the database itself. A **PL/SQL procedure** or **function** is a schema object that consists of a set of SQL statements and other PL/SQL constructs, grouped together, stored in the database, and run as a unit to solve a specific problem or to perform a set of related tasks. The principal benefit of server-side programming is that built-in functionality can be deployed anywhere.

Oracle Database can also store program units written in Java. A Java stored procedure is a Java method published to SQL and stored in the database for general use. You can call existing PL/SQL programs from Java and Java programs from PL/SQL.



It is a procedural language using sql concepts.
It is an extension to the sql.

pl/sql program

It is a collection of programming stmts and with sql concepts (queries) to perform user required tasks .

What is pl/sql?

Pl/sql is a collection of User defined objects like Programs, procedures, Functions, Triggers, Types, Packages and so on.

Advantages:

- Multiple queries are executed parallelly.
- It reduces number of hits to the database.
- In pl/sql user can create the objects according to his requirements.
- It increases n/w performance.
- Modularity (dividing a big task into smaller modules)
- Enhansibility (It can easily accept the future changes)
- Reusability

2 Categories of Programs

- Anonymous Blocks (Programs)
- Sub Programs (Stored Procedures and Functions)

i) Anonymous Blocks

It is a program which is not saved inside the DataBase.

2 types of anonymous blocks

a) Static Programs

It is not accepting runtime input values and Always generate same output, Input value is fixed.

b) Dynamic Programs

It will accept RunTime Input values and Generates different outputs based on different input values.

STRUCTURE OF PL/SQL PROGRAM:

```

DECLARE           [ Optional ]
<declaration stmts >;
BEGIN             [Mandatory ]
<Assignment stmt>;
<Output stmts>;
<Data processing stmts>;
EXCEPTION        [Optional ]
<Error handling stmts>;
END;              /* end of program */

```

DECLARE block

It contains declaration stmts. Used to declare variables. So these variables get memory space from the database engine based on their datatype and size.

Syntax:

```
var_name  DATATYPE(size);
```

Ex: v_enoint;

```
v_name    varchar2(20);
v_sal    number(5);
v_jdate    date;
```

BEGIN block

Assignment statements

Used to store values into the declared variables by using assignment operator [:=] or by using SELECT stmt with INTO keyword.

Syntax-1:

BY USING **ASSIGNMENT** OPERATOR

```
var_name:=value / expression;
```

Ex: v_eno:=7654;

Ex:

```
declare
x    int;
y    int;
z    int;
begin
x:=100;
y:=200;
z:=x+y;
```

Syntax-2:

By Using **SELECT Query with INTO keyword**

```
select col1, col2, ...,coln INTO var_name1, varname_2,...var-n
from table
where <cond>;
```

Ex: select ename,sal,hiredate INTO V_name,v_sal,v_jdate
from emp where empno=v_eno;

Out Put stmts

Used to display values of variables and normal messages.
Oracle provides a predefined output function as follows.

DBMS_OUTPUT.PUT_LINE('normal messages' or var_name);

Ex: dbms_output.put_LINE(' employee information ');
dbms_output.put_LINE('-----');
dbms_output.put_line(v_eno);
dbms_output.put_line(v_name);
dbms_output.put_line(v_sal);
dbms_output.put_line(v_jdate);

Data Processing stmts

Any sql query is known as data processing stmt.

EXCEPTION Block

It is used to display user friendly error messages instead of system generated error messages.

END;

Indicates end of program.

HOW TO "COMPILE" AND "EXECUTE" THE PL/SQL PROGRAM IN sql *plus WINDOW?

/ Used to compile and execute the pl/sql program in SQLPLUS environment.

-- It is a single line comment symbol

/* text */ It is a multi line comment symbol

NOTE:

By default any program or procedure should not display output. To display output,

SET SERVEROUTPUT ON;

It enable oracle engine to display output from any pl/sql program or procedure.

This is valid for current session.

Ex: write a program to display welcome message .

```
begin
dbms_output.put_line('welcome to oracle pl/sql');
end;
```

Ex: write a program to display addition , average, max and min of 3000 and 5000?

```
DECLARE
X    INT;
Y    INT:=5000;
S    INT;
A    NUMBER(7,2);
MX   INT;
MN   INT;
BEGIN
X:=3000;
S:=X+Y;
```

```

A:=S/2;
SELECT GREATEST(X,Y) INTO MX FROM DUAL;
SELECT LEAST(X,Y) INTO MN      FROM DUAL;
DBMS_OUTPUT.PUT_LINE(' SUM=');
DBMS_OUTPUT.PUT_LINE(S);
DBMS_OUTPUT.PUT_LINE('AVERAGE=');
DBMS_OUTPUT.PUT_LINE(A);
DBMS_OUTPUT.PUT_LINE('HIGHER VALUE=');
DBMS_OUTPUT.PUT_LINE(MX);
DBMS_OUTPUT.PUT_LINE('Least VALUE=');
DBMS_OUTPUT.PUT_LINE(MN);
END;
/

```

sample execution:

```

SUM=
8000
AVERAGE=
4000
HIGHER VALUE=
5000
MIN VALUE=
3000

```

PL/SQL procedure successfully completed.

NOTE:

How can i display a normal mesg after that the value of variable immediately in the same line?

By using concatenation Operator [||]

It will display the value or message after the previous message or value.

```

DECLARE
X      INT;
Y      INT:=5000;
S      INT;
A      NUMBER(7,2);
MX     INT;
MN     INT;
BEGIN
X:=3000;
S:=X+Y;
A:=S/2;
SELECT GREATEST(X,Y) INTO MX FROM DUAL;
SELECT LEAST(X,Y) INTO MN      FROM DUAL;
DBMS_OUTPUT.PUT_LINE(' SUM='||s);
--DBMS_OUTPUT.PUT_LINE(S);
DBMS_OUTPUT.PUT_LINE('AVERAGE='||a);
--DBMS_OUTPUT.PUT_LINE(A);
DBMS_OUTPUT.PUT_LINE('HIGHER VALUE='||mx);
--DBMS_OUTPUT.PUT_LINE(MX);

```

```

DBMS_OUTPUT.PUT_LINE('MIN VALUE='||mn);
--DBMS_OUTPUT.PUT_LINE(MN);
END;
/

```

sample execution:

```

anonymous block completed
SUM=8000
AVERAGE=4000
HIGHER VALUE=5000
MIN VALUE=3000

```

Types of pl/sql programs: 2

1) Static programs

Always generate same output Since the input value in the program is fixed and constant.

2) Dynamic programs

Generates different output values based on different input values. In this case the program can accept runtime input values.

STATIC PROGRAMS

Ex: write a program to display the employee information of empID 7654?

```

declare
veno int:=7654;
vname varchar2(20);
vsal number(5);
vjob varchar2(20);
vhiredate date;
vcomm number(4);
vdeptno number(3);
BEGIN
select ename,sal,job,hiredate,comm,deptno INTO
      vname,vsal,vjob,vhiredate,vcomm,vdeptno
from emp
where empno=veno;
dbms_output.put_line(' Info of 7654');
dbms_output.put_line('-----');
dbms_output.put_line(vname);
dbms_output.put_line(vsal);
dbms_output.put_line(vjob);
dbms_output.put_line(vhiredate);
dbms_output.put_line(vcomm);
dbms_output.put_line(vdeptno);
END;
/

```


sample output:

Info of 7654
MARTIN
1250
SALESMAN
28-SEP-81
1400
30

PL/SQL procedure successfully completed.

Ex:

```
declare
veno int:=7654;
vname  varchar2(20);
vsal  number(5);
vjob  varchar2(20);
vhiredate  date;
vcomm  number(4);
vdeptno  number(3);
BEGIN
select ename,sal,job,hiredate,comm,deptno INTO
      vname,vsal,vjob,vhiredate,vcomm,vdeptno
from emp
where empno=veno;
dbms_output.put_line(' Info of Emp id:      7654');
dbms_output.put_line('=====')
=====);
dbms_output.put_line('Name of emp: '||vname);
dbms_output.put_line('Salary of emp: '||vsal);
dbms_output.put_line('Designation of emp: '||vjob);
dbms_output.put_line('Join date of emp: '||vhiredate);
dbms_output.put_line('Commission of emp: '||vcomm);
dbms_output.put_line('Deptno of emp: '||vdeptno);
dbms_output.put_line('=====')
=====);
END;
/
```

output:

Info of 7654
=====

Name of emp:	MARTIN
Salary of emp:	1250
Designation of emp:	SALESMAN
Join date of emp:	28-SEP-81
Commission of emp:	1400
Deptno of emp:	30

=====

PL/SQL procedure successfully completed.

Ex: Write a program to display number of customers from the city " Delhi " ?

```
DECLARE
VCITY  VARCHAR2(20):='Delhi';
CUST_CNT  INT;
BEGIN
SELECT COUNT(*) INTO CUST_CNT FROM CUST_DTLS WHERE
CITY=VCITY;
DBMS_OUTPUT.PUT_LINE(' NUMBER OF CUSTOMERS FROM '||VCITY||' =
'||CUST_CNT);
END;
```

Ex:

Write a program to display number of male customers and number of female customers?

```
declare
male_cnt  int;
female_cnt  int;
begin
select count(*) into male_cnt from cust_dtls where gender='M';
select count(*) into female_cnt from cust_dtls where gender='F';
dbms_output.put_line(' Number of males= '||male_cnt);
dbms_output.put_line(' Number of Females='||female_cnt);
end;
```

Assignments:

- 1) Write a program to display designation of empid 7788?
- 2) Write a program to display the city and mobile number of customer id " cust-5"?
- 3) Write a program to display the location of department " RESEARCH "?

Dynamic Programs

By using **&(Substitution)** operator we will make a program as a dynamic program.

Instead of assigning a constant value in to a variable , While the execution of the program, the program has to take a value from the end user and that value stored it into a variable.

```
varname:=&varname;
```

Ex: write a program to display the details of employee for the given empno?

```

declare
v_eno number(4);
v_ename varchar2(20);
v_sal number(5);
v_job varchar2(20);
v_hiredate date;
v_comm int;
v_mgr_code int;
v_deptno int;
BEGIN
v_eno:='&v_eno';
select ename,sal,job,hiredate,comm,mgr,deptno INTO
v_ename,v_sal,v_job,v_hiredate,v_comm,v_mgr_code,
v_deptno
from emp
where empno=v_eno;
dbms_output.put_line(v_eno||' Employee information ');
dbms_output.put_line
('=====');
dbms_output.put_line
('EMPNAME : EMP-SALARY :EMP-DESG : EMP-JOINDATE : comm :
manager code : deptno');
dbms_output.put_line
('=====');
dbms_output.put_line
(v_ename||' '||v_sal||' '||v_job||' '||v_hiredate||'
'||v_comm||' '||v_mgr_code||' '||v_deptno);
dbms_output.put_line
('=====');
end;
```

Ex: using %rowtype

```

declare
v_eno number(4);
r emp%rowtype;
BEGIN
v_eno:='&v_eno';
select * INTO r from emp
where empno=v_eno;
dbms_output.put_line(v_eno||' Employee information ');
dbms_output.put_line('=====');

dbms_output.put_line
(r.ename||' '||r.sal||' '||r.job||' '||r.mgr||' '||r.hiredate||' '||r.comm||'
'||r.deptno);
dbms_output.put_line('=====');
end;
```

output:

7654 information

```
=====
MARTIN          1250 SALESMAN 28-SEP-81
=====
```

output:

7788 information

```
=====
SCOTT           3000 ANALYST  19-APR-87
=====
```

Ex: Write a program to display the number of emps in the given deptno?

```
declare
  vjno number(2);
  e_count int;
begin
  vjno:=&vjno;
  select count(empno) into e_count from emp where deptno=vjno;
  dbms_output.put_line(' number of emps in Department : '||vjno||' : =
'||e_count);
end;
```

output:

number of emps in 20 = 5

Ex:

number of emps in 30 = 6

Ex:

write a program to display the dept details under which the employee is working with given id?

```
declare
  veno number(4);
  vjno number(4);
  vname varchar2(20);
  vloc varchar2(20);
begin
  veno:=&veno;
  select * into vjno,vname,vloc from dept
  where deptno=(select deptno from emp where empno=veno);
  dbms_output.put_line(' department details of employee : '||veno);
  dbms_output.put_line(vjno||' '||vname||' '||vloc);
end;
```

Ex output:

```
department details of employee : 7902
20    RESEARCH DALLAS
```

Ex output:

```
department details of employee : 7788
20    RESEARCH DALLAS
```

Assignment

- 1) write a program to display the total salary i am paying to deptno 30 employees?
- 2) write a program to display the "number of male customers" from the given city?
- 3) Write a program to display the number of emps working under given deptno?
- 4) write a program to find the number of emps working with given designation?
- 5)
WRITE A PROGRAM TO DISPLAY NUMBER OF PRODUCTS HAVING WARRENTY MANUFACTURED IN THE YEAR 2014?

```
DECLARE
nop int;
begin
select count(*) into nop from prod_dtls
where mfg between '01-jan-14' and '31-dec-14'
and
warrenty is not null;
dbms_output.put_line(' NUmber of products having warrenty='|| nop);
end;
```

Ex

write a program to display customer account details for the given customer name?

```
declare
vcname varchar2(20);
vactno number(12);
vacttype varchar2(10);
vopendate date;
vactbal number;
vccode varchar2(10);
begin
```

```

vcname:='&vcname';
select actno,act_type,act_open_date,act_bal,cust_code into
       vactno,vacttype,vopendate,vactbal,vccode
       from cust_act_dtls
       where cust_code =(select cno from cust_dtls where cname=vcname);

dbms_output.put_line('Account details of '||upper(vcname));
dbms_output.put_line('*****');
dbms_output.put_line('Account Number='||vactno);
dbms_output.put_line('Account Type='||vacttype);
dbms_output.put_line('Account Open Date='||vopendate);
dbms_output.put_line('Account Balance='||vactbal);
dbms_output.put_line('Customer Code='||vccode);
end;

```

Note:

Generally, we need to declare a variable with column data type otherwise we will get compatibility issues.

To store a complete record in to a variable we need declare a variable as TABLE BASED RECORD type variable.

TYPE COMPATIBILITY KEYWORDS

We can declare the variables dynamically/ Implicitly as column DATA type (using **%TYPE**) and record type (by using **%ROWTYPE**). This eliminates Data type and size incompatibility issues.

To Declare variables implicitly

%TYPE

It is used to declare a variable of column datatype dynamically.

Syntax: var_name table_name.col_name%type;

Ex: vdnno emp.deptno%type;

In the above the %TYPE is taking the data type and size of the column DEPTNO from the table EMP and these things will be applicable to the variable.

Ex: vacctno cust_act_dtls.actno%type;

PL/SQL RECORDS: 3 types

i) Table based records %ROWTYPE

This record type variable supports all columns from the table

ii) Cursor based records %rowtype

This record type variable supports all columns from the cursor

iii) User defined Records TYPE with keyword IS RECORD(.....);

This record type variable supports all columns from the user defined record structure.

%ROWTYPE

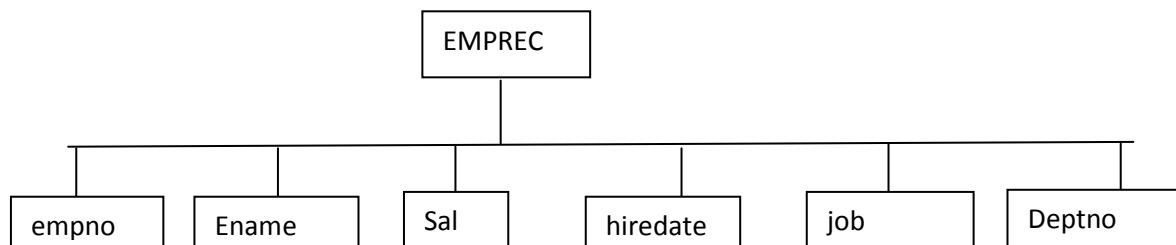
Used to declare a Table Based RECORD TYPE variable, which will supports all columns from that table.

syn:

```
var_name    tablename%ROWTYPE;
```

Ex: emp_rec emp%rowtype;

Ex: prod_info prod_dtls%rowtype;



Ex: select * into emp_rec from emp where empno=7788;

How to access individual values from record type variable?

Syntax:

```
record_var_name.colname;
```

Ex: dbms_output.put_line(' emp salary :'|| emp_rec.sal);

Ex: dbms_output.put_line(' emp Info :'|| emp_rec);----->Don't specify like this

Ex:

Write a program to display the information of employee for the given employee id?

```
declare
veno emp.empno%type; --Dynamic declaration of variable
e_rec emp%rowtype;    --Declaring table based record type variable
begin
veno:=&veno;
select * into e_rec from emp where empno=veno;
dbms_output.put_line(' given employee id: '||veno);
dbms_output.put_line(' Information of '||veno);
dbms_output.put_line
(e_rec.ename||' '||e_rec.sal||' '||e_rec.job||' '||e_rec.hiredate||'
'||e_rec.comm||' '||e_rec.deptno);
end;
```

Ex:
write a program to display the information of product for the given product id?

```
declare
vpid prod_dtls.pid%type;
p_rec prod_dtls%rowtype;

begin
vpid:='&vpid';
select * into p_rec from prod_dtls where pid=vpid;
dbms_output.put_line(' information of prodid: '||vpid);
dbms_output.put_line(p_rec.pname||' '||p_rec.cost||' '||p_rec.mfg||'
'||p_rec.warranty||' '||p_rec.prod_comp_id);
end;
```

Ex
write a program to display the information of a customer for the given customer number?

```
declare
vcustid cust_dtls.custid%type;
cust_rec cust_dtls%rowtype;
begin
vcustid:='&vcustid';
select * into cust_rec from cust_dtls where custid=vcustid;
dbms_output.put_line(' given customer id: '||vcustid);
dbms_output.put_line(' customer name='||cust_rec.custname);
dbms_output.put_line(' customer city='||cust_rec.custcity);
dbms_output.put_line(' customer gender='||cust_rec.custgender);
dbms_output.put_line(' customer mobile
number='||cust_rec.custmobile);
end;
```

```

declare
    vcustid      customers_br1.cid%type;
    cust_rec     customers_br1%rowtype;
begin
    vcustid:='&vcustid';
    select * into cust_rec from customers_br1 where cid=vcustid;
/*
    dbms_output.put_line(' given customer id: '||vcustid);
    dbms_output.put_line(' customer name='||cust_rec.cname);
    dbms_output.put_line(' customer city='||cust_rec.city);
    dbms_output.put_line(' customer gender='||cust_rec.gender);
    dbms_output.put_line(' customer mobile number='||cust_rec.mobile); */
    dbms_output.put_line('GIVEN ID:'||vcustid);
    dbms_output.put_line('-----');
    dbms_output.put_line('NAME      CITY      GENDER      MOBILE-
NUMBER'      );
    dbms_output.put_line('-----');
    dbms_output.put_line(cust_rec.cname||'      '||cust_rec.city||'
'||cust_rec.gender||'      '||cust_rec.mobile);
end;

```

SUB PROGRAMS

PROCEDURES FUNCTIONS

With Oracle Database, you can store programs in the database, so commonly used code can be written and tested once and then accessed by any application that requires it. Program units that reside in the database also ensure that when the code is invoked the data is processed consistently, which leads to ease and consistency of the application development process.

Schema-level, or standalone subprograms such as functions (which return a value) and procedures (which do not return a value) are compiled and stored in an Oracle Database. Once compiled, they become **stored procedure** or **stored function** schema objects, and can be referenced or called by any applications connected to Oracle Database. At invocation, both stored procedures and functions can accept parameters.

Procedures and functions follow the basic PL/SQL block structure, which consists of the following elements:

- A declarative part, sometimes starting with the keyword **DECLARE**, identifies variables and constants used in the application logic. This part is optional.
- An executable part, starting with **BEGIN** and ending with **END**, contains the application logic. This part is mandatory.
- An exception-handling part, starting with **EXCEPTION**, handles error conditions that may be raised in the executable part of the block. This part is optional.

The general form of a PL/SQL block follows. Note also that each stored program unit has a header that names the unit and identifies it as either a function, procedure, or a package.

STORED PROCEDURES or PL/SQL PROCEDURES

A named program which is stored / created under the data base is known stored procedure.

It is also known as a subprogram to perform a task or set of tasks.

Features of a Procedure:-

- It can be stored as a precompiled object.
- Means it will be compiled once and executed any number of times.

- Procedures provides reusability.
- Procedures can be easily enhansible for future requirements.
- Procedures are EXPLICITLY executed by the user.
- Procedures are executed by using the command EXECUTE / EXEC.
- Procedures may or may not take arguments
- Procedures may called from other procedures or functions
- By Default,Procedure cannot return any value to the calling object(Program or procedure or function).

Procedures are 2 types

a) Static Procedure

It will not contains any argument variables and it wont take any values from the user at runtime.
Always display same output.

b) Dynamic procedure

A procedure with argument variables is known as dynamic procedure.

It will take runtime input values from the user and display different output values for different users.

PROCEDURE CONTAINS 2 PARTS

i) Procedure specification

It contains creation of procedure information like proc_name and argument list

ii) Procedure body

It contains the code for the functionality of procedure.

Syntax:

/*Procedure specification*/

```
create [or replace] PROCEDURE <proc_name> [(arg_var1 datatype,
arg_var2 datatype,.....)]
IS / AS
```

/* Procedure body */

```
<declaration stmts>;
BEGIN
<assignment stmts>;
<data processing stmts>;
<output stmts>;
EXCEPTION
<error handling stmts>;
END <proc_name>;
```

/ (to compile the procedure in SQL * PLUS window)

Procedure created.

HOW TO EXECUTE THE PROCEDURE?

We can execute a procedure **implicitly and also explicitly**.

i)Executing procedure implicitly

By writing procedure calling statement as follows

```
Proc_name(arg_val1,arg_val2,. . . .);
```

ii)Executing a procedure explicitly by using EXECUTE command

```
EXECUTE <proc_name>[(arg_val1, arg_val2,.....)];
```

STATIC PROCEDURE

Ex: write a procedure which is finding number of emps working under dept 30?

```
create procedure proc_ecount_30
is
e_count int;
begin
select count(*) into e_count from emp where deptno=30;
dbms_output.put_line(' Number of emps under deptno 30 are
'||e_count);
end proc_ecount_30;
```

Ex: execute proc_ecount_30; /* Executing procedure explicitly */

How do i execute a procedure implicitly?

By making a call to the procedure we can implicitly execute the procedure.

--Calling a procedure from any program

```
BEGIN
proc_ecount_30; --Procedure calling stmt
dbms_output.put_line(' calling procedure is finished');
dbms_output.put_line(' Program Execution is finished');
end;
```

DYNAMIC PROCEDURES

--WRITING THE PROCEDURE TO DISPLAY THE NUMBER OF EMPS IN THE GIVEN DEPTNO?

```
CREATE PROCEDURE PROC_ecount_did(VDNO EMP.DEPTNO%TYPE)
IS
CNT INT;
```

```

BEGIN
SELECT COUNT(*) INTO CNT FROM EMP WHERE DEPTNO=VDNO;
DBMS_OUTPUT.PUT_LINE(' NUMBER OF EMPS IN DEPT '||VDNO||' ARE :
'||CNT);
END PROC_ecount_did;

```

SAMPLE EXECUTIONS:

```

EXEC PROC_2(10);
EXEC PROC_2(20);
EXEC PROC_2(30);

```

anonymous block completed
NUMBER OF EMPS IN DEPT 10 ARE 2

anonymous block completed
NUMBER OF EMPS IN DEPT 20 ARE 5

anonymous block completed
NUMBER OF EMPS IN DEPT 30 ARE 6

ex:

```

declare
vdno int;
begin
vdno:=&vdno;
proc_ecount_did(vdno); --Procedure calling statement
dbms_output.put_line(' Exec Finished');
end;

```

WRITING PROCEDURE CALLING STATEMENTS IN A PROGRAM:-

For the following program we need to create 2 procedures called p_add and p_multi.

```

declare
x int;
y int;
begin
x:=&x;
y:=&y;
dbms_output.put_line(' adding '||x||' and '||y);
Dbms_output.put_line(' Making a call to P_ADD procedure');
p_add(x,y);
dbms_output.put_line(' Multiplication of '||x||' and '||y);
dbms_output.put_line(' make a call to P_MULTI procedure');
p_multi(x,y);
dbms_output.put_line(' 2 tasks successfully completed');
end;

```

STATIC PROCEDURE:

Ex: write a procedure to display the customer details and his account details for the custid "cust-5"?

```
create or replace procedure proc_CUST_ACT_DTLS
is
v_cname          cust_dtls.cname%type;
v_gender         cust_dtls.gender%type;
v_city           cust_dtls.city%type;
v_actno          cust_act_dtls.actno%type;
v_act_type       cust_act_dtls.act_type%type;
v_act_bal        cust_act_dtls.act_bal%type;
begin
select c.cname,c.gender,c.city,ca.actno,ca.act_type,ca.act_bal INTO
      V_CNAME,v_gender,v_city,v_actno,v_act_type,v_act_bal
from cust_dtls c , cust_act_dtls ca
where c.cno='cust-5'
      and
      c.cno=ca.cust_code;
dbms_output.put_line(' Details of customer 5');
dbms_output.put_line
(v_cname||' '||v_gender||' '||v_city||' '||v_actno||' '||v_act_type||' '||v_act_bal);
end proc_CUST_ACT_DTLS;
```

DYNAMIC PROCEDURE:

Ex:
write a procedure to display the customer details and his account details for the GIVEN custid ?

```
create or replace procedure proc_3(VCID  cust_dtls.cno%type)
is
v_cname          cust_dtls.cname%type;
v_gender         cust_dtls.gender%type;
v_city           cust_dtls.city%type;
v_actno          cust_act_dtls.actno%type;
v_act_type       cust_act_dtls.act_type%type;
v_act_bal        cust_act_dtls.act_bal%type;
begin
select c.cname,c.gender,c.city,ca.actno,ca.act_type,ca.act_bal INTO
      V_CNAME,v_gender,v_city,v_actno,v_act_type,v_act_bal
from cust_dtls c , cust_act_dtls ca
where c.cno=VCID
      and
      c.cno=ca.cust_code;
dbms_output.put_line(' Details of customer '||VCID);
dbms_output.put_line
(v_cname||' '||v_gender||' '||v_city||' '||v_actno||' '||v_act_type||' '||v_act_bal);
end proc_3;
```

STATIC PROCEDURE:

Ex:

write a procedure to display the employee details and dept details for the empno 7902?

```
create or replace procedure proc_4
is
v_ename emp.ename%type;
v_job emp.job%type;
v_sal emp.sal%type;
v_hiredate emp.hiredate%type;
v_dno emp.deptno%type;
v_dname dept.dname%type;
v_loc dept.loc%type;
begin
select e.ename,e.sal,e.job,e.hiredate,e.deptno,d.dname,d.loc INTO
      v_ename,v_sal,v_job,v_hiredate,v_dno,v_dname,v_loc
from emp e , dept d
where e.empno=7902 and e.deptno=d.deptno;
dbms_output.put_line('Details of empno= 7902');
dbms_output.put_line
(v_ename||' '||v_job||' '||v_sal||' '||v_hiredate||' '||v_dno||' '||v_dname||'
' ||v_loc);
end proc_4;
```

DYNAMIC PROCEDURES:

EX: write a procedure to display the details of employee for the given employee id?

```
create or replace procedure proc_5 ( vempid emp.empno%type)
is
emp_rec emp%rowtype;
begin
select * into emp_rec from emp where empno=vempid;
dbms_output.put_line(' employee details of '||vempid);
dbms_output.put_line ('*****');
dbms_output.put_line
(emp_rec.ename||','||emp_rec.sal||','
||emp_rec.job||','||emp_rec.hiredate);
end proc_5;
```

sample output:

```
employee details of 7902
FORD,3000,ANALYST,03-DEC-81
```

```
anonymous block completed
employee details of 7788
SCOTT,3000,ANALYST,19-APR-87
```

anonymous block completed
employee details of 7654
MARTIN,1250,SALESMAN,28-SEP-81

Ex: write a procedure to display the customer mobile number and name of the given custid if he is male and from the city delhi?

desc cust_dtls
Name Null Type

CUSTID CHAR(4)
CUSTNAME VARCHAR2(20)
CUSTCITY VARCHAR2(10)
CUSTGENDER CHAR(1)
CUSTMOBILE NUMBER(10)

```
create or replace procedure proc_5(vcid cust_dtls.cno%type) is
    --vmobile cust_dtls.mobile%type;
    vcustname cust_dtls.cname%type;
begin
    select cname into vcustname from cust_dtls where cno=vcid and
gender='M' and
city='Delhi';
    dbms_output.put_line(' give customer id=||vcid);
    dbms_output.put_line(' he is male and he is from delhi');
    --dbms_output.put_line('mobile number is -||vmobile);
    dbms_output.put_line(' His name is '||vcustname);
end proc_5;
```

Ex:
write a procedure to display number of emps working under given department name?

Ex:
write a procedure to display the number of customers from the given city with given gender?

Ex:
write a procedure to display the number of accounts of given account type?

HOW DO I CREATE A RECORD ?

By using 3-methods. (3 types of plsql records)

i) Table based records

It supports all columns from the table.

syn: varname tablename%ROWTYPE;

ii) Cursor Based Records

It supports all columns from the cursor.

syn: varname cursorname%ROWTYPE;

iii) User Defined records:

A record is a collection of columns.

A table based record is a collection of all columns from the table.

A user defined record is a collection of columns specified in the record structure.

It supports the columns specified in the user defined structure.

User can define his required record structure by using the keyword "TYPE".

SYN: creating user defined record structure

```
TYPE <typename> IS RECORD(  
    col1  datatype(size),  
    col2  datatype(size),  
    :      :      :  
    :      :      :  
);
```

Ex:

```
TYPE emprec IS RECORD(  
    empname emp.ename%type,  
    esal    emp.sal%type,  
    ejob    emp.job%type  
);
```

SYNTAX: Declaring variables as user defined record type.

```
varname    <typename>;
```

Ex: r emprec; /* Here Oracle engine reserves only 3 column length memory */

 erec emp%rowtype; /* Here oracle engine reserves the memory required for all columns from the table emp */

Ex:

Write a procedure to display the name,sal,job,hiredate,deptno for the given employee id?

```
create or replace procedure p_emp( vno emp.empno%type)  
is
```

```

type emprec is record(
    name emp.ename%type,
    esal emp.sal%type,
    desg emp.job%type,
    jdate emp.hiredate%type,
    dno emp.deptno%type
);

rec emprec;
begin
select ename,sal,job,hiredate,deptno into rec
from emp where empno=veno;
dbms_output.put_line(' Ename='||rec.name);
dbms_output.put_line(' EmpSal='||rec.esal);
dbms_output.put_line(' Emp Job Title='||rec.desg);
dbms_output.put_line(' Emp join date='||rec.jdate);
dbms_output.put_line(' Emp Dept No='||rec.dno);
end p_emp;

```

Advantages:

- i) It is reducing the number of declarations of variables.
- ii) It is reducing the amount of memory reserved for Record based variable.

Ex: Write a procedure to display the name,sal,job,dno,dname,loc for the given employee id?

```

create or replace procedure p_emp_dept( veno emp.empno%type)
is
type edrec is record(
    name emp.ename%type,
    esal emp.sal%type,
    desg emp.job%type,
    dno emp.deptno%type,
    dname dept.dname%type,
    city dept.loc%type
);

r edrec;          /* User Defined Record Based Variable */
begin
select e.ename,e.sal,e.job,e.deptno,d.dname,d.loc into r
from emp e , dept d where e.empno=veno and e.deptno=d.deptno;
dbms_output.put_line(' Ename='||r.name);
dbms_output.put_line(' EmpSal='||r.esal);
dbms_output.put_line(' Emp Job Title='||r.desg);
dbms_output.put_line(' Emp Dept No='||r.dno);
dbms_output.put_line(' Emp dept name='||r.dname);
dbms_output.put_line(' Emp dept city='||r.city);
end p_emp_dept;

```

Ex:

Write a procedure to display the name, cost and warranty of the product for the given product id?

Ex:

Write a procedure to display pname, cost, mfg, exp, companyName, Dealer address, phonenumber for the given product id?

Ex:

write a procedure to display customer name, actno, act balance and act name for the given customer id?

```
create or replace procedure p_cust_act_info( vcid cust_dtls.cno%type)
is
type custrec is record(
    cname cust_dtls.cname%type,
    actno cust_act_dtls.actno%type,
    actname act_types.act_name%type,
    bal cust_act_dtls.act_bal%type
);

rec custrec;
begin
select cd.cname, cad.actno, at.act_name, cad.act_bal INTO rec
from cust_dtls cd Inner join cust_act_dtls cad
ON cd.cno=cad.cust_code
Inner Join act_types at
ON cad.act_type=at.act_type
WHERE cd.cno=vcid;

dbms_output.put_line(' Cust name='||rec.cname);
dbms_output.put_line(' Cust Act No='||rec.actno);
dbms_output.put_line(' Cust Act Name='||rec.actname);
dbms_output.put_line(' Account Bal='||rec.bal);
end p_cust_act_info;
```

CONDITIONAL STATEMENTS

These are used to execute a set of statements based on condition result. The conditional statements are used to compare values.

1) simple if

It is having only true part. If the condition is true then it executes the stmts.

Syntax:

```
if (condition) then
  stmt;
  stmt;
:
end if;
```

2) if -- then-- else

It is having both true and false parts. If the condition is true then it executes the true part and if the condition was failed then it executes the false part.

Syntax:

```
if (condition) then
  stmt;
  stmt;
:
ELSE
  stmt;
  stmt;
:
end if;
```

3) COMPUND IF

checking if condition with in other if condition.

Syntax:

```
if (cond-1) then
  stmts;
  stmts;
  if (cond-2) then
    stmt;
    stmts;
    :
  else
    stmt;
    :
  end if; --closing of cond-2
else
```

```
stmt;  
:  
end if; --closing of cond-1
```

4) **ELSIF construct**

checking if condition with in else part of other if.

Syntax:

```
if (cond) then  
  stmts;  
  :  
else  
  if ( cond) then  
    stmts;  
    :  
    else  
      stmts;  
      :  
    end if  
  stmts;  
  stmts;  
  :  
end if;  
  
or  
  
if (cond) then  
  stmt;  
  stmt;  
  :  
  ELSIF(cond) then  
    stmt;  
    stmt;  
    else  
      stmts;  
      :  
  END IF;
```

Ex:

Creating a procedure to display biggest of 2 integers with simple if logic?

```
CREATE OR REPLACE PROCEDURE PROC_BIGINT_1( X INT, Y INT)  
IS  
BEGIN  
  DBMS_OUTPUT.PUT_LINE('X='||X||' '||'Y='||Y);  
  IF (X>Y) THEN  
  
    DBMS_OUTPUT.PUT_LINE('FIRST VALUE IS BIG');  
  END IF;  
  DBMS_OUTPUT.PUT_LINE('end of execution');  
END PROC_BIGINT_1;
```

```
EXEC PROC_BIGINT_1(10,20);  
EXEC PROC_BIGINT_1(100,20);
```

Ex:

Creating a procedure to display biggest of 2 integers with if-then-else logic?

```
CREATE OR REPLACE PROCEDURE PROC_BIGINT_2( X INT, Y INT)  
IS  
BEGIN  
DBMS_OUTPUT.PUT_LINE('X='||X||' '||Y='||Y);  
IF (X>Y) THEN  
    DBMS_OUTPUT.PUT_LINE('FIRST VALUE IS BIG');  
else  
    DBMS_OUTPUT.PUT_LINE('Second VALUE IS BIG');  
END IF;  
    DBMS_OUTPUT.PUT_LINE('end of execution');  
END PROC_BIGINT_2;  
  
exec proc_bigint_2(20,10);  
exec proc_bigint_2(100,100);
```

Ex:

Creating a procedure to display biggest of 2 integers with Elself logic?

```
CREATE OR REPLACE PROCEDURE PROC_BIGINT_3( X INT, Y INT)  
IS  
BEGIN  
DBMS_OUTPUT.PUT_LINE('X='||X||' '||Y='||Y);  
IF (X>Y) THEN  
    DBMS_OUTPUT.PUT_LINE('FIRST VALUE IS BIG');  
elseif(x=y) then  
    dbms_output.put_line(' x and y are equal');  
else  
    DBMS_OUTPUT.PUT_LINE('Second VALUE IS BIG');  
END IF;  
    DBMS_OUTPUT.PUT_LINE('end of execution');  
END PROC_BIGINT_3;  
  
exec proc_bigint_3(4,5);  
exec proc_bigint_3(5,5);  
exec proc_bigint_3(14,5);
```

Ex:

Write a program to display biggest of integers if any one given value is null?

```
CREATE OR REPLACE PROCEDURE PROC_BIGINT_4( X INT, Y INT)  
IS
```

```

a int;
b int;
BEGIN
DBMS_OUTPUT.PUT_LINE('X='||X||' '||Y='||Y);
if x is null then
dbms_output.put_line(' Enter a valid value in the First position');
else
a:=x;
end if;
if y is null then
dbms_output.put_line(' Enter a valid value in the second position');
else
b:=y;
end if;

IF (a>b) THEN
    DBMS_OUTPUT.PUT_LINE('FIRST VALUE IS BIG');
elseif(a=b) then
    dbms_output.put_line(' Values are equal');
else
DBMS_OUTPUT.PUT_LINE('Second VALUE IS BIG');
END IF;
    DBMS_OUTPUT.PUT_LINE('end of execution');
END PROC_BIGINT_4;

```

Ex:

write a procedure to find which type of customers(either male customers or female customers) less in count, from the given city?

```

create or replace procedure proc_Gender_cnt
(vcity cust_dtls.city%type) is
    v_mcnt      int;
    v_fcntint;
begin
select count(*) into v_mcnt
from cust_dtls where city=vcity and gender='M';
select count(*) into v_fcnt
from cust_dtls where city=vcity and gender='F';
dbms_output.put_line('Given city Name:- '||vcity);
dbms_output.put_line('Male count:- '||v_mcnt);
dbms_output.put_line('Female count:- '||v_fcnt);
if(v_mcnt < v_fcnt) then
dbms_output.put_line
('Males are less in count than females');
elseif(v_mcnt=v_fcnt) then
dbms_output.put_line
('Males and females are equal in number');
else
dbms_output.put_line
('females are less in count than males');

```

```
end if;  
end proc_gender_cnt;
```

Ex:

write a procedure to find the higher number of customers from the given 2 cities?

Ex:

write a procedure to find the total balance amount in the given account type?

Ex:

Write a procedure to display the number of products from the given 2 company names and find out from which company we have less number of products?

Ex:

Write a procedure to display the number of mobile products from the given 2 company names?

Ex:

Write a procedure to display the total number of customers opted for given account type?

Ex:

write a procedure to display the higher total funds from the given 3 types of accounts?

LOOPS

Loops are used to execute the set of statements repeatedly.

Types: 3

1) simple loop

Syntax:

```
LOOP  
  stmt;  
  stmt;  
  :  
  EXIT WHEN(condition);  
  stmt;  
  stmt;  
  :  
END LOOP;
```

- No condition check before entering in to the loop
- Before exit from the loop it check condition, if the condition is true then loop execution terminates, if the condition fails then loop execution is continue.

2) FOR LOOP

Syntax:

```
FOR<varname> IN [ REVERSE ] range_of_values  
LOOP  
  stmt;  
  stmt;  
  :  
END LOOP;
```

Ex: for x in 1..5
 loop
 dbms_output.put_line(x);
 end loop;

output:

1
2
3
4
5

Ex: for x in REVERSE 1..5
 loop
 dbms_output.put_line(x);
 end loop;

OUTPUT:

5
4
3
2
1

- No condition check before entering in to the loop
- The for loop variable need not be declared.
- In operator takes the values from first value to last value from the range
- Reverse operator enables IN to take the values from last to first
- Range is specified using ..(dot dot) operator.

3) WHILE LOOP

Syntax:

```
WHILE ( cond )  
loop  
  stmt;  
  stmt;  
  :  
end loop;
```

- Condition check before entering into the loop
- If the condition failed then the loop execution terminates.

Ex: write a procedure to display the sequence of values from 1 to 10 using above loops?

create or replace procedure proc_seqnos is

```
x      int:=1;  
begin  
  --x:=1;  
  dbms_output.put_line(' By using  simple loop');  
  dbms_output.put_line('-----');  
  loop  
    dbms_output.put_line(x);  
    x:=x+1;  
    exit when(x > 10);  
  end loop;  
  DBMS_OUTPUT.PUT_LINE(' END  OF SIMPLE LOOP');
```

```

DBMS_OUTPUT.PUT_LINE(' ');

dbms_output.put_line(' By using for loop');
dbms_output.put_line('-----');

for y in 1..10
loop
dbms_output.put_line(y);
end loop;
DBMS_OUTPUT.PUT_LINE(' END  OF FOR LOOP');
DBMS_OUTPUT.PUT_LINE(' ');

```

--Display reverse numbers from 10 to 1 using For Loop

```

dbms_output.put_line(' By using for loop , Displaying 1 to 10 values in
reverse');
dbms_output.put_line('-----');

for y in REVERSE 1..10
loop
dbms_output.put_line(y);
end loop;
DBMS_OUTPUT.PUT_LINE(' END  OF FOR LOOP');
DBMS_OUTPUT.PUT_LINE(' ');

```

```

dbms_output.put_line(' current value of x: ' || x);

```

```

dbms_output.put_line(' By using while loop');
dbms_output.put_line('-----');
x:=1;
dbms_output.put_line(' New value of x: ' || x);
while(x<=10)
loop
dbms_output.put_line(x);
x:=x+1;
end loop;
DBMS_OUTPUT.PUT_LINE(' END  OF WHILE LOOP');
end proc_seqnos;

```

Ex: write a procedure to display sequential even numbers from given number TO GIVEN NUMBER?

```

create or replace procedure proc_even(S INT,e int)
is
r int;
nd int;
ST INT;
begin
ST:=S;
nd:=e;
if(st <= nd) then
dbms_output.put_line(' EVEN NUMBERS FROM '||St||' TO '||nd);

```

```

dbms_output.put_line('*****');
while (st<=nd)
loop
select mod(st,2) into r from dual;
if(r=0) then
dbms_output.put_line('Even-'||st);
end if;
st:=st+1;
end loop;
else
dbms_output.put_line(' Invalid Range ');
end if;
end proc_even;

```

Ex: write a procedure to insert the records like radius, pi value and area values into the table "circle" for the radius range 1 to 10?

```

create table circle
(
radiusnumber(2),
pi    number(3,2),
area  number(6,2)
);

create or replace procedure proc_circle_area is
r    int;
pi    constant number(3,2):=3.14;
a    number(7,2);
begin
r:=1;
for x in 1..10
loop
a:=pi*r*r;
insert into circle values(r,pi,a);
r:=r+1;
end loop;
end proc_circle_area;

```

sample output:

```
exec proc_circle_area;
```

```
select * from circle;
```

```

anonymous block completed
  RADIUS      PI      AREA
-----
      1      3.14      3.14

```

2	3.14	12.56
3	3.14	28.26
4	3.14	50.24
5	3.14	78.5
6	3.14	113.04
7	3.14	153.86
8	3.14	200.96
9	3.14	254.34
10	3.14	314

10 rows selected

Ex: write a procedure to display reverse number for the given number?

```
create or replace procedure proc_revno(n number)
is
rn number;
begin
for x in reverse 1..length(n)
loop
rn:=rn||substr(n,x,1);
end loop;
dbms_output.put_line(' Given number= '||n);
dbms_output.put_line(' Reverse number= '||rn);
end proc_revno;
```

n	1..length(n) rn	x	substr(n,x,1)	rn substr(n,x,1)	
6329	1..4	4	substr(6329,4,1)	9	9
6329	1..3	3	substr(6329,3,1)	9 2	92
6329	1..2	2	substr(6329,2,1)	92 3	
	923				
6329	1..1	1	substr(6329,1,1)	923 6	
	9236				

EX:

--display employee names who is getting the given salary?

```
create or replace procedure proc_ename(vsal number)
is
vename varchar2(10);
begin
select ename into vename from emp where sal=vsal;
dbms_OUTPUT.PUT_LINE(VENAME);
END PROC_ENAME;
```

```
EXEC PROC_ENAME(10000);
```

ERRORS LIKE:

01422. 00000 - "exact fetch returns more than requested number of rows"

01403. 00000 - "no data found"

....

NOTE:

General programs or procedures cannot be able to display multiple records from a table or multiple values from a column.

For Multiple records-----Explicit cursors

For multiple values from a column---Using VARRAYS

EXCEPTIONS

Exception is known as an error which can be handled by the user by displaying user friendly error messages.

Exception types: 2

i) Pre defined exceptions / Named Exceptions

These can be managed by data base engine as follows.

- o While execution if there exists any error then it will look for exception block
- o With in that it will look for corresponding exception handler
- o Then it executes the stmts with in the exception handler.

Exception handlers:

i) TOO_MANY_ROWS

It can be raised when the select stmts is selecting data from multiple records.

ii) NO_DATA_FOUND

It can be raised if there is no matched data from the table.

iii) ZERO_DIVIDE

It can be raised if a value is divided by zero

iv) CURSOR_ALREADY_OPEN

It can be raised if we are trying to open a cursor which is already opened.

v) VALUE_ERROR

It can be raised if there is incompatibility between declared variable and receiving value by that variable.

EXAMPLE PROCEDURES:

Ex:

Write a procedure to display employee names and job for the given salary?

```
CREATE OR REPLACE PROCEDURE PROC_no_EXCEP(VSAL NUMBER)
IS
```

```

VNAME EMP.ENAME%TYPE;
VJOB EMP.JOB%TYPE;
BEGIN
SELECT ename,job INTO vname,vjob from emp where sal=vsal;
dbms_output.put_line('Name of emp:'||vname||'; Desg of emp: '||vjob);
dbms_output.put_line(' End of procedure ');
end proc_no_excep;

SET SERVEROUTPUT ON;

EXEC PROC_NO_EXCEP(5000);

SELECT ENAME ,JOB ,SAL FROM EMP;

EXEC PROC_NO_EXCEP(800);

EXEC PROC_NO_EXCEP(10000);

```

Ex: Above procedure with exceptions

```

CREATE OR REPLACE PROCEDURE PROC_EXCEP(VSAL NUMBER)
IS

VNAME EMP.ENAME%TYPE;
VJOB EMP.JOB%TYPE;

BEGIN

SELECT ename,job INTO vname,vjob from emp where sal=vsal;
dbms_output.put_line('Name of emp: '||vname||' ; Job of emp: '||vjob);
dbms_output.put_line(' End of procedure ');

EXCEPTION
WHEN TOO_MANY_ROWS THEN
DBMS_OUTPUT.PUT_LINE(' SAL IS DUPLICATED');
DBMS_OUTPUT.PUT_LINE(' END OF TOO_MANY_ROWS--Exception');
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('REQUIRED SAL IS NOT AVAILABLE');
DBMS_OUTPUT.PUT_LINE(' END OF NO DATA FOUND');

end proc_excep;

EXEC PROC_EXCEP(800);----Only one employee is getting sal-display
output

EXEC PROC_EXCEP(8000);---No emp getting this salary , so raise an error
no_data_found

```


EXEC PROC_EXCEP(1250);---Multiple emps getting the salary, so raise an error Too_many_rows.

Ex:

Write a procedure to display the employee details who is working with given designation?

```
create or replace procedure proc_emp_dtls_desg(vjob emp.job%type)
is
r_emp emp%rowtype;
begin
select * into r_emp from emp where job=vjob;
dbms_output.put_line(' The given job is '||vjob);
dbms_output.put_line(r_emp.empno||' '||r_emp.ename||' '||r_emp.job||'
'||r_emp.sal||
' '||r_emp.hiredate||' '||r_emp.comm||'
'||r_emp.deptno);
EXCEPTION
WHEN TOO_MANY_ROWS THEN
DBMS_OUTPUT.PUT_LINE(' MULTIPLE EMPS WORKING AS
'||VJOB);
DBMS_OUTPUT.PUT_LINE(' I can display if there exists 1 emp with
the given job,
Otherwise i am unable to display multiple records');
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No EMPS WORKKING AS '||VJOB);
end proc_emp_dtls_desg;
```

sample outputs:

anonymous block completed
MULTIPLE EMPS WORKING LIKE SALESMAN

PROCEDURE PROC_EMP_DTLS_DESG compiled
anonymous block completed
MULTIPLE EMPS WORKING LIKE SALESMAN
I can display if there exists 1 emp with the given job, Otherwise i am unable
to display multiple records

anonymous block completed
The given job is PRESIDENT
7839 KING PRESIDENT 5000 17-NOV-81 2000 10

Ex: write a procedure to display customer account detials for the given customer id?

If customer have multiple account or no account or no custid then display corresponding message?

2) User defined exceptions / Unnamed Exceptions

Here user has to manage the exceptions. User has to RAISE the exception if he is expecting any error. The exception is to be raised by using exception variable name. The exception variable acts as an EXCEPTION HANDLER in an exception block.

i) declare exception type of variable in declare block

syn: varname exception;

Ex: sal_miss exception;
comm_miss exception;
no_exp exception;

ii) Raise the exception using the keyword RAISE in begin block

syn: RAISE < exptype variable name >;

Ex: RAISE COMM_MISS;

iii) Define the exception in exception block

syn: WHEN < exp handler > THEN
stmt;
stmts;
:

ex: when comm_miss then
dbms_output.put_line('comm value is null');

Ex: Write a program to display commission of employee for the given employee id?

```
create or replace procedure proc_e_comm(veno emp.empno%type) is
vcomm int;
begin
select comm into vcomm from emp where empno=veno;
dbms_output.put_line('comm of '||veno||' is '||vcomm);
exception
when too_many_rows then
dbms_output.put_line(' Duplicate emp id existed ');
when no_data_found then
dbms_output.put_line(' Emp id not existed ');
end proc_e_comm;
```

Ex: write a procedure to display the commission of the employee for the given employee id?

```

        create or replace procedure proc_emp_comm( veno
emp.empno%type)
    is
        vcomm      emp.comm%type;
        comm_miss exception; /* Declaring Exception type variable , Then is
acting as exception Handler.*/
    begin          /* we can define exception handler in exception
block */
        select comm into vcomm from emp where empno=veno;

        if vcomm is null then
            RAISE comm_miss;
        else
            dbms_output.put_line(' comm of '||veno||' is '||vcomm);
        end if;
    exception
    when no_data_found then
        dbms_output.put_line(' No employee with id'||veno);
    when Too_many_rows then
        dbms_output.put_line(' Duplicate empid existed');
    when comm_miss then
        dbms_output.put_line(' Employee not getting commission ');
    end proc_emp_comm;

```

sample output:

```

PROCEDURE PROC_EMP_COMM compiled
anonymous block completed
comm of 7654 is 1400

```

```

anonymous block completed
comm of 7788 is 2000

```

```

anonymous block completed
comm of employee is null

```

```

anonymous block completed
No employee with given id

```

Ex: Write a procedure to display employee information , if any column value is null display corresponding message for the given employee id?

```

create or replace procedure proc_e1_comm1( veno emp.empno%type)
    is
        r emp%rowtype;
    begin
        select * into r from emp where empno=veno;
        dbms_output.put_line(r.ename||';'||r.sal||';'||r.job||';'||r.hiredate);
        if r.mgr is null then
            dbms_output.PUt_line(' Mgr code is null for empid '||veno);
        else
            dbms_output.put_line(' mgr code is '||r.mgr);

```

```

end if;
if r.comm is null then
dbms_output.PUT_line(' comm is null for empid '||veno);
else
dbms_output.put_line(' comm is '||r.comm);
end if;
if r.deptno is null then
dbms_output.PUT_line(' deptno is null for empid '||veno);
else
dbms_output.put_line(' deptno is '||r.deptno);
end if;
exception
when no_data_found then
dbms_output.put_line(' No employee with id'||veno);
when Too_many_rows then
dbms_output.put_line(' Duplicate empid existed');
end proc_e1_comm1;

```

ex: write a procedure to display the mfg, exp and warrenty of the given product id?
If any one is null then display corresponding message?

Ex: write a procedure to display the account number, account type, account balance, account open date for given customer id?

User Named Exception PRAGMA EXCEPTION_INIT,
RAISE_APPLICATION_ERROR

PL/SQL pragma EXCEPTION_INIT

pragma EXCEPTION_INIT : Pragma is a keyword directive to execute proceed at compile time. pragma EXCEPTION_INIT function take this two argument,

1. **exception_name**
2. **error_number**

You can define pragma EXCEPTION_INIT in DECLARE BLOCK on your program.

```
PRAGMA EXCEPTION_INIT(exception_name, -error_number);
```

exception_name and *error_number* define on yourself, where exception_name is character string up to 2048 bytes support and error_number is a negative integer range from -20000 to -20999.

Syntax

```
DECLARE
    user_define_exception_name EXCEPTION;
    PRAGMA EXCEPTION_INIT(user_define_exception_name, -
error_number);
BEGIN
    statement(s);
    IF condition THEN
        RAISE user_define_exception_name;
    ENDIF;
EXCEPTION
    WHEN user_define_exception_name THEN
        User defined statement (action) will be taken;
END;
```

Example Code

user-named_exp.sql

```
SQL>edit user-named_exp
DECLARE
    myex EXCEPTION;
    PRAGMA EXCEPTION_INIT(myex, -20015);
    n NUMBER :=&n;
BEGIN
    FOR i IN 1..n LOOP
        dbms_output.put.line(i);
        IF i=n THEN
            RAISE myex;
        ENDIF;
    ENDLOOP;
EXCEPTION
    WHEN myex THEN
        dbms_output.put.line('loop finish');
END;
/
```

Example Result

```
SQL>@user-named_exp
```

```
n number &n= 5
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
loop finish
```

PL/SQL procedure successfully operation.

RAISE_APPLICATION_ERROR

In PL/SQL RAISE_APPLICATION_ERROR function use to assign exception name and exception error code. Define

RAISE_APPLICATION_ERROR function syntax,

```
raise_application_error(error_number, error_message);
```

Example Code

raise_app_error.sql

```
SQL>edit user-named_exp
```

```
DECLARE
```

```
    myex EXCEPTION;
```

```
    n NUMBER :=&n;
```

```
BEGIN
```

```
    FOR i IN 1..n LOOP
```

```
        dbms_output.put_line(i);
```

```
        IF i=n THEN
```

```
            RAISE myex;
```

```
        ENDIF;
```

```
    ENDLOOP;
```

```
EXCEPTION
```

```
    WHEN myex THEN
```

```
RAISE_APPLICATION_ERROR(-20015,'loop finish');  
END;  
/
```

CURSORS

QUERY EXECUTION PROCESS:

query-----> sql stmt executor--> Oracle engine--> oracle DB--> DD-->actual data-->
-----> temp memory area (CURSOR)--->cache memory-----> CLIENT
[from view]

In the above the required data can be processed in the temporary memory area (cursor), and then displayed to the client . This is the case if we are selecting the data from the table.

After cursor memory , it maintains a copy of data in cache, if we are selecting the data from view.

CURSORS

It is a temp memory area, is used to perform intermediate operations before output displayed to the client.

Types of cursors: 2

1) Implicit cursors

A cursor which is used by sql query. This type can be assigned and managed by DB engine.

Properties:

SQL%ISOPEN It returns true if the file is opened for processing

SQL%FOUND It returns true if the matched data found

SQL%NOTFOUND It returns true if there is no matched data

SQL%ROWCOUNT It returns an integer value which represents the number of affected records.

2) Explicit cursors

These are declared and defined and managed by user. Generally we can use explicit cursors to display multiple records from the table through programs or procedures.

This process consists the following steps.

--declare and define the cursor in DECLARE block

SYN: CURSOR <cursor_name> IS SELECT from where....order by.....;

Ex: cursor c_mgr is select * from emp where job='MANAGER';

--Open the cursor for processing in BEGIN block

syn: OPEN <cur_name>;

Ex: OPEN c_mgr;

--Fetch data from the cursor in the begin block

syn: FETCH <cur_name> INTO <varname>;

This fetch operation should be done repeatedly until all records are processed in, the cursor.

Ex: LOOP
 FETCH c_mgr INTO REC;
 stmt;
 stmt;
 EXIT WHEN (c_mgr%NOTFOUND);
 stmt;
 stmt;
END LOOP;

--Close the cursor , to release the memory occupied by the cursor.

syn: CLOSE <cur_name>;

Ex: close c_mgr;

Explicit cursor properties

<cur_name>%ISOPEN It returns true if the file is opened for processing

<cur_name>%FOUND It returns true if there exists matched data

<cur_name>%NOTFOUND It returns true if there is no matched data

<cur_name>%ROWCOUNT It returns an integer value represents number of effected records.

Ex:

write a procedure to find the number of records affected if the emps are updated for the given job category with 10% of increment in their salary with in the table emp_copy?

```
create or replace procedure proc_update( vjob emp_copy.job%type)
is
begin
update emp_copy set sal=sal+(0.10*sal) where job=vjob;
if (sql%found) then
dbms_output.put_line(' updation is success');
dbms_output.put_line(' number of records updated='||sql%rowcount);
else
dbms_output.put_line('updation not performed');
dbms_output.put_line(' number of records updated='||sql%rowcount);
end if;
end proc_update;
```

Ex: write a procedure to display the employee details from given deptno?

/* Procedure with Explicit Cursor */

```
create or replace procedure proc_emp_dtls_dno (vdno
emp.deptno%type)
is
cursor c is select * from emp where deptno=vdno;
e_rec emp%rowtype;
begin
open c;
dbms_output.put_line(' Details of emps under the deptno: '||vdno);
dbms_output.put_line('-----');
loop
fetch c into e_rec;
exit when c%notfound;
dbms_output.put_line(e_rec.empno||';'||e_rec.ename||';'||e_rec.sal||';'||
e_rec.job||';'||e_rec.deptno);
end loop;
close c;
end proc_emp_dtls_dno;
```

CURSOR BASED RECORD VARIABLE

If we are fetching data from multiple tables multiple records, then we need to declare cursor based record variable . It supports all columns from the cursor.

syn: varname <cursor_name>%rowtype;

Ex: write a procedure to display the employee details along with his dept info from given deptno?

```
/* Procedure with Explicit Cursor */

create or replace procedure proc_emp_dept_dtls_dno (vdno
emp.deptno%type)
is
cursor c is select e.empno,e.ename,e.sal,e.job,d.deptno,d.dname,d.loc
from emp e, dept d
where e.deptno=vdno and e.deptno=d.deptno;

/* DECLARING CURSOR BASED RECORD type variable, Which
SUPPORTS ALL COLUMNS FROM THE CURSOR */

/* IT IS REQUIRED IF WE ARE SELECTING DATA FROM MULTIPLE
TABLES IN TO THE CURSOR */

/* Declaring Cursor type variable--> varname
cur_name%rowtype; */

c_rec c%rowtype;
begin
open c;
dbms_output.put_line(' Details of emps under the deptno: '||vdno);
dbms_output.put_line(' -----');
loop
fetch c into c_rec;
exit when c%notfound;
dbms_output.put_line(c_rec.empno||';'||c_rec.ename||';'||c_rec.sal||';'||
c_rec.job||';'||c_rec.deptno||';'||c_rec.dname||';'||c_rec.loc);
end loop;
close c;
end proc_emp_dept_dtls_dno;
```

Ex: write a procedure to display the employee details of given job?

```
create or replace procedure proc_emp_info_with_givenjob( vjob
emp.job%type)
is
cursor c_emp is select * from emp where job=vjob;
r emp%rowtype;

begin
open c_emp;
dbms_output.put_line(' the given job title: '||vjob);
dbms_output.put_line('all '||vjob||' information');
LOOP
FETCH c_emp INTO r;
exit when (c_emp%notfound);
dbms_output.put_line(r.empno||' '||r.ename||' '||r.job||' '||r.sal||'
'||r.hiredate||' '||r.deptno);
```

```

end loop;
close c_emp;
end proc_emp_info_with_givenjob;

```

sample output:

```

PROCEDURE PROC_EMP_INFO_GIVENJOB compiled
anonymous block completed
the given job title:  MANAGER
all  MANAGER information
7566 JONES MANAGER 3272.5 02-APR-81 20
7698 BLAKE MANAGER 3135 01-MAY-81 30
7782 CLARK MANAGER 2695 09-JUN-81 10

```

```

PROCEDURE PROC_EMP_INFO_GIVENJOB compiled
anonymous block completed
the given job title:  SALESMAN
all  SALESMAN information
7499 ALLEN SALESMAN 1600 20-FEB-81 30
7521 WARD SALESMAN 1250 22-FEB-81 30
7654 MARTIN SALESMAN 1250 28-SEP-81 30
7844 TURNER SALESMAN 1500 08-SEP-81 30

```

Ex: write a procedure to display the employee names and salaries, their department names and locations?

```

create or replace procedure proc_emp_dept is
cursor c1 is select e.ename,e.sal,d.dname,d.loc from emp e,dept d
where e.deptno=d.deptno;
cur_var c1%rowtype; /* DECLARING CURSOR TYPE VARIABLE */
begin
open c1;
dbms_output.put_line('EMP-NAME      EMP-SALARY
Department name      department-Location');
dbms_output.put_line('=====
=====');
LOOP
FETCH c1 INTO cur_var;
EXIT WHEN (c1%notfound);
dbms_output.put_line(cur_var.ename||'          '||cur_var.sal||'
'||cur_var.dname||
',          '||cur_var.loc);

end loop;
close c1;
end proc_emp_dept;

```

NOTE:

cursor type variable will supports all columns with in the cursor. No matter the columns are from which table.

sample output:

EMP-NAME =====	EMP-SALARY =====	Department name =====	department-Location =====
SMITH	800	RESEARCH	DALLAS
ALLEN	1600	SALES	CHICAGO
WARD	1250	SALES	CHICAGO
JONES	2975	RESEARCH	DALLAS
MARTIN	1250	SALES	CHICAGO
BLAKE	2850	SALES	CHICAGO
CLARK	2450	ACCOUNTING	NEW YORK
SCOTT	3000	RESEARCH	DALLAS
KING	5000	ACCOUNTING	NEW YORK
TURNER	1500	SALES	CHICAGO
ADAMS	1100	RESEARCH	DALLAS
JAMES	950	SALES	CHICAGO
FORD	3000	RESEARCH	DALLAS
MILLER	1300	ACCOUNTING	NEW YORK
dinesh	3400	OPERATIONS	BOSTON

--Write a procedure to display customer name,city, actno, balance,act_name of all customers?

create or replace procedure proc_cust_act_info
is

cursor c_acts is select cd.cname,cd.city,cad.actno,cad.act_bal,at.act_name
from cust_dtls cd, cust_act_dtls cad, act_types at
where cd.cno=cad.cust_code and cad.act_type=at.act_type;

rec c_acts%rowtype; /* Declaring a cursor based variable , supports all
columns from cursor */

begin

open c_acts;

dbms_output.put_line(' Customers and their Accounts information ');

dbms_output.put_line('CustName---custCity--- Account No: Account Bal
Account Name');

dbms_output.put_line('-----');

loop

fetch c_acts into rec;

EXIT when (c_acts%notfound);

dbms_output.put_line(rec.cname||'---'||rec.city||'---'||rec.actno||'

'||rec.act_bal||' '||rec.act_name);

end loop;

close c_acts;

end proc_cust_act_info;

exec proc_cust_act_info;

Using Ref Cursors

The example below uses a ref cursor to return a subset of the records in the EMP table.

The following procedure opens a query using a SYS_REFCURSOR output parameter. Notice the cursor is not closed in the procedure. It is up to the calling code to manage the cursor once it has been opened.

```
CREATE OR REPLACE
PROCEDURE get_emp_rs (p_deptno      IN  emp.deptno%TYPE,
                     p_recordset OUT SYS_REFCURSOR) AS
BEGIN
    OPEN p_recordset FOR
        SELECT ename,
               empno,
               deptno
        FROM   emp
        WHERE  deptno = p_deptno
        ORDER BY ename;
END GetEmpRS;
/
```

The resulting cursor can be referenced from PL/SQL as follows.

```
SET SERVEROUTPUT ON SIZE 1000000
DECLARE
    l_cursor  SYS_REFCURSOR;
    l_ename   emp.ename%TYPE;
    l_empno   emp.empno%TYPE;
    l_deptno  emp.deptno%TYPE;
BEGIN
    get_emp_rs (p_deptno      => 30,
                p_recordset => l_cursor);
```

```
LOOP
    FETCH l_cursor
    INTO  l_ename, l_empno, l_deptno;
    EXIT WHEN l_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(l_ename || ' | ' || l_empno || ' | '
|| l_deptno);
END LOOP;
CLOSE l_cursor;
END;
```

FUNCTIONS

It is database object which is used to perform a task and it should return a value to the calling program.

- Functions are stored as pre compiled object.
- Functions are also known as sub programs.
- Functions are calling from other procedure or functions or by using select stmts.
- Functions are may or may not take the arguments
- FUNCTIONS ARE IMPLICITLY EXECUTED.
- FUNCTIONS SHOULD return a value to the calling program.

Function also contains 2 parts. i.e Specification and Body like procedure.

Syntax

```
create or replace function<func_name> ( argvar datatype, argvar  
datatype..... )  
RETURN<datatype>  
IS
```

```
<declaration stmts>;  
begin  
<assignment stmts>;  
<data processing stmts>;  
<output stmts>;  
Exception  
<error handling stmts>;  
.....  
RETURN(value/var_name);  
END<func_name>;
```

```
Ex:  create or replace function f_add( x int, y int)  
      return number  
      is  
      res int;  
      begin  
      res:=x+y;  
      return(res);  
      end f_add;
```

Note:

A Function can be called through any SELECT or PROGRAM or Procedure.

```
Ex:  select func_add(100,200) from dual;  
      300
```


Ex:

CALLING FUNCTION BY USING PROCEDURE:-

```
create or replace procedure p_add( a int, b int)
is
    res int;
begin
    res:=f_add(a,b);
    dbms_output.put_line(' addition='||res);
end p_add;
```

```
exec p_add(20,30);
```

```
select f_add(100,400) from dual;
```

```
declare
s1 int;
s2 int;
r int;
begin
s1:=&s1;
s2:=&s2;
    dbms_output.put_line(' calling function from a procedure through
program');
    dbms_output.put_line(' ');
    p_add(s1,s2); /* Procedure calling stmt */
    dbms_output.put_line(' completed');
    dbms_output.put_line('-----');
    dbms_output.put_line(' calling function from program directly');
    r:=f_add(s1,s2); /* function calling stmt */
    dbms_output.put_line(' result='||r);
    dbms_output.put_line(' completed');
end;
```

Example:

```
create or replace function f_add( x int, y int)
return number
is
    res int;
begin
    res:=x+y;
    return(res);
end f_add;
```

```
/* calling function using SELECT */
```

```
select f_add(32,23) from dual;
```

```

/* calling function using PROCEDURE */

create or replace procedure proc_fsampl(x int, y int) is
r int;
begin
r:=f_add(x,y);
dbms_output.put_line(' output from function is '||r);
end proc_fsampl;

exec proc_fsampl(10,20);

/* calling function using PROGRAM OR ANONYMOUS BLOCK */

declare
a int;
b int;
output int;
begin
a:=&a;
b:=&b;
output:=f_add(a,b);
dbms_output.put_line(' result from function='||output);
end;

```

Ex: write procedure to display esal,bonus value and final salary (basic sal+ bonus)
for the given employee id?

Ex: Do the above for all employees?

Ex: Do the above for all given job category of emps?

[Common Functionality is " Finding Bonus Based on salary Range"]

Bonus is to be calculated as follows

sal<1000	2%
>=1000 &< 2000	5%
>=2000 &<3000	10%
>=3000 &<5000	20%
>=5000	25%

*** Creating a function to calculate the Bonus?

```

create or replace function func_bonus(s emp.sal%type)
return number
is
bonusnumber(7,2);
begin
if (s <1000) then

```

```

bonus:=0.02*s;
end if;
if (s>=1000 and s<2000) then
bonus:=0.05*s;
end if;
if (s>=2000 and s<3000) then
bonus:=0.10*s;
end if;
if (s>=3000 and s<5000) then
bonus:=0.20*s;
end if;
if (s>=5000) then
bonus:=0.25*s;
end if;
return(bonus);
end func_bonus;

```

```

create or replace procedure proc_emp_fsal(veno emp.empno%type)
is
vsal emp.sal%type;
b number(7,2);
fsal number(7,2);
begin
select sal into vsal from emp where empno=veno;
b:=func_bonus(vsal); /* function calling stmt */
fsal:=vsal+b;
dbms_output.put_line(' The salary details of '||veno);
dbms_output.put_line('-----');
dbms_output.put_line(' salary      bonus      final salary');
dbms_output.put_line(' vsal||'      ||b||'      ||fsal);
end proc_emp_fsal;

```

Ex: Perform above for all emps:

```

create or replace procedure proc_emps_fsal
is
cursor c is select empno,sal from emp;
veno emp.empno%type;
vsal emp.sal%type;
b number(7,2);
fsal number(7,2);
begin
open c;
dbms_output.put_line('veno salary      bonus      final salary');
dbms_output.put_line('-----');
loop
fetch c into veno,vsal;

```

```

b:=func_bonus(vsal); /* function calling stmt */
fsal:=vsal+b;
exit when (c%notfound);
dbms_output.put_line( 'veno ||' '||vsal||'          '||b||'          '||fsal);
end loop;
close c;
end proc_emps_fsal;

```

Ex: write a procedure to display the above output for all emps with in given job category?

```

create or replace procedure proc_emps_fsal_job(vjob emp.job%type)
is
cursor c is select empno,sal from emp where job=vjob;
veno emp.empno%type;
vsal emp.sal%type;
b      number(7,2);
fsal   number(7,2);
begin
open c;
dbms_output.put_line('veno      salary          bonus          final salary');
dbms_output.put_line('-----');
loop
fetch c into veno,vsal;
b:=func_bonus(vsal); /* function calling stmt */
fsal:=vsal+b;
exit when (c%notfound);
dbms_output.put_line( 'veno ||' '||vsal||'          '||b||'          '||fsal);
end loop;
close c;
end proc_emps_fsal_job;

```

Ex: write a procedure and functions to display the productname, cost, discount, final price for the given product id?

Ex: write a procedure and functions to display the productname, cost, discount, final price for all products?

Ex: write a procedure and functions to display the productname, cost, discount, final price for the given category of products?

Discount calculation is as follows

cost< 5000	no discount
>=5000 &< 10000	5%
>=10000 &< 20000	10%
>=20000 &< 40000	20%
>40000	30%

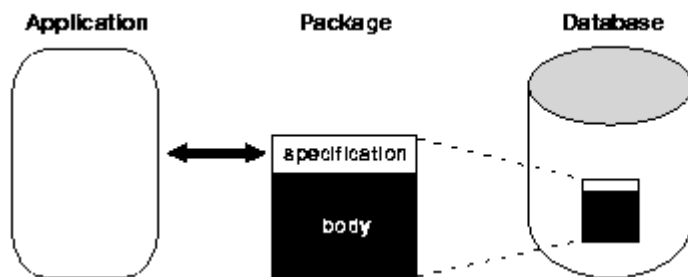
How do i delete function?

```
drop function <func_name>;
```

PACKAGES

It is a data base object which logically groups the related objects. It reduces the search time for the required object according to required action.

Package Interface



It contains 2 parts

1) package specification:

It contains function calling stmts and procedure calling stmts , global variable declarations.

syn-1:package specification

The standard package specification has this form:

```
CREATE OR REPLACE PACKAGE package_name AS
type definitions for records, index-by tables
constants
exceptions
global variable declarations
    procedure procedure_1(arg1, ...);
    ...
    function function_1(arg1,...) return datat_ype;
    ...
END package_name;
```

The **package body** contains the code that implements these subprograms, the code for all private subprograms that can only be invoked from within the package, and the queries for the cursors. You can change the implementation details inside the package body without invalidating the calling applications.

The package body has this form:

```

CREATE OR REPLACE PACKAGE BODY package_name AS
  PROCEDURE procedure_1(arg1,...) IS
    BEGIN
      ...
    EXCEPTION
      ...
    END procedure_1;
  ...
  FUNCTION function_1(arg1,...) RETURN data_type IS
result_variabledata_type
    BEGIN
      ...
      RETURN result_variable;
    EXCEPTION
      ...
    END function_1;
  ...
END package_name;

```

Ex: write a package which contains the procedures and functions to calculate sal, bonus, final salaries for given empid , for all emps and for given job category employees?

```

CREATE OR REPLACE PACKAGE PKG_Emp_FSAL
AS
  PROCEDURE PROC_EMP_FSAL(VENO EMP.EMPNO%TYPE);
  PROCEDURE PROC_EMPs_Fsal;
  PROCEDURE PROC_EMPs_FSAL_JOB( vjob emp.job%type);
  FUNCTION FUNC_bonus(S EMP.SAL%TYPE)
  RETURN NUMBER;
END PKG_EMP_FSAL;

/

package created.

```

EX:package body

```

CREATE OR REPLACE PACKAGE BODY PKG_EMP_FSAL IS

  /* PROC_EMP_FSAL */

  Procedure proc_emp_fsal(veno emp.empno%type)
  is
    vsal emp.sal%type;
    b number(7,2);
    fsal number(7,2);
  begin
    select sal into vsal from emp where empno=veno;

```

```

        b:=func_bonus(vsal); /* function calling stmt */
        fsal:=vsal+b;
        dbms_output.put_line(' The salary details of '||vno);
        dbms_output.put_line('-----');
        dbms_output.put_line
(' salary          bonus          final salary');
        dbms_output.put_line
( vsal||'          '||b||'          '||fsal);
        end proc_emp_fsal;

/* function to calcualte the BONUS */

Function func_bonus(s emp.sal%type)
return number
is
bonusnumber(7,2);
begin
if (s <1000) then
bonus:=0.02*s;
end if;
if (s>=1000 and s<2000) then
bonus:=0.05*s;
end if;
if (s>=2000 and s<3000) then
bonus:=0.10*s;
end if;
if (s>=3000 and s<5000) then
bonus:=0.20*s;
end if;
if (s>=5000) then
bonus:=0.25*s;
end if;
return(bonus);
end func_bonus;

/* Procedure to display the sal info of all emps */

Procedure proc_emps_fsal
is
cursor c is select empno,sal from emp;
vno emp.empno%type;
vsal emp.sal%type;
b number(7,2);
fsal number(7,2);
begin
open c;

        dbms_output.put_line('-----');
        dbms_output.put_line
(' salary          bonus          final salary');
        dbms_output.put_line('*****');

        loop
        fetch c into vno,vsal;

```



```

        dbms_output.put_line(' The salary details of '||vno);
        b:=func_bonus(vsal); /* function calling stmt */
        fsal:=vsal+b;
        EXIT WHEN (C%NOTFOUND);
        dbms_output.put_line
( vsal||'          '||b||'          '||fsal);
        END LOOP;
        CLOSE C;
        end proc_emps_fsal;

        /* procedure  proc_emps_fsal_job */

        procedure proc_emps_fsal_job(vjob emp.job%type) is
        cursor c is select empno,sal from emp where job=vjob;
        vsal  emp.sal%type;
        vno  emp.empno%type;
        incrval  number(7,2);
        fsal  number(7,2);
        begin
        open c;
        loop
        fetch c into vno,vsal;
        incrval:=func_bonus(vsal); /* function calling stmt */
        fsal:=vsal+incrval;

        EXIT WHEN c%NOTFOUND;
        dbms_output.put_line(' sal,incrval,final salary of '||vno);
        dbms_output.put_line(vsal||' , '||incrval||' , '||fsal);
        end loop;
        close c;
        end proc_emps_fsal_job;

        END PKG_EMP_FSAL;

/

```

HOW TO EXECUTE A PROCEDURE FROM A PACKAGE?

syn: exec package_name.proc_name(argval, argval.....);

Ex: exec pkg_emp_fsal.proc_emp_fsal(7654);

How to drop a package?

drop package <pkg_name>;

Ex: write a procedure to display the final product cost

- a) for the given product-id,
- b) for all products

c) given category of products ?

The discount is applied based on following ranges?

<5000	5%
>=5000 & <10000	10%
>=10000 & <20000	15%
>=20000	20%

Ex: create a package for the above?

Ex: write a procedure to display the final amount paid to the customer by adding interest amount for the principal amount?

Do the above process for given customer id?

Do it for all customers?

Do it for given type of account holders?

The interest is calculated based on the following?

<1000	no interest
>=1000 & < 25000	7.5%
>=25000 & < 50000	8.00%
>=50000 & < 100000	9.00%
>=100000 & <=600000	10.00%
>600000	Not applicable

COLLECTIONS

An Oracle PL/SQL collection is a single-dimensional array; it consists of one or more elements accessible through an index value.

Collections are used in some of the most important performance optimization features of PL/SQL, such as

- **BULK COLLECT.** SELECT statements that retrieve multiple rows with a single fetch, increasing the speed of data retrieval.
 - **FORALL.** Inserts, updates, and deletes that use collections to change multiple rows of data very quickly
 - **Table functions.** PL/SQL functions that return collections and can be called in the FROM clause of a SELECT statement.
- You can also use collections to work with lists of data in your program that are not stored in database tables.

This article introduces you to collections and gives you a solid foundation in both collection syntax and features.

Collection Concepts and Terminology

Before exploring collections, it is helpful to have a common collections vocabulary that includes the following terms.

Index value. The location of the data in a collection. Index values are usually integers but for one type of collection can also be strings.

Element. The data stored at a specific index value in a collection. Elements in a collection are always of the same type (all of them are strings, dates, or records). PL/SQL collections are *homogeneous*.

Sparse. A collection is sparse if there is at least one index value between the lowest and highest defined index values that is not defined. For example, a sparse collection has an element assigned to index value 1 and another to index value 10 but nothing in between. The opposite of a sparse collection is a *dense* one.

Method. A collection method is a procedure or function that either provides information about the collection or changes the contents of the collection. Methods are attached to the collection variable with dot notation (object-oriented syntax), as in *my_collection.FIRST*.

Types of Collections

Collections were first introduced in Oracle7 Server and have been enhanced in several ways through the years and across Oracle Database versions. There are now three types of collections to choose from, each with its own set of characteristics and each best suited to a different circumstance.

Associative array. The first type of collection available in PL/SQL, this was originally called a “PL/SQL table” and can be used only in PL/SQL blocks. Associative arrays can be sparse or dense and can be indexed by integer or string.

Nested table. Added in Oracle8 Database, the nested table can be used in PL/SQL blocks, in SQL statements, and as the datatype of columns in tables. Nested tables can be sparse but are almost always dense. They can be indexed only by integer. You can use the MULTISSET operator to perform set operations and to perform equality comparisons on nested tables.

Varray. Added in Oracle8 Database, the varray (variable-size array) can be used in PL/SQL blocks, in SQL statements, and as the datatype of columns in tables. Varrays are always dense and indexed by integer. When a varray type is defined, you must specify the maximum number of elements allowed in a collection declared with that type.

You will rarely encounter a need for a varray (How many times do you know in advance the *maximum* number of elements you will define in your collection?). The associative array is the most commonly used collection type, but nested tables have some powerful, unique features (such as MULTISSET operators) that can simplify the code you need to write to use your collection.

Collection means a group of values.

If a variable is able to store group of values at a time then it is known as Collection Type variable.

Ex: Table based record variable.
 User defined record variable.
 Array type variable.

TYPES OF ARRAYS:

VARRAYS [Variable sized Array]

ASSOCIATIVE ARRAYS [No size specification]

NESTED TABLES [Multidimensional Array]

VARRAYS--Single Dimensional Array

Variable sized array. Array is able to store multiple values with same data type at continuous memory locations. Here we must specify the size. and the specified size is fixed.

Defining array strucutre:

TYPE <typename> IS VARRAY(size) OF <Array_element_Data_type>;

Ex: type empsal is varray(20) of number;

or

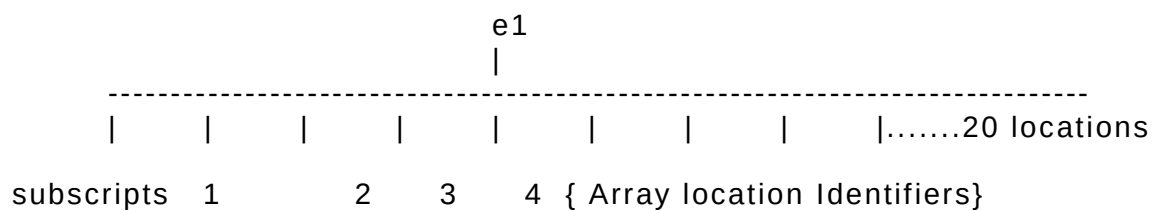
```
type empsal is varray(20) of emp.sal%type;
```

How do i declare array type variable?

```
VARNAME      Array_type_name;
```

```
Ex:  e1      empsal;  
     e2      empsal;
```

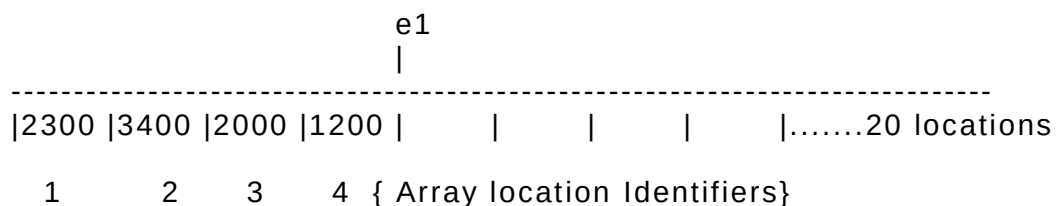
For the Array variable e1 , the memory assigned is as follows.
Each memory location is identified with SUBSCRIPTS value. Generally
subscripts are integers beginning with 1,....



How do i assign values into array type variable?

```
syn:      array_var_name  := array_name(val1,val2,.....);
```

```
Ex:      e1:=empsal(2300,3400,2000,1200,.....);
```



Accessing / displaying values from Array type variable:

```
syn:  Array_var_name(subscriptNumber)
```

```
Ex:
```

```
e1(3)
```

e1(1)

By using any loop:

```
declare
x      int:=1;

for i in e1.first..e1.last
loop
dbms_output.put_line(e1(x));
x:=x+1;
end loop;
```

Methods related with arrays:

COUNT--Holds number of array locations

FIRST--takes value from first location

LAST--indicates last location of array

NEXT--indicates next location

PRIOR--Indicates previous location

Ex: Display employee names and salaries using VARRAY?

```
declare
type esal is varray(5) of number;
type enames is varray(5) of varchar(10);
vsal esal;
vename enames;
x      int:=1;
begin

vsal:=esal(1250,4500,2300,7600,1000);
vename:=enames('Ajay','Bhagath','Xavier','SMITH','SCOTT');

/* displaying data from array type variable */

for a in VSAL.FIRST..VSAL.LAST --[ 1..VSAL.COUNT ]
loop
dbms_output.put_line(vename(x)||' is getting sal: '||Vsal(x));
x:=x+1;
end loop;
dbms_output.put_line(' end of program');
end;
```

Note:

In the above examples,

esal.FIRST .. esal.LAST --Represents from the first location of vsal to the last location of vsal.

The following are methods used along with Array type variables.

Collection Methods

A variety of methods exist for collections, but not all are relevant for every collection type.

- EXISTS(n) - Returns TRUE if the specified element exists.
- COUNT - Returns the number of elements in the collection.
- LIMIT - Returns the maximum number of elements for a VARRAY, or NULL for nested tables.
- FIRST - Returns the index of the first element in the collection.
- LAST - Returns the index of the last element in the collection.
- PRIOR(n) - Returns the index of the element prior to the specified element.
- NEXT(n) - Returns the index of the next element after the specified element.
- EXTEND - Appends a single null element to the collection.
- EXTEND(n) - Appends n null elements to the collection.
- EXTEND(n1,n2) - Appends n1 copies of the n2th element to the collection.
- TRIM - Removes a single element from the end of the collection.
- TRIM(n) - Removes n elements from the end of the collection.
- DELETE - Removes all elements from the collection.
- DELETE(n) - Removes element n from the collection.
- DELETE(n1,n2) - Removes all elements from n1 to n2 from the collection.

MULTISET Conditions

Oracle provides MULTISET conditions against collections, including the following.

MULTISET UNION joins the two collections together, doing the equivalent of a UNION ALL between the two sets.

```
SET SERVEROUTPUT ON
DECLARE
  TYPE t_tab IS TABLE OF NUMBER;
  l_tab1 t_tab := t_tab(1,2,3,4,5,6);
  l_tab2 t_tab := t_tab(5,6,7,8,9,10);
BEGIN
  l_tab1 := l_tab1 MULTISET UNION l_tab2;

  FOR i IN l_tab1.first .. l_tab1.last LOOP
    DBMS_OUTPUT.put_line(l_tab1(i));
  END LOOP;
END;
```

```
/
1
2
3
4
5
6
5
6
7
8
9
10
```

PL/SQL procedure successfully completed.

SQL>

BULK COLLECT

It improves the performance of database. It eliminates the context switching between program area and cursor area.

By using this we can take all values from the cursor at a time and control coming to program area and save that collected values in to array type variable.

Ex:

Write a procedure to Display all employee names with out BULK COLLECT and varrays?

```
create or replace procedure proc_enames
is
cursor c is select ename from emp;
vname emp.ename%type;
begin
open c;
dbms_output.put_line(' All Employee Names ');
dbms_output.put_line('=====');
loop
```



```

fetch c into vename;
exit when c%notfound;
dbms_output.put_line(vename);
end loop;
close c;
end proc_enames;

```

Ex: write a procedure to display employee names from emp using varrays and bulk collect?

BULK COLLECT:

It reduces number of context switches from program Memory to cursor Memory and vice versa. There by it is Increasing DB performance.

Ex: write a procedure to display employee names with BULK COLLECT

create or replace procedure proc_names_sal is

/* using explicit cursors */

cursor c is select ename,sal from emp;

/* creating array type structure for storing employee names */

type enames is varray(25) of emp.ename%type;

/* creating array type structure for storing employee salaries */

type esal is varray(25) of emp.sal%type;

vename enames;

vsal esal;

x int:=1;

begin

open c;

fetch c bulk collect into vename,vsal;

close c;

/* displaying data from array type variable */

dbms_output.put_line(' All Employee Names and salaries ');

dbms_output.put_line('=====');

```

        for i in Vename.FIRST..Vename.LAST --[ 1..VSAL.COUNT ]
        loop
            dbms_output.put_line
(vename(x)||' is getting salary is '||vsal(x));
            x:=x+1;
        end loop;
        dbms_output.put_line(' end of program');

    end proc_names_sal;

```

exec proc_names_sal;

sample output:

```

PROCEDURE PROC_NAMES compiled
anonymous block completed
SMITH
ALLEN
WARD
JONES
MARTIN...

```

ii) ASSOCIATIVE ARRAYS

It is also a single Dimensional array.

These arrays are not having any limit for size like VARRAYS.

we can also index the locations of Associative array with STRING VALUES. (Oracle 9i Release 2 onwards)

					Associative	

of size)						N is size(NO specification
INT	subscripts	1		2	3.....	
String	subscripts	abc		xyz	stu	

Ex:

```

asso(1)
asso(abc)

```

Both represents first location of associative array.

```
syn: TYPE <type names > IS TABLE OF <element type>
      INDEX BY PLS_INTEGER / BINARY_INTEGER / VARCHAR2;
```

Ex: Write a procedure to display employee names and salaries using Associative arrays, bulk collect?

create or replace procedure proc_name_sal is

```
/* using explicit cursors */
```

```
    cursor c is select ename,sal from emp;
```

```
/* creating array type structures for storing employee names and salaries */
```

```
    type esal is TABLE of emp.sal%type INDEX BY PLS_INTEGER;
    type enames is TABLE of emp.ename%type INDEX BY
PLS_INTEGER;
```

```
    vsal esal;
    vename enames;
    x int:=1;
    begin
```

```
    open c;
```

```
    fetch c bulk collect      into vename,vsal;
    close c;
```

```
/* displaying data from array type variable */
```

```
    for i in Vename.FIRST..Vename.LAST --[ 1..VSAL.COUNT ]
    loop
    dbms_output.put_line(vename(x)||' is getting sal :'||vsal(x));
    x:=x+1;
    end loop;
    dbms_output.put_line(' end of program');
```

```
    end proc_name_sal;
```

```
exec proc_name_sal;
```

sample output:

```
anonymous block completed
SMITH is getting sal :800
ALLEN is getting sal :1600
WARD is getting sal :1250
JONES is getting sal :2975
MARTIN is getting sal :1250
BLAKE is getting sal :2850
CLARK is getting sal :2450
```

SCOTT is getting sal :3000
KING is getting sal :5000
TURNER is getting sal :1500
ADAMS is getting sal :1100
JAMES is getting sal :950
FORD is getting sal :3000
MILLER is getting sal :1300
end of program

Ex:

Write a procedure to display all product names, cost using bulk collect and associative Arrays?

iv) Nested Tables

Used to store table like records in the array. Each array location divided into parts. In these parts we can store multiple data types of values.

To perform above, do the following.

a) creating an array of record type structure.

--define record type structure
--define array with record type structure

----> To define a PL/SQL type for nested tables, use the

Nested table syntax:

TYPE type_name IS TABLE OF rec_type_variablename [NOT NULL];

Ex: Display empid, ename and salaries of all emps with given job?

create or replace procedure proc_empjob_rec(vjob emp.job%type) is

cursor c_emp is select empno, ename, sal from emp where job=vjob;

```
TYPE e_rec IS RECORD(  
    eid    emp.empno%type,  
    name  emp.ename%type,  
    salary emp.sal%type  
);
```

TYPE e_arr IS TABLE OF e_rec INDEX BY BINARY_INTEGER;

emp_rec e_arr;

x int:=1;

```

begin

open c_emp;
fetch c_emp bulk collect into emp_rec;
close c_emp;
dbms_output.put_line( 'Information of '||vjob||'s');
dbms_output.put_line('-----');

for i in 1..emp_rec.count
loop
dbms_output.put_line(emp_rec(x).eid||';'||emp_rec(x).name||';'||emp_rec(x).s
alary);
x:=x+1;
end loop;
end proc_empjob_rec;

```

EX:write a procedure to display
pid,pname,cost,mfg,comp_name,comp_country for all products?

```

SET SERVEROUTPUT ON
DECLARE
  TYPE country_rec IS RECORD (
                                iso_code VARCHAR2(5),
                                name      VARCHAR2(50)
                              );

```

/* Indexing by integer and no limit for the array */

```

TYPE country_arr IS TABLE OF country_rec
  INDEX BY BINARY_INTEGER;

```

country_arr					
1		2		3	

					
iso_code	name	iso_code	name	iso_code	name

```

v_country country_arr;
BEGIN

```

```

v_country(1).iso_code := 'UK';
v_country(1).name      := 'United Kingdom';
v_country(2).iso_code := 'US';
v_country(2).name      := 'United States of America';

```

```

v_country(3).iso_code := 'FR';
v_country(3).name     := 'France';
v_country(4).iso_code := 'DE';
v_country(4).name     := 'Germany';

FOR i IN 1 .. 4
LOOP
  IF v_country(i).iso_code = 'DE' THEN
    DBMS_OUTPUT.PUT_LINE('ISO code "DE" = ' || v_country(i).name);

  END IF;
END LOOP;

END;
/

```

Ex:

/* Indexing Associative array by using string values */

```

SET SERVEROUTPUT ON
DECLARE
  TYPE country_tab IS TABLE OF VARCHAR2(50)
    INDEX BY VARCHAR2(5);

  t_country country_tab;
BEGIN

  t_country('abc') := 'United Kingdom';
  t_country('xyz') := 'United States of America';
  t_country('FR')  := 'France';
  t_country('DE')  := 'Germany';

  -- Find country name for ISO code "DE"
  DBMS_OUTPUT.PUT_LINE('ISO code "DE" = ' || t_country('DE'));

END;

```

```

SET SERVEROUTPUT ON
DECLARE
  TYPE country_tab IS TABLE OF VARCHAR2(50)
    INDEX BY VARCHAR2(5);

  t_country country_tab;
BEGIN

  t_country('abc') := 'United Kingdom';
  t_country('xyz') := 'United States of America';

```

```

t_country('zzz') := 'France';
t_country('xxx') := 'Germany';

-- Find country name for ISO code "DE"
DBMS_OUTPUT.PUT_LINE('ISO code "zzz" = ' || t_country('zzz'));

END;

```

PL/SQL Procedure Parameters

IN, OUT and IN OUT parameters

```

create or replace procedure <name> ( Formal Parameters List )
is
.....
.....
end <name>;

```

Exec <Proc_name>(Actual parameters....List);

IN Parameter

How to use IN parameter properly?

Here are the rules about IN parameters:

- A formal IN parameter acts like constant. It can not be assigned with new values.
- An actual IN parameter can take a value or a variable.
- An actual IN parameter is passed by reference to the specified value or the value of the specified variable.
- An actual IN parameter will not receive any value from the formal parameter.

Ex: write a procedure to display name of given employee id?

```

create or replace procedure proc_ename(veid IN emp.empno%type)
is
vename varchar2(20);
begin
select ename into vename from emp where empno=veid;
dbms_output.put_line(' name of emp='||vename);
--veid:=7654;
end proc_ename;

```

Ex:

```

CREATE OR REPLACE PROCEDURE WELCOME AS
  SITE CHAR(80) := 'FYICenter.com';
  PROCEDURE WELCOME_PRINT(S IN CHAR) AS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Welcome to ' || S);
    -- S := 'Google.com'; -- Not allowed
  END;
BEGIN
  WELCOME_PRINT('MySpace.com');
  WELCOME_PRINT(SITE);
END;
/

```

```

SQL> EXECUTE WELCOME;
Welcome to MySpace.com
Welcome to FYICenter.com

```

OUT Parameter

How to use OUT parameter properly?

Here are the rules about OUT parameters:

- A formal OUT parameter acts like an un-initialized variable. It must be assigned with new values before the end of the procedure or function.
- An actual OUT parameter must be a variable.
- An actual OUT parameter will not pass any value to the formal parameter.
- An actual OUT parameter will receive a copy of the value from the formal parameter at the end of the procedure or function.

Ex:

```

CREATE OR REPLACE PROCEDURE WELCOME AS
  SITE CHAR(40) := 'FYICenter.com';
  MESSAGE CHAR(80);
  PROCEDURE WELCOME_PRINT(S IN CHAR, M OUT CHAR) AS
  BEGIN
    M := 'Welcome to ' || S;
  END;
BEGIN
  WELCOME_PRINT('MySpace.com', MESSAGE);
  DBMS_OUTPUT.PUT_LINE(MESSAGE);
  WELCOME_PRINT(SITE, MESSAGE);
  DBMS_OUTPUT.PUT_LINE(MESSAGE);
END;
/

```

```

SQL> EXECUTE WELCOME;
Welcome to MySpace.com
Welcome to FYICenter.com

```


Ex:

```
create or replace procedure proc_e_sal
(veid in emp.empno%type,vsal out emp.sal%type)
is
begin
select sal into vsal from emp where empno=veid;
end proc_e_sal;
```

/* calling Program */

```
declare
veid emp.empno%type;
vsal emp.sal%type;
begin
veid:=&veid;
proc_e_sal(veid,vsal);
dbms_output.put_line(' emp salary= '||vsal);
end;
```

Ex: write a procedure to "return" Name of employee for the given employee id?

```
create or replace procedure proc_ename_out(veno IN int,vename OUT
emp.ename%type)
is
begin
select ename into vename from emp where empno=veno;
dbms_output.put_line(' Vename value returned to Calling Program');
end proc_ename_out;
```

Ex: /* calling above Procedure from a Program */

```
declare
veid int;
empname varchar2(20);
Begin
veid:=&veid;
--Procedure calling stmt
Proc_ename_out(veid,empname);
dbms_output.put_line('name of '||veid|| ' is '||empname);
end;
```

IN OUT Parameter

How to use IN OUT parameter properly?

Here are the rules about IN OUT parameters:

A formal IN OUT parameter acts like an initialized variable.

An actual IN OUT parameter must be a variable.

An actual IN OUT parameter passes a copy of its value to the formal parameter when entering the procedure or function.

An actual IN OUT parameter will receive a copy of the value from the formal parameter at the end of the procedure or function.

Ex:

```
CREATE OR REPLACE PROCEDURE SWAP_TEST AS
  A NUMBER := 3;
  B NUMBER := 8;
  PROCEDURE MY_SWAP(X IN OUT NUMBER,Y IN OUT NUMBER) AS
    T NUMBER;
  BEGIN
    T := X;
    X := Y;
    Y := T;
  END MY_SWAP;
BEGIN
  MY_SWAP(A,B);
  DBMS_OUTPUT.PUT_LINE('A = ' || TO_CHAR(A));
  DBMS_OUTPUT.PUT_LINE('B = ' || TO_CHAR(B));
END;
/
```

```
SQL> EXECUTE SWAP_TEST;
```

```
A = 8
```

```
B = 3
```

TRIGGERS

TRIGGER is a data base object which can be enabled or executed implicitly for any dml operation on the table.

Generally it can be used to take a audit information of table data.

Trigger contains 3 parts:

i) trigger event

Any dml (insert or update or delete) operation is known as trigger event

ii) trigger restriction

It controls the trigger execution that is before or after dml operation.

iii) Trigger action

It implements or represent the logic or functionality of the trigger, that is what it is doing for dml operation.

Types of triggers: 12

At row level

FOR EACH ROW--It enable the trigger execution for each record inside the table.

The following are record level triggers

- before insert
- after insert
- before update
- after update
- before delete
- after delete

At statement level--Trigger is executed only once for any number affected records for a dml operation.

- before insert
- after insert
- before update
- after update
- before delete
- after delete

:NEW--It represents the new values into a column[insert & After update]

:OLD--It represents the old values from a column [Before update & before delete]

Syntax:

```
create or replace trigger <trig_name>
[before / after]
[ insert / update /delete ] of <col1>,<col2>.....
on <table name>
FOR EACH ROW
declare
-----
-----
BEGIN
-----
-----
-----
END <trig_name>;

/

trigger created.
```

Ex: table

```
create table employee
(
  ename varchar2(20),
  city varchar2(20)
);
```

```
insert into employee values('Kiran','Delhi');
insert into employee values('madhu','Delhi');
insert into employee values('DINESH','HYD');
insert into employee values('smith','texas');
```

```
select * from employee;
```

ENAME	CITY
Kiran	Delhi
madhu	Delhi
DINESH	HYD
smith	texas

Ex: write a trigger to insert or update the data of employee in upper case?

[Generally to maintain uniform data in the business Database]

```
create or replace trigger trig_upper
```

```

before
insert or update on employee
for each row
begin
:new.ename:=upper(:new.ename);
:new.city:=upper(:new.city);
end trig_UPPER;

/

```

Ex: write a trigger to maintain Audit transaction details of employee table data?

```

create table emp_audit_New
(
    ename      varchar2(20),
    city       varchar2(20),
    opname     varchar2(30),
    opdt       date
);

```

```

create table emp_audit_Changed
(
    ename      varchar2(20),
    city       varchar2(20),
    opname     varchar2(30),
    opdt       date
);

```

```

create table emp_audit_delete
(
    ename      varchar2(20),
    city       varchar2(20),
    opname     varchar2(30),
    opdt       date
);

```

```

create or replace trigger trig_emp_Audit
before
insert or update or delete on employee
for each row
declare
opname      varchar2(32);

begin

if inserting then
opname:='inserted';

```

```

        insert into emp_audit_New
values(:new.ename,:new.city,opname,sysdate);
    end if;

    if updating then
        opname:='Before Update-oldvalues';
        insert into emp_audit_Changed values
        (:old.ename,:old.city,opname,sysdate);

        opname:='After Update--new values';
        insert into emp_audit_Changed values
        (:new.ename,:new.city,opname,sysdate);
    end if;

    if deleting then
        opname:='deleting';
        insert into emp_audit_delete values
        (:old.ename,:old.city,opname,sysdate);
    end if;

    end trig_emp_Audit;

/

```

sample examples:

```
SQL> select * from employee;
```

ENAME	CITY
A	CHENNAI
b	chennai
s	blore
X	DALLAS
Y	TEXAS

```
SQL> select * from trig_emp2;
```

ENAM	CITY	OPNAME	OPDT
y	texas	insert	08-JAN-13

```
SQL> delete from employee where ename='b';
```

1 row deleted.

```
SQL> select * from trig_emp2;
```

ENAM	CITY	OPNAME	OPDT
y	texas	insert	08-JAN-13
b	chennai	delete	08-JAN-13

```
SQL> select * from employee;
```

ENAME	CITY
A	CHENNAI
s	blore
X	DALLAS
Y	TEXAS

NOTE:

system table: USER_TRIGGERS

It contains the information of triggers created in the database.

CASE statement:

The CASE statement begins with the keyword CASE. The keyword is followed by a selector.

The selector expression can be arbitrarily complex. For example, it can contain function calls. Usually, however, it consists of a single variable. The selector expression is evaluated only once. The value it yields can have any PL/SQL datatype other than BLOB, BFILE, an object type, a PL/SQL record, an index-by-table, a varray, or a nested table.

The selector is followed by one or more WHEN clauses, which are checked sequentially.

The value of the selector determines which clause is executed. If the value of the selector equals the value of a WHEN-clause expression, that WHEN clause is executed.

The ELSE clause works similarly to the ELSE clause in an IF statement. In the last example, if the grade is not one of the choices covered by a WHEN clause, the ELSE clause is selected, and the phrase 'No such grade' is output. The ELSE clause is optional. However, if you omit the ELSE clause, PL/SQL adds the following implicit ELSE clause:

```
ELSE RAISE CASE_NOT_FOUND;
```

If the CASE statement selects the implicit ELSE clause, PL/SQL raises the predefined exception CASE_NOT_FOUND. So, there is always a default action, even when you omit the ELSE clause.

The keywords END CASE terminate the CASE statement. These two keywords must be separated by a space. The CASE statement has the following form:

SYNTAX:

CASE selector

```
WHEN expression1 THEN sequence_of_statements1;
WHEN expression2 THEN sequence_of_statements2;
```

```

...
    WHEN expressionN THEN sequence_of_statementsN;
[ELSE sequence_of_statementsN+1;]
END CASE;

```

Example:

```

    declare
    grade char;
    begin
    dbms_output.put_line(' Select anyone Grade from the following');
    dbms_output.put_line(' A B C D F');
    grade:='&grade';

```

CASE grade

```

    WHEN 'A' THEN dbms_output.put_line('Excellent');
    WHEN 'B' THEN dbms_output.put_line('Very Good');
    WHEN 'C' THEN dbms_output.put_line('Good');
    WHEN 'D' THEN dbms_output.put_line('Fair');
    WHEN 'F' THEN dbms_output.put_line('Poor');
    ELSE dbms_output.put_line('Invalid grade selection');
END CASE;
END;

```

An alternative to the CASE statement is the CASE expression, where each WHEN clause is an expression.

Searched CASE Statement:-

PL/SQL also provides a searched CASE statement, which has the form:

CASE

```

    WHEN search_condition1 THEN sequence_of_statements1;
    WHEN search_condition2 THEN sequence_of_statements2;
    ...
    WHEN search_conditionN THEN sequence_of_statementsN;
[ELSE sequence_of_statementsN+1;]
END CASE;

```

The searched CASE statement has no selector. Also, its WHEN clauses contain search conditions that yield a Boolean value, not expressions that can yield a value of any type. An example follows:

CASE

```

    WHEN grade = 'A' THEN dbms_output.put_line('Excellent');
    WHEN grade = 'B' THEN dbms_output.put_line('Very Good');
    WHEN grade = 'C' THEN dbms_output.put_line('Good');
    WHEN grade = 'D' THEN dbms_output.put_line('Fair');
    WHEN grade = 'F' THEN dbms_output.put_line('Poor');
    ELSE dbms_output.put_line('No such grade');
END CASE;

```


Ex: with out FORALL

```
create table emp1
(
  eid number(4),
  empname varchar2(20),
  sal number(5),
  dept number(2)
);

create or replace procedure p1(vdno emp.deptno%Type)
is
  cursor c is select empno,ename,sal,deptno from emp where deptno=vdno;
  type e_rec is record
    (
      empno emp.empno%type,
      ename emp.ename%type,
      sal emp.sal%type,
      deptno emp.deptno%type
    );
  type e_arr is table of c%rowtype index by pls_integer;
  rec e_arr;
  a int:=1;
  begin
    open c;
    fetch c bulk collect into rec;
    close c;
    /* inserting row by row */
    for x in 1..rec.count
    loop
      insert into emp1 values(rec(a).empno,rec(a).ename,rec(a).sal,rec(a).deptno);
      a:=a+1;
    end loop;
    commit;
  end p1;
```

```
FORALL indx IN 1 .. l_eligible_ids.COUNT
31   UPDATE employees emp
32   SET emp.salary =
33       emp.salary
34       + emp.salary * increase_salary.increase_pct_in
35   WHERE emp.employee_id = l_eligible_ids (indx);
36 END increase_salary;
```

Ex: with FORALL

It is used to improve the performance while doing dml operation inside the loop.

Instead of for loop we are using FORALL keyword.

```
create table emp2
```

```
(  
  eid number(4),  
  empname varchar2(20),  
  sal number(5),  
  dept number(2)  
);
```

DECLARE

```
  TYPE subset_rt IS RECORD  
  ( empno emp.empno%TYPE  
    , ename emp.ename%TYPE  
    , hiredate emp.hiredate%TYPE  
    , deptno emp.deptno%TYPE );
```

```
  TYPE subset_aat IS TABLE OF subset_rt  
    INDEX BY PLS_INTEGER;
```

```
  aa_subset subset_aat;
```

BEGIN

```
  /* Some "source" data... */  
  SELECT ROWNUM, owner, created, 20  
  BULK COLLECT INTO aa_subset  
  FROM all_objects  
  WHERE ROWNUM <= 10;
```

```
  /* Record-based insert and subset of columns... */  
  FORALL i IN 1 .. aa_subset.COUNT  
    INSERT INTO (SELECT empno, ename, hiredate, deptno FROM emp)  
    VALUES aa_subset(i);
```

```
  DBMS_OUTPUT.PUT_LINE(TO_CHAR(SQL%ROWCOUNT) || ' rows  
inserted.');
```

```
  END;  
  /
```

UTL_FILE Example

UTL_FILE is an oracle pl/sql package that is supplied to allow PL/SQL to read and create text files in the file system.UTL_FILE can only read and create text files. Specifically, it cannot be used to read or create binary files.UTL_FILE is an appropriate tool for creating reports in the form of flat file from the database . UTL_FILE is also used for reading files.

The following steps must be followed in order to run UTL_FILE package.

Step 1) Connect as sys database user

Step 2) Create a directory:

```
SQL> CREATE OR REPLACE DIRECTORY utl_file_dir AS 'E:\PLSQL';  
Directory created.
```

In the above query utl_file_dir is the logical name for the path 'E:\PLSQL'.We can mention the logical name utl_file_dir inside the program in uppercase within single quotes (utl_file_dir is mapped to the directory 'E:\PLSQL')We can create any number of logical path names(DBA directories) in oracle 10g.

Step 3) Grant read and write on the directory

```
SQL> grant read,write on directory utl_file_dir to scott;  
Grant succeeded.
```

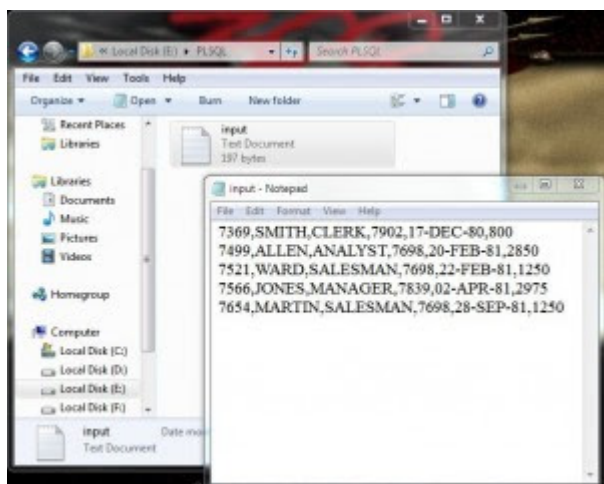
Step 4) connect as scott database user

```
SQL>CONNECT SCOTT/*****
```

Example 1:

The following is the procedure to read a text file in the file system and inserting the text file contents(rows) into a table that is present in oracle 10g database.

Step 1) Create a text file called input.txt in windows directory 'E:\PLSQL' as shown in the below screen shot.



Step 2) Check whether any contents present in emp12 table.

```
SQL>CREATE TABLE EMP12(EMPNO NUMBER(10), ENAME
VARCHAR2(15), JOB VARCHAR2(15), MGR NUMBER(4), HIREDATE
DATE, SAL NUMBER(7,2));
```

Table created.

```
SQL> desc emp12;
```

Name	Null?	Type
EMPNO		NUMBER(10)
ENAME		VARCHAR2(15)
JOB		VARCHAR2(15)
MGR		NUMBER(4)
HIREDATE		DATE
SAL		NUMBER(7,2)

```
SQL> select * from emp12;
```

no rows selected

Step 3) Execute the following procedure.

This PL/ SQL procedure will insert data(rows) from the text file called 'input.txt' to the database table emp12 in scott schema.

```
SQL> conn scott/tiger
```

Connected.

```
SQL> show user
```

USER is "SCOTT"

```
SQL>create or replace procedure read_file is
```

```
f_line varchar2(2000);
```

```
f utl_file.file_type;
```

```
f_dir varchar2(250);
```

```
fname varchar2(50);
```

```

Comma1 varchar(10);
Comma2 varchar(10);
Comma3 varchar(10);
Comma4 varchar(10);
Comma5 varchar(10);

f_empno emp.empno%type;
f_ename emp.ename%type;
f_job emp.job%type;
f_mgr emp.mgr%type;
f_hiredate emp.hiredate%type;
f_sal emp.sal%type;

begin

f_dir := 'E:\PLSQL';

fname := 'input.txt';

f := utl_file.fopen('UTL_FILE_DIR',fname,'r'); --opening the file using fopen
function

loop

begin

utl_file.get_line(f,f_line);

--using a loop continuously get the file's content using get_line function

exception

when no_data_found then

exit;

end;

Comma1 := INSTR(f_line, ',' ,1 , 1);

Comma2 := INSTR(f_line, ',' ,1 , 2);

Comma3 := INSTR(f_line, ',' ,1 , 3);

```

```

Comma4 := INSTR(f_line, ',' ,1 , 4);
Comma5 := INSTR(f_line, ',' ,1 , 5);

--Each field in the input record is delimited by commas.

--We need to find the location of two commas in the line.

--and use the locations to get the field from the line.

f_empno := to_number(SUBSTR(f_line, 1, Comma1-1));
f_ename := SUBSTR(f_line, Comma1+1, Comma2-Comma1-1);
f_job := SUBSTR(f_line, comma2+1, Comma3-Comma2-1);
f_mgr := to_number(SUBSTR(f_line, comma3+1, Comma4-Comma3-1));
f_hiredate := to_date(SUBSTR(f_line, comma4+1, Comma5-Comma4-1),'dd-
mon-yyyy');
f_sal := to_number(SUBSTR(f_line, comma5+1),'99999');

dbms_output.put_line(f_empno || ' ' || f_ename || ' ' || f_job || ' ' || f_mgr || ' ' ||
f_hiredate || ' ' || f_sal);

insert into emp12 VALUES (f_empno,f_ename,f_job,f_mgr,f_hiredate,f_sal);

end loop;

utl_file.fclose(f);

commit;

end;

/

```

Procedure created.

```
SQL> execute read_file;
```

PL/SQL procedure successfully completed.

Step 4) check the emp12 table that the rows from external text file gets inserted or not.

```
SQL> select * from emp12;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL
-------	-------	-----	-----	----------	-----

7369 800	SMITH	CLERK	7902	17-DEC-80
7499 2850	ALLEN	ANALYST	7698	20-FEB-81
7521 1250	WARD	SALESMAN	7698	22-FEB-81
7566 2975	JONES	MANAGER	7839	02-APR-81
7654 1250	MARTIN	SALESMAN	7698	28-SEP-81

Example 2:

The following is the procedure to write a database table contents to a text file.

This PL/SQL Procedure write the contents of the database table 'emp' in the scott schema to a text file called 'emp_table.txt' in the windows directory 'E:\PLSQL'

```
SQL> conn scott/tiger
```

Connected.

```
SQL> show user
```

USER is "SCOTT"

Step 1)Execute the following procedure.

create or replace procedure write_file is

```
file1 utl_file.file_type;
```

```
cursor empc is
```

```
select * from emp;
```

```
employ empc%rowtype;
```

```
stmt varchar2(300);
```

```
head varchar2(300);
```

```
line varchar2(300);
```

```

begin

file1 := utl_file.fopen('UTL_FILE_DIR','emp_table.txt','w');

utl_file.put_line(file1,'Report Generated on: ' || sysdate);

utl_file.new_line(file1);

head:='EMPNO    ENAME      JOB                MGR
HIREDATE          SAL                COMM          DEPTNO';

UTL_FILE.PUTF(file1, head);

utl_file.new_line(file1);

line:=
'=====
=====';

UTL_FILE.PUTF(file1, line);

utl_file.new_line(file1);

for employ in empc loop

stmt := rpad(employ.empno,10,' ') ||
rpad(employ.ename,20,' ') ||
rpad(employ.job,20,' ') ||
rpad(nvl(to_char(employ.mgr),' '),30,' ') ||
rpad(employ.hiredate,30,' ') ||
rpad(employ.sal,30,' ') ||
rpad(nvl(to_char(employ.comm),' '),25,' ') ||
rpad(employ.deptno,8,' ');

utl_file.PUTF(file1, stmt);

utl_file.new_line(file1);

end loop;

utl_file.fclose(file1);

end;

/

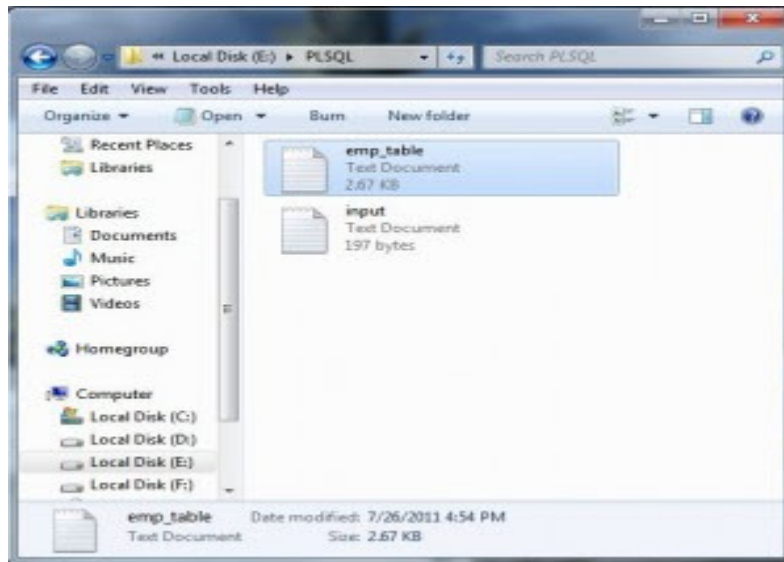
```


Procedure created.

```
SQL> execute write_file;
```

PL/SQL procedure successfully completed.

Step 2) After execution of the above procedure a new file emp_table.txt is created in the location 'E:\PLSQL' as shown in the below screenshot.



Step 3) Open the text document emp_table.txt to view the output.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	ANALYST	7698	20-FEB-81	2850	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-81	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

UTL FILE Read Example

create or replace procedure read_file
is

```
f_line  varchar2(2000);  
f       utl_file.file_type;  
f_dir   varchar2(250);  
fname   varchar2(50);  
Comma1  varchar(10);  
Comma2  varchar(10);  
Comma3  varchar(10);  
Comma4  varchar(10);
```

```

Comma5    varchar(10);
f_empno   emp.empno%type;
f_ename   emp.ename%type;
f_job     emp.job%type;
f_mgr     emp.mgr%type;
f_hiredate emp.hiredate%type;
f_sal     emp.sal%type;

begin

f_dir := 'E:\PLSQL';
fname := 'input.txt';
f := utl_file.fopen('UTL_FILE_DIR',fname,'r'); --opening the file using fopen
function
loop

utl_file.get_line(f,f_line);
--using a loop continuously get the file's content using get_line function
exception
when no_data_found then
exit;
end;
Comma1 := INSTR(f_line, ',',1,1);
Comma2 := INSTR(f_line, ',',1,2);
Comma3 := INSTR(f_line, ',',1,3);
Comma4 := INSTR(f_line, ',',1,4);
Comma5 := INSTR(f_line, ',',1,5);

--Each field in the input record is delimited by commas.
--We need to find the location of two commas in the line.
--and use the locations to get the field from the line.

f_empno := to_number(SUBSTR(f_line, 1, Comma1-1));
f_ename := SUBSTR(f_line, Comma1+1, Comma2-Comma1-1);
f_job := SUBSTR(f_line, comma2+1, Comma3-Comma2-1);
f_mgr := to_number(SUBSTR(f_line, comma3+1, Comma4-Comma3-1));
f_hiredate := to_date(SUBSTR(f_line, comma4+1, Comma5-Comma4-1),'dd-
mon-yyyy');
f_sal := to_number(SUBSTR(f_line, comma5+1),'99999');

dbms_output.put_line(f_empno || ' ' || f_ename || ' ' || f_job || ' ' || f_mgr || ' ' ||
f_hiredate || ' ' || f_sal);
insert into emp12 VALUES (f_empno,f_ename,f_job,f_mgr,f_hiredate,f_sal);
end loop;

utl_file.fclose(f);
commit;
end;
/
Procedure created.

SQL> execute read_file;
PL/SQL procedure successfully completed.

```

UTL FILE WRITE Example:

Example 2:

The following is the procedure to write a database table contents to a text file.

This PL/SQL Procedure write the contents of the database table 'emp' in the scott schema to a text file called 'emp_table.txt' in the windows directory 'E:\PLSQL'

```
SQL> conn scott/tiger
Connected.
```

```
SQL> show user
USER is "SCOTT"
```

Step 1)Execute the following procedure.

```
create or replace procedure write_file
is
```

```
file1 utl_file.file_type;
cursor empc is select * from emp;
employ empc%rowtype;
stmt varchar2(300);
head varchar2(300);
line varchar2(300);
```

```
begin
```

```
file1 := utl_file.fopen('UTL_FILE_DIR','emp_table.txt','w');
utl_file.put_line(file1,'Report Generated on: ' || sysdate);
utl_file.new_line(file1);
head:='EMPNO    ENAME      JOB                MGR      DEPTNO';
HIREDATE          SAL          COMM          DEPTNO';
UTL_FILE.PUTF(file1, head);
utl_file.new_line(file1);
line:=
'=====
=====';
UTL_FILE.PUTF(file1, line);
utl_file.new_line(file1);
```

```
for employ in empc
loop
stmt := rpad(employ.empno,10,' ') ||
rpad(employ.ename,20,' ') ||
rpad(employ.job,20,' ') ||
rpad(nvl(to_char(employ.mgr),' '),30,' ') ||
rpad(employ.hiredate,30,' ') ||
rpad(employ.sal,30,' ') ||
rpad(nvl(to_char(employ.comm),' '),25,' ') ||
rpad(employ.deptno,8,' ');
utl_file.PUTF(file1, stmt);
```

```
utl_file.new_line(file1);  
end loop;  
utl_file.fclose(file1);  
end;  
/
```

Procedure created.

SQL> execute write_file;

PL/SQL procedure successfully completed.

Step 2) After execution of the above procedure a new file emp_table.txt is created in the location 'E:\PLSQL' as shown in the below screenshot.

Step 3) Open the text document emp_table.txt to view the output.

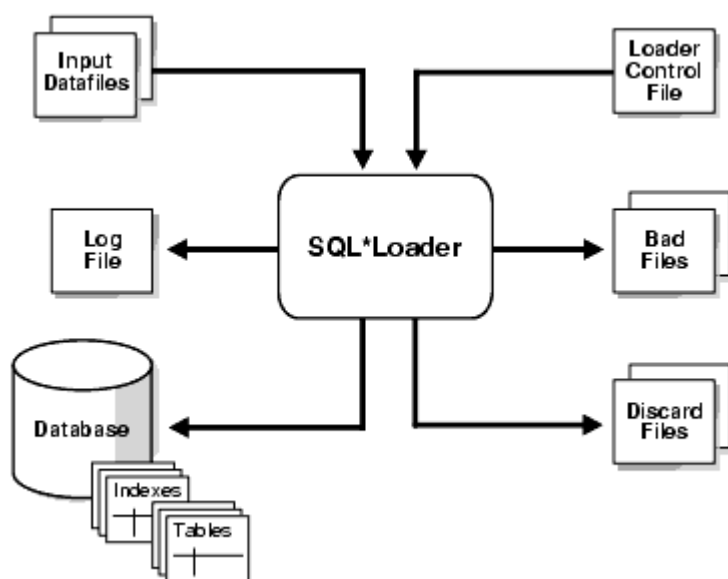
SQL * LOADER

SQL*Loader loads data from external files into tables of an Oracle database. It has a powerful data parsing engine that puts little limitation on the format of the data in the datafile. You can use SQL*Loader to do the following:

- Load data across a network if your data files are on a different system than the database.
- Load data from multiple datafiles during the same load session.
- Load data into multiple tables during the same load session.
- Specify the character set of the data.
- Selectively load data (you can load records based on the records' values).
- Manipulate the data before loading it, using SQL functions.
- Generate unique sequential key values in specified columns.
- Use the operating system's file system to access the datafiles.
- Load data from disk, tape, or named pipe.
- Generate sophisticated error reports, which greatly aid troubleshooting.
- Load arbitrarily complex object-relational data.
- Use secondary datafiles for loading LOBs and collections.

A typical SQL*Loader session takes as input a control file, which controls the behavior of SQL*Loader, and one or more datafiles. The output of SQL*Loader is an Oracle database (where the data is loaded), a log file, a bad file, and potentially, a discard file.

Figure 6-1 SQL*Loader Overview



SQL*Loader Parameters

SQL*Loader is invoked when you specify the `sqlldr` command and, optionally, parameters that establish session characteristics.

In situations where you always use the same parameters for which the values seldom change, it can be more efficient to specify parameters using the following methods, rather than on the command line:

- Parameters can be grouped together in a parameter file. You could then specify the name of the parameter file on the command line using the `PARFILE` parameter.
- Certain parameters can also be specified within the SQL*Loader control file by using the `OPTIONS` clause.

Parameters specified on the command line override any parameter values specified in a parameter file or `OPTIONS` clause.

SQL*Loader Control File

The control file is a text file written in a language that SQL*Loader understands. The control file tells SQL*Loader where to find the data, how to parse and interpret the data, where to insert the data, and more.

Although not precisely defined, a control file can be said to have three sections.

The first section contains session-wide information, for example:

- Global options such as `bindsize`, `rows`, `records to skip`, and so on
- `INFILE` clauses to specify where the input data is located
- Data to be loaded

The second section consists of one or more `INTO TABLE` blocks. Each of these blocks contains information about the table into which the data is to be loaded, such as the table name and the columns of the table.

The third section is optional and, if present, contains input data.

Some control file syntax considerations to keep in mind are:

- The syntax is free-format (statements can extend over multiple lines).
- It is case insensitive; however, strings enclosed in single or double quotation marks are taken literally, including case.
- In control file syntax, comments extend from the two hyphens (`--`) that mark the beginning of the comment to the end of the line. The optional third section of the control file is interpreted as data rather than as control file syntax; consequently, comments in this section are not supported.

- The keywords `CONSTANT` and `ZONE` have special meaning to SQL*Loader and are therefore reserved. To avoid potential conflicts, Oracle recommends that you do not use either `CONSTANT` or `ZONE` as a name for any tables or columns.

Input Data and Datafiles

SQL*Loader reads data from one or more files (or operating system equivalents of files) specified in the control file. From SQL*Loader's perspective, the data in the datafile is organized as *records*. A particular datafile can be in fixed record format, variable record format, or stream record format. The record format can be specified in the control file with the `INFILE` parameter. If no record format is specified, the default is stream record format.

Note:

If data is specified inside the control file (that is, `INFILE *` was specified in the control file), then the data is interpreted in the stream record format with the default record terminator.

Fixed Record Format

A file is in fixed record format when all records in a datafile are the same byte length. Although this format is the least flexible, it results in better performance than variable or stream format. Fixed format is also simple to specify. For example:

```
INFILE datafile_name "fix n"
```

This example specifies that SQL*Loader should interpret the particular datafile as being in fixed record format where every record is *n* bytes long.

[Example 6-1](#) shows a control file that specifies a datafile that should be interpreted in the fixed record format. The datafile in the example contains five physical records. Assuming that a period (.) indicates a space, the first physical record is [001,...cd,.] which is exactly eleven bytes (assuming a single-byte character set). The second record is [0002,fghi,\n] followed by the newline character (which is the eleventh byte), and so on. Note that newline characters are not required with the fixed record format.

Note that the length is always interpreted in bytes, even if character-length semantics are in effect for the file. This is necessary because the file could contain a mix of fields, some of which are processed with character-length semantics and others which are processed with byte-length semantics. See [Character-Length Semantics](#).

Example 6-1 Loading Data in Fixed Record Format

```
load data
```

```
infile 'example.dat' "fix 11"
into table example
fields terminated by ',' optionally enclosed by '"'
(col1, col2)
```

example.dat:

```
001,  cd, 0002,fghi,
00003,lmn,
1, "pqrs",
0005,uvwxy,
```

Variable Record Format

A file is in variable record format when the length of each record in a character field is included at the beginning of each record in the datafile. This format provides some added flexibility over the fixed record format and a performance advantage over the stream record format. For example, you can specify a datafile that is to be interpreted as being in variable record format as follows:

```
INFILE "datafile_name" "var n"
```

In this example, *n* specifies the number of bytes in the record length field. If *n* is not specified, SQL*Loader assumes a length of 5 bytes. Specifying *n* larger than 40 will result in an error.

[Example 6-2](#) shows a control file specification that tells SQL*Loader to look for data in the datafile `example.dat` and to expect variable record format where the record length fields are 3 bytes long. The `example.dat` datafile consists of three physical records. The first is specified to be 009 (that is, 9) bytes long, the second is 010 bytes long (that is, 10, including a 1-byte newline), and the third is 012 bytes long (also including a 1-byte newline). Note that newline characters are not required with the variable record format. This example also assumes a single-byte character set for the datafile.

The lengths are always interpreted in bytes, even if character-length semantics are in effect for the file. This is necessary because the file could contain a mix of fields, some processed with character-length semantics and others processed with byte-length semantics. See [Character-Length Semantics](#).

Example 6-2 Loading Data in Variable Record Format


```
load data
infile 'example.dat' "var 3"
into table example
fields terminated by ',' optionally enclosed by '"'
(col1 char(5),
 col2 char(7))

example.dat:

009hello,cd,010world,im,
012my,name is,
```

Stream Record Format

A file is in stream record format when the records are not specified by size; instead SQL*Loader forms records by scanning for the *record terminator*. Stream record format is the most flexible format, but there can be a negative effect on performance. The specification of a datafile to be interpreted as being in stream record format looks similar to the following:

```
INFILE datafile_name ["str terminator_string"]
```

The *terminator_string* is specified as either '*char_string*' or *X'hex_string'* where:

- '*char_string*' is a string of characters enclosed in single or double quotation marks
- *X'hex_string'* is a byte string in hexadecimal format

When the *terminator_string* contains special (nonprintable) characters, it should be specified as an *X'hex_string'*. However, some nonprintable characters can be specified as ('*char_string*') by using a backslash. For example:

- \n indicates a line feed
- \t indicates a horizontal tab
- \f indicates a form feed
- \v indicates a vertical tab
- \r indicates a carriage return

If the character set specified with the NLS_LANG parameter for your session is different from the character set of the datafile, character strings are converted to the character set of the datafile. This is done before SQL*Loader checks for the default record terminator.

Hexadecimal strings are assumed to be in the character set of the datafile, so no conversion is performed.

On UNIX-based platforms, if no *terminator_string* is specified, SQL*Loader defaults to the line feed character, `\n`.

On Windows NT, if no *terminator_string* is specified, then SQL*Loader uses either `\n` or `\r\n` as the record terminator, depending on which one it finds first in the datafile. This means that if you know that one or more records in your datafile has `\n` embedded in a field, but you want `\r\n` to be used as the record terminator, you must specify it.

[Example 6-3](#) illustrates loading data in stream record format where the terminator string is specified using a character string, `'\\n'`. The use of the backslash character allows the character string to specify the nonprintable line feed character.

Example 6-3 Loading Data in Stream Record Format

```
load data
infile 'example.dat' "str '\\n'"
into table example
fields terminated by ',' optionally enclosed by '"'
(col1 char(5),
 col2 char(7))

example.dat:
hello,world,|
james,bond,|
```

Logical Records

SQL*Loader organizes the input data into physical records, according to the specified record format. By default a physical record is a logical record, but for added flexibility, SQL*Loader can be instructed to combine a number of physical records into a logical record.

SQL*Loader can be instructed to follow one of the following logical record-forming strategies:

- Combine a fixed number of physical records to form each logical record.
- Combine physical records into logical records while a certain condition is true.

Data Fields

Once a logical record is formed, field setting on the logical record is done. Field setting is a process in which SQL*Loader uses control-file field specifications to determine which parts of logical record data correspond to which control-file fields. It is possible for two or more field specifications to claim the same data. Also, it is possible for a logical record to contain data that is not claimed by any control-file field specification.

Most control-file field specifications claim a particular part of the logical record. This mapping takes the following forms:

- The byte position of the data field's beginning, end, or both, can be specified. This specification form is not the most flexible, but it provides high field-setting performance.
- The strings delimiting (enclosing and/or terminating) a particular data field can be specified. A delimited data field is assumed to start where the last data field ended, unless the byte position of the start of the data field is specified.
- The byte offset and/or the length of the data field can be specified. This way each field starts a specified number of bytes from where the last one ended and continues for a specified length.
- Length-value datatypes can be used. In this case, the first *n* number of bytes of the data field contain information about how long the rest of the data field is.

Control File Contents

The SQL*Loader control file is a text file that contains data definition language (DDL) instructions. DDL is used to control the following aspects of a SQL*Loader session:

- Where SQL*Loader will find the data to load
- How SQL*Loader expects that data to be formatted
- How SQL*Loader will be configured (memory management, rejecting records, interrupted load handling, and so on) as it loads the data
- How SQL*Loader will manipulate the data being loaded

To create the SQL*Loader control file, use a text editor such as vi or xemacs.

In general, the control file has three main sections, in the following order:

- Session-wide information
- Table and field-list information
- Input data (optional section)

Example 8-1 Sample Control File

```
1  -- This is a sample control file
2  LOAD DATA
3  INFILE 'sample.dat'
4  BADFILE 'sample.bad'
5  DISCARDFILE 'sample.dsc'
6  APPEND
7  INTO TABLE emp
8  WHEN (57) = '.'
9  TRAILING NULLCOLS
10 (hiredate SYSDATE,
    deptno POSITION(1:2) INTEGER EXTERNAL(2)
        NULLIF deptno=BLANKS,
    job   POSITION(7:14) CHAR TERMINATED BY WHITESPACE
        NULLIF job=BLANKS "UPPER(:job)",
    mgr   POSITION(28:31) INTEGER EXTERNAL
        TERMINATED BY WHITESPACE, NULLIF mgr=BLANKS,
    ename POSITION(34:41) CHAR
        TERMINATED BY WHITESPACE "UPPER(:ename)",
    empno POSITION(45) INTEGER EXTERNAL
        TERMINATED BY WHITESPACE,
    sal   POSITION(51) CHAR TERMINATED BY WHITESPACE
        "TO_NUMBER(:sal,'$99,999.99')",
    comm  INTEGER EXTERNAL ENCLOSED BY '(' AND '%'
        "(:comm * 100"
)
```

In this sample control file, the numbers that appear to the left would not appear in a real control file. They are keyed in this sample to the explanatory notes in the following list:

1. This is how comments are entered in a control file. See [Comments in the Control File](#).
2. The **LOAD DATA** statement tells SQL*Loader that this is the beginning of a new data load. See [Appendix A](#) for syntax information.
3. The **INFILE** clause specifies the name of a datafile containing data that you want to load. See [Specifying Datafiles](#).
4. The **BADFILE** clause specifies the name of a file into which rejected records are placed. See [Specifying the Bad File](#).
5. The **DISCARDFILE** clause specifies the name of a file into which discarded records are placed. See [Specifying the Discard File](#).
6. The **APPEND** clause is one of the options you can use when loading data into a table that is not empty. See [Loading Data into Nonempty Tables](#).

To load data into a table that is empty, you would use the **INSERT** clause. See [Loading Data into Empty Tables](#).

7. The **INTO TABLE** clause enables you to identify tables, fields, and datatypes. It defines the relationship between records in the datafile and tables in the database. See [Specifying Table Names](#).
8. The **WHEN** clause specifies one or more field conditions. SQL*Loader decides whether or not to load the data based on these field conditions. See [Loading Records Based on a Condition](#).
9. The **TRAILING NULLCOLS** clause tells SQL*Loader to treat any relatively positioned columns that are not present in the record as null columns. See [Handling Short Records with Missing Data](#).
10. The remainder of the control file contains the field list, which provides information about column formats in the table being loaded.
Comments in the Control File

Comments can appear anywhere in the command section of the file, but they should not appear within the data. Precede any comment with two hyphens, for example:

```
--This is a comment
```

All text to the right of the double hyphen is ignored, until the end of the line.

Sample execution of SQLLDR from command window

```
C:\Program Files>cd oracle
```

```
C:\Program Files\Oracle>cd temp
```

```
C:\Program Files\Oracle\temp>sqlldr  
scott/tiger@127.0.0.1/pdb1 data=dependents.txt  
control=dataload.txt
```

ORACLE QUESTIONS

1) Can we create a New table with any old table structure ?

A: Yes we can. That is while creation of new table ,use any false condition as below example.

Ex:

```
create table emp_new
as
select * from emp_old where 1=2;
```

2) What is Materialized view?

A: A view which contains physical data dynamically is known as materialized view.

Materialized views are created to perform analysis on running business data.

3) How to enable or disable constraint on any table?

A: By using ALTER command , we can enable or disable the following constraints.

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE KEY
- CHECK CONSTRAINT

Ex: alter table <table_name>

ENABLE / DISABLE <constraint_name>;

4) Can we add a constraint if the column having invalid data according to constraint?

A: Yes we can add a constraint without checking existing data ,by using NO VALIDATE CONSTRAINT.

[But before doing above , the table created with DIFFERABLE column. On this column

we can add NO VALIDATE constraint]

5) What is ROWID and its use?

A: It is a pseudo column ,which contains physical address of each record.

6) How to find nth highest salary?

A: select distinct *
from(select sal,dense_rank() over(order by sal desc) rank from emp)
where rank=n

Note: In the above example , n value is any integer value like 1,2,3,4.. . .

7) What is the difference between TRUNCATE and DELETE operations?

A: Truncate command delete all the records from the table permanently.
Delete command delete all or some of records from the table , but we can restore deleted records using rollback.

8) What is the difference between Inner join and Correlated join?

A: Inner join will display only matched data from both tables according to join condition.
OUTER join will display complete data from any one table and only matched data according to join condition.

9) We are not allowed to perform one dml operation on COMPOSITE VIEW, what is that?

A: Insert operation is not allowed on any composite view.

10) Can we add ON DELETE CASCADE to the existing FOREIGN KEY column?

A: No .

11) What is CORRELATED subquery, Is it increasing performance?

A: A sub query which depends on Outer query result is known as Correlated subquery.
It will decrease performance.

12) What is the use index , why it is and what is the difference between view and index?

A: Index is a data base object which contains ordered data from column ,physically.
Index is useful to search for the required data as much as fast.

View is a logical table.
Index is like a physical table.

13) what is GROUP BY clause?

A: Group by clause internally makes logical groups based on distinct values from column.

14) what is the difference between HAVING and WHERE clauses?

A: Where clause is used to filter table data.
Having clause is used to filter GROUP BY output.

15) what are the differences between PROCEDURES AND FUNCTIONS?

A: By default, Procedures cannot return any value.

By default, functions return a value.

Procedures cannot be called using SELECT QUERY
we can call a function from any other object.

16) What is the instead of triggers?

A: Instead of triggers are implemented on composite views only.

17) What is FOR EACH ROW in a trigger?

A: It makes the trigger as ROW LEVEL TRIGGER.

18) What is the use of BULK COLLECT?

A: Bulk collect reduces the CONTEXT SWITCH between cursor area and program area.
so it improves performance of pl/sql program.

19) What are PL/SQL TABLES?

A: Multi dimensional arrays are known as PL/SQL tables, to store a group of records
with same structure at contiguous memory locations.

20) How do I create user defined record structure?

A: By using TYPE, WE CAN create user defined record.

21) What is a local procedure?

A: A procedure which is created inside a package is known as local procedure.

22) What is a cursor?

A: A cursor is a pointer to the temporary memory, to perform intermediate operations
before output delivered to the client.

23) How can I see the source code of any database object?

A: select
dbms_metadata.get_ddl('object_type','Object_name')
from dual;

1) Select the answer for the following query

DROP TABLE CUST_DTLS;

- a) Delete customer table records
- b) Delete customer table structure
- c) Delete table structure and records
- d) None of the above

2) View is a

- a) contains physical data
- b) contains logical data
- c) contains no data
- d) all of above

3) What is the output from below query, if emp table has 10 records?

```
SELECT 1000 FROM DUAL;
```

- a) No output
- b) 1000 printed one time
- c) 1000 printed 1000 times
- d) 1000 printed 10 times

4) Can we select data from INDEX?

- a) YES
- b) NO
- c) SOME TIMES
- d) None of above

5) What is a CORRELATED subquery?

- a) A query depends on outer query
- b) A query depends on inner query
- c) Both a & b
- d) None

6) How to make updates and inserts at a time
(using single command)?

- a) UPDATE
- b) INSERT
- c) MERGE
- d) UPSERT

7) What is the difference between Inner join and Outer join?

- a) Innerjoin get all data from all tables & Outer join get only data from one table
- b) Inner join get matched data from all tables based on join condition & Outer join gets all data from one table and get matched data from other table
- c) Inner join get un-matched data and outer join get matched data.
- d) none of above

8) What is the command to enable a Primary Key constraint?

- a) create command with alter keyword
- b) ALTER command with ENABLE CONSTRAINT
- c) INSERT command WITH ADD keyword
- d) None

9) Can i perform insert operation on a JOIN VIEW?

- a) YES
- b) NO

10) Write a query to get top-n records from a table?

- a) select top-n from <table_name>;
- b) select * from <table_name> where col=top-n;
- c) select * from <table_name> where rownum<=n;
- d) select * from <table_name>;

11) How do i restore deleted table from recyclebin?

- a) Using RESTORE command
- b) Using ROLLBACK command
- c) Using FLASHBACK command
- d) Using GETBACK command

12) What is the output from the below query?

```
SELECT COMM,NVL(comm,'not available') from emp;
```

- a) Prints commission values and the string at null values
- b) prints all commissions with given string
- c) No output
- d) Prints ERROR message

13) Can i use HAVING clause without GROUP BY clause in a query?

- a) YES
- b) NO

14) What is the use of COUNT(colname) function?

- a) Prints number of values from the column
- b) prints only number of not null values
- c) Prints number of records from a table
- d) prints number of null records

15) What is the function to find "nth " occurrence of a "char"?

- a) SUBSTR()
- b) INSTR()
- c) LTRIM()
- d) RTRIM()

16) How to declare a variable dynamically in PLSQL program?

- a) %ROWTYPE
- b) TYPE
- c) %CURSOR
- d) %TYPE

17) What is a stored procedure?

- a) a program saved in database permanently

- b) a program not saved in database
- c) a program executed implicitly
- d) a program not executed at all.

18) What is the use of explicit cursor?

- a) To get multiple tables in to program
- b) To get multiple objects in to program
- c) To get multiple records in to a program
- d) To get multiple users into a program

19) How can i see the names of all PROCEDURES?

- a) select * from tab;
- b) select * from procedures;
- c) select * from user_programs;
- d) select object_name from user_procedures;

20) Can i call a procedure from a SELECT query?

- a) yes
- b) NO
- c) May be
- d) None

SQL FAQs

1. Explain: a.) Integrity Constraints

It can be called as a declarative way in order to define a business rule for a table's column

2) Index

- It can be called as an optional structure which is associated with a table for direct access to the rows
- Index can be created for one or more columns in a table

3) Extent

- It can be defined as a specific number of contiguous data blocks in single allocation.
- It is used to store a specific type of information.

4) List the types of joins used in writing SUBQUERIES.

- Self join
- Outer Join
- Equi-join

5) List the various Oracle database objects.

- TABLES
- VIEWS
- INDEXES
- SYNONYMS
- SEQUENCES
- TABLESPACES

6) Tell about the use of i.)Rename ii.)Alias.

- Rename - It is a permanent name provided to a table or column.
- Alias - It is a temporary name provided to a table or column which gets over after the execution of SQL statement.

7) What is a view?

- It is virtual table which is defined as a stored procedure based on one or more tables.

8) What are the various components of physical database structure of Oracle database?

Oracle database comprises of three kinds of files:

- Datafiles,
- Redo log files,
- Control files.

9) List out the components of logical database structure of Oracle database.

- Tablespaces
- Database's schema objects.

10) What do you mean by a tablespace?

- These are the Logical Storage Units into which a database is divided.
- It is used to group together the related logical structures.

11) What is a synonym? What are its various types?

A synonym can be called as an alias for a table, view, sequence or program unit. It is basically of two types:

- Private - Only the owner can access it.
- Public - Can be accessed by any database user.

12) What are the uses of synonyms?

- Mask the real name and owner of an object.
- Provide public access to an object
- Provide location transparency for tables, views or program units of a remote database.
- Simplify the SQL statements for database users.

13) What do you mean by a deadlock?

- When two processes are waiting to update the rows of a table which are locked by another process, the situation is called a deadlock.
- The reasons for it to happen are:
 - lack of proper row lock commands.
 - Poor design of front-end application
- It reduces the performance of the server severely.
- These locks get automatically released automatically when a commit/rollback operation is performed or any process is killed externally.

14) What suggestions do you have to reduce the network traffic?

Following are some of the actions which can be taken to reduce the network traffic:

- Use snapshots to replicate data.
- Use remote procedure calls.
- Replicate data in distributed environment.

15) What is a sub query? What are its various types?

- Sub Query also termed as Nested Query or Inner Query is used to get data from multiple tables.
- A sub query is added in the where clause of the main query.

There can be two types of subqueries:

16) Explain me about Correlated sub query ?

- It can reference column in a table listed in the from list of the outer query but is not as independent as a query. .

17) Will you be able to store pictures in the database? Explain.

- Yes, pictures can be stored in the database using Long Raw Data type.

18) Differentiate between: a.) TRANSLATE and REPLACE.

- Translate is used to substitute character by character.
- Replace is used to substitute a single character with a word.

19) What are the various Oracle Database objects?

Various database objects are as follows:

- Tables – This is a set of elements organized in vertical and horizontal fashion.
- Tablespaces – This is a logical storage unit in Oracle.

- Views – It is virtual table derived from one or more tables.
- Indexes – This is a performance tuning method to process the records.
- Synonyms – This is a name for tables.
- Sequences.

20) What is the purpose of Save Points in Oracle database?

- Save Points are used to divide a transaction into smaller phases.
- It enables rolling back part of a transaction.
- Maximum 5 save points are allowed in Oracle Database.
- Whenever an error is encountered, it is possible to rollback from the point where the SAVEPOINT has been saved.

PL/SQL FAQs

1) Differentiate between % ROWTYPE and TYPE RECORD.

Ans:

% ROWTYPE is used when a query returns an entire row of a table or view. TYPE RECORD, on the other hand, is used when a query returns column of different tables or views.

Eg. TYPE r_emp IS RECORD (sno smp.smpno%type,sname smp sname %type)

e_rec smp ROWTYPE

Cursor c1 is select smpno,dept from smp;

e_rec c1 %ROWTYPE

2) Explain uses of cursor.

Ans:

Cursor is a named private area in SQL from which information can be accessed. They are required to process each row individually for queries which return multiple rows.

3) Show code of a cursor for loop.

Ans:

Cursor declares %ROWTYPE as loop index implicitly. It then opens a cursor, gets rows of values from the active set in fields of the record and shuts when all records are processed.

Eg. FOR smp_rec IN C1 LOOP

totalsal=totalsal+smp_recsal;

ENDLOOP;

4) Explain the uses of database trigger.

Ans:

A PL/SQL program unit associated with a particular database table is called a database trigger. It is used for :

1)Audit data modifications.

2)Log events transparently.

3)Enforce complex business rules.

4)Maintain replica tables

5)Derive column values

6)Implement Complex security authorizations

5) What are the two types of exceptions.

Ans:

Error handling part of PL/SQL block is called Exception. They have two types : user_defined and predefined.

6) Explain about following predefined exceptions.

Ans:

ZERO_DIVIDE

CURSOR_ALREADY_OPEN
VALUE_ERROR

7) Explain about Raise_application_error.

Ans:

It is a procedure of package DBMS_STANDARD that allows issuing of user_defined error messages from database trigger or stored sub-program.

8) Show how functions and procedures are called in a PL/SQL block.

Ans:

Function is called as a part of an expression.

total:=calculate_sal('b644')

Procedure is called as a statement in PL/SQL.

calculate_bonus('b644');

9) Differentiate between Syntax and runtime errors.

Ans:

A syntax error can be easily detected by a PL/SQL compiler. For eg, incorrect spelling.

A runtime error is handled with the help of exception-handling section in an PL/SQL block. For eg, SELECT INTO statement, which does not return any rows.

10) Explain Commit, Rollback and Savepoint.

Ans:

For a COMMIT statement, the following is true:

Other users can see the data changes made by the transaction.

The work done by the transaction becomes permanent.

A ROLLBACK statement gets issued when the transaction ends, and the following is true.

The work done in a transition is undone as if it was never issued.

It undoes all the work done by the user in a transaction. With SAVEPOINT, only part of transaction can be undone.

11) Define Implicit and Explicit Cursors.

Ans:

A cursor is implicit by default. The user cannot control or process the information in this cursor.

If a query returns multiple rows of data, the program defines an explicit cursor. This allows the application to process each row sequentially as the cursor returns it.

12) When is a declare statement required?

Ans:

DECLARE statement is used by PL/SQL anonymous blocks such as with stand alone, non-stored procedures. If it is used, it must come first in a stand alone file.

13) How many triggers can be applied to a table?

Ans:

A maximum of 12 triggers can be applied to one table.

14) What is the importance of SQLCODE and SQLERRM?

Ans:

SQLCODE returns the value of the number of error for the last encountered error whereas SQLERRM returns the message for the last error.

15) If a cursor is open, how can we find in a PL/SQL Block?

Ans:

the %ISOPEN cursor status variable can be used.

16) Show the two PL/SQL cursor exceptions.

Ans:

Cursor_Already_Open

Invalid_cursor

17) What operators deal with NULL?

Ans:

NVL converts NULL to another specified value.

var:=NVL(var2,'Hi');

IS NULL and IS NOT NULL can be used to check specifically to see whether the value of a variable is NULL or not.

18) Does SQL*Plus also have a PL/SQL Engine?

Ans:

No, SQL*Plus does not have a PL/SQL Engine embedded in it. Thus, all PL/SQL code is sent directly to database engine. It is much more efficient as each statement is not individually stripped off.

19) Explain 3 basic parts of a trigger.

Ans:

A triggering statement or event.

A restriction

An action

20) Show the cursor attributes of PL/SQL.

Ans:

%ISOPEN : Checks if the cursor is open or not

%ROWCOUNT : The number of rows that are updated, deleted or fetched.

%FOUND : Checks if the cursor has fetched any row. It is true if rows are fetched

%NOT FOUND : Checks if the cursor has fetched any row. It is True if rows are not fetched.

21) How would you reference column values BEFORE and AFTER you have inserted and deleted triggers?

Ans:

Using the keyword "new.column name", the triggers can reference column values by new collection. By using the keyword "old.column name", they can reference column values by old collection.

22) What are the uses of SYSDATE and USER keywords?

Ans:

SYSDATE refers to the current server system date. It is a pseudo column. USER is also a pseudo column but refers to current user logged onto the session. They are used to monitor changes happening in the table.

23) How does ROWID help in running a query faster?

Ans:

ROWID is the logical address of a row, it is not a physical column. It composes of data block number, file number and row number in the data block. Thus, I/O time gets minimized retrieving the row, and results in a faster query.

24) What does fetching a cursor do?

Ans:

Fetching a cursor reads Result Set row by row.

25) What does closing a cursor do?

Ans:

Closing a cursor clears the private SQL area as well as de-allocates memory

26) Differ between Anonymous blocks and sub-programs.

Ans:

Anonymous blocks are unnamed blocks that are not stored anywhere but sub-programs are compiled and stored in database. They are compiled at runtime.

27) Differ between DECODE and CASE.

Ans:

DECODE and CASE statements are very similar, but CASE is extended version of DECODE. DECODE does not allow Decision making statements in its place.

```
select decode(totalsal=12000,'high',10000,'medium') as decode_tesr from  
smp where smpno in (10,12,14,16);
```

This statement returns an error.

You can watch my youtube training videos using below link.

www.youtube.com/durgasoft/Dineshoraclevideos

https://www.youtube.com/results?search_query=oracle+videos+by+dinesh